

FR-6

Money Runway

เงินพอถึงวันไหน

เข้าใจระบบทั้งหมดตั้งแต่ศูนย์ จนอธิบายโค้ดและตอบ

คำถามอาจารย์ได้ เหมือนเป็นคนสร้างระบบนี้เอง

ภาค ก · พื้นฐานทั้งระบบ (สำหรับคนไม่เคยเขียนโค้ด)

ภาค ข · FR-6 ละเอียดทุกส่วน

โค้ดจริงพร้อมคำอธิบายทีละบรรทัด



วิธีใช้เล่มนี้

เล่มนี้ทำให้คนที่รับผิดชอบ FR-6 Money Runway เข้าใจระบบทั้งหมด ตั้งแต่ "แอปนี้สร้างด้วยอะไร" ไปจนถึง "บรรทัดนี้ของโค้ดทำอะไร" แม้ไม่เคยเขียนโปรแกรมมาก่อน

เล่มนี้แบ่งเป็น 2 ภาค

📌 ภาค ก · พื้นฐานทั้งระบบ

เหมือนกันทุกเล่ม (FR-1 ถึง FR-6) สอนตั้งแต่ศูนย์: เครื่องมือที่ใช้ (VS Code, Node.js, Git), ภาษา TypeScript, React, การทำแอปและเว็บจากโค้ดชุดเดียว, เซิร์ฟเวอร์ Supabase, การเชื่อม AI, การออกแบบหน้าจอ (UX/UI), การทดสอบ และ GitHub

★ ภาค ข · FR-6 Money Runway

เฉพาะ FR ของเล่มนี้ เขียนในมุม "ฉันเป็นคนสร้างระบบนี้" มีหน้าจอจริง หลักการ ตัวอย่างคิดมือ โค้ดจริงพร้อมคำอธิบายทีละบรรทัด การทดสอบ เหตุผลการออกแบบ คำถามอาจารย์ แบบฝึกหัด และสรุปที่น่าเสนอ

เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม

โค้ดทุกก้อนในเล่มนี้ ดึงมาจากไฟล์จริงในโปรเจกต์ ไม่ได้พิมพ์ใหม่ เลขบรรทัดทางซ้ายจึงตรงกับในไฟล์ทุกตัว ทุกก้อนมีแถบสำแบบนี้:

src/domain/runway.ts

ตัวอย่าง · บรรทัด 17-23

```
17 * so the status is "no_spending" and no date is shown.
18 */
19 import { addDays, bkkDayKey, daysBetween } from './dates';
20 import { confirmedOnly } from './summary';
21 import type { Transaction } from './types';
22
23 export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:17

- 1 **แตะชื่อไฟล์ หรือ "เปิดใน VS Code (เว็บ)"** ระบบจะเปิด VS Code ในเบราว์เซอร์ (vscode.dev) พร้อมโปรเจกต์ MindPay และกระโดดไปที่บรรทัดนั้นเลย ไม่ต้องติดตั้งอะไร ใช้ได้ทั้งคอมและแท็บเล็ต ครั้งแรกอาจถามให้เข้าสู่ระบบ GitHub ก่อน
- 2 **แตะ "GitHub"** เปิดไฟล์เดียวกันบนเว็บไซต์ GitHub และไฮไลต์บรรทัดเหล่านั้น (ใช้ได้บนมือถือ)
- 3 **ถ้าโคลนโปรเจกต์ลงเครื่องแล้ว** (วิธีอยู่ในบท A3) เปิด VS Code กด **Ctrl+P** (Mac: **Cmd+P**) แล้วพิมพ์ข้อความตัวหนาที่มุมขวา เช่น **src/domain/runway.ts:17** แล้วกด Enter จะเปิดไฟล์และไปที่บรรทัด 17 ทันที

ชื่อไฟล์สีเขียวขีดเส้นใต้ในเนื้อหา เช่น [src/domain/runway.ts](#) ก็แตะเปิดได้เหมือนกัน

📌 ลิงก์ล็อกไว้ที่เวอร์ชันของเล่มนี้

ทุกลิงก์ชี้ไปที่โค้ดเวอร์ชันวันที่ 3 ต.ค. 2569 (commit 646b485) ซึ่งเป็นเวอร์ชันที่ใช้เขียนเล่มนี้ ถ้าทีมแก้ไขโค้ดหลังจากนี้ ลิงก์ในเล่มยังเปิดเวอร์ชันเดิมที่เลขบรรทัดตรงกับในเล่ม ถ้าอยากดูเวอร์ชันล่าสุด ให้เปลี่ยนไปที่ branch main ใน VS Code หรือ GitHub

อ่านคำอธิบายโค้ดอย่างไร

หลังโค้ดแต่ละก้อนจะมีตาราง "อ่านทีละบรรทัด" คอลัมน์ซ้ายคือเลขบรรทัด คอลัมน์ขวาคือคำอธิบายเป็นภาษาคน ให้เปิดโค้ดกับตารางคู่กัน อ่านบรรทัดในโค้ดก่อน แล้วค่อยอ่านคำอธิบาย ถ้าเจอคำที่ไม่รู้จัก ให้กลับไปดูบท A4 ถึง A7 ซึ่งสอนพื้นฐานภาษาตั้งแต่ศูนย์

บรรทัด	ความหมาย
17	ตัวอย่าง: คำอธิบายของบรรทัด 17 จะอยู่แถวนี้
18–20	ถ้าหลายบรรทัดทำงานเรื่องเดียวกัน จะอธิบายรวมกันเป็นช่วง

กล่องสีต่าง ๆ ในเล่ม

≈ เปรียบเทียบ

เทียบกับของในชีวิตจริง ให้เห็นภาพก่อนลงรายละเอียด

★ จำให้ได้

เรื่องสำคัญที่อาจารย์มักถาม

!ระวัง

จุดที่คนมักเข้าใจผิด หรือบีกที่เคยเกิดขึ้นจริง

i เพิ่มเติม

รายละเอียดเสริม ข้ามได้ถ้ารีบ

ถ้ามีเวลาน้อย อ่านตามนี้

เวลาที่มี	อ่านอะไร
15 นาที	บท A1 (MindPay คืออะไร), บท A2 (แผนที่ระบบ), แล้วภาค ข บทที่ 1 และบทสุดท้าย (สรุปต้นน้ำเสนอ)
1–2 ชั่วโมง	ภาค ก บท A1, A2, A4, A9 แล้วอ่านภาค ข ทั้งหมดยกเว้นตารางโค้ดที่ละบรรทัด
เตรียมตอบอาจารย์	ภาค ข บท "ทำไมฉันออกแบบแบบนี้", "ข้อจำกัด", "คำถามที่อาจารย์อาจถาม" และ "แบบฝึกหัด"
อยากอธิบายโค้ดได้จริง	อ่านทั้งเล่มตามลำดับ เปิด VS Code คู่กัน และลองทำแบบฝึกหัดโดยไม่ดูเฉลย

i ตัวเลขในเล่ม

ตัวอย่างคิดมือและเฉลยแบบฝึกหัดทุกข้อ ตรวจโดยรันฟังก์ชันจริงของแอปแล้ว ภาพหน้าจอตายจากเว็บเวอร์ชันจริงในโหมดทดลอง (ข้อมูลตัวอย่าง) วันที่ 9 ต.ค. 2569 ข้อมูลตัวอย่างสร้างตามวันที่เปิดแอป ถ้าเปิดดูเองตัวเลขจะต่างไปเล็กน้อย แต่หลักการเหมือนเดิมทุกอย่าง

สารบัญ

วิธีใช้เล่มนี้	1
เล่มนี้แบ่งเป็น 2 ภาค	1
เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม	1
อ่านคำอธิบายโค้ดยังไ้	1
กล่องสีต่าง ๆ ในเล่ม	2
ถ้ามีเวลาน้อย อ่านตามนี้	2
พื้นฐานทั้งระบบ MINDPAY	9
MindPay คืออะไร และทีมเราสร้างอะไร	10
ปัญหาที่ทีมเราอยากแก้	10
6 ฟีเจอร์หลัก (Functional Requirements)	10
หน้าตาแอปโดยรวม	10
ใช้แอปได้ 3 แบบ	11
ตัวเลขของโปรเจกต์	12
คำศัพท์ที่จะเจอบ่อยในเล่มนี้	12
แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังไ้	14
ระบบมี 4 ส่วนใหญ่	14
ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น	14
ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน	15
ตามข้อมูลที่ไ้ใช้ AI: สแกนสลิป 1 ใบ	19
เครื่องมือ: เขียนโค้ดที่ไไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์	20
กล่องเครื่องมือของทีมเรา	20
ติดตั้งเครื่องมือ (ถ้าครั้งเดียว)	20
เอาโค้ด MindPay มาไว้ในเครื่อง	20
ไฟล์ package.json: บัตรประชาชนของโปรเจกต์	21
คำสั่งที่ใช้บ่อย	23
รู้จักหน้าจั่ว VS Code	24
คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า	24
ส่วนเสริม (Extensions) ที่แนะนำ	25
ไ้ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ	25
ฝึกใช้: ตามรอย Money Runway ใน 5 นาที	25
พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไ้เคยเขียนเลย	26
โปรแกรมคืออะไร	26
1. ค่า (value) และชนิดของค่า	26
2. ตัวแปร: กล่องที่มีชื่อ	26
3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ	26
4. เงื่อนไข if: ถ้า... ให้ทำ...	27
5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก	27
6. Array: รายการของหลายค่า	28
7. Object: ของ 1 ชิ้นที่มีหลายช่อง	29
8. วงซ้ำ (loop)	30
9. ข้อความ (string) ที่ไ้ใช้บ่อย	30
10. คณิตศาสตร์ที่ไ้ใช้	30
แบบฝึกหัด A4	31
TypeScript: ภาษาที่ทีมเราไ้ใช้ และชนิดข้อมูลของ MindPay	32
ทำไมต้อง TypeScript	32
การบอกชนิดข้อมูล (type annotation)	32
ไฟล์ชนิดข้อมูลกลาง: src/domain/types.ts	32
ชนิดข้อมูลแบบพิเศษที่ไ้เจอบ่อย	34

import และ export: แคร่โค้ดระหว่างไฟล์	34
งานที่ต้องรอ: async, await และ Promise	35
จัดการความผิดพลาด: try, catch, throw	35
ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)	36
อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์	36
Regular expression (regex) อ่านยังงี้	38
แปลงสตางค์กลับเป็นข้อความบาท	38
React: วาดหน้าจอจากข้อมูล	40
แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล	40
Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน	40
State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่	42
Hook ที่ใช้ในโปรเจกต์	43
Context: แคร่ข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ	44
ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังงี้	45
React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ	47
React Native คืออะไร	47
Expo: ชุดเครื่องมือที่ทำให้ React Native ย่างขึ้น	48
Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า	48
อ่านโค้ด: _layout.tsx โครงของทั้งแอป	49
แถบเมนูล่าง: (tabs)/_layout.tsx	52
โค้ดที่ต่างกันระหว่างมือถือกับเว็บ	54
จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง	55
วิธีอ่านโค้ด MindPay ด้วยตัวเอง	57
ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น	57
รูปแบบที่เจอซ้ำทั้งแอป (ตารางโกง)	57
ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)	58
แผนที่: อยากรู้เรื่องนี้ ไปเปิดไฟล์ไหน	59
เซิร์ฟเวอร์และฐานข้อมูล: Supabase	60
ฐานข้อมูลคืออะไร	60
Supabase ให้อะไรกับเราบ้าง	60
Migration: ไฟล์สร้างฐานข้อมูล	61
Row Level Security (RLS): หัวใจความปลอดภัย	63
กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม	65
แอปต่อ Supabase ยังงี้: src/data/supabase.ts	65
การสมัครและเข้าสู่ระบบ	66
Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน	67
เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด	72
การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังงี้	75
AI ในแอปคืออะไร	75
ทำไมแอปไม่เรียก AI เอง	75
เส้นทางของคำถาม 1 ข้อถึงน้องกล้า	75
ฝั่งแอป: src/services/coach.ts	76
ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละอุสทัด	77
ตัวช่วยกลาง: supabase/functions/_shared/common.ts	79
เลือก AI ตัวไหน: Claude หรือ Gemini	82
ส่งคำขอไปที่ Claude: callClaude	83
ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)	85
หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้	86
UX/UI: การออกแบบหน้าจอและ Design System	87
หลักการออกแบบของ MindPay	87
Design System คืออะไร	87
สี: Green + Gold	87

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ	89
ตัวอักษร	90
ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)	91
Accessibility: ทุกคนใช้ได้	93
แอนิเมชัน: สวยแต่ไม่กินแบต	93
เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)	94
การทดสอบ: พิสูจน์ว่าระบบทำงานถูก	95
ทำไมต้องเขียนเทสต์	95
Unit test: ตรวจสอบฟังก์ชันทีละตัว	95
E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง	97
Manual test: ทดสอบบนมือถือจริง	99
Traceability: ผูก Requirement → โค้ด → เทสต์	100
ตรวจสอบคุณภาพอื่น ๆ	100
Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ	101
Git: ระบบบันทึกประวัติ	101
ใช้ Git ประจำวัน	101
ประวัติของ MindPay บน GitHub	102
GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่ push	102
Secrets บน GitHub	106
ไฟล์เอกสารใน repo	106
6 FR ทำงานร่วมกันยังใจ และคำถามระดับระบบ	108
สายข้อมูลของทั้งระบบ	108
ใครใช้ของใคร	108
ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)	108
คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้	109
คำศัพท์รวม	110
FR-6 MONEY RUNWAY	112
วิธีศึกษา FR-6 ให้เข้าใจจริง	113
เป้าหมาย: ตอบ 5 คำถามนี้ได้ โดยไม่ต้องเปิดเล่ม	113
แผนอ่าน 3 ระดับ	113
ตัวอย่าง 2 ชุดที่ใช้ทั้งเล่ม (อย่าสับสน)	113
แผนที่: สไลด์หน้าไหน อ่านเพิ่มบทไหน	114
ทริกอ่านโค้ดใน VS Code ที่ใช้บ่อยที่สุด	114
วิธีอ่านแต่ละบทให้ได้ผล	115
FR-6 ในหน้าเดียว	116
ตัวอย่างสั้น ๆ (วันนี้ 8 ต.ค. 2569)	116
อยู่ตรงไหนในแอป	116
3 ข้อที่ต้องจำ	116
ปัญหาที่জননয়াকแก้	118
ทำไมใช้ชื่อ "Runway"	118
เป้าหมายที่จัดตั้งไว้ตอนออกแบบ	118
Requirement นิยาม และเงื่อนไขผ่าน	119
เงื่อนไขผ่าน (Acceptance criteria) → โค้ด → เทสต์	119
วัตถุดิบ 4 อย่างของการคำนวณ	121
① ยอดคงเหลือ (Balance)	121
② เงินสำรองขั้นต่ำ (Floor หรือ Low line)	121
③ เงินในกระเป๋าออม (Reserved)	121
④ ค่าเฉลี่ยรายจ่ายต่อวัน (Average daily spend)	121
ค่าเฉลี่ยรายจ่ายต่อวัน	122
หน้าต่าง 7 วัน	122

กฎข้อ 1: วันที่ไม่ได้ใช้เงิน ต้องนับด้วย	122
กฎข้อ 2: ผู้ใช้ใหม่ หาดด้วยจำนวนวันที่ใช้มาจริง	122
กฎข้อ 3: นับเฉพาะ "รายจ่าย" ที่ "ยืนยันแล้ว"	122
กฎข้อ 4: ปัดเป็นสตางค์	122
ทำไมเลือก 7 วัน	122
สูตร Money Runway ที่ละขั้น	124
ภาพให้เห็น: ยอดเงินที่คาดว่าจะยังใช้วันละ \$145 ทุกวัน	124
สถานะ 5 แบบ และกรณีพิเศษ	126
ต้นไม้เงินและวงแหวน	126
กรณีพิเศษที่ฉันต้องคิดเพื่อ (ตรวจสอบด้วยฟังก์ชันจริงทุกข้อ)	126
ตัวหน้าจ่อ: หน้าหลัก	128
สถานะต่าง ๆ บนหน้าจอจริง	128
ตัวหน้าจ่อ: แท็บ "เงินพอถึง"	130
เครื่องมือ "ลองปรับ" "เช็กก่อนจ่าย" และ "ถามโค้ช"	133
เงินสำรอง และกระเป๋าออม	135
เบื้องหลัง: ข้อมูลไหลอย่างไร	137
ทำไม Runway ไม่เก็บในฐานข้อมูล	137
เทคนิคที่ 1: เก็บเงินเป็น "สตางค์" (จำนวนเต็ม)	137
เทคนิคที่ 2: นับวันตามเวลาไทย	137
เทคนิคที่ 3: เริ่มวันใหม่ตอนเที่ยงคืน	138
แผนที่โค้ดของ FR-6	139
อ่านโค้ด: src/domain/runway.ts ทั้งไฟล์	140
ส่วนหัว: import ค่าคงที่ และชนิดข้อมูล	140
ฟังก์ชันที่ 1: averageDailyExpense (ค่าเฉลี่ยรายจ่ายต่อวัน)	140
ชนิดของผลลัพธ์ Runway	141
ฟังก์ชันที่ 2: computeRunway (สูตรหลัก)	142
ฟังก์ชันที่ 3: safeDailySpend (วันนี้ใช้ได้ประมาณเท่าไร)	143
ฟังก์ชันที่ 4: endOfMonthDay (วันสิ้นเดือน)	143
ฟังก์ชันที่ 5: runwayAfterPurchase (เช็กก่อนจ่าย)	144
ฟังก์ชันที่ 6: runwayWithReduction (ลองปรับ)	144
ไล่โค้ดแบบคอมพิวเตอร์: ค่าของตัวแปรทุกบรรทัด	146
ฟังก์ชันที่ 1: averageDailyExpense กับรายการ 9 รายการ	146
ฟังก์ชันที่ 2: computeRunway ที่ละบรรทัด (ยอด \$3,500 สำรอง \$500 เหลือ \$145)	147
ฟังก์ชันที่ 3 และ 4: safeDailySpend กับ endOfMonthDay	148
ฟังก์ชันที่ 5 และ 6: เช็กก่อนจ่าย และลองปรับ	149
ลองรันเองใน VS Code (5 นาที)	150
พจนานุกรมสัญลักษณ์: อ่าน runway.ts ออกทุกตัวอักษร	152
1. โครงของไฟล์	152
2. ค่าคงที่ ตัวแปร และชนิดข้อมูล	152
3. ฟังก์ชัน	152
4. อีอเบเจกต์และอาร์เรย์	153
5. เมธอดของอาร์เรย์ (ใช้ต่อกันเป็นสาย)	153
6. การเปรียบเทียบและการตัดสินใจ	154
7. คลิตศาสตร์และวันที่	154
อ่านโค้ด: หน้าจ่อแท็บเงินพอถึง (runway.tsx)	155
ส่วนที่ 1: import และฟังก์ชันช่วยแสดงจำนวนวัน	155
ส่วนที่ 2: เตรียมข้อมูลของหน้า	155
ส่วนที่ 3: หัวหน้าจ่อและการ์ดเขียวพร้อมวงแหวน	156
ส่วนที่ 4: การ์ด "คำนวณยังไ้"	158

ส่วนที่ 5: เส้นเวลา 30 วัน	159
ส่วนที่ 6: ลองปรับ	160
ส่วนที่ 7: เช็กก่อนจ่าย และส่งต่อให้โค้ช	161
อ่านโค้ด: การ์ดบนหน้าหลัก และ useMoney	163
ตารางป้ายสถานะ	163
ตัวเลขที่หน้าหลักดึงมาใช้	163
การ์ดเขียว (hero) ส่วนของ Runway	164
useMoney: จุดที่ประกอบทุกอย่างเข้าด้วยกัน	166
อ่านโค้ด: กระปุกออม (goals)	168
กฎของกระปุกออม	168
หยอดหรือถอนจากหน้าจอ	169
ตารางในฐานข้อมูล	170
อ่านโค้ด: แลปเลื่อน ต้นไม้เงิน และสิ่งที่ส่งให้โค้ช	172
Slider: แลปเลื่อนที่เขียนเอง	172
treeHealth: ในทอกร์เปอร์เซ็นต์	174
การ์ดเดือน Runway ของโค้ช (insights.ts)	175
ข้อมูลที่ส่งให้ AI (buildCoachContext)	176
แบบ purchaseCheck ตอนถามโค้ช (coach.tsx)	176
ช่องเงินสำรองในหน้าตั้งค่า	177
สร้าง FR-6 ตั้งแต่ศูนย์: ทำตามได้ทีละขั้น	178
ภาพรวม: 10 ขั้นตอนจากศูนย์ถึงแอปในมือผู้ใช้	178
ขั้นที่ 0: เตรียมเครื่อง (ทำครั้งเดียว)	178
ขั้นที่ 1: เขียนนิยามก่อนเขียนโค้ด	179
ขั้นที่ 2: เขียนเทสต์ก่อน (ให้มันแดงก่อน)	179
ขั้นที่ 3: เขียนสูตรให้เทสต์ผ่าน (เขียว)	181
ทดลองทำบีกเอง: ทำไมต้องทดสอบที่ขอบ	182
จากฉบับย่อสู่ของจริง (งั้นหะเก็บกวาด)	182
ขั้นที่ 4: ตรวจสอบคุณภาพก่อนไปต่อ	183
ขั้นที่ 5: ต่อสูตรเข้ากับข้อมูลจริง	184
ขั้นที่ 6: ทำหน้าจอลง และเพิ่มแท็บ	184
ขั้นที่ 7: ทดสอบแบบคนใช้จริง (E2E)	186
ขั้นที่ 8: ส่งงานขึ้น GitHub	188
ขั้นที่ 9: ปรับตาม feedback (ประวัติจริงของ FR-6 ใน Git)	189
เจาะ 1 การปรับปรุง: จากปุ่ม 4 ปุ่ม เป็นแลปเลื่อน	190
เช็กลิสต์ก่อนบอกว่า "เสร็จ" (Definition of Done)	191
การทดสอบ FR-6	192
Unit test: ทดสอบสูตรทีละเงื่อนไข	192
E2E: ทดสอบเหมือนคนใช้จริงในเบราว์เซอร์	193
เช็กลิสต์ทดสอบเองบนมือถือ (ทำได้ใน 10 นาที)	194
FR-6 เชื่อมกับ FR อื่นยังไ้	195
ทำไมฉันออกแบบแบบนี้	196
ข้อจำกัด และสิ่งที่จะทำต่อ	197
ไอเดียพัฒนาต่อ (ยังไม่ได้ทำ)	197
คำถามที่อาจารย์อาจถาม พร้อมคำตอบ	198
แบบฝึกหัดเช็กความเข้าใจ	200
สคริปต์นำเสนอ 1 นาที และคำศัพท์	202
คำศัพท์ของ FR-6	202
นำเสนอ FR-6 ด้วยสไลด์ของฉัน	203
ก่อนใช้สไลด์: 4 เรื่องที่ต้องทำ	203

แผนเวลา 10 นาที	203
สไลด์ที่ละหน้า: พูดอะไร ชี้ตรงไหน	203
เดโมสด 2 นาที (ถ้าอาจารย์ให้โชว์แอปจริง)	205
แผนสำรองเมื่อเกิดเรื่องไม่คาดคิด	206
ตอบใน 1 ประโยค (ท่องให้คล่อง)	206
เทคนิคการพูดที่ช่วยได้จริง	207
เช็กลิสต์ก่อนขึ้นพูด	207

ภาค ก

พื้นฐานทั้งระบบ MindPay

สอนตั้งแต่ศูนย์ สำหรับคนที่ไม่เคยเขียนโค้ด ภาคนี้เหมือนกับทุกเล่ม เพราะไม่ว่าจะรับผิดชอบ FR ไหน ทุกคนต้องเข้าใจว่าแอปทั้งตัวสร้างด้วยอะไร ทำงานที่ไหน และชิ้นส่วนต่างๆ คอยกันยังไ

- เครื่องมือ ภาษา TypeScript และ React ตั้งแต่พื้นฐาน
- โค้ดชุดเดียวกลายเป็นทั้งเว็บและแอป Android ได้ยังไ
- เซิร์ฟเวอร์ Supabase ฐานข้อมูล และความปลอดภัย
- การเชื่อม AI, การออกแบบ UX/UI, การทดสอบ และ GitHub

MindPay คืออะไร และทีมเราสร้างอะไร

ก่อนดูโค้ดสักบรรทัด ต้องรู้ก่อนว่าแอปนี้แก้ปัญหาอะไร ใครใช้ และมีส่วนไหนบ้าง

ปัญหาที่ทีมเราอยากแก้

นักศึกษาและคนเพิ่งเริ่มทำงาน (First Jobber) ส่วนใหญ่รับและจ่ายเงินผ่านแอปธนาคารบนมือถือ ทุกครั้งที่โอนเงินจะได้ "สลิป" เป็นรูปเก็บไว้ในแกลเลอรี แต่ปัญหาคือ

- **ขี้เกียจจด** แอปจดรายรับรายจ่ายทั่วไปต้องพิมพ์เองทุกรายการ ทั้งที่ข้อมูลอยู่ในสลิปแล้ว คนจึงเลื่อนการจดไปเรื่อย ๆ จนข้อมูลไม่ครบ
- **เห็นแค่ยอดเงิน แต่ไม่รู้ว่าพอหรือเปล่า** แอปธนาคารบอกว่าเหลือ ฿3,500 แต่ไม่ได้บอกว่าเงินก้อนนี้จะอยู่ได้ถึงวันไหน
- **รู้ตัวตอนเงินหมดแล้ว** ไม่มีใครช่วยเตือนหรือช่วยคิดก่อนจ่าย

ทีมเราจึงสร้าง **MindPay** แอปการเงินส่วนตัวที่ "ไม่ได้แค่จดว่าใช้ไปเท่าไร แต่บอกว่าเงินที่มีจะพอใช้อีกกี่วัน และช่วยคิดก่อนจ่าย" (ประโยคแรกของไฟล์ `README.md` ในโปรเจกต์)

6 ฟังก์ชันหลัก (Functional Requirements)

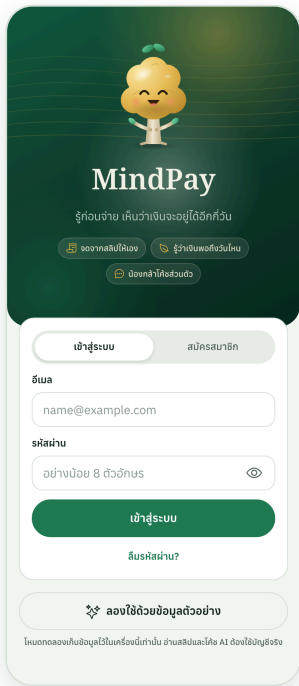
ในวิชา Software Engineering เราเรียกสิ่งที่ระบบ "ต้องทำได้" ว่า **Functional Requirement** หรือ **FR** แต่ละ FR มีเจ้าของ 1 คนในกลุ่ม MindPay มีทั้งหมด 6 ข้อ

FR	ชื่อ	ทำอะไร (ภาษาคน)	อยู่ตรงไหนในแอป
FR-1	Transaction Management	จดรายการเอง แก้ไข เลือกรหมวด และลบรายการได้	แท็บ "รายการ" และหน้า "จดรายการ"
FR-2	Overview Dashboard	สรุปรายรับ รายจ่าย ยอดคงเหลือ และยอดแยกตามหมวด	แท็บ "หน้าหลัก"
FR-3	Voice Entry	พูดว่า "ข้าว 50 บาท" แล้วแอปจดให้	ปุ่มไมค์ของบนหน้าหลัก
FR-4	Automatic Slip Detection	หาสลิปในแกลเลอรีเองแล้วให้ AI อ่านยอดเงิน วันที่ และชื่อผู้รับ	ปุ่มทวงกลางแถบล่าง "สแกนสลิป" และสแกนอัตโนมัติตอนเปิดแอป
FR-5	AI Coach (น้องกล้า)	โค้ช AI ที่อธิบายตัวเลขการเงินเป็นภาษาคน และพูดออกเสียงได้	แท็บ "โค้ช"
FR-6	Money Runway	คำนวณว่าเงินที่มีจะพอใช้อีกกี่วัน และหมดวันไหน	แท็บ "เงินพอถึง" และการ์ดเขียบบนหน้าหลัก

★ **จำให้ได้: ทุก FR ต่อกันเป็นสายเดียว**

FR-1, FR-3 และ FR-4 คือ **ทางเข้าของข้อมูล** (จดเอง พูดจด สแกนสลิป) ส่วน FR-2 และ FR-6 คือ **สมองที่คิดตัวเลข** (ยอดคงเหลือ และเงินพอถึงวันไหน) และ FR-5 คือ **ปากที่อธิบาย** ตัวเลขเหล่านั้นให้ผู้ฟัง ถ้า FR ไหนพึ่ง FR ที่อยู่ปลายน้ำจะได้ตัวเลขผิดพลาดไปด้วย

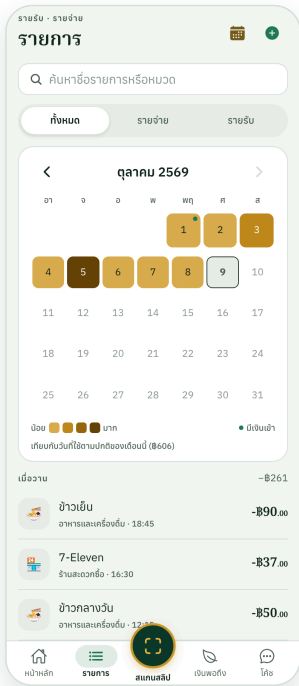
หน้าตาแอปโดยรวม



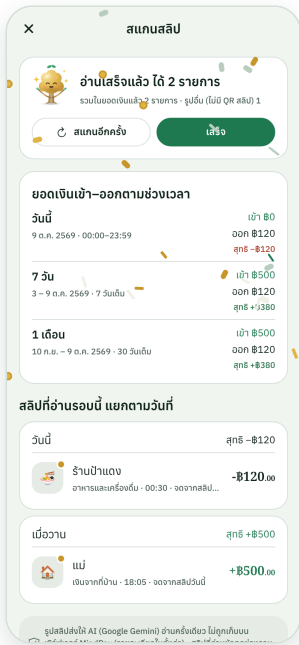
หน้าต้อนรับ: สมัคร เข้าสู่ระบบ หรือ "ลองใช้ด้วยข้อมูลตัวอย่าง"



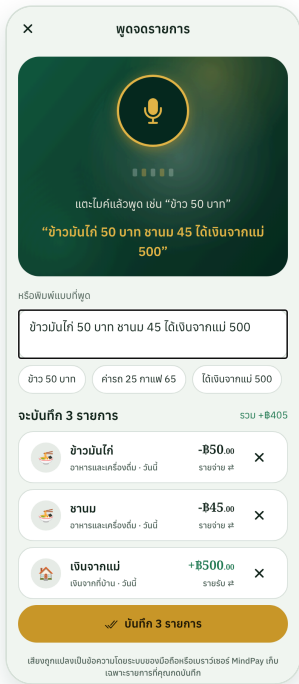
หน้าหลัก (FR-2): ยอดคงเหลือ, การ์ดเงินพอถึง (FR-6), น้องกล้าพูดสรุป



แท็บรายการ (FR-1): รายการทั้งหมด ค้นหา กรองรายการ/รายการจ่าย



สแกนสลิป (FR-4): อ่านสลิปแล้วสรุปตามช่วงเวลา



พูดจ (FR-3): พูดประโยคเดียว ได้หลายรายการ



เงินพอถึง (FR-6): 16 วัน ถึงประมาณ 25 ต.ค.

ด้านล่างของทุกหน้ามี **แถบเมนู 5 ช่อง**: หน้าหลัก, รายการ, ปุ่มทวง "สแกนสลิป" ตรงกลาง, เงินพอถึง และไอซ์ ปุ่มสแกนอยู่ตรงกลางและใหญ่ที่สุด เพราะเป็นสิ่งที่ผู้ใช้ทำบ่อยที่สุด (โค้ดอยู่ใน [src/app/\(tabs\)/_layout.tsx](#) ซึ่งจะอ่านทีละบรรทัดในบท A7)

ใช้แอปได้ 3 แบบ

แบบ	เปิดยังไง	ต่างกันยังไง
เว็บ	https://2550expo-tech.github.io/Socrates-and-Skeletons-/ (เปิดในเบราว์เซอร์ได้ทั้งคอม iPhone และ Android)	ทำได้ทุกอย่าง ยกเว้นสแกนแกลเลอรีทั้งหมดเอง (บนเว็บต้องกดเลือกรูปสลิป)
แอป Android (.apk)	ดาวน์โหลดจากหน้า Releases ของ GitHub ชื่อ latest - apk	สแกนแกลเลอรีอัตโนมัติ พุดจดด้วยไม้ค้ นื่องกล้าพุดออกเสียง
Expo Go	แอปตัวช่วยตอนพัฒนา สแกน QR จากคอมของนักพัฒนา	ใช้ทดสอบระหว่างเขียนโค้ดเท่านั้น

และไม่ว่าเปิดแบบไหน ผู้ใช้เลือกได้ 2 โหมด

- **บัญชีจริง** สมัครด้วยอีเมล ข้อมูลเก็บบนเซิร์ฟเวอร์ (Supabase) เปิดจากเครื่องไหนก็เห็นข้อมูลเดียวกัน ใช้ AI อ่านสลิปและโค้ชได้
- **โหมดทดลอง** แตะ "ลองใช้ด้วยข้อมูลตัวอย่าง" แอปสร้างรายการตัวอย่างประมาณ 75 วันให้ เก็บไว้ในเครื่องอย่างเดียว ไม่ต้องสมัคร ใช้ตอนพีรเซนต์หรือลองเล่น (AI ยังใช้ไม่ได้ในโหมดนี้)

ตัวเลขของโปรเจกต์

~18,700 บรรทัดโค้ดของแอปและเซิร์ฟเวอร์ (ไม่นับเทสท์)	157 unit test ผ่านทั้งหมด	145 ข้อตรวจในเบราว์เซอร์ (E2E) ผ่านทั้งหมด
--	-------------------------------------	--

โฟลเดอร์	มีอะไร	ไฟล์	บรรทัด
src/app	หน้าจอกทุกหน้า	19	5,266
src/ui	ชิ้นส่วนหน้าตา: ปุ่ม การ์ด สี กราฟ ภาพน้องกล้า	31	6,042
src/domain	กฎและการคำนวณทั้งหมด (สมองของแอป)	24	3,457
src/data	การเก็บและโหลดข้อมูล	9	1,019
src/services	งานที่ต้องคุยกับโลกภายนอก: แกลเลอรี ไม้ค้ เสียง AI	15	1,776
supabase	ฐานข้อมูลและโค้ดฝั่งเซิร์ฟเวอร์	8	1,160
e2e, scripts	เทสท์ในเบราว์เซอร์ และสคริปต์ช่วย (ไอคอน ภาพหน้าจอล)	10	1,867

นับจากโค้ดเวอร์ชัน 3 ต.ค. 2569 ไฟล์ .ts, .tsx, .sql และ .mjs ไม่รวมไฟล์เทสท์ (ไฟล์เทสท์อีกประมาณ 2,000 บรรทัด)

≈ **เปรียบเทียบแอปเป็นร้านอาหาร**

หน้าจอล (src/app, src/ui) คือหน้าร้านที่ลูกค้าเห็น **กฎการคำนวณ** (src/domain) คือสูตรอาหารของเชฟ **ข้อมูล** (src/data) คือตู้เย็นเก็บวัตถุดิบ **เซิร์ฟเวอร์** (supabase) คือโอดังกลางที่ทุกสาขาใช้ร่วมกัน และ **AI** คือผู้เชี่ยวชาญภายนอกที่เราโทรถามเมื่อต้องอ่านสลิปหรืออธิบายตัวเลข บกต่อไปจะวาดแผนที่ว่าทั้งหมดนี้ต่อกันยังไง

คำศัพท์ที่จะเจอบ่อยในเล่มนี้

คำ	ความหมายแบบง่าย
รายการ (Transaction)	เงินเข้าหรือเงินออก 1 ครั้ง เช่น ข้าวมันไก่ ฿50
แบบร่าง (Draft)	รายการจากสลิปที่ AI อ่านแล้วแต่ยังไม่แน่ใจ รอผู้ใช้ตรวจ ยังไม่นับในยอดเงิน

คำ	ความหมายแบบง่าย
ยืนยันแล้ว (Confirmed)	รายการที่นับในยอดเงินแล้ว
สตางค์ (satang)	หน่วยที่แอปใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท (เหตุผลอยู่บท A5)
เส้นเงินสำรอง (Floor)	เงินขั้นต่ำที่ผู้ใช้ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
น้องกล่า	มาสคอตของแอป เป็นต้นกล่าจากต้นไม้เงิน และเป็นโค้ช AI ใน FR-5
Supabase	บริการเซิร์ฟเวอร์ที่ให้ทั้งฐานข้อมูล ระบบบัญชีผู้ใช้ และที่รันโค้ดฝั่งเซิร์ฟเวอร์
Expo	เครื่องมือที่ทำให้โค้ดชุดเดียวเป็นได้ทั้งแอป Android, iPhone และเว็บ

แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังงี้

ภาพใหญ่ของ MindPay ตั้งแต่นิ้วแตะจอ จนข้อมูลไปถึงฐานข้อมูลบนคลาวด์และกลับมาเป็นตัวเลขบนหน้าจอ

ระบบมี 4 ส่วนใหญ่

① แอปบนมือถือหรือเว็บ (โค้ดในโฟลเดอร์ src/)

สร้างด้วย Expo + React Native ภาษา TypeScript โค้ดชุดเดียวได้ทั้งแอป Android, iPhone และเว็บ ทำหน้าที่แสดงหน้าจอ รับการแตะ คำนวณตัวเลขทั้งหมด (ยอดคงเหลือ, Runway) อ่าน QR บนสลิป และแปลงเสียงพูดเป็นรายการ

↕ อินเทอร์เน็ท (HTTPS) ด้วย "publishable key" ซึ่งเป็นกุญแจสาธารณะ

② Supabase: ฐานข้อมูล + ระบบบัญชีผู้ใช้ (โค้ดใน supabase/migrations/)

ฐานข้อมูล PostgreSQL เก็บตาราง profiles, transactions, savings_goals, ai_usage ทุกตารางเปิด Row Level Security (RLS) ทำให้แต่ละคนอ่านและแก้ไขได้เฉพาะแถวของตัวเอง ระบบ Auth จัดการสมัคร เข้าสู่ระบบ ยืนยันอีเมล

↕ เฉพาะงานที่ต้องใช้ AI

③ Supabase Edge Functions: โค้ดฝั่งเซิร์ฟเวอร์ (โค้ดใน supabase/functions/)

ฟังก์ชัน 2 ตัว: **parse-slip** (อ่านสลิป, FR-4) และ **coach** (โค้ชน้องกล้า, FR-5) ตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริง นับโควตาต่อวัน แล้วถือ "กุญแจลับ" ของ AI ไว้เรียกแทนแอป

↕ API key ลับ (อยู่บนเซิร์ฟเวอร์เท่านั้น)

④ AI ภายนอก: Claude (Anthropic) หรือ Gemini (Google)

อ่านรูปสลิปแล้วตอบเป็น JSON และเขียนคำอธิบายของโค้ช ระบบเลือกใช้ตัวที่มีกุญแจ ถ้ามีทั้งสองตัวจะใช้ Claude ก่อน

★ **จำให้ได้: 4 กฎของสถาปัตยกรรม (จาก README)**

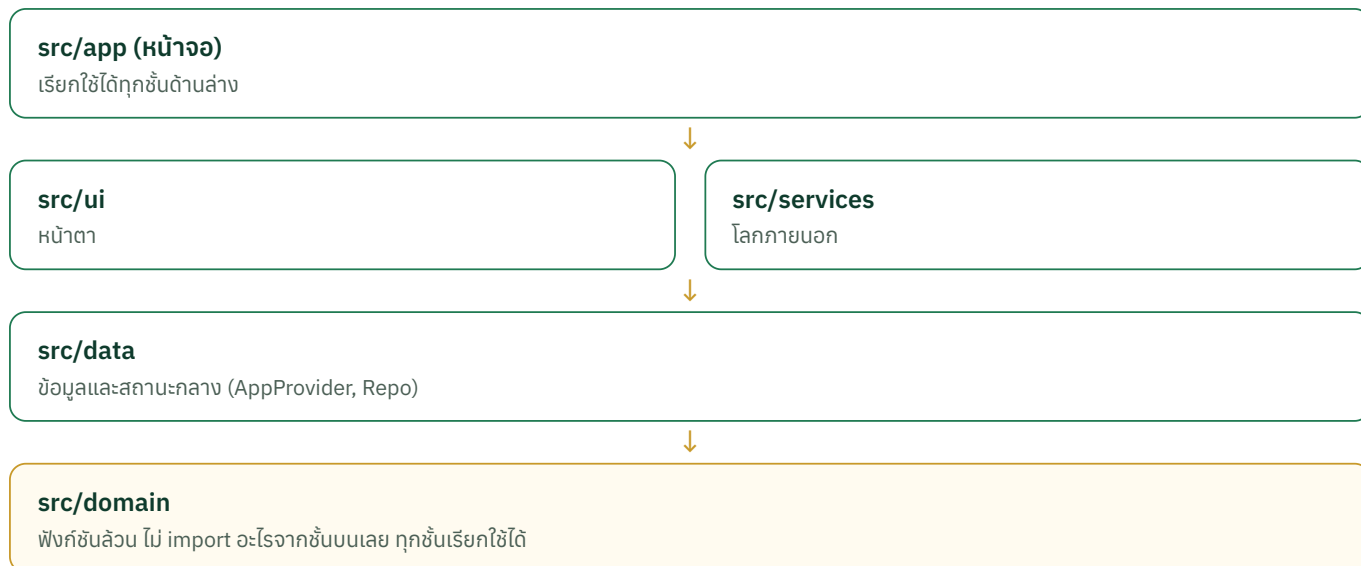
- เงินเก็บเป็นสตางค์ (จำนวนเต็ม) ไม่มีปัญหาทศนิยมเพี้ยน
- เวลาอิงเวลาไทย (UTC+7) ทุกจุด ทั้งการจัดกลุ่มรายวันและการอ่านวันที่บนสลิป
- สลิปที่ AI อ่านได้ไม่ชัดเป็น "แบบร่าง" จนกว่าผู้ใช้จะยืนยัน ไม่เข้ายอดเงินก่อน
- กุญแจ AI อยู่บนเซิร์ฟเวอร์เท่านั้น ในแอปไม่มีกุญแจลับเลยสักตัว

ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น

ถ้าเอาโค้ดทุกอย่างไปกองในไฟล์เดียว จะแก้ยากและทดสอบไม่ได้ ทีมเราจึงแยกตามหน้าที่ แต่ละโฟลเดอร์ใน src/ มีหน้าที่ชัดเจน และมีกฎว่าใครเรียกใครได้

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
หน้าจอ	src/app/	แต่ละไฟล์คือ 1 หน้า (Expo Router) วางชิ้นส่วน รับการแตะ แล้วเรียกชั้นอื่นทำงาน	<code>src/app/transaction.tsx</code> หน้าจอรายการ
ชิ้นส่วน หน้าตา	src/ui/	ปุ่ม การ์ด ช่องกรอก สี ตัวอักษร กราฟ ภาพวาด ใช้ซ้ำได้ทุกหน้า	<code>src/ui/components.tsx</code> , <code>src/ui/theme.ts</code>
กฎและการ คำนวณ	src/domain/	"สมอง" ของแอป เป็นฟังก์ชันล้วน รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอหรืออินเทอร์เน็ท จึงทดสอบได้ง่าย มี unit test ทุกไฟล์	<code>src/domain/runway.ts</code> , <code>src/domain/money.ts</code>

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
ข้อมูล	src/data/	โหลดและบันทึกข้อมูล ทั้งบนคลาวด์ (Supabase) และในเครื่อง (โหมดทดลอง) และเก็บ "สถานะกลาง" ที่ทุกหน้าใช้ร่วมกัน	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
บริการภายนอก	src/services/	งานที่คุยกับโลกนอกแอป: แคลเลอร์รูป ไมโครโฟน เสียงอ่าน และเรียก Edge Functions	<code>src/services/slips.ts</code> , <code>src/services/coach.ts</code>



≈ ทำไมแยก domain ออกมา

เหมือนแยก "สูตรคำนวณภาษี" ออกจาก "แบบฟอร์มกรอกภาษี" ถ้าสูตรเปลี่ยน แก้ทีเดียว แบบฟอร์มทุกฉบับได้ผลใหม่ทันที และเราตรวจสอบสูตรได้ด้วยการใส่ตัวเลขทดลองโดยไม่ต้องกรอกแบบฟอร์มจริง โค้ดใน `src/domain` ก็เช่นกัน มีเทสต์ 157 ข้อที่รันจบในไม่ถึง 2 วินาทีโดยไม่ต้องเปิดแอป

ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน

วิธีที่ดีที่สุดในการเข้าใจระบบ คือเลือกการกระทำหนึ่งอย่าง แล้วตามดูว่ามันวิ่งผ่านไฟล์ไหนบ้าง สมมติผู้ใช้กด "ข้ามบันทึก #85" แล้วกด "บันทึกรายการ"

1. หน้าจอรายการ `src/app/transaction.tsx`

ฟังก์ชัน `save()` แปลง "85" เป็น 8,500 สตางค์ ด้วย `parseBahtToSatang` แล้วเรียก `addTx(...)`



2. สถานะกลาง `src/data/AppProvider.tsx`

`addTx` ส่งต่อให้ `Repo` บันทึก รวผลกลับมา แล้วเพิ่มรายการใหม่เข้าไปในรายการทั้งหมด (`setTx`s)



3. Repo `src/data/repo.ts`

บัญชีจริง: แปลงชื่อช่องเป็นแบบฐานข้อมูล (`amountSatang` → `amount_satang`) แล้วส่งคำสั่ง `insert` ไปที่ Supabase · โหมดทดลอง: เก็บลงเครื่องแทน



4. ฐานข้อมูล Supabase `supabase/migrations/20260927000000_init.sql`

ตรวจสอบ: จำนวนเงินต้อง > 0, ผู้บันทึกต้องเป็นเจ้าของแถว (RLS) แล้วเพิ่มแถวใหม่ในตาราง `transactions` ส่งแถวที่บันทึกแล้วกลับไป



5. คำนวณใหม่อัตโนมัติ `useMoney()` ใน `src/data/AppProvider.tsx`

เพราะรายการเปลี่ยน React จึงคำนวณยอดคงเหลือ (FR-2) และ Money Runway (FR-6) ใหม่ด้วยฟังก์ชันใน `src/domain` แล้ววาดหน้าหลักใหม่ ตัวเลขลดลง ฿85 กันที

ดูโค้ดจริงของแต่ละขั้น (ยังไม่ต้องเข้าใจทุกคำ บท A4–A9 จะสอนทีละอย่าง ตอนนี้อยู่แค่ที่มันต่อกันยังงี้)

ขั้น 1: หน้าจอรายการเรียก `addTx`

`src/app/transaction.tsx`

บรรทัด 87–124

```
87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวบรวมยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
113       await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',
117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
```

```

121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123   }
124   leaveWhenDone();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:87

บรรทัด	ความหมาย
87	ประกาศฟังก์ชัน save ซึ่งทำงานเมื่อกดปุ่มบันทึก คำว่า async แปลว่าข้างในมีงานที่ต้องรอ (เช่น รออินเทอร์เน็ท)
88	ถ้ากำลังบันทึกอยู่แล้ว (busy) ให้ออกเลย กันคนกดปุ่มรัว ๆ แล้วได้รายการซ้ำ
89–93	แปลงข้อความในช่องเงินเป็นสตริง ถ้าแปลงไม่ได้ (เช่น พิมพ์ "abc" หรือ 0) จะได้ null แล้วขึ้นข้อความเตือนใต้ช่อง และหยุดไม่บันทึก
95	ถ้าวันที่หรือเวลาที่พิมพ์ (บนเว็บ) ผิดรูปแบบ ก็ไม่บันทึก
96–103	รวมค่าจากทุกช่องในฟอร์มเป็นก้อนเดียวชื่อ input: ประเภท จำนวนเงิน หมวด ชื่อ บันทึก และเวลา (แปลงวันและเวลาไทยเป็นรูปแบบสากลด้วย bkkToIso)
104	ตั้ง busy เป็นจริง ปุ่มจะหมุ่บรอและกดซ้ำไม่ได้
106–111	ถ้าเป็นการแก้ไขรายการเดิม เรียก updateTx แทน
113–121	ถ้าเป็นรายการใหม่ เรียก addTx พร้อมบอกว่ามาจากการจดเอง (manual) และยืนยันแล้ว (confirmed) จึงนับในยอดเงินทันที
122	ขึ้นข้อความสั้น ๆ ด้านล่าง (toast) ว่า "บันทึกรายการแล้ว"
124	ปิดหน้าฟอร์มกลับไปหน้าก่อน

ขั้น 2: AppProvider ส่งต่อให้ Repo แล้วอัปเดตรายการ

src/data/AppProvider.tsx

บรรทัด 385–392

```

385   async addTx(input) {
386     const r = need();
387     const tx = await r.insert(input);
388     if (repoRef.current !== r) return tx;
389     noteLocal(tx.id, tx);
390     setTxs((list) => [tx, ...list]);
391     return tx;
392   },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:385

บรรทัด	ความหมาย
386	หยิบ Repo ตัวที่ใช้อยู่ (บัญชีจริงหรือโหมดทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ need() จะโยน error
387	สั่ง Repo บันทึก แล้วรอผล (await) ได้รายการที่มี id จริงกลับมา
388	ถ้าระหว่างรอ ผู้ใช้สลับบัญชีไปแล้ว ไม่ต้องเอารายการนี้ไปใส่ในบัญชีใหม่
389	จดไว้ว่ามีอะไรเปลี่ยนแปลงในเครื่อง (กันรายการหายถ้าคำสั่งโหลดข้อมูลชุดใหม่พร้อมกัน)
390	เอารายการใหม่ไปไว้หน้าสุดของรายการทั้งหมด การเรียก setTxs ทำให้ React วาดทุกหน้าที่ใช้รายการนี้ใหม่

ขั้น 3: Repo ส่งคำสั่งไป Supabase

src/data/repo.ts

บรรทัด 217–222

```

217   async insert(input) {
218     const { data, error } = await db.from('transactions').insert(toRow(input)).select().single();
219     if (isUniqueViolation(error)) throw new DuplicateSlipError();

```

```

220     if (error) throw error;
221     return fromRow(data as TxRow);
222 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217

บรรทัด ความหมาย

- 218** อ่านเหมือนประโยคภาษาอังกฤษ: "จากตาราง transactions, เพิ่ม (insert) แถวนี้, แล้วเลือก (select) แถวที่เพิ่ง เพิ่มกลับมา 1 แถว (single)" toRow แปลงชื่อช่องให้ตรงกับฐานข้อมูล
- 219** ถ้าฐานข้อมูลตอบว่า "ซ้ำ" (รหัส 23505) แปลว่าสลิปนี้เคยบันทึกแล้ว โยน error แบบพิเศษให้หน้าจอบอกผู้ใช้ว่า "มีรายการจากสลิปนี้อยู่แล้ว"
- 220** error อื่น ๆ (เช่น อินเทอร์เน็ตหลุด) โยนต่อให้หน้าจอขึ้น "บันทึกไม่สำเร็จ ลองอีกครั้ง"
- 221** แปลงแถวจากฐานข้อมูลกลับเป็นรูปแบบที่แอปใช้ (amount_satang → amountSatang) แล้วส่งคืน

ขั้น 5: ตัวเลขคำนวณใหม่เอง

src/data/AppProvider.tsx

บรรทัด 442–459

```

442 /** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */
443 export function useMoney() {
444   const { profile, txs, carrySatang, goals, today } = useApp();
445   return useMemo(() => {
446     const opening = profile?.openingBalanceSatang ?? 0;
447     const floor = profile?.runwayFloorSatang ?? 50_000;
448     // Money from before the loaded window (over ~13 months ago) still counts.
449     const balance = computeBalance(opening + carrySatang, txs);
450     const average = averageDailyExpense(txs);
451     // Money in savings goals is set aside; it does not count as money to spend.
452     const reserved = reservedSatang(goals);
453     const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang:
average.averageSatang });
454     const drafts = txs.filter((t) => t.status === 'draft');
455     return { balance, average, runway, drafts, reserved, floor };
456     // `today` makes the runway and averages start over at midnight.
457     // eslint-disable-next-line react-hooks/exhaustive-deps
458   }, [profile, txs, carrySatang, goals, today]);
459 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:442

บรรทัด ความหมาย

- 443** useMoney คือ "hook" ที่ทุกหน้าเรียกใช้เมื่ออยากได้ตัวเลขเงิน (บท A6 อธิบายว่า hook คืออะไร)
- 444** ดึงข้อมูลดิบจากสถานะกลาง: โปรไฟล์ รายการ ยอดยกมา กระปุกออม และวันนี้
- 445** useMemo แปลว่า "คำนวณแล้วจำไว้" คำนวณใหม่เฉพาะเมื่อข้อมูลในบรรทัด 458 เปลี่ยน
- 446–447** ยอดเงินตอนเริ่มใช้แอป และเส้นเงินสำรอง (ถ้ายังไม่ตั้ง ใช้ 50,000 สตางค์ = ฿500)
- 449** คำนวณยอดคงเหลือ (FR-2) จากรายการทั้งหมด
- 450–453** คำนวณค่าเฉลี่ยรายจ่าย เงินในกระปุกล และ Money Runway (FR-6)
- 454** หารายการแบบร่างที่รอตรวจ (FR-4)
- 455** ส่งตัวเลขทั้งหมดออกไปให้หน้าจอใช้
- 458** รายการ "สิ่งที่ต้องจับตา" ถ้า txs เปลี่ยน (เพิ่งเพิ่มข้าวมันไก่) จะคำนวณใหม่ทั้งหมด และ today ทำให้คำนวณใหม่ตอนเที่ยงคืน

★ **จำให้ได้: ตัวเลขบนหน้าจอไม่ได้เก็บไว้ แต่คิดใหม่ทุกครั้ง**

ยอดคงเหลือ ค่าเฉลี่ย และ Runway **ไม่ได้เก็บในฐานข้อมูล** ฐานข้อมูลเก็บแค่ "รายการ" แล้วแอปคำนวณตัวเลขทั้งหมดสด ๆ จาก รายการทุกครั้งที่มีอะไรเปลี่ยน ข้อดีคือตัวเลขไม่มีทางขัดกันเอง (ไม่มีกรณียอดในฐานข้อมูลบอกรายการหนึ่ง แต่รายการบอกอีกอย่าง) คอมเมนต์บนสุดของ [src/data/AppProvider.tsx](#) เขียนไว้ว่า "Numbers shown on screen are always derived from this state ... never stored twice"

ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ

1. มีข้อกำหนดรูปใหม่ในแคปเลอร์ [src/services/AutoScanProvider.tsx](#)

อ่าน QR บนรูปในเครื่องก่อน (ฟรี ไม่ใช้อินเทอร์เน็ต) รูปใหม่ไม่มี QR ของสลิปธนาคารจะไม่ถูกส่งไปไหนเลย

↓ เฉพาะรูปที่เป็นสลิป

2. ส่งรูปไป Edge Function [src/services/slips.ts](#) → [supabase/functions/parse-slip/index.ts](#)

เซิร์ฟเวอร์ตรวจสอบว่าเข้าสู่ระบบจริง ตรวจสอบโควตา (300 ใบต่อวัน) แล้วส่งรูปให้ AI อ่าน 2 ครั้งพร้อมกันด้วย 2 รุ่น

↓

3. AI ตอบเป็น JSON

ยอดเงิน วันที่ ชื่อผู้รับ ธนาคาร และ "ความมั่นใจ" ของแต่ละช่อง เซิร์ฟเวอร์รวมผลสองครั้ง ถ้าไม่ตรงกันจะให้ผู้ใช้งานตรวจสอบ

↓

4. แอปบันทึกรายการ

AI อ่านตรงกันทั้ง 2 ครั้งและมั่นใจทุกช่อง $\geq 80\%$ → เข้ายอดเงินเลย (ผู้ใช้ปิดตัวเลือกนี้ได้ในหน้าตั้งค่า) · ไม่มั่นใจหรืออ่านไม่ตรงกัน → เป็นแบบร่าง รอผู้ใช้ตรวจสอบ แล้วตัวเลขทั้งแอปคำนวณใหม่เหมือนขั้น 5 ด้านบน

รายละเอียดของ AI อยู่ในบท A10 และรายละเอียดเพิ่มเติมของการสแกนสลิปอยู่ในเล่ม FR-4

เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์

ทุกอย่างที่ต้องมีในเครื่อง วิธีเอาโค้ด MindPay มาเปิด รันแอป และใช้ VS Code อ่านโค้ดอย่างมืออาชีพ

กล่องเครื่องมือของทีมเรา

เครื่องมือ	คืออะไร	ใช้ทำอะไรใน MindPay
VS Code	โปรแกรมเขียนโค้ด (code editor) ฟรีจาก Microsoft	เปิด อ่าน แก้โค้ดทุกไฟล์ และรับคำสั่งผ่าน terminal ในตัว
Node.js (เวอร์ชัน 22 ขึ้นไป)	ตัวรันภาษา JavaScript นอกเบราว์เซอร์	รันเครื่องมือทั้งหมด: Expo, เทสต์, สคริปต์ build
npm	ตัวจัดการ "แพ็คเกจ" (โค้ดสำเร็จรูปที่คนอื่นเขียนไว้) มากับ Node.js	ติดตั้งไลบรารี 38 ตัวที่แอปใช้ (npm install) และรับคำสั่งใน package.json
Git	ระบบเก็บประวัติการแก้ไขโค้ด ย้อนเวอร์ชันได้	บันทึกการเปลี่ยนแปลงทุกครั้งเป็น "commit"
GitHub	เว็บที่เก็บโค้ดของ Git บนคลาวด์ ทำงานเป็นทีมได้	เก็บโค้ดที่ github.com/2550expo-tech/Socrates-and-Skeletons - และ build แอปให้อัตโนมัติ (un A13)
Expo	ชุดเครื่องมือทำแอปมือถือด้วย React Native	รันแอประหว่างพัฒนา (npx expo start), ทำเว็บ, build APK, ส่งอัปเดต
Expo Go	แอปบนมือถือสำหรับลองแอปที่กำลังเขียน	สแกน QR จากคอมแล้วเห็นแอปบนมือถือทันที
Supabase Dashboard	หน้าเว็บจัดการเซิร์ฟเวอร์ที่ supabase.com	ดูตาราง รัน SQL ใส่กุญแจ AI ดู log ของ Edge Functions
เบราว์เซอร์ Chrome	-	เปิดเว็บเวอร์ชัน และใช้ DevTools (F12) ดู error

ติดตั้งเครื่องมือ (ทำครั้งเดียว)

- VS Code:** ดาวน์โหลดจาก code.visualstudio.com ติดตั้งแบบค่าเริ่มต้น (Windows ตี "Add to PATH" และ "Open with Code" ไว้ด้วย)
- Node.js:** ดาวน์โหลดรุ่น LTS (22 ขึ้นไป) จาก nodejs.org แล้วเปิด terminal พิมพ์ node -v ต้องขึ้น v22 หรือสูงกว่า
- Git:** ดาวน์โหลดจาก git-scm.com (Mac มีมาให้แล้ว) ตรวจสอบด้วย git --version
- บัญชี GitHub:** สมัครที่ github.com และขอให้เจ้าของ repo เพิ่มเป็นสมาชิกถ้าต้องแก้ไขโค้ด (อ่านอย่างเดียวไม่ต้องขอ เพราะ repo เป็นสาธารณะ)
- Expo Go** บนมือถือ (ไม่บังคับ): ดาวน์โหลดจาก Play Store หรือ App Store

เอาโค้ด MindPay มาไว้ในเครื่อง

เปิด terminal (Windows: PowerShell, Mac: Terminal หรือใช้ terminal ใน VS Code) แล้วพิมพ์ทีละบรรทัด

Terminal

คำสั่งที่พิมพ์จริง

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install
```

```
4 cp .env.example .env
5 code .
```

บรรทัด	ความหมาย
1	clone = คัดลอกโค้ดทั้งหมดพร้อมประวัติจาก GitHub มาไว้ในโฟลเดอร์ใหม่ชื่อ Socrates-and-Skeletons-
2	cd (change directory) = เข้าไปในโฟลเดอร์นั้น
3	อ่านรายชื่อแพ็คเกจใน <code>package.json</code> แล้วดาวน์โหลดทั้งหมดลงโฟลเดอร์ <code>node_modules</code> (ประมาณ 1–3 นาที ใช้พื้นที่หลายร้อย MB)
4	คัดลอกไฟล์ตั้งค่าตัวอย่างเป็น <code>.env</code> ซึ่งบอกแอปว่าเซิร์ฟเวอร์ Supabase อยู่ที่ไหน (Windows PowerShell ใช้ <code>copy .env.example .env</code>)
5	เปิด VS Code ที่โฟลเดอร์นี้ (จุด . แปลว่า "โฟลเดอร์ปัจจุบัน")

ไฟล์ .env มีอะไร

`.env.example`

บรรทัด 1–6

```
1 # Copy this file to .env and fill in the values from
2 # Supabase Dashboard -> Project Settings -> API (or the green "Connect" button).
3 # Only the PUBLISHABLE (anon) key goes here. Never put the secret/service_role key
4 # or any AI key (Gemini / Anthropic) in the app: those live in Supabase Edge Function secrets.
5 EXPO_PUBLIC_SUPABASE_URL=https://dufvcdswcaqsfbxjwat.supabase.co
6 EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_LKQP3TU8Fr3t8JD6bzg7TA_5Uy6zcm
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `.env.example:1`

บรรทัด	ความหมาย
1–4	คอมเมนต์ (บรรทัดที่ขึ้นต้นด้วย # คอมพิวเตอร์ไม่อ่าน) บอกว่าให้ใส่เฉพาะกุญแจ "publishable" ห้ามใส่กุญแจลับหรือกุญแจ AI ในแอปเด็ดขาด
5	ที่อยู่ของเซิร์ฟเวอร์ Supabase ของทีม
6	กุญแจสาธารณะ (publishable key) ใช้บอกว่า "แอปนี้เป็นของโปรเจกต์ไหน" ปลอดภัยที่จะอยู่ในแอป เพราะฐานข้อมูลใช้ RLS กันไว้ (บน A9) ชื่อที่ขึ้นต้นด้วย <code>EXPO_PUBLIC_</code> คือค่าที่ Expo ยอมใส่เข้าไปในแอป

ไฟล์ package.json: บัตรประชาชนของโปรเจกต์

ไฟล์นี้บอกว่าโปรเจกต์ชื่ออะไร ใช้แพ็คเกจอะไร และมีคำสั่งอะไรบ้าง

`package.json`

บรรทัด 1–59

```
1 {
2   "name": "mindpay",
3   "version": "1.0.0",
4   "main": "expo-router/entry",
5   "dependencies": {
6     "@expo-google-fonts/anuphan": "^0.4.1",
7     "@expo-google-fonts/noto-serif-thai": "^0.4.2",
8     "@expo/vector-icons": "^15.0.2",
9     "@react-native-community/datetimepicker": "9.1.0",
10    "@supabase/supabase-js": "^2.117.2",
11    "expo": "~57.0.25",
12    "expo-camera": "~57.0.5",
13    "expo-constants": "~57.0.19",
14    "expo-crypto": "~57.0.3",
15    "expo-file-system": "~57.0.7",
16    "expo-font": "~57.0.4",
```

```

17   "expo-haptics": "~57.0.3",
18   "expo-image-manipulator": "~57.0.20",
19   "expo-image-picker": "~57.0.20",
20   "expo-linear-gradient": "~57.0.2",
21   "expo-linking": "~57.0.11",
22   "expo-media-library": "~57.0.5",
23   "expo-router": "~57.0.23",
24   "expo-speech": "~57.0.3",
25   "expo-speech-recognition": "^57.1.0",
26   "expo-splash-screen": "~57.0.9",
27   "expo-sqlite": "~57.0.3",
28   "expo-status-bar": "~57.0.1",
29   "expo-system-ui": "~57.0.4",
30   "expo-updates": "~57.0.23",
31   "react": "19.2.3",
32   "react-dom": "19.2.3",
33   "react-native": "0.86.3",
34   "react-native-safe-area-context": "~5.7.0",
35   "react-native-screens": "~4.26.0",
36   "react-native-svg": "15.15.4",
37   "react-native-web": "~0.21.0"
38 },
39 "devDependencies": {
40   "@types/react": "~19.2.2",
41   "eslint": "^9.0.0",
42   "eslint-config-expo": "~57.0.2",
43   "playwright": "^1.56.0",
44   "typescript": "~6.0.3",
45   "vitest": "^5.0.2"
46 },
47 "scripts": {
48   "start": "expo start",
49   "android": "expo start --android",
50   "ios": "expo start --ios",
51   "web": "expo start --web",
52   "test": "vitest run",
53   "typecheck": "tsc --noEmit",
54   "lint": "expo lint",
55   "e2e:web": "EXP0_WEB_BASE_URL=/Socrates-and-Skeletons- expo export --platform web --output-dir dist &&
EXP0_WEB_BASE_URL=/Socrates-and-Skeletons- node scripts/web-home-screen.mjs dist && cp dist/index.html
dist/404.html && node e2e/web-auth.mjs dist e2e-shots",
56   "demo:video": "node e2e/demo-video.mjs dist demo-video light android && node e2e/demo-video.mjs dist demo-
video dark android && node e2e/demo-video.mjs dist demo-video light web"
57 },
58 "private": true
59 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `package.json:1`

บรรทัด	ความหมาย
2-3	ชื่อโปรเจกต์ mindpay และเวอร์ชัน
4	จุดเริ่มต้นของแอป: ให้ Expo Router เป็นคนเริ่ม แล้วมันจะไปหาหน้าจอในโฟลเดอร์ src/app เอง
5-38	dependencies แพคเกจที่แอปต้องใช้ตอนทำงานจริง ตัวเลขหลังชื่อคือเวอร์ชัน (^ = ยอมรุ่นย่อยที่ใหม่กว่า, ~ = ยอมเฉพาะรุ่นแก้บั๊ก)
6-7	ฟอนต์ไทย Anuphan (ข้อความทั่วไป) และ Noto Serif Thai (ตัวเลขเงินและหัวข้อ)
8	ไอคอนสำเร็จรูป (Icons) เช่น รูปบ้าน รูปใบไม้ในแถบล่าง
9	ตัวเลือกว่าวันที่และเวลาแบบของมือถือ (FR-1)
10	ตัวเชื่อมต่อ Supabase (ฐานข้อมูล, เข้าสู่ระบบ, Edge Functions)
11	Expo ตัวหลัก เวอร์ชัน 57

บรรทัด	ความหมาย
12	expo-camera ใช้ อ่าน QR จากรูป (ไม่ได้ใช้ถ่ายรูป แอปบล็อกสิทธิ์กล้องไว้)
14	expo-crypto ทำ "ลายนิ้วมือ" (hash) ของรูปสลิป กันสแกนซ้ำ (FR-4)
15, 18	อ่านไฟล์ และย่อ/ตัดรูปก่อนส่งให้ AI
17	expo-haptics สั่นเบา ๆ เวลากดปุ่ม
19, 22	เลือกรูปเอง และเข้าถึงแคเลเลอรี่ทั้งหมด (FR-4)
20	สีไล่ระดับ (gradient) ของการ์ดเขียว
21	เปิดลิงก์ เช่น ลิงก์ยืนยันอีเมลที่พาเข้าแอป
23	Expo Router ระบบเปลี่ยนหน้าจอ (un A7)
24–25	expo-speech ให้น้องกล่าวพูดออกเสียง (FR-5), expo-speech-recognition ฟังเสียงพูดจด (FR-3)
26	หน้าจอแรกตอนเปิดแอป (splash)
27	expo-sqlite ฐานข้อมูลเล็ก ๆ ในเครื่อง ใช้เก็บการตั้งค่าและข้อมูลโหมดทดลอง
30	expo-updates รับอัปเดตโค้ดใหม่โดยไม่ต้องลงแอปใหม่ (EAS Update)
31–33	React (ตัววาดหน้าจอ), react-dom (สำหรับเว็บ), React Native (สำหรับมือถือ)
34–35	จัดการขบจบ (รอยบาก แถบสถานะ) และหน้าจอแบบ native
36	วาดภาพเวกเตอร์ (SVG) เช่น ต้นไม้เงิน น้องกล่าว กราฟ
37	react-native-web แปลงชิ้นส่วนของมือถือเป็น HTML ให้แสดงผลบนเว็บได้
39–46	devDependencies ใช้ตอนพัฒนาเท่านั้น ไม่ได้อยู่ในแอปที่ผู้ใช้ลง: ตัวตรวจภาษา TypeScript, ESLint (ตรวจรูปแบบโค้ด), Playwright (ควบคุมเบราว์เซอร์อัตโนมัติสำหรับ E2E), Vitest (รัน unit test)
47–57	scripts คำสั่งลัด พิมพ์ npm run ชื่อ (หรือ npm test, npm start) แทนคำสั่งยาว ๆ ทางขวา
58	private = ไม่เผยแพร่เป็นแพ็คเกจให้คนอื่นติดตั้ง

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร	ผลที่ควรเห็น
npm run web	เปิดแอปเวอร์ชันเว็บในเบราว์เซอร์ แก่โค้ดแล้วหน้าเว็บเปลี่ยนเอง	เบราว์เซอร์เปิด localhost:8081
npx expo start	เปิด "dev server" มี QR ให้สแกนด้วย Expo Go	QR ใน terminal กด w = เปิดเว็บ, r = โหลดใหม่
npm test	รัน unit test ทั้งหมด	"Tests 157 passed (157)"
npm run typecheck	ให้ TypeScript ตรวจชนิดข้อมูลทั้งโปรเจกต์	ไม่มีข้อความ = ไม่มีปัญหา
npx expo lint	ตรวจรูปแบบโค้ดตามกฎ ESLint	ไม่มี error
npm run e2e:web	build เว็บแล้วรันเทสในเบราว์เซอร์ 145 ข้อ ถ่ายภาพหน้าจอเก็บใน e2e-shots	"145/145 checks passed"

! ถ้าเจอ error ตอนติดตั้ง

"node: command not found" = ยังไม่ได้ติดตั้ง Node.js หรือต้องปิดแล้วเปิด terminal ใหม่ · **npm install** ซ้ำหรือค้าง = รอได้ หรือลบโฟลเดอร์ node_modules แล้วลองใหม่ · **แอปขึ้นแต่เข้าสู่ระบบไม่ได้** = ลืมสร้างไฟล์ .env (บรรทัด 4) · ห้ามใช้ npm install ชื่อแพ็คเกจ ตรง ๆ ในโปรเจกต์ Expo ให้ใช้ npx expo install ชื่อแพ็คเกจ เพื่อให้ได้รุ่นที่เข้ากับ Expo 57 (กฎใน AGENTS.md)

รู้จักหน้าจอ VS Code

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Activity Bar	แถบไอคอนซ้ายสุด	สลับระหว่าง Explorer, Search, Source Control, Run, Extensions
Explorer	ไอคอนรูปกระดาษ (Ctrl+Shift+E)	ต้นไม้โฟลเดอร์และไฟล์ทั้งโปรเจกต์ คลิกเพื่อเปิดไฟล์
Editor	พื้นที่ใหญ่ตรงกลาง	อ่านและแก้ไขโค้ด เปิดหลายไฟล์เป็นแท็บได้ ลากแท็บไปข้าง ๆ เพื่อดู 2 ไฟล์คู่กัน
Search	ไอคอนแว่นขยาย (Ctrl+Shift+F)	ค้นคำในทุกไฟล์ เช่นค้น computeRunway จะเห็นทุกที่ที่ใช้
Source Control	ไอคอนกิ่งไม้ (Ctrl+Shift+G)	ดูไฟล์ที่แก้ไข, commit, push (un A13)
Terminal	ด้านล่าง (Ctrl+ ` ปุ่มตัวหนอนใต้ Esc)	พิมพ์คำสั่ง npm, git ได้โดยไม่ต้องออกจาก VS Code
Problems	แท็บข้าง Terminal (Ctrl+Shift+M)	รวม error จาก TypeScript และ ESLint คลิกแล้วกระโดดไปบรรทัดที่ผิด
Status Bar	แถบล่างสุด	บอก branch ของ Git, บรรทัด/คอลัมน์ที่เคอร์เซอร์อยู่, ภาษาของไฟล์

คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า

Mac ใช้ Cmd แทน Ctrl และ Option แทน Alt

คีย์	ทำอะไร	ลองกับ MindPay
Ctrl+P	เปิดไฟล์จากชื่อ พิมพ์ : เลขบรรทัด ต่อท้ายเพื่อกระโดดไปบรรทัดนั้น	พิมพ์ runway.ts : 72 แล้ว Enter
Ctrl+G	ไปบรรทัดที่ต้องการในไฟล์ที่เปิดอยู่	Ctrl+G แล้วพิมพ์ 89
F12	Go to Definition ไปดูว่าฟังก์ชันหรือตัวแปรนี้ถูกสร้างไว้ที่ไหน (หรือ Ctrl+คลิก)	ใน AppProvider.tsx คลิกที่ computeRunway แล้วกด F12
Alt+F12	Peek Definition: ดูโค้ดปลายทางในกล่องเล็ก ๆ โดยไม่ต้องเปลี่ยนไฟล์	-
Shift+F12	Find All References หาทุกที่ที่เรียกใช้สิ่งนี้	ที่ computeRunway จะเห็นทั้งในแอปและในเทสต์
Alt+← / Alt+→	ย้อนกลับ/ไปข้างหน้า หลังกระโดดไปมาด้วย F12	-
เอาเมาส์ชี้ (hover)	เห็นชนิดข้อมูลและคอมเมนต์อธิบายของสิ่งนั้น	ชี้ที่ Transaction จะเห็นทุกช่องของรายการ
Ctrl+Shift+0	รายการฟังก์ชันทั้งหมดในไฟล์นี้ (outline)	ใน runway.ts จะเห็น 6 ฟังก์ชัน
Ctrl+T	ค้นหาชื่อฟังก์ชันทั้งโปรเจกต์	พิมพ์ parseSpokenEntry
Ctrl+Shift+F	ค้นข้อความในทุกไฟล์	ค้น "เงินพอใช้อีก" เพื่อหาว่าข้อความบนจอมาจากไฟล์ไหน
Ctrl+F	ค้นในไฟล์นี้	-

คีย์	ทำอะไร	ลองกับ MindPay
F2	เปลี่ยนชื่อตัวแปรทุกที่พร้อมกันอย่างปลอดภัย	-
Ctrl+/,	เปลี่ยนบรรทัดเป็นคอมเมนต์ (ปิดชั่วคราว) หรือกลับคืน	-
Ctrl+Shift+P	Command Palette เรียกคำสั่งทุกอย่างของ VS Code	พิมพ์ "Format Document"
Ctrl+B	ซ่อน/แสดงแถบด้านข้าง ให้มีที่อ่านโค้ดมากขึ้น	-

★ เทคนิคหาว่าข้อความบนจอมาจากโค้ดตรงไหน

เห็นคำว่า "เงินพอใช้อีก 16 วัน" บนหน้าหลัก แล้วอยากรู้ว่าโค้ดอยู่ไหน: กด **Ctrl+Shift+F** พิมพ์ "เงินพอใช้อีก" จะเจอไฟล์ที่มีข้อความนี้ทั้งหมด คลิกผลลัพธ์แล้วจะไปบรรทัดนั้นทันที วิธีนี้ใช้ได้กับทุกข้อความภาษาไทยในแอป เพราะข้อความทั้งหมดเขียนอยู่ในโค้ดตรง ๆ

ส่วนเสริม (Extensions) ที่แนะนำ

กด Ctrl+Shift+X แล้วค้นชื่อ กด Install

ชื่อ	ช่วยอะไร
ESLint (Microsoft)	ขีดเส้นใต้โค้ดที่ผิดกฎทันทีที่พิมพ์ ใช้กฎเดียวกับ npx expo lint
Expo Tools	ช่วยเติมคำและตรวจไฟล์ app.json / eas.json
Vitest	มีปุ่ม ▶ ข้างเทสต์แต่ละข้อ กดรับทีละข้อได้
GitLens (ไม่บังคับ)	ชี้บรรทัดไหนก็เห็นว่าใครแก้ล่าสุด เมื่อไร ใน commit อะไร
Error Lens (ไม่บังคับ)	แสดงข้อความ error ถ้ายบบรรทัดเลย ไม่ต้องเอาเมาส์ชี้

ส่วน TypeScript ไม่ต้องลงเพิ่ม VS Code รองรับในตัวอยู่แล้ว และใช้ TypeScript รุ่นในโปรเจกต์ (node_modules/typescript) ตามไฟล์ tsconfig.json

ไม่ยากติดตั้งอะไร: ใช้ VS Code บนเว็บ

มี 2 ทางที่เปิดโค้ด MindPay ด้วย VS Code ได้ทันทีในเบราว์เซอร์

- **ลิงก์ในเล่มนี้** แต่ชื่อไฟล์บนหัวโค้ดก่อนไหนก็ได้ จะเปิด **vscode.dev** ที่ไฟล์และบรรทัดนั้น
- **จากหน้า GitHub** เปิด repo บน github.com แล้วกดปุ่ม . (จุด) บนคีย์บอร์ด หน้าเว็บจะกลายเป็น VS Code (github.dev)

VS Code บนเว็บอ่านโค้ด ค้นหา (Ctrl+Shift+F) กด F12 ไปดูฟังก์ชัน และแก้ไขได้ แต่ **รันคำสั่งไม่ได้** (ไม่มี terminal จึงรัน npm test หรือเปิดแอปไม่ได้) ถ้าจะรันแอปต้องโคลนลงเครื่องตามขั้นตอนด้านบน

ฝึกใช้: ตามรอย Money Runway ใน 5 นาที

1. Ctrl+P พิมพ์ index.tsx เลือกไฟล์ใน src/app/ (tabs) (หน้าหลัก)
2. Ctrl+F ค้น useMoney แล้วกด F12 ที่คำนั้น จะแดงไป [src/data/AppProvider.tsx](#) บรรทัด 443
3. ที่บรรทัด 453 คลิก computeRunway แล้วกด F12 จะไปที่ [src/domain/runway.ts](#) บรรทัด 72 ซึ่งเป็นสูตรของ FR-6
4. กด Shift+F12 ที่ชื่อฟังก์ชัน จะเห็นว่า มีเทสต์เรียกใช้ไป [src/domain/_tests_/domain.test.ts](#) ด้วย
5. กด Alt+← สองครั้ง ย้อนกลับไปหน้าหลัก

ทำแบบนี้ได้ แปลว่าพร้อมอ่านโค้ดส่วนไหนของแอปก็ได้แล้ว

พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย

ทุกอย่างที่ต้องรู้เพื่ออ่านโค้ด MindPay ออก: ค่า ตัวแปร เงื่อนไข ฟังก์ชัน array และ object พร้อมตัวอย่างจากโค้ดจริง

โปรแกรมคืออะไร

โปรแกรมคือ รายการคำสั่งที่คอมพิวเตอร์ทำตามทีละบรรทัด จากบนลงล่าง เหมือนสูตรทำอาหาร ทีมเราเขียนด้วยภาษา **TypeScript** ซึ่งคือภาษา **JavaScript** (ภาษาของเว็บ) ที่เพิ่ม "ชนิดข้อมูล" เข้าไป (un A5) ไฟล์ที่ลงท้ายด้วย `.ts` เป็น TypeScript ล้วน ส่วน `.tsx` คือ TypeScript ที่มีโค้ดหน้าจอบน React ปนอยู่ (un A6)

1. ค่า (value) และชนิดของค่า

ชนิด	ตัวอย่าง	ใช้ใน MindPay
number ตัวเลข	8500, -3, 0.8	จำนวนเงินเป็นสตางค์ 8500 = ฿85, ความมั่นใจ 0.8 = 80%
string ข้อความ	'ข้าวมันไก่', "2026-10-09"	ชื่อรายการ วันที่ ข้อความบนจอ (ใส่ในเครื่องหมายคำพูด ' หรือ ")
boolean จริง/เท็จ	true, false	เข้าสู่ระบบแล้วหรือยัง, ตั้งค่าเสร็จหรือยัง (onboarded)
null	null	"ไม่มีค่า" โดยตั้งใจ เช่น รายการที่จองเองไม่มีเลขอ้างอิงสลิป (slipRef: null)
undefined	undefined	"ยังไม่ได้ใส่ค่า" เช่น ไม่ได้ส่งพารามิเตอร์นั้นมา

2. ตัวแปร: กล่องที่มีชื่อ

ตัวแปรคือ "กล่องที่ติดป้ายชื่อ" ไว้ใส่ค่า เพื่อเรียกใช้ทีหลัง

ตัวแปร

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const floor = 50_000; // const = ค่าคงที่ ใส่แล้วเปลี่ยนไม่ได้
2 let total = 0; // let = เปลี่ยนค่าได้ภายหลัง
3 total = total + 8500; // ตอนนี้ total = 8500
4 const title = 'ข้าวมันไก่';
```

- const ใช้บ่อยที่สุดในโค้ด MindPay เพราะค่าส่วนใหญ่ไม่ควรเปลี่ยน ใช้ let เฉพาะตอนต้องเปลี่ยนจริง ๆ
- 50_000 คือ 50000 เครื่องหมาย _ ช่วยให้อ่านง่ายเหมือนคอมมา คอมพิวเตอร์ไม่สนใจมัน
- ข้อความหลัง // คือ **คอมเมนต์** คอมพิวเตอร์ข้าม มีไว้ให้คนอ่าน ส่วน /* ... */ และ /** ... */ เป็นคอมเมนต์หลายบรรทัด (แบบ /** VS Code จะเอาไปแสดงตอนเอาเมาส์ชี้)
- ตั้งชื่อแบบ camelCase:** คำแรกตัวเล็ก คำต่อไปขึ้นต้นตัวใหญ่ เช่น balanceSatang, averageDailyExpense คำคงที่สำคัญใช้ตัวใหญ่ทั้งหมด เช่น RUNWAY_CAP_DAYS

★ กฎการตั้งชื่อของทีมเรา: ตัวแปรเงินลงท้ายด้วย Satang

ทุกตัวแปรที่เป็นจำนวนเงิน ลงท้ายด้วย Satang เสมอ เช่น amountSatang, floorSatang เห็นชื่อแล้วรู้ทันทีว่าต้องหาร 100 ก่อนแสดงเป็นบาท กันความผิดพลาดแบบเอาบาทไปบวกกับสตางค์

3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ

เขียนว่า	ความหมาย	ตัวอย่าง → ผล
+ - * /	บวก ลบ คูณ หาร	300000 / 145 → 2068.96...
%	เศษจากการหาร	8550 % 100 → 50 (50 สตางค์)
=== !==	เท่ากัน / ไม่เท่ากัน (แบบเคร่ง ชนิตต้องตรงด้วย)	t.kind === 'income'
< <= > >=	น้อยกว่า น้อยกว่าหรือเท่ากับ ...	days < 7
&&	"และ" จริงเมื่อทั้งสองข้างจริง	d >= windowStart && d <= today
	"หรือ" จริงเมื่อข้างใดข้างหนึ่งจริง	event === 'SIGNED_IN' ...
!	"ไม่" กลับจริงเป็นเท็จ	!existing → ไม่มีรายการเดิม
ก ? ข : ค	ถ้า ก จริง ได้ ข ไม่จริงได้ ค (ternary)	kind === 'income' ? 'เพิ่มเงินเข้า' : 'จดยาการ'
ก ?? ข	ใช้ ก ถ้า ก ไม่ใช่ null/undefined ไม่จริงใช้ ข (ค่าสำรอง)	profile?.runwayFloorSatang ?? 50_000
ก?.ข	เข้าถึง ข ใน ก แต่ถ้า ก เป็น null ไม่ต้อง error ได้ undefined	profile?.openingBalanceSatang

ลองอ่านบรรทัดนี้จาก [src/data/AppProvider.tsx:447](#) ซึ่งใช้ทั้ง ?. และ ??

```
src/data/AppProvider.tsx                                บรรทัด 447-447

447    const floor = profile?.runwayFloorSatang ?? 50_000;

▶ เปิดใน VS Code (เว็บ)  GitHub                        VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:447
```

อ่านว่า: "เส้นเงินสำรอง = ค่าที่ผู้ใช้ตั้งไว้ในโปรไฟล์ แต่ถ้ายังไม่มีโปรไฟล์ (?.) หรือไม่มีค่า ให้ใช้ 50,000 สตางค์ (??)"

4. เชื้อไข if: ถ้า... ให้ทำ...

```
if / else                                                ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

1  if (spendable <= 0) {
2    // ทำส่วนนี้เมื่อเชื้อไขจริง
3  } else if (averageSatang <= 0) {
4    // ไม่จริง ถ้าเชื้อไขจริง ทำส่วนนี้
5  } else {
6    // ไม่ตรงอะไรเลย ทำส่วนนี้
7  }
```

ในโค้ด MindPay จะเห็นรูปแบบ "เช็กกรณีพิเศษก่อน แล้วออกเลย" (early return) บ่อยมาก เช่นในสูตร Money Runway

```
src/domain/runway.ts                                    บรรทัด 83-88

83  if (spendable <= 0) {
84    return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85  }
86  if (averageSatang <= 0) {
87    return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88  }
```

อ่านว่า: "ถ้าเงินที่ใช้ได้ ≤ 0 ให้ส่งผลกลับเลยว่าต่ำกว่าเงินสำรอง · ถ้าค่าเฉลี่ย ≤ 0 ส่งกลับว่ายังไม่มีรายจ่าย" พอผ่าน 2 ด้านนี้ไปได้ โค้ดด้านล่างมั่นใจได้ว่าหารได้แน่นอน ไม่หารด้วยศูนย์

5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้" เรียกใช้ซ้ำได้ รับ **พารามิเตอร์** (ของที่ส่งเข้า) และ **return** ผลลัพธ์ออกมา

ฟังก์ชัน 2 แบบ

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 // แบบที่ 1: function
2 function toBaht(satang: number): number {
3   return satang / 100;
4 }
5
6 // แบบที่ 2: arrow function (ลูกศร =>) ทำงานเหมือนกับ เขียนสั้นกว่า
7 const toBaht2 = (satang: number) => satang / 100;
8
9 toBaht(8500); // ได้ 85
```

- (satang: number) = รับ 1 พารามิเตอร์ชื่อ satang ต้องเป็นตัวเลข
- : number หลังวงเล็บ = ฟังก์ชันนี้จะคืนค่าเป็นตัวเลข
- arrow function ที่ไม่มี { } จะ return ค่าทางขวาของ => ให้อัตโนมัติ
- export หน้าฟังก์ชัน = ให้ไฟล์อื่น import ไปใช้ได้

★ Pure function: หัวใจของ src/domain

ฟังก์ชันใน src/domain ทุกตัวเป็น **pure function** คือ (1) ใส่ค่าเดิม ได้ผลเดิมทุกครั้ง และ (2) ไม่ไปทำอะไรข้างนอก ไม่เรียกอื่นเตอร์เน็ต ไม่แตะหน้าจอ เหมือนเครื่องคิดเลข จึงทดสอบง่ายมาก: ใส่ตัวเลขแล้วตรวจคำตอบ อาจารย์มักถามว่า "ทำไมแยก domain" ให้ตอบเรื่องนี้

6. Array: รายการของหลายค่า

Array คือรายการเรียงลำดับ เขียนใน [] คำนวณด้วยคอมมา ตำแหน่งแรกคือ 0

Array พื้นฐาน

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const amounts = [12000, 0, 8500, 30000];
2 amounts[0]; // 12000 (ตัวแรก)
3 amounts.length; // 4 (มีกี่ตัว)
4 amounts.includes(0); // true
```

MindPay เก็บรายการเงินทั้งหมดเป็น array ชื่อ txs (ย่อจาก transactions) และใช้ "เมธอด" ของ array ต่อไปนี้ตลอด ต้องจำให้ได้

เมธอด	ทำอะไร	ตัวอย่าง
filter	คัดเฉพาะตัวที่ผ่านเงื่อนไข ได้ array ใหม่ที่สั้นลง	txs.filter((t) => t.status === 'confirmed')
map	แปลงทุกตัว ได้ array ยาวเท่าเดิม	txs.map((t) => t.amountSatang)
reduce	รวมทุกตัวให้เหลือค่าเดียว เช่น ผลรวม	list.reduce((s, t) => s + t.amountSatang, 0)
find	หาตัวแรกที่ตรงเงื่อนไข (ไม่เจอได้ undefined)	txs.find((t) => t.id === id)
some	มีอย่างน้อย 1 ตัวที่ตรงไหม (true/false)	list.some((t) => t.slipRef === ref)
sort	เรียงลำดับ	days.sort()
slice	ตัดเอาบางส่วน	list.slice(0, 5) = 5 ตัวแรก

ตัว (t) => ... ที่ส่งเข้าไปในเมธอดเหล่านี้คือ arrow function ที่จะถูกเรียกกับสมาชิกทีละตัว t คือ "รายการที่กำลังดูอยู่"

ตัวอย่างจริง: ยอดคงเหลือคำนวณยังง (FR-2)

src/domain/summary.ts

บรรทัด 9-17

```
9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:9

บรรทัด	ความหมาย
9	สร้างฟังก์ชัน confirmedOnly: รับรายการทั้งหมด คั้นเฉพาะรายการที่ยืนยันแล้ว (กรองแบบร่างทิ้ง) ด้วย filter
11	คอมเมนต์อธิบายสูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน
12	ฟังก์ชัน computeBalance รับ 2 อย่าง: เงินตอนเริ่ม (สตางค์) และรายการทั้งหมด คั้นค่าเป็นตัวเลข
13	เอาเฉพาะรายการที่ยืนยันแล้ว แล้ว reduce รวมให้เหลือตัวเลขเดียว
14	สำหรับแต่ละรายการ t: ถ้าเป็นรายรับ บวกเข้า (+t.amountSatang) ถ้าเป็นรายจ่าย ลบออก (-t.amountSatang) แล้วเก็บผลไว้ใน sum
15	ค่าเริ่มต้นของ sum คือเงินตอนเริ่มใช้แอป

ลองคิดตาม: เริ่ม ฿3,000 (300,000 สตางค์) มีรายรับ ฿10,000 ที่ยืนยันแล้ว และรายจ่าย ฿85 กับแบบร่าง ฿200

```
sum เริ่ม          = 300,000
+ รายรับ 1,000,000 = 1,300,000
- รายจ่าย 8,500    = 1,291,500
(แบบร่าง 20,000 ถูกกรองทิ้งตั้งแต่บรรทัด 13)
ผล = 1,291,500 สตางค์ = ฿12,915.00
```

7. Object: ของ 1 ชิ้นที่มีหลายช่อง

Object คือกลุ่มของ "ชื่อช่อง: ค่า" อยู่ใน { } รายการเงิน 1 รายการใน MindPay คือ object แบบนี้

รายการ 1 รายการ (Transaction)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const tx = {
2   id: 'demo-12',
3   kind: 'expense',
4   amountSatang: 8500,
5   categoryKey: 'food',
6   title: 'ข้าวมันไก่',
7   status: 'confirmed',
8 };
9 tx.title;           // 'ข้าวมันไก่' (เข้าถึงช่องด้วยจุด)
10 tx.amountSatang;   // 8500
```

เทคนิค object ที่เจอบ่อยในโค้ด

เขียนว่า	ชื่อเรียก	ความหมาย
const { a, b } = obj;	destructuring (แกะของ)	ดึงช่อง a และ b ออกมาเป็นตัวแปรในบรรทัดเดียว เช่น const { balance, runway } = useMoney();
{ ...obj, status: 'confirmed' }	spread (กระจาย)	คัดลอกทุกช่องของ obj มาเป็น object ใหม่ แล้วเปลี่ยนเฉพาะช่อง status ของเดิมไม่ถูกแก้
[tx, ...list]	spread กับ array	array ใหม่ที่มี tx อยู่หน้าสุด ตามด้วยของเดิมทั้งหมด (ใช้ตอนเพิ่มรายการใหม่ใน un A2)
{ kind, title }	shorthand	ย่อจาก { kind: kind, title: title }

! ห้ามโค้ดชอบสร้าง object ใหม่แทนการแก้ของเดิม

React (un A6) รู้ว่าข้อมูลเปลี่ยนด้วยการดูว่า "เป็น object ตัวใหม่หรือเปล่า" ถ้าเราแก้ช่องใน object เดิมตรง ๆ React อาจไม่รู้ และหน้าจอไม่อัปเดต โค้ดจึงใช้ { ...t, status: 'confirmed' } สร้างของใหม่เสมอ (ดูตัวอย่างใน [src/data/AppProvider.tsx:418](#))

8. วงซ้ำ (loop)

for ... of

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 let total = 0;
2 for (const t of txs) {           // ทำซ้ำกับรายการทีละตัว
3   if (t.kind !== 'expense') continue; // continue = ข้ามตัวนี้ ไปตัวถัดไป
4   total += t.amountSatang;      // += แปลว่า total = total + ...
5 }
```

ส่วน for (let i = 0; i < 20; i++) คือวนแบบนับรอบ: เริ่ม i = 0 ทำซ้ำตราบที่ i < 20 และเพิ่ม i ทีละ 1 (i++) ส่วน break คือหยุดวนทันที

9. ข้อความ (string) ที่ใช้บ่อย

เขียนว่า	ผล
`เงินพอใช้อีก \${days} วัน`	template string (ใช้ backtick) แทรกค่าตัวแปรใน \${ } ได้ เช่น "เงินพอใช้อีก 16 วัน"
'2026-10-09'.slice(0, 7)	'2026-10' (ตัดตัวอักษรตำแหน่ง 0 ถึงก่อน 7)
'a,b'.split(',')	['a', 'b'] (แยกเป็น array)
' ข้าว '.trim()	'ข้าว' (ตัดช่องว่างหัวท้าย)
'KFC'.toLowerCase()	'kfc'
String(5).padStart(2, '0')	'05' (เติม 0 ข้างหน้าให้ครบ 2 ตัว ใช้กับวันที่)
s.replace(/บาง/g, '')	ลบคำว่า "บาง" ทุกที่ (/.../g คือ regular expression ดู un A5)

10. คณิตศาสตร์ที่ใช้

คำสั่ง	ผล	ใช้ที่ไหน
<code>Math.floor(20.69)</code>	20 (ปัดลงเสมอ)	จำนวนวัน Money Runway
<code>Math.round(680.2857)</code>	680 (ปัดตามปกติ)	ค่าเฉลี่ยต่อวันเป็นสตางค์
<code>Math.max(0, -500)</code>	0 (ตัวที่มากกว่า)	กั้นเงินที่ใช้ได้ติดลบ
<code>Math.min(7, 3)</code>	3 (ตัวที่น้อยกว่า)	จำนวนวันที่ใช้หารค่าเฉลี่ย ไม่เกิน 7
<code>Math.abs(-85)</code>	85 (ค่าสัมบูรณ์)	แสดงยอดเงินโดยไม่สนเครื่องหมาย

แบบฝึกหัด A4

1. ถ้า profile เป็น null ค่าของ profile?.runwayFloorSatang ?? 50_000 คืออะไร

2. `[100, 200, 300].filter((x) => x > 150).map((x) => x / 100)` ได้อะไร

3. เริ่ม ฿500 มีรายการยืนยันแล้ว: รายรับ ฿1,000 รายจ่าย ฿300 และแบบร่างรายจ่าย ฿200 computeBalance คืนกี่สตางค์

4. `const t2 = { ...t, title: 'ข้าวผัด' }` แล้ว `t.title` เปลี่ยนไหม

i เฉลย

1) $50000 \cdot 2$ [2, 3] (กรองเหลือ 200, 300 แล้วหาร 100) $\cdot 3$ $50,000 + 100,000 - 30,000 = 120,000$ สตางค์ (แบบร่างไม่นับ) $\cdot 4$) ไม่เปลี่ยน t2 เป็น object ใหม่ t ยังเป็นค่าเดิม

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay

TypeScript คือ JavaScript ที่บังคับให้บอกว่าข้อมูลแต่ละตัวเป็นชนิดอะไร ทำให้จับผิดได้ตั้งแต่ตอนพิมพ์ ก่อนผู้ใช้จะเจอ

ทำไมต้อง TypeScript

ใน JavaScript ธรรมดา ถ้าพิมพ์ `tx.amountSatang` (สะกดผิด) โปรแกรมจะรันได้ แต่ได้ค่า `undefined` แล้วยอดเงินเปลี่ยนตอนผู้ใช้กำลังใช้ ส่วน TypeScript จะขีดเส้นแดงใน VS Code กันที่ว่า "ไม่มีช่องชื่อนี้" ทีมเราเปิดโหมดเข้มงวดที่สุด (`"strict": true` ในไฟล์ `tsconfig.json`) และรัน `npm run typecheck` ก่อน build APK ทุกครั้ง

≈ เปรียบเทียบ

TypeScript เหมือนแบบฟอร์มราชการที่มีช่องบังคับว่า "ช่องนี้ต้องเป็นตัวเลข 13 หลัก" กรอกผิดเจ้าหน้าที่ก็กลับไปรู้ตอนเอกสารเข้าระบบแล้ว เมื่อ build แอป ชนิดข้อมูลทั้งหมดจะถูกตรวจสอบ เหลือ JavaScript ธรรมดา ชนิดข้อมูลจึงช่วยเราตอนเขียนโค้ดเท่านั้น ไม่ทำให้แอปช้าลง

การบอกชนิดข้อมูล (type annotation)

บอกชนิดด้วย :

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const days: number = 16;
2 const title: string = 'ข้าวมันไก่';
3 let slipRef: string | null = null; // เป็น string หรือ null ก็ได้
4 function toBaht(satang: number): number { return satang / 100; }
```

หลายครั้งไม่ต้องเขียนชนิดเอง TypeScript เดาเองได้ (type inference) เช่น `const days = 16` รู้เองว่าเป็น number เาเมาส์ชี้ใน VS Code จะเห็นชนิดที่มันเดา

ไฟล์ชนิดข้อมูลกลาง: `src/domain/types.ts`

ไฟล์ที่สำคัญที่สุดไฟล์หนึ่ง เพราะกำหนดว่า "รายการเงิน" และ "โปรไฟล์ผู้ใช้" หน้าตาเป็นยังไง ทุก FR ใช้ร่วมกัน

`src/domain/types.ts`

บรรทัด 1-55

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay drafts). */
12 export type TxStatus = 'draft' | 'confirmed';
13
14 export type TxSource = 'manual' | 'slip';
15
16 export interface Transaction {
17   id: string;
18   kind: TxKind;
19   amountSatang: number;
20   categoryKey: string;
21   title: string;
```

```

22 note: string | null;
23 /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
24 occurredAt: string;
25 source: TxSource;
26 status: TxStatus;
27 /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
28 slipRef: string | null;
29 /** SHA-256 of the processed slip image. Second line of duplicate detection. */
30 slipImageHash: string | null;
31 /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
32 ocrConfidence: number | null;
33 /** Fields the slip reader was unsure about, shown to the user during review. */
34 reviewFlags: string[];
35 createdAt: string;
36 }
37
38 /** Everything the user can type or change in the form. */
39 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
40
41 export type CoachTone = 'friend' | 'coach' | 'senior';
42
43 export interface Profile {
44   id: string;
45   displayName: string;
46   /** Money the user had when they started using MindPay. Balance = opening + income - expense. */
47   openingBalanceSatang: number;
48   /** FR-6: the "low" line. Runway counts the days until balance would reach this amount. */
49   runwayFloorSatang: number;
50   monthlyBudgetSatang: number | null;
51   coachTone: CoachTone;
52   onboarded: boolean;
53 }
54
55 export type RangeKey = 'today' | '7d' | '1m' | '6m' | '1y';

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด ความหมาย

- | | |
|-------|--|
| 1-7 | คอมเมนต์อธิบายกฎสำคัญ: เงินเก็บเป็นจำนวนเต็มสตางค์เสมอ (1 บาท = 100 สตางค์) เพราะใน JavaScript 0.1 + 0.2 ไม่เท่ากับ 0.3 |
| 9 | union type : ชนิด TxKind เป็นได้แค่ 2 ค่า คือข้อความ 'income' (รายรับ) หรือ 'expense' (รายจ่าย) เครื่องหมาย แปลว่า "หรือ" ถ้าพิมพ์ 'incom' จะขึ้นเส้นแดงทันที |
| 11-12 | สถานะของรายการ: 'draft' (แบบร่าง ยังไม่บัน) หรือ 'confirmed' (ยืนยันแล้ว บันในยอด) คอมเมนต์อ้างอิงถึง Charter ของทีมว่า ผลจากสลิปเป็นแบบร่างก่อน |
| 14 | รายการมาจากไหน: จดเอง ('manual') หรือจากสลิป ('slip') รายการที่พุดจด (FR-3) บันเป็น manual |
| 16 | interface = พิมพ์เขียวของ object บอกว่า "รายการ 1 รายการ ต้องมีช่องอะไรบ้าง และแต่ละช่องชนิดอะไร" |
| 17 | id รหัสไม่ซ้ำของรายการ (ฐานข้อมูลสร้างให้) |
| 18 | kind ใช้ชนิด TxKind จากบรรทัด 9 |
| 19 | amountSatang จำนวนเงินเป็นสตางค์ (บวกเสมอ ส่วนเข้าหรือออกดูจาก kind) |
| 20 | categoryKey รหัสหมวด เช่น 'food', 'transport' (รายชื่อหมวดอยู่ใน src/domain/categories.ts) |
| 21-22 | ชื่อรายการ และบันทึกเพิ่มเติม (string null = ไม่มีบันทึกก็ได้) |
| 23-24 | เวลาที่เกิดรายการ เก็บแบบสากล ISO 8601 เช่น "2026-09-27T01:15:00.000Z" (Z = เวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ตัวอย่างนี้คือ 08:15 น. เวลาไทย) |
| 25-26 | ที่มาและสถานะ ใช้ชนิดจากบรรทัด 14 และ 12 |

บรรทัด	ความหมาย
27-30	เลขอ้างอิงจากสลิป และ "ลายนิ้วมือ" ของรูป (SHA-256) ใช้กันสลิปซ้ำ 2 ชั้น (FR-4)
31-32	ความมั่นใจต่ำสุดที่ AI อ่านได้ (0 ถึง 1) รายการที่จัดเองเป็น null
33-34	รายชื่อช่องที่ AI ไม่แน่ใจ เช่น ['amount'] ให้หน้าตรวจสอบสลิปไฮไลต์ช่องนั้น string[] = array ของข้อความ
35	เวลาที่สร้างแถวในระบบ
38-39	TransactionInput = Transaction ที่ ไม่มี id และ createdAt (omit = ตัดช่องออก) ใช้ตอนสร้างรายการใหม่ เพราะสองช่องนี้ฐานข้อมูลเป็นคนสร้าง
41	โทนโคช 3 แบบเดิม (ไม่ได้ใช้แล้ว เพราะ 29 ก.ย. เปลี่ยนเป็นน้องกล้าเสียงเดียว แต่ยังเก็บชนิดไว้ให้ตรงกับคอลัมน์ในฐานข้อมูล)
43-53	Profile ข้อมูลผู้ใช้ 1 คน: ชื่อ, เงินตอนเริ่มใช้แอป (บรรทัด 47), เส้นเงินสำรองของ FR-6 (บรรทัด 49), จบเดือน (ไม่ตั้งก็ได้ = null), โทนโคช และตั้งค่าครั้งแรกเสร็จหรือยัง
55	ช่วงเวลาที่เลือกดูได้ในหน้าหลัก: วันนี้, 7 วัน, 1 เดือน, 6 เดือน, 1 ปี (FR-2)

ชนิดข้อมูลแบบพิเศษที่เจอบ่อย

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
A B	เป็น A หรือ B (union)	'draft' 'confirmed'
ชื่อ?: ชนิด	ช่องนี้ไม่ใช่ก็ได้ (optional)	now?: Date
ชนิด[]	array ของชนิดนั้น	Transaction[]
Omit<T, 'a'>	T ที่ตัดช่อง a ออก	Omit<Transaction, 'id' 'createdAt'>
Partial<T>	T ที่ทุกช่องไม่บังคับ ใช้ตอน "แก้บางช่อง"	updateTx(id, patch: Partial<TransactionInput>)
Pick<T, 'a' 'b'>	T ที่เอาเฉพาะช่อง a และ b	Pick<TxRow, 'kind' 'amount_satang'>
Record<K, V>	object ที่ชื่อช่องเป็นชนิด K ค่าเป็นชนิด V	Record<string, unknown>
Promise<T>	"สัญญาว่าจะได้ T ในอนาคต" (งานที่ต้องรอ)	getProfile(): Promise<Profile>
type X = ...	ตั้งชื่อเล่นให้ชนิด	type TxKind = 'income' 'expense'
x as Y	บอก TypeScript ว่า "เชื่อกัน x เป็นชนิด Y" (ใช้น้อยๆ)	data as TxRow[]
'confirmed' as const	ให้ถือว่าเป็นค่าตายตัว ไม่ใช่ string ทั่วไป	status: 'confirmed' as const
unknown	ยังไม่รู้ชนิด ต้องตรวจสอบก่อนใช้ (ปลอดภัยกว่า any)	context: unknown

import และ export: แอร์โค้ดระหว่างไฟล์

src/data/AppProvider.tsx

บรรทัด 6-17

```

6 import { Storage } from './storage';
7 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type ReactNode } from
  'react';
8 import { AppState } from 'react-native';
9 import { authErrorMessage, authLinkErrorMessage, parseAuthLink } from '../domain/auth';

```

```

10 import { bkkDayKey, msUntilNextBkkMidnight } from '../domain/dates';
11 import { applyDeposit, reservedSatang, type GoalInput, type SavingsGoal } from '../domain/goals';
12 import { averageDailyExpense, computeRunway } from '../domain/runway';
13 import { computeBalance } from '../domain/summary';
14 import type { Profile, Transaction, TransactionInput } from '../domain/types';
15 import { createCloudRepo, createDemoRepo, resetDemoData, type Repo } from './repo';
16 import { clearLinkFromAddressBar, onIncomingLink, openingLink } from './authLinks';
17 import { supabase } from './supabase';

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:6

บรรทัด ความหมาย

- | | |
|------|--|
| 6 | นำ Storage มาจากไฟล์ ./storage (จุด = โพลเดอร์เดียวกัน ไม่ต้องใส่نامสกุลไฟล์) |
| 7 | นำเครื่องมือของ React มาใช้ ชื่อที่ไม่มีจุดนำหน้า เช่น 'react' คือแพ็คเกจใน node_modules |
| 9-13 | ../domain/... = ถอยออกไป 1 โพลเดอร์ (จาก data ไป src) แล้วเข้า domain นำฟังก์ชันคำนวณมาใช้ |
| 14 | import type = นำมาเฉพาะชนิดข้อมูล (หายไปตอน build ไม่เพิ่มขนาดแอป) |
| 11 | type GoalInput ในวงเล็บปีกกา = import ทั้งฟังก์ชันและชนิดจากไฟล์เดียวกันในบรรทัดเดียว |

ฝั่งไฟล์ต้นทางต้องเขียน export หน้าสิ่งที่ยอมให้คนอื่นใช้ ส่วน export default ใช้กับ "ของหลัก" ของไฟล์ เช่นหน้าจอแต่ละหน้า (Expo Router หาหน้าจอจาก default export)

งานที่ต้องรอ: async, await และ Promise

การคุยกับอินเทอร์เน็ตใช้เวลา (อาจครั้งวินาทีหรือหลายวินาที) ถ้าแอปหยุดรอเฉย ๆ หน้าจอจะค้าง JavaScript จึงใช้ **Promise** คือ "ใบรับของ" ที่บอกว่าเดี๋ยวผลจะมา

async / await

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 async function save() {           // async = ฟังก์ชันนี้มันงานที่ต้องรอ
2   const tx = await repo.insert(input); // await = รอตรงนี้อันได้ผล แต่หน้าจอไม่ค้าง
3   setTxs((list) => [tx, ...list]); // ทำต่อเมื่อได้ผลแล้ว
4 }

```

ถ้ามีหลายงานที่ไม่ต้องรอกัน ใช้ Promise.all ทำพร้อมกัน เช่น ตอนโหลดข้อมูลครั้งแรก แอปโหลดโปรไฟล์ รายการ และกระปุกออมพร้อมกันทั้ง 3 อย่าง

src/data/AppProvider.tsx

บรรทัด 131-133

```

131 try {
132   // Goals never block the app: if they cannot be loaded, the rest still works.
133   const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(), r.listGoals().catch(() =>
    [] as SavingsGoal[])]);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:131

บรรทัด 133: เรียก 3 งานพร้อมกัน แล้วรอจนเสร็จทั้งหมด ผลออกมาเป็น array 3 ช่อง แยกใส่ตัวแปร p, list, g ส่วน .catch(() => []) ทำงานที่ 3 แปลว่า "ถ้าโหลดกระปุกออมไม่ได้ ให้ถือว่าไม่มีกระปุก" แอปส่วนอื่นยังใช้ได้ (Supabase ฝั่งเซิร์ฟเวอร์ใน Edge Function ก็ใช้ Promise.allSettled อ่านสไลด์ 2 ครั้งพร้อมกันแบบเดียวกัน un A10)

จัดการความผิดพลาด: try, catch, throw

try / catch / finally

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 try {
2   await addTx(input);           // ลองทำ

```



```

3 toast({ message: 'บันทึกรายการแล้ว' });
4 } catch (e) { // ถ้ามี error ระหว่างทาง กระโดดมาที่นี่
5 toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
6 } finally { // ถ้าเสมอ ไม่ว่าสำเร็จหรือพลาด
7 setBusy(false);
8 }

```

throw คือ "โยน error" ออกไปให้ผู้เรียกจัดการ ทีมเราสร้าง error ชนิดของตัวเองเพื่อแยกเหตุผล เช่น DuplicateSlipError (สลิปซ้ำ) ใน [src/data/repo.ts:14](#)

src/data/repo.ts

บรรทัด 14-19

```

14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:14

class ... extends Error = สร้างชนิด error ใหม่โดยต่อยอดจาก Error ธรรมดา super(...) ส่งข้อความภาษาไทยให้ Error ตัวแม่ และตั้ง name ไว้ให้หน้าจอเช็กได้ว่าเป็น error แบบไหน (ดู [src/app/transaction.tsx:126](#))

ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมเป็นเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ทดลองรันใน Node.js จริง:

```

0.1 + 0.2    → 0.30000000000000004
19.99 * 100  → 1998.9999999999998

```

ถ้าบวกเงินเป็นบาททศนิยมหลายพันรายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็น **สตางค์ (จำนวนเต็ม)** ทั้งหมด เช่น ฿85.50 เก็บเป็น 8550 จำนวนเต็มบวกกลับกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงบนจอ ฐานข้อมูลก็เก็บเป็นจำนวนเต็ม (คอลัมน์ amount_satang bigint)

อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์

ฟังก์ชัน parseBahtToSatang ใน [src/domain/money.ts](#) ใช้ทุกที่ที่ผู้ใช้กรอกจำนวนเงิน (FR-1 จดรายการ, ตั้งค่า, เช็กก่อนจ่ายของ FR-6, กระปุกออม) และใช้กับยอดเงินที่ AI อ่านจากสลิปด้วย

src/domain/money.ts

บรรทัด 1-40

```

1 /**
2  * Money helpers. All amounts are integers in satang.
3  */
4
5 /** The largest amount the app takes: ฿1,000,000,000 (the database caps goals at the same value). */
6 export const MAX_SATANG = 100_000_000_000;
7
8 /** "๑๒๓" -> "123": Thai digits typed on a Thai keyboard. */
9 export function toAsciiDigits(s: string): string {
10   return s.replace(/[\u0E50-\u0E59]/g, (d) => String(d.charCodeAt(0) - 0x0e50));
11 }
12
13 /**
14  * Parse what a user (or a slip) wrote into satang.
15  * Accepts "1,234.50", "1234.5", "฿85", "85 บาท", " 1 234.00 ", Thai digits "๘๕", and a decimal
16  * comma as typed on phones set to such regions ("85,50" = 85.50, "12,34" = 12.34; never 1,234).
17  * Other commas must group thousands ("1,234.50"); anything else ("1,23,4") is refused.
18  * Returns null for anything that is not a positive amount with at most 2 decimals, or above

```

```

19  * MAX_SATANG. With `allowZero`, "0" and "0.00" give 0 (an opening balance may be zero).
20  */
21  export function parseBahtToSatang(input: string | number | null | undefined, opts: { allowZero?: boolean } =
    {}): number | null {
22      if (input === null || input === undefined) return null;
23      const ok = (satang: number) => (satang > MAX_SATANG ? null : satang > 0 || (opts.allowZero && satang === 0)
    ? satang : null);
24      if (typeof input === 'number') {
25          if (!Number.isFinite(input) || input < 0) return null;
26          return ok(Math.round(input * 100));
27      }
28      let cleaned = toAsciiDigits(input)
29          .replace(/฿|บาท|THB|baht/gi, '')
30          .replace(/\s/g, '');
31      if (/^\d+,\d{1,2}$/.test(cleaned)) cleaned = cleaned.replace(',', '.');
32      else if (cleaned.includes(',')) {
33          if (!/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/i.test(cleaned)) return null;
34          cleaned = cleaned.replace(/,/g, '');
35      }
36      if (!/^\d+(\.\d{0,2})?$/i.test(cleaned)) return null;
37      const [whole, frac = ''] = cleaned.split('.');
38      if (whole.length > 12) return null;
39      return ok(Number(whole) * 100 + Number((frac + '00').slice(0, 2)));
40  }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/money.ts:1

บรรทัด	ความหมาย
5-6	ค่าคงที่ MAX_SATANG = 100,000,000,000 สตางค์ = ฿1,000,000,000 เป็นจำนวนเงินสูงสุดที่แอปรับ (ฐานข้อมูลตั้งเพดานเดียวกันกับค่าที่อ่านผิดใหญ่เกินจริง)
8-11	toAsciiDigits แปลงเลขไทย "๑๒๓" เป็น "123" คนที่ใช้แป้นพิมพ์ไทยอาจพิมพ์เลขไทยมา ทำงานโดยหาตัวอักษรในช่วงเลขไทย (รหัส U+0E50 ถึง U+0E59) แล้วลบด้วยรหัสของ "๐" (0x0E50) จะได้ 0-9
13-20	คอมเมนต์บอกว่ารับรูปแบบไหนได้บ้าง: "1,234.50", "1234.5", "฿85", "85 บาท", เลขไทย และทศนิยมแบบคอมมาที่มีทศนิยมบางประเทศใช้ ("85,50" = 85.50) แต่ "1,23,4" ไม่รับ
21	รับค่าเป็นข้อความหรือตัวเลขก็ได้ และมีตัวเลือก allowZero (ปกติ 0 ไม่ผ่าน เพราะรายการเงิน 0 บาทไม่มีความหมาย แต่ยอดเงินตั้งต้นเป็น 0 ได้) คืนค่าเป็นสตางค์ หรือ null ถ้าไม่ใช่จำนวนเงินที่ต้องการ
22	ไม่มีค่าเลย คืน null
23	ฟังก์ชันย่อย ok ตรวจสอบสุดท้าย: เกินเพดานไม่ผ่าน, มากกว่า 0 ผ่าน, เท่ากับ 0 ผ่านเฉพาะเมื่อ allowZero
24-27	ถ้าส่งเข้ามาเป็นตัวเลขอยู่แล้ว ต้องเป็นตัวเลขจริงและไม่ติดลบ แล้วคูณ 100 ปัดเป็นจำนวนเต็ม (ปัดเพื่อแก้ปัญหา 19.99 × 100 = 1998.999...)
28-30	ถ้าเป็นข้อความ: แปลงเลขไทย ลบสัญลักษณ์ ฿, คำว่า "บาท", THB, baht (ไม่สนตัวพิมพ์เล็กใหญ่) และลบช่องว่างทั้งหมด
31	ถ้ารูปแบบคือ "ตัวเลข,ตัวเลข 1-2 หลัก" เช่น "85,50" ถือว่าคอมมาคือจุดทศนิยม
32-35	ถ้ามีคอมมาแบบอื่น ต้องเป็นคอมมาคั่นหลักพันที่ถูกต้อง (เช่น 1,234 หรือ 1,234.50) ถึงจะลบคอมมาออก ไม่จิ้นคืน null
36	สุดท้ายต้องเหลือแค่ตัวเลข กับทศนิยมไม่เกิน 2 ตำแหน่ง ไม่จิ้นคืน null (เช่น "85.555" หรือ "abc")
37	แยกส่วนจำนวนเต็มกับทศนิยมที่จุด ถ้าไม่มีทศนิยมให้ frac เป็นข้อความว่าง
38	เกิน 12 หลักไม่รับ (กันตัวเลขยาวผิดปกติ)
39	คำนวณสตางค์ด้วยจำนวนเต็มล้วน: บาท × 100 + สตางค์ (เติม "00" แล้วตัด 2 ตัว ทำให้ ".5" กลายเป็น 50 สตางค์) ไม่มีการคูณทศนิยมเลย จึงไม่เพี้ยน

ผลจากการรับฟังค์ชันจริง

ผู้ให้พิมพ์	ได้สตางค์	เหตุผล
85	8500	-
85.5	8550	".5" → 50 สตางค์
1,234.50	123450	คอมมาคั่นหลักพันถูกต้อง
฿85 / 85 บาท / ๘๕	8500	ลบสัญลักษณ์และแปลงเลขไทย
85,50	8550	คอมมาเป็นทศนิยม
1,23,4	null	คอมมาผิดรูปแบบ
0	null	ต้องมากกว่า 0 (เว้นแต่ allowZero)
85.555	null	ทศนิยมเกิน 2 ตำแหน่ง
-5	null	มีเครื่องหมายลบ ไม่ใช่ตัวเลขล้วน
1000000000.01	null	เกินเพดาน ฿1,000,000,000

Regular expression (regex) อ่านยังงี้

บรกด 10 และ 29–36 มีของแปลก ๆ ใน / / เรียกว่า **regular expression** คือ "แพตเทิร์นสำหรับค้นหาข้อความ" ไม่ต้องจำทั้งหมด แต่อ่านออก

สัญลักษณ์	แปลว่า
\d	ตัวเลข 0–9 หนึ่งตัว
\s	ช่องว่าง (space, tab, ขึ้นบรรทัด)
+	ตัวข้างหน้ามี 1 ตัวขึ้นไป (\d+ = ตัวเลขที่หลักก็ได้)
{1,3}	ตัวข้างหน้ามี 1 ถึง 3 ตัว
?	ตัวข้างหน้ามีหรือไม่ก็ได้
^ และ \$	ต้นข้อความ และท้ายข้อความ (ต้องตรงทั้งข้อความ ไม่ใช่แค่บางส่วน)
()	จับกลุ่ม
a b	a หรือ b
[๐-๙]	ตัวอักษร 1 ตัวในช่วงเลขไทย ๐–๙
g, i (ท้าย /)	g = หาทุกจุด, i = ไม่สนตัวพิมพ์เล็กใหญ่

ตัวอย่าง `/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/` อ่านว่า: "ขึ้นต้นด้วยตัวเลข 1–3 หลัก ตามด้วย (คอมมา + ตัวเลข 3 หลัก) อย่างน้อย 1 ชุด แล้วอาจมี (จุด + ตัวเลข 1–2 หลัก) และจบข้อความ" ซึ่งก็คือรูปแบบ 1,234 หรือ 12,345,678.50

แปลงสตางค์กลับเป็นข้อความบาท

src/domain/money.tsบรรทัด 42–63

```
42 /** 123450 -> "1,234.50" */
43 export function formatSatang(satang: number, opts: { decimals?: boolean } = {}): string {
44   const decimals = opts.decimals ?? true;
```

```

45 const negative = satang < 0;
46 const abs = Math.abs(Math.round(satang));
47 // Whole baht only: round to the nearest baht (#99.99 reads #100, not #99).
48 const whole = decimals ? Math.floor(abs / 100) : Math.round(abs / 100);
49 const frac = abs % 100;
50 const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
51 const body = decimals ? `${grouped}.${String(frac).padStart(2, '0')}` : grouped;
52 return negative ? `-${body}` : body;
53 }
54
55 /** 123450 -> "#1,234.50"; with sign: "+#1,234.50" / "-#1,234.50" */
56 export function formatBaht(
57   satang: number,
58   opts: { decimals?: boolean; sign?: boolean } = {},
59 ): string {
60   const body = formatSatang(Math.abs(satang), opts);
61   if (opts.sign) return `${satang < 0 ? '-' : '+'}#${body}`;
62   return `${satang < 0 ? '-' : ''}#${body}`;
63 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/money.ts:42`

บรรทัด	ความหมาย
42–43	formatSatang แปลง 123450 เป็น "1,234.50"
44	ตัวเลือก decimals ถ้าไม่ส่งมาใช้ค่าเริ่มต้น true (แสดงสตางค์)
45–46	จำไว้ว่าติดลบไหม แล้วทำงานกับค่าบวกที่ปิดเป็นจำนวนเต็มแล้ว
47–48	ส่วนบาท: ถ้าแสดงสตางค์ด้วย ใช้ปัดลง (สตางค์จะแสดงแยก) ถ้าไม่แสดงสตางค์ ปิดตามปกติ #99.99 จึงแสดงเป็น #100 ไม่ใช่ #99
49	สตางค์คือเศษจากการหาร 100
50	ใส่คอมมามันหลักพันด้วย regex (หาตำแหน่งที่ด้านหลัมีตัวเลขเป็นชุดละ 3 ตัวพอดี)
51–52	ต่อสตางค์ 2 หลัก (5 สตางค์ → "05") และเติมเครื่องหมายลบถ้าติดลบ
55–63	formatBaht เติม "฿" ข้างหน้า และถ้าขอเครื่องหมาย (sign) จะใส่ + หรือ - เช่น formatBaht(-8500, {sign: true}) → "-฿85.00"

i ผลจริง

formatBaht(123450) → "#1,234.50" · formatBaht(-8500, { sign: true }) → "-฿85.00" ·
 formatBaht(9999, { decimals: false }) → "#100"

React: วาดหน้าจอจากข้อมูล

React คือวิธีคิดที่ว่า "หน้าจอ = ฟังก์ชันของข้อมูล" ข้อมูลเปลี่ยนเมื่อไร หน้าจอวาดใหม่เอง ไม่ต้องสั่งแก้ทีละจุด

แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล

สมัยก่อน ถ้ายอดเงินเปลี่ยน โปรแกรมเมอร์ต้องเขียนเองว่า "ไปหาข้อความยอดเงิน แก่ตัวเลข ไปหาต้นไม้ เปลี่ยนจำนวนใบ ไปหาป้ายสถานะ เปลี่ยนสี..." พลาดจุดเดียวหน้าจอก็ขัดกันเอง React เปลี่ยนวิธีคิด: **เราเขียนแค่ว่า "ถ้าข้อมูลเป็นแบบนี้ หน้าจอหน้าตาแบบนี้"** แล้ว React จะวาดใหม่ให้เองทุกครั้งที่ข้อมูลเปลี่ยน

ข้อมูล (state)

เช่น รายการทั้งหมด txs, ยอดคงเหลือ

↓ React เรียก component ใหม่

Component (ฟังก์ชันวาดหน้าจอ)

อ่านข้อมูล แล้วคืน "หน้าตา" เป็น JSX

↓ React เทียบกับของเดิม แก้เฉพาะจุดที่ต่าง

หน้าจอที่ผู้ใช้เห็น

Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน

Component คือฟังก์ชันที่ชื่อขึ้นต้นด้วยตัวใหญ่ และคืนค่าเป็น **JSX** (หน้าตาที่เขียนคล้าย HTML อยู่ในโค้ด) หน้าจอทั้งหน้าก็เป็น component และปุ่มเล็ก ๆ ก็เป็น component ซ้อนกันเป็นชั้น ๆ

Component เล็กที่สุด

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 function Hello({ name }: { name: string }) {
2   return <Text>สวัสดี {name}</Text>;
3 }
4
5 // เรียกใช้ที่อื่น:
6 <Hello name="มิ่งท" />    // แสดง "สวัสดี มิ่งท"
```

ส่วน	ความหมาย
<Text>...</Text>	JSX แยกเปิดและปิด เหมือน HTML (Text คือชิ้นส่วนแสดงข้อความของ React Native)
{name}	วงเล็บปีกกาใน JSX = "ใส่ค่าจากโค้ด JavaScript ตรงนี้"
{ name }: { name: string }	props คือค่าที่ผู้เรียกส่งเข้ามา (เหมือนพารามิเตอร์) ในที่นี้แค่เอาช่อง name ออกมา
<Hello name="มิ่งท" />	ใช้ component เหมือนแท็ก ส่ง props เป็น attribute แท็กที่ไม่มีของข้างในปิดด้วย />

ตัวอย่างจริง: T ชิ้นส่วนตัวหนังสือที่ทุกหน้าใช้

ทุกข้อความในแอปใช้ <T> แทนการตั้งขนาดและสีเอง เพื่อให้ทั้งแอปหน้าตาเหมือนกัน (กฎใน Design System un A11)

src/ui/components.tsx

บรรทัด 36–90

```
36 type Variant = 'display' | 'h1' | 'h2' | 'h3' | 'body' | 'small' | 'micro' | 'label';
37
38 const VARIANT_FONT: Record<Variant, string> = {
39   display: fonts.serif,
```

```

40  h1: fonts.serif,
41  h2: fonts.sansBold,
42  h3: fonts.sansSemi,
43  body: fonts.sans,
44  small: fonts.sans,
45  micro: fonts.sansMedium,
46  label: fonts.sansSemi,
47  };
48
49  export function T({
50    v = 'body',
51    color,
52    style,
53    children,
54    numberOfLines,
55    center,
56    selectable,
57    accessibilityRole,
58  }): {
59    v?: Variant;
60    color?: string;
61    style?: StyleProp<TextStyle>;
62    children: ReactNode;
63    numberOfLines?: number;
64    center?: boolean;
65    selectable?: boolean;
66    accessibilityRole?: 'header' | 'text' | 'alert';
67  } {
68    const theme = useTheme();
69    const size = v === 'label' ? type.micro : type[v];
70    return (
71      <Text
72        accessibilityRole={accessibilityRole}
73        numberOfLines={numberOfLines}
74        selectable={selectable}
75        style={[
76          size,
77          {
78            fontFamily: VARIANT_FONT[v],
79            color: color ?? (v === 'small' || v === 'micro' ? theme.inkSoft : theme.ink),
80            textAlign: center ? 'center' : undefined,
81          },
82          // Labels are text: inkSoft (inkFaint is for placeholders, rules and quiet icons, below 4.5:1).
83          v === 'label' && { letterSpacing: 0.8, textTransform: 'uppercase', color: color ?? theme.inkSoft },
84          style,
85        ]}
86      >
87        {children}
88      </Text>
89    );
90  }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:36

บรรทัด ความหมาย

- 36** ระดับตัวหนังสือมี 8 แบบ ตั้งแต่ใหญ่สุด display (ยอดเงินบนการ์ด) ไปจนเล็กสุด micro
- 38–47** ตารางจับคู่ว่าแต่ละระดับใช้ฟอนต์อะไร: หัวข้อใหญ่ใช้ Noto Serif Thai (มีหัว) ที่เหลือใช้ Anuphan น้ำหนักต่างกัน Record<Variant, string> บังคับให้ต้องใส่ครบทั้ง 8 ระดับ
- 49–57** ประกาศ component ชื่อ T พร้อมแกะ props ออกมา v = 'body' แปลว่าถ้าไม่ระบุระดับ ใช้ body
- 58–67** ชนิดของ props แต่ละตัว (เครื่องหมาย ? = ไม่บังคับ) children คือสิ่งที่อยู่ระหว่าง <T> กับ </T> ส่วน accessibilityRole นอกโปรแกรมอ่านหน้าจอสำหรับคนตาบอดว่าเป็นหัวข้อหรือข้อความแจ้งเตือน

บรรทัด	ความหมาย
68	เรียก useTheme() เอาชุดสีของธีมที่ผู้ใช้เลือก (สว่าง/มืด, ปากทอง/ม่วง ฯลฯ)
69	เลือกขนาดและความสูงบรรทัดจากตาราง type (label ใช้ขนาดเท่า micro)
70–86	คืน JSX: ชิ้นส่วน Text ของ React Native พร้อม style ที่ซ้อนกันเป็นชั้น (array ของ style ตัวหลังทับตัวหน้า)
78	ฟอนต์ตามระดับ
79	สี: ถ้าผู้เรียกส่งสีมาใช้สีนั้น ไม่จับข้อความเล็กใช้สีรอง (inkSoft) ข้อความอื่นใช้สีหลัก (ink)
80	จัดกึ่งกลางถ้าส่ง center
82–83	ระดับ label (หัวข้อเล็กตัวพิมพ์ใหญ่ เช่น "MONEY RUNWAY") ห่างตัวอักษรกว้างขึ้น และใช้สีที่ผ่านเกณฑ์อ่านง่าย
84	style ที่ผู้เรียกส่งมาเองใส่ท้ายสุด จึงทับค่าทั้งหมดได้
87	แสดงสิ่งที่อยู่ข้างใน (ข้อความ)

ใช้งานแบบนี้: `<T v="h3">จดรายการ</T>` หรือ `<T v="small" color={theme.watch}>อ่านวันที่ไม่ชัด</T>`

State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่

ตัวแปรธรรมดาในฟังก์ชันจะหายไปทุกครั้งที่ React วาดใหม่ ถ้าอยากให้ component "จำ" ค่าได้ ต้องใช้ **state** ผ่าน hook ชื่อ `useState`

useState

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const [amount, setAmount] = useState(''); // ค่าเริ่มต้น = ข้อความว่าง
2 // amount = ค่าปัจจุบัน
3 // setAmount = ฟังก์ชันเปลี่ยนค่า เรียกแล้ว React วาดหน้าจอใหม่
4
5 <AmountField value={amount} onChangeText={setAmount} />
6 // ผู้ใช้พิมพ์ "8" → setAmount('8') → วาดใหม่ ช่องแสดง 8
```

หน้าจอดรายการ (FR-1) ใช้ state จำค่าทุกช่องในฟอร์ม ดูจากโค้ดจริง:

src/app/transaction.tsx

บรรทัด 35–46

```
35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:35

บรรทัด	ความหมาย
35	ประเภทเริ่มต้น: ถ้าแก็รรายการเดิม ใช้ประเภทเดิม ไม่จับดูว่าเปิดมาจากปุ่ม "เพิ่มเงินเข้า" ใหม่ (เป็นรายรับ) ไม่จับเป็นรายจ่าย
36	state ประเภท (รายรับ/รายจ่าย) <code>useState<TxKind></code> บอกว่าเก็บได้แค่ 'income' หรือ 'expense'
37	state ช่องจำนวนเงิน เก็บเป็นข้อความ ตามที่ผู้ใช้พิมพ์ (แปลงเป็นสตริงตอนกดบันทึก) ถ้าแก็รรายการเดิม เต็มค่าเดิมให้

บรรทัด	ความหมาย
38–39	ชื่อรายการ และหมวด
40	จำว่าผู้ใช้เลือกหมวดเองแล้วหรือยัง (ถ้ายัง แอปจะเอาหมวดจากชื่อที่พิมพ์ให้)
41–42	วันและเวลา (ค่าเริ่มต้นคือตอนนี้ ตามเวลาไทย)
43	บันทึกเพิ่มเติม
44–46	สถานะของหน้าจอเอง: กำลังบันทึกอยู่ไหม, กำลังถามยืนยันการลบไหม, ข้อความ error ใต้ช่องเงิน

! กฎของ Hook

ฟังก์ชันที่ขึ้นต้นด้วย use เรียกว่า **hook** มีกฎ 2 ข้อ: (1) เรียกได้เฉพาะใน component หรือใน hook อื่น (2) ต้องเรียกที่ระดับบนสุดของ component ห้ามอยู่ใน if หรือ loop เพราะ React จำ state ตามลำดับการเรียก ESLint ของโปรเจกต์ตรวจกฎนี้ให้อัตโนมัติ

Hook ที่ใช้ในโปรเจกต์

Hook	ทำอะไร	ตัวอย่างใน MindPay
useState	จำค่าที่เปลี่ยนได้ เปลี่ยนแล้ววาดใหม่	ทุกช่องในฟอร์ม, รายการทั้งหมดใน AppProvider
useEffect	ทำงาน "ข้างเคียง" หลังวาดเสร็จ เช่น ตั้งเวลา ฟัง event โหลดข้อมูล และเก็บกวาดเมื่อเลิกใช้	ตั้งนาฬิกาให้วันเปลี่ยนตอนเที่ยงคืน
useMemo	คำนวณแล้วจำผลไว้ คำนวณใหม่เฉพาะเมื่อข้อมูลที่ระบุเปลี่ยน	useMoney() คำนวณยอดเงินและ Runway
useCallback	จำ "ฟังก์ชัน" ไว้ไม่ให้สร้างใหม่ทุกครั้ง	loadAll, activateRepo
useRef	กล่องจำค่าที่เปลี่ยนแล้ว ไม่ ต้องวาดใหม่	จำว่ากำลังใช้บัญชีไหนอยู่ (activeUserRef)
useContext	หยิบข้อมูลที่ "แชร์ทั้งแอป" มาใช้	useApp()
hook ของเรา	รวม hook หลายตัวเป็นชิ้นเดียว ตั้งชื่อขึ้นต้นด้วย use เอง	useMoney, useTheme, useToast

ตัวอย่าง useEffect: วันใหม่เริ่มตอนเที่ยงคืนตามเวลาไทย

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();

```



```

198 // First open on a different day from the last one.
199 Storage.getItem(LAST_DAY_KEY)
200   .then((last) => {
201     if (last && last !== todayRef.current) setNewDay(true);
202     return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203   })
204   .catch(() => {});
205 const sub = AppState.addEventListener('change', (s) => {
206   if (s === 'active') check();
207 });
208 return () => {
209   clearTimeout(timer);
210   sub.remove();
211 };
212 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลากรุงเทพ ทั้งตอนเปิดแอปค้างไว้ (ใช้นาฬิกาจับเวลา) และตอนกลับมาเปิดแอปตอนเช้า
184	useEffect(() => {...}, []) array ว่างท้ายสุดแปลว่า "ทำครั้งเดียวตอน component เริ่มทำงาน"
185	ตัวแปรเก็บนาฬิกาจับเวลา
186–196	ฟังก์ชัน check: ดูวันที่ตอนนี้ตามเวลาไทย ถ้าไม่ตรงกับ "วันนี้" ที่จำไว้ แปลว่าข้ามวันแล้ว ให้อัปเดตวันนี้ (setToday) ซึ่งทำให้ตัวเลขทุกอย่างที่ขึ้นกับวันนี้คำนวณใหม่ (เช่น ค่าเฉลี่ย 7 วันของ FR-6)
191	ตั้งธง "วันใหม่" ให้น้องกล่าวกักตักและสรุปเมื่อวาน
192	จดวันที่ล่าสุดลงเครื่อง
194–195	ยกเลิกนาฬิกาเดิม แล้วตั้งใหม่ให้ปลุกตอนเที่ยงคืนครั้งหน้า (+1 วินาทีเพื่อ msUntilNextBkkMidnight คำนวณว่าเหลืออีกกี่มิลลิวินาที
197	เช็กกันทีหนึ่งครั้งตอนเริ่ม
198–204	อ่านวันที่เปิดแอปครั้งสุดท้ายจากเครื่อง ถ้าไม่ใช่วันนี้ แปลว่าเป็นการเปิดครั้งแรกของวัน
205–207	ฟังเหตุการณ์ "แอปกลับมาอยู่บนหน้าจอ" (เช่น เปิดแอปค้างไว้ข้ามคืน แล้วเข้ามาสลับกลับมา) แล้วเช็กวันใหม่อีกครั้ง
208–211	ฟังก์ชันเก็บกวาด (cleanup): ถ้า component นี้ถูกปิด ให้ยกเลิกนาฬิกาและเลิกฟังเหตุการณ์ ไม่ให้ทำงานค้างกินแบตเตอรี่

Context: แשרข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ

ถ้าหน้าหลัก หน้ารายการ และหน้าเงินพอถึง ต้องใช้รายการเงินชุดเดียวกัน จะส่ง props ต่อกันเป็นทอด ๆ ก็ยุ่ง React มี **Context** เหมือน "ประกาศเสียงตามสาย" ตัวให้ข้อมูล (Provider) อยู่บนสุด ใครอยู่ข้างในจะหยิบไปใช้ได้เลย MindPay มี Provider หลายชั้นครอบทั้งแอปไว้ใน [src/app/_layout.tsx](#) (บท A7) ตัวสำคัญที่สุดคือ **AppProvider**

src/data/AppProvider.tsx

บรรทัด 87–121

```

87 const AppContext = createContext<AppContextValue | null>(null);
88
89 export function useApp() {
90   const ctx = useContext(AppContext);
91   if (!ctx) throw new Error('useApp must be used inside AppProvider');
92   return ctx;
93 }
94
95 /** Same as useApp, but null outside the provider (for small shared pieces such as the companion). */
96 export function useAppMaybe() {
97   return useContext(AppContext);
98 }

```

```

99
100 export function AppProvider({ children }: { children: ReactNode }) {
101   const [status, setStatus] = useState<AuthStatus>('loading');
102   const [repo, setRepo] = useState<Repo | null>(null);
103   const [userId, setUserId] = useState<string | null>(null);
104   const [profile, setProfile] = useState<Profile | null>(null);
105   const [txs, setTxs] = useState<Transaction[]>([]);
106   const [carrySatang, setCarry] = useState(0);
107   const [goals, setGoals] = useState<SavingsGoal[]>([]);
108   const [loadError, setLoadError] = useState<string | null>(null);
109   const [refreshing, setRefreshing] = useState(false);
110   const [authNotice, setAuthNotice] = useState<AuthNotice | null>(null);
111   const [today, setToday] = useState(() => bkkDayKey(Date.now()));
112   const [newDay, setNewDay] = useState(false);
113   const todayRef = useRef(today);
114   const repoRef = useRef<Repo | null>(null);
115   /** The account whose data is loaded (or being loaded): prevents loading the same account twice. */
116   const activeUserRef = useRef<string | null>(null);
117   const handledLinks = useRef(new Set<string>());
118   /** Each load gets a number; an older load that finishes late is ignored (it may be another account's). */
119   const loadGen = useRef(0);
120   /** Changes made on this phone while a load is running (null = deleted), put back over what it returns. */
121   const localChanges = useRef(new Map<string, Transaction | null>());

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:87

บรรทัด	ความหมาย
87	สร้าง Context เปล่า ๆ ไว้ (เริ่มต้นเป็น null)
89–93	useApp(): hook ที่ทุกหน้าใช้หีบข้อมูลกลาง ถ้าเรียกนอก Provider จะ error ทันที (กันลืมครอบ)
95–98	แบบที่ไม่ error แต่คืน null สำหรับชิ้นส่วนเล็กที่อาจอยู่นอก Provider
100	ประกาศ component AppProvider ซึ่งครอบหน้าจอทั้งหมด (children)
101	สถานะการเข้าสู่ระบบ: กำลังโหลด / ยังไม่เข้า / พร้อม
102–103	Repo ที่ใช้อยู่ (คาลังหรือเครื่อง) และ id ของผู้ใช้
104–107	ข้อมูลหลักของแอป: โพรไฟล์, รายการทั้งหมด, ยอดยกมาจากรายการเก่ามาก, กระปุกออม
108–110	ข้อความ error ตอนโหลด, คำล้งรีเฟรชใหม่, ข้อความแจ้งเรื่องบัญชี (เช่น "สมัครสำเร็จ")
111	วันนี้ตามเวลาไทย (ฟังก์ชันใน useState ทำงานครั้งเดียวตอนเริ่ม)
112	ธงวันใหม่
113–121	ค่าที่ใช้ useRef เพราะไม่ต้องวาดหน้าจอใหม่เมื่อเปลี่ยน: วันนี้ (ไว้ใช้ในนาฬิกา), Repo ปัจจุบัน, บัญชีที่กำลังโหลด, ลิงก์อีเมลที่จัดการแล้ว, เลขรอบการโหลด และการเปลี่ยนแปลงในเครื่องระหว่างโหลด

ส่วนท้ายของ AppProvider (บรรทัด 325–439) รวมข้อมูลและฟังก์ชันทั้งหมด (เช่น addTx, removeTx, saveProfile) เป็น object เดียว แล้วส่งให้ทุกหน้าผ่าน <AppContext.Provider value={value}> ที่บรรทัด 439

ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไง

รูปแบบที่ทุกหน้าใช้

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 export default function SomeScreen() {
2   const theme = useTheme(); // สีของธีม
3   const { txs, addTx } = useApp(); // ข้อมูลและคำสั่งกลาง
4   const { balance, runway } = useMoney(); // ตัวเลขที่คำนวณแล้ว
5   const [open, setOpen] = useState(false); // state ของหน้านี้เอง
6

```

```

7   return (
8     <Screen>
9       <T v="h1">ยอด {formatBaht(balance)}</T>
10      {runway.days !== null ? <T>เงินพอใช้อีก {runway.days} วัน</T> : null}
11    </Screen>
12  );
13 }

```

บรรทัด `{runway.days !== null ? ... : null}` คือ **การแสดงผลแบบมีเงื่อนไข** ใน JSX: ถ้ามีจำนวนวัน แสดงข้อความ ถ้าไม่มี (null) ไม่แสดงอะไร จะเจอรูปแบบนี้ทุกหน้า อีกแบบที่เจอบ่อยคือ `{list.map((x) => <Row key={x.id} ... />)}` แสดงรายการหลายแถวจาก array (ต้องใส่ key ที่ไม่ซ้ำ ให้ React รู้ว่าแถวไหนเป็นแถวไหน)

★ สรุป A6 ใน 4 ประโยค

(1) Component คือฟังก์ชันที่คืนหน้าตา (JSX) (2) props คือของที่ส่งเข้า component, state คือความจำของ component (3) เปลี่ยน state ด้วย `setXxx` แล้ว React วาดใหม่เอง (4) ข้อมูลที่ทั้งแอปใช้ร่วมกันอยู่ใน `AppProvider` หยิบใช้ด้วย `useApp()` และตัวเลขเงินหยิบด้วย `useMoney()`

React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ

ทำไมทีมเราเขียนโค้ดครั้งเดียว แล้วได้ทั้งแอป Android, iPhone และเว็บ และการเปลี่ยนหน้าจอด้วย Expo Router ทำงานยังไ้

React Native คืออะไร

React (บท A6) เดิมใช้ทำเว็บ ซึ่งวาดหน้าจอด้วย HTML แต่แอปมือถือไม่มี HTML **React Native** จึงเปลี่ยน "ชิ้นส่วนพื้นฐาน" ให้เป็นของจริงบนมือถือ เราเขียน `<View>` แล้วบน Android กลายเป็นกล่องของ Android บน iPhone กลายเป็นกล่องของ iOS แอปจึงสั้นเหมือนแอปที่เขียนด้วยภาษาของมือถือโดยตรง

สำหรับเว็บ มีแพ็คเกจ **react-native-web** (บรรทัด 37 ใน package.json) แปลชิ้นส่วนเดียวกันเป็น HTML เช่น `<View>` → `<div>` `<Text>` → `<span>` โค้ดชุดเดียวจึงได้ 3 แพลตฟอร์ม

ชิ้นส่วน React Native	คล้ายกับ (เว็บ)	ใช้ทำอะไร
View	div	กล่องสำหรับจัดวาง (การ์ด แถว คอลัมน์)
Text	span / p	ข้อความ (ข้อความทุกตัวต้องอยู่ใน Text)
Pressable	button	พื้นที่ที่แตะได้ มี onPress, onPressIn, onPressOut
ScrollView / FlatList / SectionList	หน้าที่เลื่อนได้	เลื่อนเนื้อหายาว ๆ (FlatList/SectionList เหมาะกับรายการยาวมาก เพราะวาดเฉพาะแถวที่เห็น)
TextInput	input	ช่องพิมพ์ (ห่ออยู่ใน Field / AmountField ของเรา)
Image	img	รูปภาพ
SafeAreaView	-	ไม่ให้เนื้อหาไปซ้อนทับรอยบาก แถบสถานะ หรือขอบล่างของมือถือ
Animated.View	-	กล่องที่ขยับได้ (แอนิเมชัน)

การจัดวางด้วย style

React Native ไม่มีไฟล์ CSS แต่ใส่ `style={{ ... }}` เป็น object ตรง ๆ ชื่อเหมือน CSS แต่เป็น camelCase เช่น `backgroundColor` หน่วยเป็นตัวเลขเฉย ๆ (หมายถึง "จุด" ที่ปรับตามความละเอียดจอเอง) การจัดวางใช้ **Flexbox**:

style	ความหมาย
<code>flexDirection: 'row'</code>	เรียงลูกในแนวนอน (ค่าเริ่มต้นคือแนวตั้ง 'column')
<code>flex: 1</code>	ขยายเต็มพื้นที่ที่เหลือ
<code>alignItems: 'center'</code>	จัดลูกให้อยู่กลาง (แกนขวาง)
<code>justifyContent: 'space-between'</code>	กระจายลูกให้ชิดหัวท้าย (แกนหลัก)
<code>gap: 8</code>	ระยะห่างระหว่างลูกแต่ละตัว
<code>padding / margin</code>	ระยะห่างด้านใน / ด้านนอก
<code>borderRadius</code>	มุมโค้ง

ตัวอย่างจากชิ้นส่วน Row ที่ใช้ทั้งแอป: บรรทัดเดียวจบ

```

189 export function Row({
190   children,
191   gap = space.sm,
192   style,
193   align = 'center',
194   justify,
195 }): {
196   children: ReactNode;
197   gap?: number;
198   style?: StyleProp<ViewStyle>;
199   align?: ViewStyle['alignItems'];
200   justify?: ViewStyle['justifyContent'];
201 }) {
202   return <View style={[{ flexDirection: 'row', alignItems: align, justifyContent: justify, gap }, style]}>
    {children}</View>;
203 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:189

ussกัถ 202: สร้าง View ที่เรียงลูกแนวนอน (row) จัดกลางแนวดิ่ง และเว้นระยะ gap ค่าเริ่มต้น 8 แล้วต่อด้วย style ที่ผู้เรียกส่งมา

Expo: ชุดเครื่องมือที่ทำให้ React Native ข่ายขึ้น

React Native เปล่า ๆ ต้องติดตั้ง Android Studio, Xcode และตั้งค่านามมาย **Expo** ห่อทุกอย่างให้ และมีโมดูลสำเร็จรูป (expo-camera, expo-speech ฯลฯ) ที่ใช้ได้เลย สิ่งที Expo ให้ทีมเรา:

- **Expo Go + dev server:** แก่โค้ดแล้วเห็นผลบนมือถือภายในไม่กี่วินาที
- **Expo Router:** ระบบเปลี่ยนหน้าจอจากโครงสร้างไฟล์ (หัวข้อถัดไป)
- **expo export:** ทำเว็บเวอร์ชันเป็นไฟล์นิ่ง ๆ เอาไปวางบนเว็บไซต์ได้
- **prebuild + EAS Build:** สร้างโปรเจกต์ Android/iOS และไฟล์ APK
- **EAS Update:** ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้วโดยไม่ต้องลง APK ใหม่

! Expo ทีมเราใช้เวอร์ชัน 57

Expo เปลี่ยนเร็วมาก ทุกเวอร์ชันมีคำสั่งที่เปลี่ยนชื่อหรือย้ายที่ ไฟล์ AGENTS.md ในโปรเจกต์จึงเตือนว่าก่อนใช้ API ของ Expo ให้เปิดเอกสารของเวอร์ชัน 57 (docs.expo.dev/versions/v57.0.0/) ไม่ใช่จำจากที่เคยเห็น

Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า

ใน src/app/ ทุกไฟล์คือหน้าจอ ชื่อไฟล์คือที่อยู่ (URL) ของหน้านั้น บนเว็บพิมพ์ที่อยู่ได้ตรง ๆ เลย

ไฟล์	ที่อยู่	หน้าจอ	FR
_layout.tsx	-	โครงกั๊งแอป: ฟอนต์, Provider, กฎว่าเข้าหน้าไหนได้ (ไม่ใช่หน้าจอ)	ทุก FR
welcome.tsx	/welcome	สมัคร เข้าสู่ระบบ สัมผัสผ่าน ลองใช้ข้อมูลตัวอย่าง	บัญชี
onboarding.tsx	/onboarding	ตั้งค่าเงินครั้งแรก (ยอดเงินตอนนี้, เงินสำรอง, งบ)	FR-2, FR-6
(tabs)/_layout.tsx	-	แถบเมนูล่าง 4 แท็บ + ปุ่มสแกน	-
(tabs)/index.tsx	/	หน้าหลัก	FR-2 (+FR-6)
(tabs)/transactions.tsx	/transactions	แท็บรายการ ปฏิทินการใช้จ่าย	FR-1, FR-2
(tabs)/runway.tsx	/runway	แท็บเงินพอถึง	FR-6

ไฟล์	ที่อยู่	หน้าจอ	FR
(tabs)/coach.tsx	/coach	แท็บโค้ชน้องกล้า	FR-5
transaction.tsx	/transaction	จด/แก้/ลบ/ตรวจสอบสลิป 1 รายการ	FR-1, FR-4
voice.tsx	/voice	พูดจด	FR-3
scan.tsx	/scan	สแกนสลิป	FR-4
drafts.tsx	/drafts	สลิปรอยืนยัน	FR-4
settings.tsx	/settings	ตั้งค่า (เงินสำรอง, ธีม, เสียงน้องกล้า ฯลฯ)	หลาย FR
goals.tsx, goal.tsx	/goals, /goal	กระปุกออม	FR-6
recap.tsx	/recap	สรุปเดือนแบบเรื่องเล่า	FR-2
achievements.tsx, skins.tsx	/achievements, /skins	เหรียญ การกิจ ตูสกินน้องกล้า	FR-5 (เสริม)
whatsnew.tsx	/whatsnew	มีอะไรใหม่	-

- โฟลเดอร์ในวงเล็บ เช่น (tabs) เป็น "กลุ่ม" ไม่ไฟล์ในที่อยู่ ใช้จัดหน้าที่อยู่ใต้แถบเมนูเดียวกัน
- index.tsx คือหน้าแรกของโฟลเดอร์นั้น (ที่อยู่ /)
- ไฟล์ _layout.tsx ไม่ใช่หน้าจอ แต่เป็น "กรอบ" ที่ครอบหน้าจอในโฟลเดอร์เดียวกัน

เปลี่ยนหน้าในโค้ด

โค้ด	ผล
router.push('/scan')	เปิดหน้าสแกนซ้อนขึ้นมา (กดกลับได้)
router.push({ pathname: '/transaction', params: { kind: 'income' } })	เปิดหน้าจดยานการ พร้อมส่งค่า kind=income (ปุ่ม "เพิ่มเงินเข้า")
useLocalSearchParams()	หน้าปลายทางอ่านค่าที่ส่งมา (src/app/transaction.tsx:28)
router.navigate('/runway')	ไปที่แท็บเงินพอถึง
goBack()	กลับหน้าก่อน ถ้าไม่มี (เช่น เปิดลิงก์ตรงบนเว็บ) กลับหน้าหลัก (src/ui/nav.ts)

อ่านโค้ด: _layout.tsx โครงของทั้งแอป

ไฟล์นี้ทำงานเป็นไฟล์แรกเมื่อเปิดแอป ทุกหน้าจออยู่ข้างในมัน

src/app/_layout.tsx	บรรทัด 1-51
<pre> 1 import { useFonts } from 'expo-font'; 2 import { Stack } from 'expo-router'; 3 import * as SplashScreen from 'expo-splash-screen'; 4 import { StatusBar } from 'expo-status-bar'; 5 import { useCallback, useState, type ReactNode } from 'react'; 6 import { Platform, View } from 'react-native'; 7 import { SafeAreaProvider } from 'react-native-safe-area-context'; 8 import { 9 Anuphan_400Regular, 10 Anuphan_500Medium, 11 Anuphan_600SemiBold, </pre>	

```

12 Anuphan_700Bold,
13 } from '@expo-google-fonts/anuphan';
14 import { NotoSerifThai_600SemiBold, NotoSerifThai_700Bold } from '@expo-google-fonts/noto-serif-thai';
15 import { AppProvider, useApp } from '../data/AppProvider';
16 import { AutoScanProvider } from '../services/AutoScanProvider';
17 import { AuthNoticeHost } from '../ui/AuthNotice';
18 import { CelebrationProvider } from '../ui/effects';
19 import { ToastProvider } from '../ui/feedback';
20 import { LaunchIntro } from '../ui/LaunchIntro';
21 import { UpdateBanner } from '../ui/UpdateBanner';
22 import { mix, useTheme } from '../ui/theme';
23
24 // The native splash stays until the opening animation (LaunchIntro) takes over.
25 SplashScreen.preventAutoHideAsync().catch(() => {});
26
27 export default function RootLayout() {
28   const [fontsLoaded, fontError] = useFonts({
29     Anuphan_400Regular,
30     Anuphan_500Medium,
31     Anuphan_600SemiBold,
32     Anuphan_700Bold,
33     NotoSerifThai_600SemiBold,
34     NotoSerifThai_700Bold,
35   });
36   return (
37     <SafeAreaProvider>
38       <WebFrame>
39         <CelebrationProvider>
40           <AppProvider>
41             <ToastProvider>
42               <AutoScanProvider>
43                 <Navigator ready={fontsLoaded || !!fontError} />
44               </AutoScanProvider>
45             </ToastProvider>
46           </AppProvider>
47         </CelebrationProvider>
48       </WebFrame>
49     </SafeAreaProvider>
50   );
51 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:1

บรรทัด ความหมาย

- 1-22** นำเครื่องมือมาใช้: โหลดฟอนต์, Stack (การซ้อนหน้า), หน้าจอเปิดแอป, แถบสถานะ, ฟอนต์ Anuphan 4 น้ำหนัก และ Noto Serif Thai 2 น้ำหนัก, Provider ต่าง ๆ ของเรา
- 24-25** สั่งให้หน้าจอเปิดแอป (splash สีเขียวรูปน้องกล้า) ค้างไว้ก่อน จนกว่าแอนิเมชันเปิดแอปจะมารับช่วง .catch(() => {}) = ถ้าทำไม่ได้ก็ไม่เป็นไร
- 27** export default = นี่คือ component หลักของไฟล์ Expo Router จะเรียกตัวนี้
- 28-35** โหลดฟอนต์ทั้ง 6 ตัว ได้ผลว่าโหลดเสร็จหรือยัง (fontsLoaded) หรือมี error ไหม
- 36-50** ซ้อน Provider เป็นชั้น ๆ เหมือนตุ๊กตาแม่ลูกดก ชั้นนอกสุดถึงชั้นในสุด: ขอบจอปลอดภัย → กรอบเว็บ → ระบบพลลวง → **ข้อมูลกลาง (AppProvider)** → ข้อความแจ้งเตือน (toast) → **สแกนสลิปอัตโนมัติ (FR-4)** → ตัวเปลี่ยนหน้า ชั้นในใช้ของชั้นนอกได้ เช่น AutoScanProvider ใช้ข้อมูลจาก AppProvider ได้เพราะอยู่ข้างใน
- 43** ส่ง ready = ฟอนต์โหลดเสร็จ หรือโหลดพังก็ให้ไปต่อ (ไม่ค้างหน้าขาวตลอดไป)

src/app/_layout.tsx

บรรทัด 53-73

```

53 /** On a computer browser, show the app at phone width in the middle of the screen. */
54 function WebFrame({ children }: { children: ReactNode }) {

```

```

55   const theme = useTheme();
56   if (Platform.OS !== 'web') return <>{children}</>;
57   return (
58     <View style={{ flex: 1, alignItems: 'center', backgroundColor: theme.dark ? mix(theme.bg, '#000000', 0.55)
: theme.surfaceAlt }}>
59       <View
60         style={{
61           flex: 1,
62           width: '100%',
63           maxWidth: 440,
64           backgroundColor: theme.bg,
65           overflow: 'hidden',
66           boxShadow: '0 0 40px rgba(0, 0, 0, 0.18)',
67         }}
68       >
69         {children}
70       </View>
71     </View>
72   );
73 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:53

บรรทัด ความหมาย

- 53–54** WebFrame: บนคอม แสดงแอปกว้างเท่ามือถือคือไว้ตรงกลางจอ
- 56** ถ้าไม่ใช่เว็บ (Platform.OS !== 'web') ไม่ต้องทำอะไร คั่นของข้างในตรง ๆ (<>...</> คือกล่องที่มองไม่เห็น)
- 58** พื้นหลังเต็มจอ จัดกลาง สีเข้มขึ้นเล็กน้อยในโหมดมืด
- 59–68** กล่องแอป กว้างสูงสุด 440 จุด มีเงารอบ ๆ ให้ดูเหมือนมือถือวางบนโต๊ะ

src/app/_layout.tsx

บรรทัด 75–126

```

75 function Navigator({ ready }: { ready: boolean }) {
76   const theme = useTheme();
77   const { status, repo, profile } = useApp();
78   const loading = !ready || status === 'loading';
79   const [introDone, setIntroDone] = useState(false);
80   const finishIntro = useCallback(() => setIntroDone(true), []);
81
82   const signedIn = status === 'ready' && !!repo;
83   const needsOnboarding = signedIn && !profile && !profile.onboarded;
84   const inApp = signedIn && !needsOnboarding;
85
86   return (
87     <
88       <StatusBar style={!introDone || theme.dark ? 'light' : 'dark'} />
89       {loading ? null : <Screens signedIn={signedIn} needsOnboarding={needsOnboarding} inApp={inApp} />}
90       {loading || !introDone ? null : <AuthNoticeHost />}
91       {loading || !introDone ? null : <UpdateBanner />}
92       <LaunchIntro ready={!loading} fontsReady={ready} onDone={finishIntro} />
93     </>
94   );
95 }
96
97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding: boolean; inApp:
boolean }) {
98   const theme = useTheme();
99   return (
100     <
101       /* Screens slide in from the side; scanning and adding a transaction rise from the bottom. */
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg }, animation:
'slide_from_right' }}>
103       <Stack.Protected guard={!signedIn}>

```



```

104     <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105   </Stack.Protected>
106   <Stack.Protected guard={needsOnboarding}>
107     <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108   </Stack.Protected>
109   <Stack.Protected guard={inApp}>
110     <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />
111     <Stack.Screen name="transaction" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
112   />
113   <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation: 'slide_from_bottom' }} />
114   <Stack.Screen name="voice" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
115   <Stack.Screen name="achievements" />
116   <Stack.Screen name="skins" />
117   <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation: 'fade' }} />
118   <Stack.Screen name="goals" />
119   <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
120   <Stack.Screen name="goal" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
121   <Stack.Screen name="drafts" />
122   <Stack.Screen name="settings" />
123 </Stack.Protected>
124 </Stack>
125 );
126 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/_layout.tsx:75`

บรรทัด	ความหมาย
75–78	Navigator ดูสถานะจากข้อมูลกลาง: เข้าสู่ระบบหรือยัง มี Repo โหม โปรไฟล์ตั้งค่าเสร็จไหม และยังโหลดอยู่ไหม
79–80	จำว่าแอนิเมชันเปิดแอปจบหรือยัง
82–84	คำนวณ 3 สถานะ: signedIn (เข้าสู่ระบบแล้ว), needsOnboarding (เข้าแล้วแต่ยังไม่ตั้งค่าเงิน), inApp (เข้าแล้วและตั้งค่าแล้ว ใช้แอปได้เต็มที่)
88	สไลด์คอนแทกสถานะด้านบน (สว่างบนพื้นเข้ม)
89	ถ้ายังโหลดอยู่ ยังไม่แสดงหน้าจอ
90–91	หลังแอนิเมชันจบ แสดงหน้าต่างแจ้งเรื่องบัญชี และแถบ "มีอัปเดตใหม่"
92	แอนิเมชันเปิดแอป (น้องกล้าโผล่) ช้อนอยู่บนสุดจนจบ
97–102	Stack = หน้าจอซ้อนกันเป็นกอง หน้าใหม่ทับหน้าเก่า กดกลับก็หิบบนหน้าออก ตั้งค่าทุกหน้า: ไม่มีหัวหน้าสำเร็จรูป, ติดตามริม, เลื่อนเข้ามาจากขวา
103–105	Stack.Protected = ประตูป้าย: หน้า welcome เข้าได้เฉพาะตอน ยังไม่ เข้าสู่ระบบ
106–108	หน้า onboarding เข้าได้เฉพาะตอนเข้าแล้วแต่ยังไม่ตั้งค่าเงิน
109–122	หน้าในแอปทั้งหมดเข้าได้เฉพาะตอนพร้อมใช้งาน บางหน้าแสดงแบบ modal (เลื่อนขึ้นจากล่าง) เช่น จดรายการ พูดจต, บางหน้าเต็มจอ เช่น สแกนสลิป สรุปเดือน

★ ผลของ Stack.Protected

ผู้ใช้ไม่ต้องทำอะไร พอสมัครเสร็จ สถานะเปลี่ยนเป็น needsOnboarding ยามจะปิดประตู welcome และเปิดประตู onboarding แอปจึงพาไปหน้าตั้งค่าเงินเอง พอตั้งค่าเสร็จก็ไปหน้าหลักเอง และกดย้อนกลับก็ไม่เจอหน้าสมัครอีก (ตรงกับ acceptance criteria ในเอกสาร Traceability)

แถบเมนูล่าง: (tabs)/_layout.tsx

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด 23–28: ตารางของ 4 แท็บ แต่ละแท็บมีชื่อไทย ไอคอนตอนไม่ได้เลือก และไอคอนตอนเลือก (แบบทึบ) ชื่อช่อง (index, transactions, runway, coach) ตรงกับชื่อไฟล์ในโฟลเดอร์ (tabs)

src/app/(tabs)/_layout.tsx

บรรทัด 134–186

```
134   {routes.slice(0, half).map((r, i) => tab(r, i))}
135   <View style={{ width: 76, alignItems: 'center' }}>
136     <View style={{ width: 60, height: 60, marginTop: -26, alignItems: 'center', justifyContent: 'center'
137   }}>
138       <PulseRing size={60} active times={auto.state.phase === 'scanning' ? 'always' : 3} color=
139   {theme.accent} />
140       <Animated.View style={{ transform: [{ scale: press.scale }] }}>
141         <Pressable
142           accessibilityRole="button"
143           accessibilityLabel="สแกนสลิป"
144           onPressIn={press.onPressIn}
145           onPressOut={press.onPressOut}
146           onPress={() => {
147             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium).catch(() => {});
148             router.push('/scan');
149           }}
150           style={{
151             width: 60,
152             height: 60,
153             borderRadius: 30,
154             backgroundColor: theme.hero,
155             borderWidth: 3,
156             borderColor: theme.accent,
157             alignItems: 'center',
158             justifyContent: 'center',
159             shadowColor: theme.accent,
160             shadowOpacity: 0.35,
161             shadowRadius: 10,
162             shadowOffset: { width: 0, height: 4 },
163             elevation: 8,
164           }}
165         >
166           <Ionicons name="scan" size={26} color={theme.heroAccent} />
167         </Pressable>
168       </Animated.View>
169     </View>
170     <Text style={{ type: micro, { fontFamily: fonts.sansSemi, color: theme.ink, marginTop: 4 }}}>สแกน
171     สลิป</Text>
172   </View>
173   {routes.slice(half).map((r, i) => tab(r, i + half))}
174 </View>
175 );
176 }
177
178 export default function TabsLayout() {
179   // Records today's visit and hands out บัตรค่า's skins for finished missions and events.
180   useKlaSync();
181   return (
182     <Tabs tabBar={({props}) => <TabBar {...props} />} screenOptions={{ headerShown: false }}>
```

```

180     <Tabs.Screen name="index" />
181     <Tabs.Screen name="transactions" />
182     <Tabs.Screen name="runway" />
183     <Tabs.Screen name="coach" />
184   </Tabs>
185 );
186 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:134

บรรทัด ความหมาย

134 วาดแท็บครั้งแรก (หน้าหลัก, รายการ) ทางซ้าย

135–169 ปุ่มทองตรงกลาง: วงกลม 60 จุด ลอยขึ้นมา 26 จุด (marginTop: -26) มีวงแหวนกระพือม (PulseRing) 3 รอบตอนเปิดแอป และกระพือมตลอดระหว่างสแกนอัตราโน้ต

144–147 ตะแล้มมือถือสั่นแรงปานกลาง และเปิดหน้าสแกน

170 วาดแท็บครั้งหลัง (เงินพอถึง, โค้ช) ทางขวา

175–186 component หลัก: ใช้แถบเมนูที่ทีมออกแบบเอง (tabBar=) แถบของสำเร็จรูป และประกาศ 4 แท็บ useKlaSync() บันทึกว่าวันนี้เปิดแอป (ใช้นับภารกิจ "เปิดแอป 30 วันติด")

โค้ดที่ต่างกันระหว่างมือถือกับเว็บ

บางอย่างทำได้ต่างกัน เช่น มือถือเก็บข้อมูลด้วย SQLite แต่เว็บใช้ localStorage ของเบราว์เซอร์ ทีมเราแก้ด้วย **ไฟล์คู่**: ตั้งชื่อ storage.ts สำหรับมือถือ และ storage.web.ts สำหรับเว็บ ตอน build ระบบ (Metro bundler) เลือกไฟล์ให้เองตามแพลตฟอร์ม ที่เหลือของแอป import แค่ './storage'

src/data/storage.ts

บรรทัด 1–12

```

1 /**
2  * Small key-value store used for app settings
3  * and demo data.
4  * Phone: SQLite-backed store (survives app
5  * restarts).
6  * Web: see storage.web.ts (browser
7  * localStorage).
8  */
9 import KV from 'expo-sqlite/kv-store';
10
11 export const Storage = {
12   getItem: (key: string) => KV.getItem(key),
13   setItem: (key: string, value: string) =>
14     KV.setItem(key, value),
15   removeItem: (key: string) =>
16     KV.removeItem(key),
17 };

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/data/storage.ts:1

src/data/storage.web.ts

บรรทัด 1–24

```

1 /** Web version of storage.ts: the browser's
2  * localStorage, wrapped so it never throws. */
3 export const Storage = {
4   async getItem(key: string): Promise<string |
5     null> {
6     try {
7       return window.localStorage.getItem(key);
8     } catch {
9       return null;
10     },
11   async setItem(key: string, value: string):
12     Promise<void> {
13     try {
14       window.localStorage.setItem(key, value);
15     } catch {
16       // Private mode or storage full: keep
17       // working in memory.
18     }
19   },
20   async removeItem(key: string): Promise<void> {
21     try {
22       window.localStorage.removeItem(key);
23     } catch {
24       // ignore
25     }
26   },
27 };

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/data/storage.web.ts:1

ทั้งสองไฟล์มี getItem, setItem, removeItem หน้าตาเดียวกัน ฉบับเว็บห่อด้วย try/catch เพราะเบราว์เซอร์ใหม่ดส่วนตัวอาจห้ามเก็บข้อมูล คู่ไฟล์แบบนี้มีทั้งหมด 5 คู่:

ไฟล์	มือถือ	เว็บ
data/storage	SQLite (expo-sqlite)	localStorage
data/sessionStorageSetup	ที่เก็บการเข้าสู่ระบบของ Supabase บนมือถือ	ของเบราว์เซอร์
services/gallery	อ่านแกลเลอรีทั้งหมด (expo-media-library)	ให้ผู้ใช้เลือกรูปเอง (เบราว์เซอร์ไม่อนุญาตให้เว็บเปิดแกลเลอรี)
services/speech	ตัวฟังเสียงของมือถือ (Google/Apple)	Web Speech API ของเบราว์เซอร์
services/tts	เสียงอ่านของมือถือ (expo-speech)	speechSynthesis ของเบราว์เซอร์

ส่วนที่ต่างเล็กน้อยใช้ Platform.OS ในไฟล์เดียวกัน เช่น หน้าจอดรายการบนเว็บให้พิมพ์วันที่ แต่บน Android เปิดปฏิทินของเครื่อง ([src/app/transaction.tsx:288](#))

Native module แบบไม่บังคับ

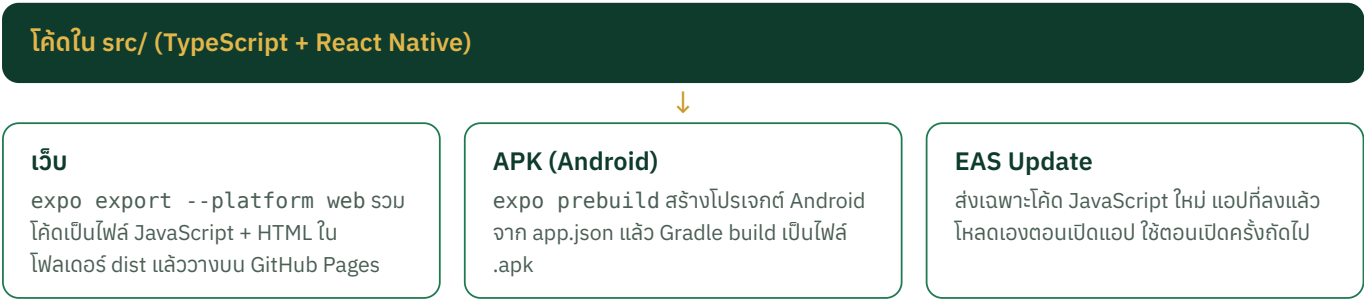
src/services/speech.tsบรรทัด 1-24

```
1 /**
2  * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech
3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15 interface NativeSpeech {
16   start(options: Record<string, unknown>): void;
17   stop(): void;
18   abort(): void;
19   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
20   isRecognitionAvailable(): boolean;
21   addListener(event: string, listener: (e: never) => void): Subscription;
22 }
23
24 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด 24 ใช้ requireOptionalNativeModule = "ขอโมดูลฟังเสียง ถ้าไม่มีก็ได้ null ไม่ต้อง crash" เพราะแอปที่ลงไว้ก่อนเพิ่มฟีเจอร์พูดจดไม่มีโมดูลนี้ แต่ยังได้รับโค้ดใหม่ผ่าน EAS Update (หัวข้อถัดไป) ถ้าเรียกแบบบังคับ แอปเก่าจะค้างปิด แบบนี้แอปเก่าแค่แสดงช่องพิมพ์แทนไม้

จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง



ทั้ง 3 ทางทำอัตโนมัติทุกครั้งที push โค้ดขึ้น branch main บน GitHub (บน A13) ไฟล์ตั้งค่าของแอปคือ app.json ส่วนสำคัญ:

ค่าใน app.json	ความหมาย
"name": "MindPay", "slug": "mindpay"	ชื่อแอปที่เห็นใต้ไอคอน
"version": "1.0.0" + "runtimeVersion": { "policy": "appVersion" }	EAS Update ส่งโค้ดใหม่ให้เฉพาะแอปที่เวอร์ชันตรงกัน ถ้าเพิ่ม native module ใหม่ ต้องเพิ่มเวอร์ชัน แล้วลง APK ใหม่
"android.package"	รหัสแอป com.socratesandskeletons.mindpay
"blockedPermissions"	บล็อกสิทธิ์กล้อง วิทยุโอ เสียงในเครื่อง ที่ไม่ได้ใช้ (ความเป็นส่วนตัว)
"plugins"	ตั้งค่า native ของโมดูล เช่น ข้อความขอสิทธิ์แคลเลอริ์/ไมโครโฟนเป็นภาษาไทย
"scheme": "mindpay"	ลิงก์ที่ขึ้นต้นด้วย mindpay:// เปิดแอปได้ (ใช้กับลิงก์ยืนยันอีเมล)
"updates"	ที่อยู่ของเซิร์ฟเวอร์ EAS Update และให้เช็คอัปเดตทุกครั้งทีเปิดแอป
"web": { "output": "single" }	เว็บเป็นหน้าเดียว (single page app) ให้ JavaScript เปลี่ยนหน้าเอง

ไฟล์ app.config.js เพิ่มค่าเฉพาะตอน build เว็บ: เว็บอยู่ที่ /Socrates-and-Skeletons- / บน GitHub Pages จึงต้องตั้ง baseUrl แต่ห้ามใส่ค่านี้นในแอปมือถือ (แอปจะหาฟอนต์และรูปผิดที่)

วิธีอ่านโค้ด MindPay ด้วยตัวเอง

โค้ดทั้งแอปมีเกือบ 19,000 บรรทัด ไม่มีใครจำได้หมด แต่ทุกไฟล์ใช้รูปแบบซ้ำ ๆ กัน รู้รูปแบบแล้วจะอ่านไฟล์ไหนก็ออก

ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น

- อ่านคอมเมนต์บนสุดของไฟล์ก่อน** ทุกไฟล์ในโปรเจกต์เริ่มด้วยคอมเมนต์ `/** ... */` บอกว่าไฟล์นี้ทำอะไร เป็นของ FR ไหน และมีกฎอะไรสำคัญ (ภาษาอังกฤษง่าย ๆ ใช้ Google Translate ช่วยได้)
- ดูบรรทัด `import`** บอกว่าไฟล์นี้พึ่งใครบ้าง ถ้า `import` จาก `../domain` แปลว่ามีการคำนวณ ถ้าจาก `../data/AppProvider` แปลว่าใช้ข้อมูลกลาง
- กด `Ctrl+Shift+O`** ดูรายชื่อฟังก์ชันทั้งหมดในไฟล์ เหมือนดูสารบัญ
- หาฟังก์ชันที่มี `export`** นั่นคือ "ประตูหน้าบ้าน" ที่ไฟล์อื่นเรียก ฟังก์ชันที่ไม่มี `export` คือผู้ช่วยภายใน
- อ่านชนิดข้อมูลก่อนอ่านตัวโค้ด** ดูว่ารับอะไรเข้า คืนอะไรออก ก็เข้าใจไปครึ่งหนึ่งแล้ว
- หาเทสต์ของมัน** กด `Shift+F12` ที่ชื่อฟังก์ชัน ถ้ามีเทสต์ใน `__tests__` จะเห็นตัวอย่างการใช้งานจริงพร้อมคำตอบที่ถูกต้อง อ่านง่ายกว่าตัวโค้ดเสียอีก

รูปแบบที่เจอซ้ำทั้งแอป (ตารางโยง)

เห็นอะไร	แปลว่าอะไร
ชื่อลงท้าย <code>Satang</code>	จำนวนเงินเป็นสตางค์ (จำนวนเต็ม) หาร 100 ได้บาท
ชื่อขึ้นต้น <code>bkk</code> เช่น <code>bkkDayKey</code> , <code>bkkToIso</code>	ฟังก์ชันจัดการวันเวลาแบบเวลาไทย (กรุงเทพ UTC+7) อยู่ใน <code>src/domain/dates.ts</code>
ข้อความวันที่ <code>'2026-10-09'</code>	"day key" วันที่ไทยรูปแบบ ปี-เดือน-วัน (ค.ศ.) เอามาเทียบมากกว่า/น้อยกว่ากันได้ตรง ๆ เพราะเรียงตามตัวอักษรก็ตรงกับเรียงตามเวลา
<code>useApp()</code>	หยิบข้อมูลกลาง (โปรไฟล์ รายการ คำสั่ง <code>addTx</code> ฯลฯ)
<code>useMoney()</code>	หยิบตัวเลขที่คำนวณแล้ว: ยอดคงเหลือ ค่าเฉลี่ย <code>Runway</code> แบบร่าง
<code>useTheme()</code> แล้วตามด้วย <code>theme.ink</code> , <code>theme.primary</code>	สีจากธีมที่ผู้ใช้เลือก (ห้ามใส่รหัสสีเองในหน้าจอ)
<code><T v="h3"></code> , <code><Money satang={...}></code> , <code><Card></code> , <code><Button></code>	ชิ้นส่วนหน้าตาจาก <code>src/ui/components.tsx</code>
<code>space.lg</code> , <code>radius.md</code>	ระยะห่างและมุมโค้งมาตรฐาน (16 จุด, 14 จุด) จาก <code>src/ui/theme.ts</code>
<code>toast({ message })</code>	ข้อความแจ้งเตือน ๆ ด้านล่างจอ
<code>router.push('/xxx')</code>	เปิดหน้า <code>src/app/xxx.tsx</code>
<code>accessibilityLabel="..."</code>	ข้อความที่โปรแกรมอ่านหน้าจออ่านให้คนตาบอดฟัง และเทสต์ E2E ใช้หาปุ่ม
<code>status === 'confirmed'</code> หรือ <code>confirmedOnly(txs)</code>	นับเฉพาะรายการที่ยืนยันแล้ว
<code>Reveal</code> , <code>usePressSpring</code> , <code>useReduceMotion</code>	แอนิเมชัน (ค่อย ๆ โผล่, ปุ่มยุบตอนกด, เคารพ "ลดการเคลื่อนไหว")
คอมเมนต์มีรหัส <code>TC-12</code>	รหัสเทสต์ ตรงกับในเอกสาร <code>docs/TRACEABILITY.md</code>
ไฟล์ชื่อ <code>xxx.web.ts</code>	ฉบับเว็บของไฟล์ <code>xxx.ts</code>

ชื่อแบบ snake_case เช่น amount_satang

ชื่อคอสมันในฐานข้อมูล (ฝั่งแอปใช้ camelCase amountSatang) แปลงกันใน

[src/data/repo.ts](#)

ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) แต่ผู้ใช้ชาวไทย (เร็วกว่า 7 ชั่วโมง) ถ้าไม่แปลง รายการตอน 06:00 น. ของวันที่ 8 จะถูกนับเป็นวันที่ 7 (เพราะ UTC ยังเป็น 23:00 ของวันที่ 7) ทุก FR จึงใช้ฟังก์ชันจาก [src/domain/dates.ts](#)

[src/domain/dates.ts](#)

บรรทัด 1-51

```

1 /**
2  * Date helpers fixed to Thailand time (Asia/Bangkok, UTC+7, no daylight saving).
3  *
4  * Everything the user sees ("today", "this week", the date printed on a slip)
5  * is a Bangkok calendar day, so all grouping and range logic goes through
6  * `bkkDayKey`. Because Thailand has no DST, a fixed +7h offset is exact.
7  */
8 import type { RangeKey } from './types';
9
10 export const BKK_OFFSET_MS = 7 * 60 * 60 * 1000;
11 export const DAY_MS = 24 * 60 * 60 * 1000;
12
13 /** Instant -> "YYYY-MM-DD" in Bangkok. */
14 export function bkkDayKey(at: Date | string | number): string {
15   const ms = new Date(at).getTime() + BKK_OFFSET_MS;
16   return new Date(ms).toISOString().slice(0, 10);
17 }
18
19 /** "YYYY-MM-DD" (Bangkok day) -> the UTC instant of 00:00 that day in Bangkok. */
20 /** Milliseconds until the next 00:00 in Bangkok (when "วันนี้" starts over). */
21 export function msUntilNextBkkMidnight(nowMs: number = Date.now()): number {
22   const next = bkkDayStart(addDays(bkkDayKey(nowMs), 1)).getTime();
23   return Math.max(0, next - nowMs);
24 }
25
26 export function bkkDayStart(dayKey: string): Date {
27   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) - BKK_OFFSET_MS);
28 }
29
30 /** Add whole days to a "YYYY-MM-DD" key. */
31 export function addDays(dayKey: string, days: number): string {
32   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) + days * DAY_MS)
33     .toISOString()
34     .slice(0, 10);
35 }
36
37 /** Whole days from a to b (both "YYYY-MM-DD"). */
38 export function daysBetween(a: string, b: string): number {
39   return Math.round((Date.parse(`${b}T00:00:00Z`) - Date.parse(`${a}T00:00:00Z`)) / DAY_MS);
40 }
41
42 /** Combine a Bangkok day and "HH:MM" into an ISO instant. */
43 export function bkkToIso(dayKey: string, time: string | null = '12:00'): string {
44   const t = time && /^d{1,2}:d{2}$/.test(time) ? time.padStart(5, '0') : '12:00';
45   return new Date(Date.parse(`${dayKey}T${t}:00.000Z`) - BKK_OFFSET_MS).toISOString();
46 }
47
48 /** "HH:MM" of an instant in Bangkok. */
49 export function bkkTime(at: Date | string): string {
50   return new Date(new Date(at).getTime() + BKK_OFFSET_MS).toISOString().slice(11, 16);
51 }

```

▶ เปิดใน VS Code (เร็ว) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → [src/domain/dates.ts:1](#)

บรรทัด	ความหมาย
1-7	คอมเมนต์: ทุกอย่างที่ผู้ใช้เห็น (วันนี้, สัปดาห์นี้, วันที่บนสลิป) เป็นวันตามปฏิทินกรุงเทพ และเพราะไทยไม่มีการปรับเวลาฤดูร้อน (daylight saving) การบวก 7 ชั่วโมงตายตัวจึงถูกต้องเสมอ
10	7 ชั่วโมงในหน่วยมิลลิวินาที ($7 \times 60 \times 60 \times 1000 = 25,200,000$)
11	1 วันในหน่วยมิลลิวินาที (86,400,000)
13-17	bkkDayKey : รับเวลาแบบไหนก็ได้ (Date, ข้อความ, ตัวเลข) บวก 7 ชั่วโมง แล้วตัดเอา 10 ตัวอักษรแรกของรูปแบบสากล ได้วันที่ไทย เช่น <code>bkkDayKey('2026-10-07T23:00:00Z')</code> → <code>'2026-10-08'</code> (23:00 UTC = 06:00 น. ของวันที่ 8 เวลาไทย)
20-24	msUntilNextBkkMidnight : เหลืออีกกี่มิลลิวินาทีถึงเที่ยงคืนไทยครั้งหน้า (เอาวันพรุ่งนี้ 00:00 ลบเวลาตอนนี้) ใช้ตั้งนาฬิกาใน AppProvider (un A6)
26-28	bkkDayStart : วันที่ไทย → เวลาสากลของ 00:00 น. วันนั้น (ลบ 7 ชั่วโมงกลับ)
30-35	addDays : บวกหรือลบจำนวนวันจากวันที่ เช่น <code>addDays('2026-10-08', 20)</code> → <code>'2026-10-28'</code> ข้ามเดือนข้ามปีให้เอง
37-40	daysBetween : นับจำนวนวันจาก a ถึง b เช่น <code>daysBetween('2026-10-08', '2026-10-31')</code> → 23
42-46	bkkToIso : เอาวันที่ไทย + เวลา "HH:MM" มารวมเป็นเวลาสากลสำหรับเก็บในฐานข้อมูล ถ้าเวลาผิดรูปแบบใช้ 12:00 แทน เช่น <code>bkkToIso('2026-10-08', '06:00')</code> → <code>'2026-10-07T23:00:00.000Z'</code>
48-51	bkkTime : เวลาสากล → "HH:MM" ตามเวลาไทย ใช้แสดงเวลาของรายการ

แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน

อยากรู้	เปิดไฟล์
หน้าจอนี้หน้าตาแบบนี้เพราะอะไร	<code>src/app/ชื่อหน้า.tsx</code> (ตารางใน un A7)
ตัวเลขนี้คิดยังไง	<code>src/domain/</code> : <code>src/domain/summary.ts</code> (ยอด), <code>src/domain/runway.ts</code> (Runway), <code>src/domain/insights.ts</code> (การ์ดโค้ช)
ข้อมูลโหลด/บันทึกยังไง	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
ตารางในฐานข้อมูลมีคอลัมน์อะไร	<code>supabase/migrations/20260927000000_init.sql</code>
AI ได้รับคำสั่งอะไร	<code>supabase/functions/coach/index.ts</code> , <code>supabase/functions/parse-slip/index.ts</code>
สี ขนาด ฟอนต์	<code>src/ui/theme.ts</code> , <code>docs/DESIGN_SYSTEM.md</code>
ฟีเจอร์นี้ตอน requirement ข้อไหน มีทดสอบอะไร	<code>docs/TRACEABILITY.md</code>
ทีมใช้ AI ช่วยเขียนตรงไหน ตรวจอะไรไปแล้ว	<code>docs/AI_USAGE_LOG.md</code>
build / deploy อัตโนมัติ	<code>.github/workflows/*.yaml</code>

★ **ถ้าอ่านแล้วงง**
ใส่ `console.log('ค่า', ตัวแปร)` ไว้ในโค้ดแล้วเปิดแอปด้วย `npm run web` กด F12 ในเบราว์เซอร์ แتب Console จะเห็นค่าที่พิมพ์ออกมา (อย่าลืมลบก่อน commit) หรือเขียนเทสต์เล็ก ๆ ลองส่งค่าเข้าฟังก์ชันใน `src/domain` แล้วดูผล

เซิร์ฟเวอร์และฐานข้อมูล: Supabase

ข้อมูลของผู้ใช้เก็บที่ไหน เก็บยังไง แอปต่อเซิร์ฟเวอร์ยังไง และทำไมคนอื่นแอบดูข้อมูลเราไม่ได้ ทั้งที่กุญแจอยู่ในแอป

ฐานข้อมูลคืออะไร

ฐานข้อมูล (database) คือที่เก็บข้อมูลแบบมีระเบียบ คิดว่าเป็น **สมุด Excel บนคลาวด์** ที่มีหลายชิต แต่ละชิตเรียกว่า **ตาราง (table)** แต่ละแถวคือข้อมูล 1 ชัน (row) แต่ละคอลัมน์คือช่องข้อมูล (column) เช่น ตาราง transactions มี 1 แถวต่อรายการเงิน 1 รายการ

id	user_id	kind	amount_satang	title	status	occurred_at
a1b2...	มีนัท	expense	8500	ข้าวมันไก่	confirmed	2026-10-08T05:10Z
c3d4...	มีนัท	income	1000000	ค่าขนมจากที่บ้าน	confirmed	2026-10-01T02:00Z
e5f6...	ต้น	expense	125000	โอนให้ นาย ส***	draft	2026-10-07T11:42Z

ตัวอย่างสมมติ คอลัมน์ user_id จริงเป็นรหัสยาว (UUID) ไม่ใช่ชื่อ

ภาษาที่ใช้สั่งฐานข้อมูลคือ **SQL** คำสั่งพื้นฐานมี 4 คำ

SQL	ความหมาย	ในแอปเขียนว่า (supabase-js)
SELECT * FROM transactions	อ่านข้อมูล	db.from('transactions').select('*')
INSERT INTO transactions ...	เพิ่มแถว	db.from('transactions').insert(row)
UPDATE transactions SET ... WHERE id = ...	แก้แถว	db.from('transactions').update(row).eq('id', id)
DELETE FROM transactions WHERE id = ...	ลบแถว	db.from('transactions').delete().eq('id', id)

แอปไม่ได้เขียน SQL ตรง ๆ แต่ใช้ไลบรารี supabase-js ซึ่งแปลงคำสั่งเป็นคำขอผ่านอินเทอร์เน็ต แล้วเซิร์ฟเวอร์แปลงเป็น SQL ให้อีกที

Supabase ให้อะไรที่มเราบ้าง

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
PostgreSQL	ฐานข้อมูลแบบตาราง ที่นิยมที่สุดตัวหนึ่งของโลก	เก็บ 4 ตาราง: profiles, transactions, savings_goals, ai_usage
Auth	ระบบบัญชีผู้ใช้สำเร็จรูป	สมัคร เข้าสู่ระบบ ส่งรหัส 6 หลักทางอีเมล สัมรหัสผ่าน
REST API อัตโนมัติ	สร้าง "ประตู" ให้เข้าถึงตารางผ่านอินเทอร์เน็ตให้เอง	แอปอ่าน/เขียนตารางได้โดยไม่ต้องเขียนเซิร์ฟเวอร์เอง
Row Level Security	กฎความปลอดภัยรายแถว	แต่ละคนเห็นเฉพาะแถวของตัวเอง
Edge Functions	ที่รันโค้ด TypeScript ฝั่งเซิร์ฟเวอร์ (Deno)	parse-slip และ coach ซึ่งถือกุญแจ AI (un A10)
Dashboard	หน้าเว็บจัดการทั้งหมด	ดูตาราง รัน SQL ตั้ง Secrets ดู log

Migration: ไฟล์สร้างฐานข้อมูล

แทนที่จะคลิกสร้างตารางในหน้าเว็บ ทิมเราเขียนคำสั่ง SQL ไว้ในไฟล์ เรียกว่า **migration** เรียงตามวันที่ในชื่อไฟล์ ใครจะสร้างฐานข้อมูลใหม่ก็แค่รันไฟล์ตามลำดับ ได้ฐานข้อมูลเหมือนกันทุกตัว และเก็บประวัติใน Git ได้

ไฟล์	ทำอะไร
<code>supabase/migrations/20260927000000_init.sql</code>	สร้างตารางหลัก 3 ตาราง กฎ RLS และ trigger (27 ก.ย. 2569)
<code>supabase/migrations/20260927000100_harden.sql</code>	แก้ตามคำแนะนำของตัวตรวจความปลอดภัย Supabase Advisor
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	เพิ่มตารางกระปุกออม (FR-6)
<code>supabase/migrations/20261001000000_atomic_ai_quota.sql</code>	ฟังก์ชันนับโควตา AI แบบกันแย่งกัน และเพดานจำนวนเงิน

ตาราง profiles: ผู้ใช้ 1 คน = 1 แถว

supabase/migrations/20260927000000_init.sqlบรรทัด 1-22

```
1 -- MindPay database schema (Supabase / PostgreSQL)
2 -- Run once: Supabase Dashboard -> SQL Editor -> paste -> Run
3 -- or with the CLI: supabase db push
4 --
5 -- Security model: every row belongs to one user and Row Level Security (RLS)
6 -- makes sure a user can only ever read or change their own rows, even though
7 -- the app talks to the database directly with the public (publishable) key.
8 --
9 -- -----
10 -- profiles: one row per user (created automatically at sign-up)
11 -- -----
12 create table if not exists public.profiles (
13   id uuid primary key references auth.users (id) on delete cascade,
14   display_name text not null default '',
15   opening_balance_satang bigint not null default 0,
16   runway_floor_satang bigint not null default 50000 check (runway_floor_satang >= 0), -- FR-6 low line,
   default $500
17   monthly_budget_satang bigint check (monthly_budget_satang is null or monthly_budget_satang > 0),
18   coach_tone text not null default 'friend' check (coach_tone in ('friend', 'coach', 'senior')), -- FR-5
19   onboarded boolean not null default false,
20   created_at timestamptz not null default now(),
21   updated_at timestamptz not null default now()
22 );
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:1

บรรทัด	ความหมาย
1-7	คอมเมนต์ SQL (ขึ้นต้นด้วย --) อธิบายวิธีรัน และหลักความปลอดภัย: ทุกแถวเป็นของผู้ใช้คนเดียว RLS ทำให้อ่านแก้ไขได้เฉพาะของตัวเอง แม้แอปจะคุยกับฐานข้อมูลตรง ๆ ด้วยกุญแจสาธารณะ
12	สร้างตาราง profiles ในส่วน public (ถ้ายังไม่มี)
13	id ชนิด uuid (รหัสยาวไม่ซ้ำ) เป็น primary key (ตัวระบุแถว) และ อ้างอิง ผู้ใช้ในระบบ Auth ถ้าลบผู้ใช้ โปรไฟล์ถูกลบตาม (on delete cascade)
14	ชื่อเล่น ห้ามว่าง (not null) ค่าเริ่มต้นเป็นข้อความว่าง
15	เงินตอนเริ่มใช้แอป เป็นสตางค์ ชนิด bigint (จำนวนเต็มขนาดใหญ่)
16	เส้นเงินสำรองของ FR-6 ค่าเริ่มต้น 50,000 สตางค์ (\$500) และมี check บังคับว่าต้อง ≥ 0 ถ้าแอปส่งค่าติดลบมา ฐานข้อมูลปฏิเสธเอง

บรรทัด	ความหมาย
17	จบเดือน ไม่ตั้งก็ได้ (null) ถ้าตั้งต้องมากกว่า 0
18	โทนโคช (ของเดิม FR-5) ต้องเป็น 1 ใน 3 ค่าเท่านั้น
19	ตั้งค่าเงินครั้งแรกเสร็จหรือยัง (ใช้กับ Stack.Protected ใน un A7)
20-21	เวลาสร้างและแก้ไขล่าสุด ใส่เวลาปัจจุบันให้เอง (now())

ตาราง transactions: รายการเงิน (FR-1)

supabase/migrations/20260927000000_init.sql
บรรทัด 24-53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and ocr_confidence <= 1)),
41   review_flags text[] not null default '{}',
42   created_at timestamptz not null default now(),
43   updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id, occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_idx
51   on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_idx
53   on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

▶ เปิดใน VS Code (เว็บ)
GitHub
VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
28	id สร้างให้อัตโนมัติด้วยฟังก์ชันสุ่ม (gen_random_uuid())
29	user_id เจ้าของรายการ ค่าเริ่มต้นคือ auth.uid() = id ของคนที่ส่งคำขอมา แอปจึงไม่ต้องส่ง user_id เอง (และปลอมเป็นคนอื่นไม่ได้)
30	ประเภทต้องเป็น income หรือ expense เท่านั้น (กฎเดียวกับ TxKind ในแอป)
31	จำนวนเงินเป็นสตางค์ ต้องมากกว่า 0
32	รหัสหมวด
33-34	ชื่อยาวไม่เกิน 120 ตัวอักษร บันทึกไม่เกิน 500 (ตรงกับ maxLength ในหน้าจอรายการ)
35	เวลาที่เกิดรายการ ชนิด timestamptz (เวลาพร้อมเขตเวลา)

บรรทัด	ความหมาย
36–37	ที่มา (manual/slip) และสถานะ (draft/confirmed) ค่าเริ่มต้นคือยืนยันแล้ว
38–39	เลขอ้างอิงสลิป และ hash ของรูป สำหรับกันซ้ำ (FR-4)
40	ความมั่นใจของ AI ตัวเลข 0.00 ถึง 1.00
41	ช่องที่ต้องตรวจ เป็น array ของข้อความ (text[])
46	index = สารบัญของตาราง ช่วยให้ค้น "รายการของผู้ใช้คนนี้ เรียงตามเวลาล่าสุด" ได้เร็ว ไม่ต้องไล่ทุกแถว
48–53	unique index = ห้ามซ้ำ: ผู้ใช้คนเดียวกันจะมีเลขอ้างอิงสลิปซ้ำกันไม่ได้ และ hash รูปซ้ำไม่ได้ ถ้ามีข้อ 2 เครื่องสแกนสลิปใบเดียวกัน พร้อมกัน ฐานข้อมูลปฏิเสธใบที่ 2 เอง (ตอบรหัส 23505 ที่ repo.ts แปลงเป็น DuplicateSlipError)

ตาราง ai_usage: นับการใช้ AI

supabase/migrations/20260927000000_init.sql	บรรทัด 55–65
<pre> 55 -- ----- 56 -- ai_usage: counts calls to the AI so one account cannot run up the bill. 57 -- Written only by the Edge Functions (service role), readable by the owner. 58 -- ----- 59 create table if not exists public.ai_usage (60 id bigserial primary key, 61 user_id uuid not null references auth.users (id) on delete cascade, 62 kind text not null check (kind in ('slip', 'coach')), 63 created_at timestamptz not null default now() 64); 65 create index if not exists ai_usage_user_day_idx on public.ai_usage (user_id, kind, created_at desc); </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:55

ทุกครั้งที่ AI ถูกเรียก (อ่านสลิป 1 ใบ หรือถามโค้ช 1 ครั้ง) Edge Function จะเพิ่ม 1 แถว เพื่อบันทึกว่าวันนี้ใช้ไปกี่ครั้งแล้ว กันบัญชีเดียวใช้ AI จนค่าใช้จ่ายบานปลาย (ผู้ใช้อ่านของตัวเองได้ แต่เขียนได้เฉพาะเซิร์ฟเวอร์)

Row Level Security (RLS): หัวใจความปลอดภัย

≈ เปรียบเทียบ: ตู้ออกเกอร์ในห้องสมุด

กุญแจสาธารณะ (publishable key) ในแอปเหมือน "บัตรเข้าห้องสมุด" ใครมีก็เดินเข้าห้องได้ แต่ละคนมีล็อกเกอร์ของตัวเอง RLS คือ "ยามประจำล็อกเกอร์" ที่ดูบัตรประชาชน (การเข้าสู่ระบบ) ทุกครั้ง และเปิดให้เฉพาะล็อกเกอร์ที่มีชื่อเรา แม้แขกเกอร์จะขโมยบัตรเข้าห้องสมุด (กุญแจในแอป) ไปได้ ก็เปิดล็อกเกอร์คนอื่นไม่ได้

supabase/migrations/20260927000000_init.sql	บรรทัด 67–87
<pre> 67 -- ----- 68 -- Row Level Security 69 -- ----- 70 alter table public.profiles enable row level security; 71 alter table public.transactions enable row level security; 72 alter table public.ai_usage enable row level security; 73 74 drop policy if exists "own profile read" on public.profiles; 75 create policy "own profile read" on public.profiles 76 for select using (auth.uid() = id); 77 drop policy if exists "own profile update" on public.profiles; 78 create policy "own profile update" on public.profiles 79 for update using (auth.uid() = id) with check (auth.uid() = id); 80 81 drop policy if exists "own transactions" on public.transactions; </pre>	

```

82 create policy "own transactions" on public.transactions
83   for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
84
85 drop policy if exists "own ai usage read" on public.ai_usage;
86 create policy "own ai usage read" on public.ai_usage
87   for select using (auth.uid() = user_id);

```

▶ เปิดใน VS Code (เร็ว) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:67

บรรทัด ความหมาย

- | | |
|-------|---|
| 70-72 | เปิด RLS ทั้ง 3 ตาราง พอเปิดแล้ว ค่าเริ่มต้นคือ ห้ามทุกอย่าง ต้องมี policy อนุญาตทีละเรื่อง |
| 74-76 | policy "own profile read": อ่านโปรไฟล์ได้ เฉพาะแถวที่ id ตรงกับคนที่เข้าสู่ระบบ (auth.uid() = id) drop policy if exists ด้านบนคือลบของเก่าก่อน ทำให้รับไฟล์เข้าได้ |
| 77-79 | แก้ไขโปรไฟล์เฉพาะของตัวเอง using = ตรวจสอบแถวเดิม, with check = ตรวจสอบแถวหลังแก้ไข (กันแก้ไข id ให้เป็นของคนอื่น) ไม่มี policy insert/delete เพราะโปรไฟล์สร้างโดย trigger เท่านั้น |
| 81-83 | รายการเงิน: ทำได้ทุกอย่าง (for all = อ่าน เพิ่ม แก้ ลบ) เฉพาะแถวที่ user_id เป็นของตัวเอง |
| 85-87 | ai_usage อ่านได้อย่างเดียวและเฉพาะของตัวเอง |

ทีมทดสอบเรื่องนี้ด้วยแผน MT-10 "บัญชี A ไม่เห็นรายการของบัญชี B" และรันตัวตรวจ Security Advisor ของ Supabase แล้วแจ้งเตือนไม่เหลือคำเตือนเรื่องตาราง ไฟล์ harden.sql เปลี่ยน auth.uid() เป็น (select auth.uid()) เพื่อให้ฐานข้อมูลคำนวณครั้งเดียวต่อคำสั่ง แทนที่จะคำนวณทุกแถว (เร็วขึ้นเมื่อมีรายการเยอะ)

supabase/migrations/20260927000100_harden.sql

บรรทัด 1-25

```

1 -- Hardening after running the Supabase security/performance advisors on the first migration.
2
3 -- 1. Fixed search_path on trigger functions (advisor 0011).
4 create or replace function public.touch_updated_at() returns trigger
5 language plpgsql set search_path = '' as $$
6 begin
7   new.updated_at := now();
8   return new;
9 end $$;
10
11 -- 2. The sign-up trigger must not be callable through the REST API (advisors 0028/0029).
12 revoke execute on function public.handle_new_user() from public, anon, authenticated;
13
14 -- 3. Evaluate auth.uid() once per query instead of once per row (advisor 0003).
15 drop policy if exists "own profile read" on public.profiles;
16 create policy "own profile read" on public.profiles
17   for select using ((select auth.uid()) = id);
18
19 drop policy if exists "own profile update" on public.profiles;
20 create policy "own profile update" on public.profiles
21   for update using ((select auth.uid()) = id) with check ((select auth.uid()) = id);
22
23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25   for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เร็ว) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:1

Trigger: งานที่ฐานข้อมูลทำเองอัตโนมัติ

supabase/migrations/20260927000000_init.sql

บรรทัด 89-118

```

89 -- -----
90 -- Triggers
91 -- -----

```

```
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95     new.updated_at := now();
96     return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101     for each row execute function public.touch_updated_at();
102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104     for each row execute function public.touch_updated_at();
105
106 -- Create the profile row when a user signs up.
107 create or replace function public.handle_new_user() returns trigger
108 language plpgsql security definer set search_path = public as $$
109 begin
110     insert into public.profiles (id, display_name)
111     values (new.id, coalesce(new.raw_user_meta_data ->> 'display_name', ''))
112     on conflict (id) do nothing;
113     return new;
114 end $$;
115
116 drop trigger if exists on_auth_user_created on auth.users;
117 create trigger on_auth_user_created after insert on auth.users
118     for each row execute function public.handle_new_user();
```

▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:89

บรรทัด	ความหมาย
92-97	ฟังก์ชัน touch_updated_at (เขียนด้วยภาษา plpgsql ของ PostgreSQL) ตั้งเวลาแก้ไขล่าสุดเป็นตอนนี้
99-104	trigger = "ทุกครั้งก่อนแก้ไขแถวในตาราง profiles หรือ transactions ให้เรียกฟังก์ชันนั้น" updated_at จึงถูกต้องเสมอ ไม่ต้องพึ่งแอป
107-114	ฟังก์ชัน handle_new_user: สร้างแถวโปรไฟล์ใหม่ โดยเอาชื่อเล่นที่ผู้ใช้กรอกตอนสมัคร (เก็บใน raw_user_meta_data) ถ้าไม่มีใช้ข้อความว่าง และถ้ามีแถวนี้อยู่แล้วไม่ต้องทำ (on conflict do nothing) security definer = รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน (เพราะผู้ใช้ไม่มีสิทธิ์ insert โปรไฟล์เอง)
116-118	trigger บนตารางผู้ใช้ของระบบ Auth: หลังจากมีคนสมัครใหม่ ให้สร้างโปรไฟล์ทันที แอปจึงไม่ต้องสร้างเอง

กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม

	Publishable key (anon)	Service role key (secret)
อยู่ที่ไหน	ในแอป (.env, eas.json) เผยแพร่ได้	บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น (Supabase ใสให้เอง)
ทำอะไรได้	ทุกคำขอต้องผ่าน RLS เห็นเฉพาะข้อมูลของคน que เข้าสู่ระบบ	ข้าม RLS ได้ทั้งหมด อ่านเขียนทุกแถว
ใช้ทำอะไร	แอปอ่าน/เขียนรายการของผู้ใช้	ตรวจตัวตนผู้ใช้ และนับโควตา AI ใน Edge Function
ถ้ารั่ว	ไม่เป็นไร (ออกแบบมาให้เปิดเผยได้)	อันตรายมาก ต้องเปลี่ยนกฎแฉทันที

กฎแฉ AI (Claude หรือ Gemini) ก็เป็นความลับระดับเดียวกับ service role เก็บใน Supabase → Edge Functions → Secrets เท่านั้น

แอปต่อ Supabase ยังไง: src/data/supabase.ts

```
src/data/supabase.ts                                     บรรทัด 1-38
1 /**
2  * Supabase client. Keys come from environment variables (see .env.example).
3  * The publishable key is safe to ship in the app: Row Level Security in the
```

```

4  * database decides what each signed-in user can read and write.
5  * AI keys are NOT here; they live only in the Edge Functions.
6  *
7  * detectSessionInUrl is off: email links are handled by AppProvider (via
8  * authLinks.ts) the same way on the web and in the phone app.
9  */
10 import './sessionStorageSetup';
11 import './authLinks'; // records the address the web page was opened with
12 import { createClient, type SupabaseClient } from '@supabase/supabase-js';
13 import { AppState } from 'react-native';
14
15 const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
16 const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
17
18 /** null when the app was built without Supabase settings: only demo mode is available. */
19 export const supabase: SupabaseClient | null =
20   url && key
21     ? createClient(url, key, {
22       auth: {
23         storage: localStorage,
24         autoRefreshToken: true,
25         persistSession: true,
26         detectSessionInUrl: false,
27       },
28     })
29     : null;
30
31 if (supabase) {
32   AppState.addEventListener('change', (state) => {
33     if (state === 'active') supabase.auth.startAutoRefresh();
34     else supabase.auth.stopAutoRefresh();
35   });
36 }
37
38 export const cloudAvailable = supabase !== null;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/supabase.ts:1

บรรทัด	ความหมาย
1-9	คอมเมนต์: หมายเหตุจากตัวแปรสภาพแวดล้อม (.env) หมายเหตุ publishable ปลอดภัยเพราะ RLS และหมายเหตุ AI ไม่อยู่ที่นี่
10	สิ่งที่เกิดการเข้าสู่ระบบ (บนมือถือใช้ SQLite เก็บไว้ ปิดแอปแล้วยังเข้าสู่ระบบอยู่)
11	จุดที่อยู่ของหน้าเว็บตอนเปิด (ไว้จัดการลิงก์ยืนยันอีเมล)
15-16	อ่านที่อยู่เซิร์ฟเวอร์และหมายเหตุจาก .env (process.env.EXPO_PUBLIC_...)
18-29	ถ้ามีทั้งสองค่า สร้าง "client" สำหรับคุยกับ Supabase ถ้าไม่มี (build แบบไม่มีการตั้งค่า) ได้ null แล้วแอปใช้ได้เฉพาะโหมดทดลอง
23-26	ตั้งค่าการเข้าสู่ระบบ: เก็บ session ไว้ (persistSession), ต่ออายุ token อัตโนมัติ (token ของ Supabase มีอายุสั้น ค่าปกติคือ 1 ชั่วโมง), และไม่ให้ไลบรารีอ่านลิงก์ในแถบตัวเอง (AppProvider จัดการเองเพื่อให้เว็บกับแอปทำงานเหมือนกัน)
31-36	ต่ออายุ token เฉพาะตอนแอปเปิดอยู่บนหน้าจอ พอแอปถูกพับไว้หยุดต่ออายุ (ประหยัดแบตเตอรี่และเน็ต)
38	บอกส่วนอื่นของแอปว่าใช้บัญชีจริงได้ไหม

การสมัครและเข้าสู่ระบบ

1. กรอกชื่อเล่น อีเมล รหัสผ่าน ≥ 8 ตัว `src/app/welcome.tsx`

↓ `supabase.auth.signUp`

2. Supabase สร้างผู้ใช้

trigger `handle_new_user` สร้างแอดโปรไฟล์ให้กับที แล้วส่งอีเมลที่มีรหัส 6 หลักและลิงก์ยืนยัน

↓ `supabase.auth.verifyOtp` (กรอกรหัส) หรือแตะลิงก์ในอีเมล

3. ได้ session

แอปเก็บ token ไว้ในเครื่อง AppProvider เห็นเหตุการณ์ SIGNED_IN แล้วสร้าง CloudRepo โหลดข้อมูล

↓

4. โปรไฟล์ยังไม่ onboarded

Stack.Protected พาไปหน้าตั้งค่าเงิน กรอกรายละเอียดตอนนี้และเงินสำรอง แล้วเข้าหน้าหลัก

`src/app/welcome.tsx`

บรรทัด 148–172

```
148     const displayName = name.trim();
149     const { data, error } = await supabase!.auth.signUp({
150       email: address,
151       password,
152       options: { data: { display_name: displayName }, emailRedirectTo: authRedirectUrl() },
153     });
154     if (error) return setNotice({ text: authErrorMessage(error), tone: 'error' });
155     if (isExistingAccountSignUp(data.user)) {
156       setMode('signin');
157       setPassword('');
158       setNotice({
159         text: 'อีเมลนี้มีบัญชีอยู่แล้ว เข้าสู่ระบบด้วยรหัสผ่านเดิม หรือกด "ลืมรหัสผ่าน" เพื่อตั้งใหม่',
160         tone: 'info',
161         action: 'forgot',
162       });
163       return;
164     }
165     if (data.session) {
166       showAuthNotice({ kind: 'signed_up', name: displayName });
167       resetForm();
168       return;
169     }
170     setSent({ kind: 'confirm', email: address, name: displayName });
171     setResendIn(RESEND_WAIT_S);
172   });
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → `src/app/welcome.tsx:148`

บรรทัด	ความหมาย
148	ตัดช่องว่างหัวท้ายของชื่อเล่น
149–153	ส่งสมัคร: ส่งอีเมล รหัสผ่าน ชื่อเล่น (เก็บใน metadata ของผู้ใช้ ซึ่ง trigger จะคัดลอกไปใส่โปรไฟล์) และที่อยู่ลิงก์ในอีเมลจะพากลับมา
154	ถ้า error (เช่น รหัสผ่านสั้นเกิน) แสดงข้อความภาษาไทยที่บอกวิธีแก้
155–164	กรณีพิเศษ: อีเมลนี้เคยสมัครแล้ว Supabase จะตอบเหมือนสำเร็จ (เพื่อไม่ให้คนนอกรู้ว่าอีเมลไหนมีบัญชี) แอปตรวจจบได้แล้วบอกให้ไปเข้าสู่ระบบหรือกดลืมรหัสผ่าน แทนที่จะขึ้น "สมัครสำเร็จ" หลอก ๆ
165–169	ถ้าโปรเจกต์ปิดการยืนยันอีเมลไว้ จะได้ session ทันที ขึ้นหน้าต่าง "สมัครบัญชีสำเร็จ"
170–171	ไม่ขึ้นไปหน้ากรอกรหัส 6 หลัก และเริ่มนับถอยหลังก่อนให้กดส่งรหัสใหม่ได้

Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน

หน้าจอไม่เคยคุยกับ Supabase หรือที่เก็บในเครื่องโดยตรง แต่คุยผ่าน **Repo** (ย่อจาก repository แปลว่าคลัง) ซึ่งมี 2 แบบที่หน้าตาเหมือนกันทุกประการ

src/data/repo.tsบรรทัด 1-38

```
1 /**
2  * Data access. The screens never talk to Supabase or storage directly; they use
3  * a Repo. Two implementations:
4  *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5  *   - DemoRepo: on-device only, starts with sample data (for trying the app
6  *                 without an account and as a fallback during a presentation)
7  */
8 import { Storage } from './storage';
9 import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from './supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;
31   update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32   remove(id: string): Promise<void>;
33   confirmMany(ids: string[]): Promise<void>;
34   listGoals(): Promise<SavingsGoal[]>;
35   insertGoal(input: GoalInput): Promise<SavingsGoal>;
36   updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37   removeGoal(id: string): Promise<void>;
38 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์: หน้าจอใช้ Repo เสมอ มี 2 ตัว คือ CloudRepo (Supabase, บัญชีจริง) และ DemoRepo (เก็บในเครื่อง มีข้อมูลตัวอย่าง ใช้ลองแอปหรือสำรองตอนพรีเซนต์)
14-19	error เฉพาะสำหรับสลิปซ้ำ (อธิบายในบท A5)
21	interface Repo = สัญญาว่า Repo ทุกตัวต้องทำสิ่งเหล่านี้ได้
22	บอกว่าเป็นแบบไหน
23-24	อ่านและแก้ไขโปรไฟล์
25-29	โหลดรายการประมาณ 13 เดือนล่าสุด และ "ยอดยกมา" ของรายการที่เก่ากว่านั้น (ยังต้องนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30-33	เพิ่ม แก้ ลบ รายการ และยืนยันหลายรายการพร้อมกัน (สลิปยืนยัน)

≈ ทำไมต้องมี 2 แบบ

เหมือนปลั๊กไฟ: หน้าจอคือเครื่องใช้ไฟฟ้า Repo คือเต้าเสียบ จะต่อกับไฟบ้าน (Supabase) หรือแบตเตอรี่สำรอง (เครื่อง) หน้าจอไม่ต้องรู้ เสียบแล้วใช้ได้เหมือนกัน ทีมเราจึงพรีเซนต์ได้แม้อินเทอร์เน็ตในห้องเรียนล่ม และ E2E ทดสอบได้โดยไม่แตะบัญชีจริง

แปลงชื่อระหว่างแอปกับฐานข้อมูล

src/data/repo.ts

บรรทัด 44–93

```

44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;
58   created_at: string;
59 };
60
61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;

```

บรรทัด ความหมาย

- 44–59** TxRow = หน้าตาของแถวในฐานข้อมูล ใช้ชื่อแบบ snake_case (ค้นด้วยขีดล่าง) ตามธรรมเนียมของ SQL
- 61–76** fromRow: แกวฐานข้อมูล → Transaction ของแอป ใช้ Number (. . .) ครอบตัวเลข เพราะ bigint บางครั้งส่งมาเป็นข้อความ และ review_flags ที่เป็น null ให้เป็น array ว้าง
- 78–93** toRow: กลับทาง ใส่เฉพาะช่องที่มีค่า (ไม่ใช่ undefined) เพราะตอนแก้รายการ เราส่งแค่บางช่อง ถ้าใส่ทุกช่อง ช่องที่ไม่ได้ส่งจะถูกเขียนทับเป็นคำว่าว่าง

โหลดรายการจากคลาวด์

src/data/repo.ts

บรรทัด 166–216

```

166 export function createCloudRepo(userId: string): Repo {
167   const db = supabase!;
168   return {
169     mode: 'cloud',
170     async getProfile() {
171       const { data, error } = await db.from('profiles').select('*').eq('id', userId).single();
172       if (error) throw error;
173       return profileFromRow(data as ProfileRow);
174     },
175     async updateProfile(patch) {
176       const { data, error } = await db.from('profiles').update(profileToRow(patch)).eq('id',
userId).select().single();
177       if (error) throw error;
178       return profileFromRow(data as ProfileRow);
179     },
180     async listTransactions() {
181       // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182       const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183       const all: Transaction[] = [];
184       for (let from = 0; ; from += 1000) {
185         const { data, error } = await db
186           .from('transactions')
187           .select('*')
188           .gte('occurred_at', since)
189           .order('occurred_at', { ascending: false })
190           // A unique tiebreaker, so rows with the same time are never skipped or read twice across pages.
191           .order('id', { ascending: true })
192           .range(from, from + 999);
193         if (error) throw error;
194         all.push(...(data as TxRow[]).map(fromRow));
195         if (!data || data.length < 1000) break;
196       }
197       // Older confirmed money still counts in the balance: add it up (two columns only).
198       let carrySatang = 0;
199       for (let from = 0; ; from += 1000) {
200         const { data, error } = await db
201           .from('transactions')
202           .select('kind,amount_satang')
203           .eq('status', 'confirmed')
204           .lt('occurred_at', since)
205           .order('occurred_at', { ascending: false })
206           // A unique tiebreaker, so rows with the same time are never skipped or read twice across pages.
207           .order('id', { ascending: true })
208           .range(from, from + 999);
209         if (error) throw error;
210         for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
211           carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -Number(r.amount_satang);
212         }

```

```

213     if (!data || data.length < 1000) break;
214   }
215   return { txs: all, carrySatang };
216 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:166

บรรทัด	ความหมาย
166–169	สร้าง CloudRepo ของผู้ใช้คนนี้ supabase! (เครื่องหมายตกใจ) บอก TypeScript ว่า "มั่นใจว่าไม่ใช่ null"
170–174	อ่านโปรไฟล์: จากตาราง profiles เลือกทุกคอลัมน์ ที่ id เท่ากับผู้ใช้ เอาแถวเดียว (RLS ก็กันไว้อีกชั้นอยู่แล้ว)
175–179	แก้โปรไฟล์แล้วขอแถวใหม่กลับมา
182	คำนวณวันที่ย้อนหลัง 400 วัน (ประมาณ 13 เดือน พอสำหรับช่วง "1 ปี" ในหน้าหลัก)
184–196	โหลดทีละหน้า หน้าละ 1,000 แถว (API จำกัดจำนวนแถวต่อครั้ง) เรียงจากใหม่ไปเก่า ถ้าได้น้อยกว่า 1,000 แปลว่าหมดแล้ว หยุด
190–191	เรียงด้วย id เป็นตัวตัดสินรอง ถ้ามีหลายรายการเวลาเดียวกัน จะไม่หลุดหรือซ้ำข้ามหน้า (บ๊ิกที่ทีมเจอและแก้ตอนตรวจทั้ง repo)
197–214	ขอยอดยกมา: รายการที่ ขึ้นขั้นแล้ว และเก่ากว่า 400 วัน โหลดมาแค่ 2 คอลัมน์ (ประเภทและจำนวน) แล้วรวมเป็นตัวเลขเดียว รายรับบวก รายจ่ายลบ ยอดคงเหลือจึงยังถูกต้องแม้ผู้ใช้แอปมาหลายปี

โมดูลทดลอง: DemoRepo

src/data/repo.ts

บรรทัด 264–305

```

264 // v2: rebalanced sample data (v1 could show a negative balance). Old v1 data is simply ignored.
265 const DEMO_KEY = 'mindpay.demo.v2';
266
267 interface DemoState {
268   profile: Profile;
269   txs: Transaction[];
270   goals?: SavingsGoal[];
271 }
272
273 const demoProfile: Profile = {
274   id: 'demo',
275   displayName: 'ผู้ทดลองใช้',
276   openingBalanceSatang: DEMO_OPENING_BALANCE_SATANG,
277   runwayFloorSatang: 50_000,
278   monthlyBudgetSatang: 1_000_000,
279   coachTone: 'friend',
280   onboarded: true,
281 };
282
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
284
285 export async function resetDemoData() {
286   await Storage.removeItem(DEMO_KEY);
287 }
288
289 export function createDemoRepo(): Repo {
290   let cache: DemoState | null = null;
291
292   async function load(): Promise<DemoState> {
293     if (cache) return cache;
294     try {
295       const raw = await Storage.getItem(DEMO_KEY);
296       if (raw) cache = JSON.parse(raw) as DemoState;
297     } catch {
298       cache = null;
299     }
300     if (!cache) {
301       cache = { profile: demoProfile, txs: buildSampleTransactions() };

```

```

302     await save();
303   }
304   return cache;
305 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:264

บรรทัด 265: ข้อมูลโหมดทดลองเก็บในเครื่องด้วยชื่อ mindpay.demo.v2 (v2 เพราะข้อมูลชุดแรกเคยทำให้ยอดติดลบ จึงเปลี่ยนชุดใหม่)
 บรรทัด 273–281 คือโปรไฟล์ของผู้ทดลองใช้: เงินตั้งต้น ฿3,000 เส้นสำรอง ฿500 จบ ฿10,000 บรรทัด 292–305: ครั้งแรกที่เปิดยัง
 ไม่มีข้อมูล จะสร้างรายการตัวอย่าง (buildSampleTransactions ใน [src/domain/sample.ts](#)) ประมาณ 75 วันย้อนหลังจากวันนี้
 แล้วเก็บไว้ ข้อมูลตัวอย่างจึงสดใหม่เสมอไม่ว่าเปิดวันไหน

เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด

src/data/AppProvider.tsx

บรรทัด 271–318

```

271 // Decide the starting mode once, then follow Supabase auth changes.
272 useEffect(() => {
273   let cancelled = false;
274   (async () => {
275     // An email link that opened the app wins over the saved mode.
276     if (await handleAuthLink(await openingLink())) return;
277     const mode = await Storage.getItem(MODE_KEY).catch(() => null);
278     if (mode === 'demo') {
279       if (!cancelled) await activateRepo(createDemoRepo(), 'demo');
280       return;
281     }
282     if (!supabase) {
283       if (!cancelled) setStatus('signedOut');
284       return;
285     }
286     const { data } = await supabase.auth.getSession();
287     if (cancelled) return;
288     const id = data.session?.user.id;
289     if (id) {
290       if (activeUserRef.current !== id) await activateRepo(createCloudRepo(id), id);
291     } else if (!activeUserRef.current) setStatus('signedOut');
292   })();
293
294   const sub = supabase?.auth.onAuthStateChange((event, session) => {
295     // Supabase warns against calling other supabase methods inside this callback
296     // (it can deadlock the auth lock), so the actual work is deferred.
297     setTimeout(() => {
298       // A password-reset code signs in with the PASSWORD_RECOVERY event instead of SIGNED_IN.
299       // Opened offline with an expired token, the session only comes back later as TOKEN_REFRESHED:
300       // that signs in too, but only while nothing is open (never over the demo or another account).
301       const signedIn =
302         event === 'SIGNED_IN' || event === 'PASSWORD_RECOVERY' || (event === 'TOKEN_REFRESHED' &&
!activeUserRef.current);
303       if (signedIn && session && activeUserRef.current !== session.user.id) {
304         Storage.setItem(MODE_KEY, 'cloud').catch(() => {});
305         activateRepo(createCloudRepo(session.user.id), session.user.id);
306       }
307       if (event === 'SIGNED_OUT' && repoRef.current?.mode === 'cloud') clear();
308     }, 0);
309   });
310   const stopLinks = onIncomingLink((url) => {
311     handleAuthLink(url);
312   });
313   return () => {
314     cancelled = true;
315     sub?.data.subscription.unsubscribe();
316     stopLinks();
317   };

```

```
318 }, [activateRepo, clear, handleAuthLink]);
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:271

บรรทัด	ความหมาย
272	ทำครั้งเดียวตอนแอปเริ่ม
275–276	ถ้าแอปถูกเปิดจากลิงก์ในอีเมล (ยืนยันสมัคร/ตั้งรหัสใหม่) จัดการลิงก์ก่อน ถ้าเข้าสู่ระบบสำเร็จก็จบ
277–281	อ่านโหมดที่จำไว้ในเครื่อง ถ้าเป็นโหมดทดลอง ใช้ DemoRepo
282–285	build ที่ไม่มี Supabase: ไปหน้าต้อนรับ
286–291	ถามว่ามี session ค้างอยู่ไหม (เคยเข้าสู่ระบบไว้) ถ้ามี ใช้ CloudRepo ของคนนั้น ไม่มีก็ไปหน้าต้อนรับ
294–309	ฟังเหตุการณ์ของระบบ Auth: เข้าสู่ระบบ (SIGNED_IN), กู้รหัสผ่าน (PASSWORD_RECOVERY) หรือ token กลับมาหลังออฟไลน์ → เปิดบัญชีนั้น · ออกจากระบบ (SIGNED_OUT) → ล้างข้อมูล ทั้งหมดห่อด้วย setTimeout(..., 0) เพราะ Supabase เตือนว่าเรียกคำสั่งอื่นในนี้ตรง ๆ อาจค้าง
310–312	ฟังลิงก์ที่เข้ามาระหว่างแอปเปิดอยู่
313–317	เก็บกวาดเมื่อปิด: เลิกฟังทุกอย่าง

src/data/AppProvider.tsx

บรรทัด 127–167

```
127 const loadAll = useCallback(async (r: Repo) => {
128   const gen = ++loadGen.current;
129   localChanges.current = new Map();
130   setLoadError(null);
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(), r.listGoals().catch(() =>
134   [] as SavingsGoal[])]);
135     if (gen !== loadGen.current || repoRef.current !== r) return;
136     // A slip saved by the automatic scan during the load is not lost when the list arrives.
137     const changes = localChanges.current;
138     const fresh = list.txs.filter((t) => !changes.has(t.id));
139     const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
140     setProfile(p);
141     setTxs([...kept, ...fresh]);
142     setCarry(list.carrySatang);
143     setGoals(g);
144     setStatus('ready');
145   } catch (e) {
146     if (gen !== loadGen.current || repoRef.current !== r) return;
147     setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
148     setStatus('ready');
149   }
150 }, []);
151
152 const activateRepo = useCallback(
153   async (r: Repo, id: string) => {
154     if (activeUserRef.current !== id) {
155       // Another account (or leaving the demo): never show or use the previous one's data meanwhile.
156       setProfile(null);
157       setTxs([]);
158       setCarry(0);
159       setGoals([]);
160     }
161     activeUserRef.current = id;
162     repoRef.current = r;
163     setRepo(r);
164     setUserId(id);
165     await loadAll(r);
166   },
```

```
166     [loadAll],  
167   );
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด ความหมาย

127–128	loadAll: โหลดข้อมูลทั้งหมด แต่ละครั้งได้เลขรอบใหม่ (gen)
129–130	เริ่มจัดการเปลี่ยนแปลงในเครื่องใหม่ และล้าง errorเก่า
133	โหลด 3 อย่างพร้อมกัน (บท A5)
134	ถ้ามีการโหลดรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทั้งผลรอบนี้ (กันข้อมูลบัญชีเก่าโผล่ในบัญชีใหม่)
135–140	ถ้าระหว่างโหลด สแกนสลิปอัตโนมัติบันทึกรายการใหม่ไป เอารายการนั้นใส่กลับไว้ด้านหน้า ไม่ให้หายเพราะผลโหลดมากับ
139–143	ใส่ข้อมูลลง state ทั้งหมด แล้วบอกว่าพร้อม (หน้าจอลoadใหม่)
144–147	ถ้าโหลดพลาด เก็บข้อความ error ให้หน้าหลักแสดงปุ่ม "ลองอีกครั้ง"
151–167	activateRepo: เปลี่ยนไปใช้ Repo ของบัญชีใหม่ ถ้าเป็นคนละบัญชีกับเดิม ล้างข้อมูลเก่าออกก่อนทันที แล้วจำ Repo และโหลดข้อมูล

★ สรุปบท A9

ฐานข้อมูลมี 4 ตาราง ทุกตารางเปิด RLS ให้แต่ละคนเห็นเฉพาะของตัวเอง เงินเก็บเป็นสตางค์และมี check กันค่าผิด สลิปซ้ำๆถูกกันด้วย unique index ระดับฐานข้อมูล แอปคุยกับฐานข้อมูลผ่าน Repo ซึ่งมีแบบคลาวด์และแบบในเครื่อง และ AppProvider เป็นคนเลือกว่าจะใช้แบบไหน

การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไง

MindPay ใช้ AI 2 งาน คืออ่านสลิป (FR-4) และโค้ชน้องกล้า (FR-5) บทนี้อธิบายเส้นทางตั้งแต่แอปถามจนได้คำตอบ ทีละบรรทัด

AI ในแอปคืออะไร

AI ที่ใช้คือ โมเดลภาษาขนาดใหญ่ (Large Language Model, LLM) เช่น Claude ของบริษัท Anthropic และ Gemini ของ Google เราส่ง "ข้อความ" (และรูป) ไปทางอินเทอร์เน็ต มันตอบกลับเป็นข้อความ บริษัทเหล่านี้คิดเงินตามจำนวนที่ใช้ จึงต้องมี **API key** (กุญแจ) ไว้ระบุว่าเป็นคนจ่าย

คำ	ความหมาย
API	"ประตู" ให้โปรแกรมคุยกับโปรแกรม ส่งคำขอเป็น JSON ทาง HTTPS แล้วได้คำตอบเป็น JSON
API key	รหัสลับยาว ๆ ที่แนบไปกับคำขอ ใครได้กุญแจไป ก็ใช้ AI โดยให้เราจ่ายเงินได้
System prompt	"คำสั่งประจำตัว" ที่บอก AI ว่าต้องเป็นใคร ตอบยังไง ห้ามอะไร (ผู้ใช้ไม่เห็น)
Token	หน่วยนับข้อความของ AI (ประมาณคำหรือพยางค์) ใช้คิดราคาและจำกัดความยาวคำตอบ
JSON Schema	แบบฟอร์มที่บังคับให้ AI ตอบเป็นโครงสร้างตายตัว เช่น ต้องมีช่อง amount, dateIso
JSON	รูปแบบข้อมูลที่เขียนเหมือน object ของ JavaScript เช่น { "text": "สวัสดี" }

ทำไมแอปไม่เรียก AI เอง

! ห้ามใส่กุญแจ AI ไว้ในแอปเด็ดขาด

ไฟล์ APK หรือเว็บที่ผู้ใช้โหลดไป ใครก็แกะดูข้างในได้ ถ้ากุญแจ AI อยู่ในแอป คนแกะจะเอาไปใช้ฟรีโดยไม่ให้ทีมเราจ่าย (หรือทำให้กุญแจโดนแบน) ทีมเราจึงเก็บกุญแจไว้บนเซิร์ฟเวอร์ใน **Edge Functions** เท่านั้น แอปเรียก Edge Function แล้ว Edge Function ไปเรียก AI แทน

การมีเซิร์ฟเวอร์คั่นกลางให้ประโยชน์อีกหลายอย่าง

- 1. **ตรวจตัวตน:** เฉพาะผู้ใช้ที่เข้าสู่ระบบแล้วเท่านั้นที่ใช้ AI ได้
- 2. **จำกัดโควตา:** อ่านสลิปได้ 300 ใบ/วัน/คน ถามโค้ชได้ 60 ครั้ง/วัน/คน กันค่าใช้จ่ายบานปลาย
- 3. **เปลี่ยน AI ได้โดยไม่ต้องอัปเดตแอป:** สลับ Claude กับ Gemini แค่เปลี่ยน Secret บนเซิร์ฟเวอร์
- 4. **ทำความเข้าใจคำตอบ:** ตรวจสอบและแก้คำตอบของ AI ก่อนส่งถึงแอป (เช่น ปี พ.ศ. ที่ AI สืมแปลง)
- 5. **ความเป็นส่วนตัว:** ควบคุมได้ว่าส่งอะไรให้ AI บ้าง

เส้นทางของคำถาม 1 ขอลงมือกล้า

1. แอป `src/app/(tabs)/coach.tsx`

ผู้ใช้พิมพ์ "ชื่อรองเก่า 1,290 ได้ไหม" แอปคำนวณ "ข้อเท็จจริง" เอง (ยอดเงิน ค่าเฉลี่ย Runway ก่อนและหลังซื้อ) ด้วยโค้ดใน `src/domain`

↓ `askCoach()` ใน `src/services/coach.ts`

2. `supabase.functions.invoke('coach')`

ส่ง `{context, question}` พร้อม token การเข้าสู่ระบบของผู้ใช้ ทาง HTTPS



3. Edge Function `coach` `supabase/functions/coach/index.ts`

ตรวจสอบ token → ตรวจสอบขนาดคำขอ → ตรวจสอบโควตา → สร้างข้อความให้ AI (system prompt + DATA + QUESTION)

↓ `กุญแจลับ (Secret)`

4. Claude หรือ Gemini

เขียนคำตอบภาษาไทย ไม่เกิน 90 คำ ใช้เฉพาะตัวเลขที่ได้รับ

↑ คำตอบย้อนกลับทางเดิม

5. แอปแสดงและอ่านออกเสียง

น้องกล้ายับปากพร้อมขับไต่เตล (FR-5)

ฝั่งแอป: `src/services/coach.ts`

`src/services/coach.ts`

บรรทัด 1-25

```
1 /** FR-5: ask น้องกล้า, the AI coach (Edge Function "coach"). */
2 import { FunctionsHttpError } from '@supabase/supabase-js';
3 import { supabase } from '../data/supabase';
4
5 export class CoachError extends Error {}
6
7 export async function askCoach(params: { context: unknown; question: string }): Promise<string> {
8   if (!supabase) throw new CoachError('น้องกล้าตอบคำถามได้เมื่อเข้าสู่ระบบด้วยบัญชีจริง ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้กล้าเล่าให้ฟังได้');
9   const { data, error } = await supabase.functions.invoke('coach', { body: params });
10   if (error) {
11     if (error instanceof FunctionsHttpError) {
12       const body = await (error.context as Response).json().catch(() => null);
13       if (body?.error?.code === 'quota') throw new CoachError('วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่นะ');
14       if (body?.error?.code === 'not_configured') {
15         throw new CoachError('โ้ช AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านล่างบนได้เลย');
16       }
17       if (body?.error?.code === 'busy') throw new CoachError('ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่นะ');
18       if (body?.error?.code === 'unauthorized') throw new CoachError('เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง');
19     }
20     throw new CoachError('ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้งนะ');
21   }
22   const text = (data as { text?: string }).text;
23   if (!text) throw new CoachError('น้องกล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ');
24   return text;
25 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: `Ctrl+P` → `src/services/coach.ts:1`

บรรทัด ความหมาย

2 นำชนิด error ของ Supabase ที่บอกว่า "เซิร์ฟเวอร์ตอบกลับมั่วว่าผิดพลาด" มาใช้

บรรทัด	ความหมาย
5	สร้าง error ชนิดของโค้ชเอง ข้อความของมันเป็นสิ่งที่จะแสดงให้ผู้ใช้งานเห็น
7	ฟังก์ชัน askCoach รับข้อมูลสรุป (context) และคำถาม คืนข้อความคำตอบ
8	ไม่มี Supabase (เช่น build ที่ไม่ได้ตั้งค่า) → บอกผู้ใช้งานว่าต้องใช้บัญชีจริง
9	เรียก Edge Function ชื่อ coach ส่ง context และ question ไป ไสวาร์แบบ token การเข้าสู่ระบบให้เอง
10-21	ถ้าผิดพลาด อ่านรหัส error ที่เซิร์ฟเวอร์ส่งมา แล้วแปลงเป็นข้อความภาษาไทยที่เป็นมิตร: ครบโควตา (quota), ยังไม่ใส่กุญแจ AI (not_configured), AI ไม่ว่าง (busy), หมดอายุการเข้าสู่ระบบ (unauthorized) อย่างอื่นบอกให้ลองใหม่
22-24	ดึงข้อความคำตอบ ถ้าไม่มีก็แจ้ง ถ้ามีส่งกลับไปให้หน้าโค้ช

ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่บรรทัด

Edge Function เขียนด้วย TypeScript เหมือนแอป แต่รันบนเซิร์ฟเวอร์ของ Supabase ด้วยโปรแกรมชื่อ **Deno** (คล้าย Node.js) ฟังก์ชันหนึ่งตัวคือไฟล์เดอร์หนึ่งใน supabase/functions/ มีไฟล์ index.ts และไฟล์เดอร์ _shared เก็บโค้ดที่ใช้ร่วมกัน

supabase/functions/coach/index.ts บรรทัด 1-26

```

1 // FR-5: the AI coach is น้องกล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError, requireUser,
  takeQuota } from '../_shared/common.ts';
6 import { plainText } from '../_shared/helpers.ts';
7
8 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
9 /** A question and a summary of totals: far below this. */
10 const MAX_BODY_BYTES = 64_000;
11
12 const SYSTEM = `You are น้องกล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for students and
  first-jobbers.
13 You are a small golden money-tree sapling and the user's personal money coach.
14
15 Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
16 - Speak natural, everyday Thai. Call yourself "กล้า". Do not use ครับ orค่ะ; soft endings such as "นะ" are fine.
17 - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short spoken sentences
  separated by spaces,
18   no emoji, no bullet points, no markdown, no lists, no headings.
19
20 Rules:
21 - Reply in Thai, at most 90 words.
22 - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the data cannot
  answer, say what is missing.
23 - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the decision to
  the user.
24 - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
25 - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the runway before and
  after.
26 - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:1

บรรทัด	ความหมาย
1-4	คอมเมนต์: โค้ชคือน้องกล้า อธิบายตัวเลขที่ยืนยันแล้ว คำตอบจะถูกอ่านออกเสียงจึงเขียนแบบภาษาพูด แอปส่งเฉพาะยอดรวมตามหมวด (ไม่มีรูปสลิป ชื่อคน หรือเลขบัญชี)
5-6	นำตัวช่วยจาก _shared มาใช้ (นามสกุล .ts ต้องเขียนเต็มใน Deno)

8 โควตาต่อวัน อ่านจาก Secret COACH_DAILY_LIMIT ถ้าไม่ได้ตั้งใช้ 60

10 ขนาดคำขอสุงสุด 64,000 ไบต์ (คำถามกับตัวเลขสรุปเล็กกว่านี้มาก กันคนส่งข้อมูลขยะก้อนใหญ่)

12-26 **System prompt** (เขียนเป็นภาษาอังกฤษเพราะ AI ทำตามคำสั่งภาษาอังกฤษได้แม่นยำกว่า แต่สั่งให้ตอบเป็นไทย) บอกว่า: เป็นน้องกล้า มาสคอตต้นกล้าสีทอง · น้ำเสียงสุ่ม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ · คำตอบถูกอ่านออกเสียง จึงห้ามอีโมจิ ห้ามรายการ ห้าม markdown · ตอบไม่เกิน 90 คำ · **ใช้เฉพาะตัวเลขในส่วน DATA ห้ามแต่งตัวเลขขึ้นมาเอง** · โครงคำตอบ: ตัวเลขบอกอะไร → สำคัญ ยังไง → คำแนะนำเล็ก ๆ 1-2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง · **ห้ามตำหนิ** · ถ้าถาม "ซื้อได้ไหม" ให้ใช้ตัวเลข purchaseCheck บอก Runway ก่อนและหลัง · ไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำลงทุน ทุยืม ทรัพย์สิน

supabase/functions/coach/index.ts

บรรทัด 28-76

```
28 Deno.serve(async (req) => {
29   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
30   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
31
32   const userId = await requireUser(req);
33   if (!userId) return fail('unauthorized', 'Sign in again', 401);
34
35   if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large', 'Request is
too large', 413);
36   let body: { context?: unknown; question?: string };
37   try {
38     body = await req.json();
39   } catch {
40     return fail('bad_request', 'Body must be JSON', 400);
41   }
42   if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object', 400);
43   const question = (body.question ?? '').toString().slice(0, 400).trim();
44   const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46   if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
47
48   try {
49     if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50       return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51     }
52     const text = await callAI({
53       task: 'coach',
54       system: SYSTEM,
55       maxTokens: 500,
56       content: [
57         {
58           type: 'text',
59           text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของเงินช่วงนี้ และบอกสิ่งที่ควรรู้'`,
60         },
61       ],
62     });
63     return json({ text: plainText(text) });
64   } catch (e) {
65     if (e instanceof NotConfiguredError) {
66       console.error('coach: AI key missing or refused', e.message);
67       return fail('not_configured', 'AI key is not set on the server or was refused', 503);
68     }
69     if (e instanceof BusyError) {
70       console.warn('coach: AI busy', e.message);
71       return fail('busy', 'The AI service is busy, try again shortly', 503);
72     }
73     console.error('coach failed', e);
74     return fail('coach_error', 'The coach is unavailable right now', 502);
75   }
76 });
```

▶ เปิดใน VS Code (เร็ว) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:28

บรรทัด	ความหมาย
28	Deno.serve = "เปิดร้าน" รอรับคำขอ ทุกคำขอที่เข้ามาจะเรียกฟังก์ชันนี้ 1 ครั้ง
29	คำขอแบบ OPTIONS คือเบราว์เซอร์ "ถามทาง" ก่อนส่งจริง (กฎ CORS ของเว็บ) ตอบ ok พร้อม header ที่อนุญาต
30	รับเฉพาะ POST อย่างอื่นตอบ 405
32-33	ตรวจตัวตน: อ่าน token จากคำขอแล้วถาม Supabase ว่าเป็นใคร ไม่ผ่าน → 401 (แอปจะบอก "เซสชันหมดอายุ")
35	คำขอใหญ่เกิน → 413 (ตรวจจากหัวคำขอ ก่อนอ่านเนื้อหา จึงไม่เปลืองหน่วยความจำ)
36-42	อ่านเนื้อหาเป็น JSON ถ้าอ่านไม่ได้หรือไม่ใช่ object → 400
43	คำถามตัดเหลือไม่เกิน 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุปแปลงเป็นข้อความ JSON ตัดเหลือไม่เกิน 6,000 ตัวอักษร (คุมค่าใช้จ่าย)
46	ไม่มีกุญแจ AI ใน Secrets → 503 not_configured
49-51	ตรวจโควตา และนับเพิ่ม 1 ในคำสั่งเดียว เกิน 60 ครั้งใน 24 ชั่วโมง → 429
52-62	เรียก AI ส่ง system prompt, จำกัดคำตอบ 500 token, และข้อความที่มี 2 ส่วน: DATA: ตามด้วยตัวเลข และ QUESTION: ตามด้วยคำถาม (ถ้าผู้ใช้ไม่ได้ถาม ใช้คำถามตั้งต้น "สรุปสถานะการเงินของฉันช่วงนี้...")
63	ล้าง markdown ที่ AI อาจใส่มา (plainText) แล้วตอบแอปเป็น {"text": "..."}
64-75	จัดการ error ตามชนิด: กุญแจผิด/ไม่มี → 503 not_configured, AI ไม่ว่าง → 503 busy, อื่น ๆ → 502 และบันทึก log ไว้ดูใน Dashboard (บันทึกเฉพาะชนิด error ไม่บันทึกคำถามหรือคำตอบของผู้ใช้)

ตัวช่วยกลาง: supabase/functions/_shared/common.ts

supabase/functions/_shared/common.ts	บรรทัด 17-83
<pre> 17 /** Read one environment variable; a name the runtime refuses counts as "not set". */ 18 const env = (name: string) => { 19 try { 20 return Deno.env.get(name); 21 } catch { 22 return undefined; 23 } 24 }; 25 const geminiKey = () => readKey(env, GEMINI_KEY_NAMES); 26 const claudeKey = () => readKey(env, CLAUDE_KEY_NAMES); 27 28 export const corsHeaders = { 29 'Access-Control-Allow-Origin': '*', 30 'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type', 31 'Access-Control-Allow-Methods': 'POST, OPTIONS', 32 }; 33 34 export function json(body: unknown, status = 200) { 35 return new Response(JSON.stringify(body), { 36 status, 37 headers: { ...corsHeaders, 'Content-Type': 'application/json' }, 38 }); 39 } 40 41 /** Error body the app understands: { error: { code, message } } */ 42 export function fail(code: string, message: string, status: number) { 43 return json({ error: { code, message } }, status); 44 } 45 46 /** Returns the signed-in user's id, or null when the request has no valid session. */ 47 export async function requireUser(req: Request): Promise<string null> { </pre>	

```

48 const token = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '');
49 if (!token) return null;
50 const { data, error } = await admin().auth.getUser(token);
51 if (error || !data.user) return null;
52 return data.user.id;
53 }
54
55 function admin() {
56   return createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!, {
57     auth: { persistSession: false },
58   });
59 }
60
61 /** Thrown when no AI key is set in the function secrets, or the key was refused. */
62 export class NotConfiguredError extends Error {}
63
64 /** The AI service is rate-limiting us (e.g. Gemini free tier) or overloaded: try again shortly. */
65 export class BusyError extends Error {}
66
67 /** A daily limit from the function settings; a missing or non-number setting uses the default. */
68 export function envLimit(name: string, fallback: number): number {
69   const n = Number(Deno.env.get(name));
70   return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
71 }
72
73 /**
74  * Daily cap per user so a leaked account or a bug cannot run up the AI bill.
75  * Returns false when the user is over the limit; otherwise records one use.
76  * Counting and recording happen in one database call under a per-user lock
77  * (take_ai_quota), so many requests at once cannot all slip under the limit.
78  */
79 export async function takeQuota(userId: string, kind: 'slip' | 'coach', dailyLimit: number) {
80   const { data, error } = await admin().rpc('take_ai_quota', { p_user: userId, p_kind: kind, p_limit:
dailyLimit });
81   if (error) throw error;
82   return data === true;
83 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:17

บรรทัด	ความหมาย
17–24	อ่านค่า Secret จากเซิร์ฟเวอร์ (Deno.env.get) ถ้าชื่อแปลกจนอ่านไม่ได้ ถือว่าไม่ได้ตั้ง
25–26	หากุญแจ Gemini และ Claude จากรายชื่อที่ข้อมรับ (ดู readKey ด้านล่าง)
28–32	header CORS: ยอมให้เว็บจากทุกที่เรียกได้ (ความปลอดภัยมาจาก token ไม่ใช่จากที่อยู่เว็บ)
34–39	json() สร้างคำตอบ HTTP ที่เป็น JSON พร้อม header
41–44	fail() สร้างคำตอบ error รูปแบบเดียวกันทุกครั้ง {"error": {"code", "message"}} ที่แอปอ่าน code ได้ (บรรทัด 13–18 ของ coach.ts ฝั่งแอป)
46–53	requireUser: ตัดคำว่า "Bearer " ออกจาก header Authorization ได้ token แล้วถาม Auth ว่า token นี้เป็นของใคร คืน id ผู้ใช้ หรือ null
55–59	admin(): client ที่ใช้ service role key (ข้าม RLS ได้) มีเฉพาะบนเซิร์ฟเวอร์
61–65	error 2 ชนิด: ไม่ได้ตั้งกุญแจ/กุญแจใช้ไม่ได้ และ AI ไม่ว่าง (ติดลิมิตหรือโหลดเกิน)
67–71	อ่านโควตาจาก Secret ถ้าไม่ใช่ตัวเลขบวกใช้ค่าตั้งต้น
73–83	takeQuota: เรียกฟังก์ชัน SQL take_ai_quota ในฐานข้อมูล (rpc = เรียกฟังก์ชันระยะไกล) ซึ่งนับและบันทึกในขั้นตอนเดียว

ทำไมนับโควตาต้อง "ทำในขั้นตอนเดียว"

supabase/migrations/20261001000000_atomic_ai_quota.sql

บรรทัด 1-44

```
1 -- Atomic daily AI quota (audit 1 Oct 2569).
2 -- Before: the Edge Functions counted today's uses and inserted a new one in two
3 -- separate calls, so many requests sent at once could all pass the limit.
4 -- Now counting and recording happen in one call, under a lock per user and kind.
5 -- Called only by the Edge Functions with the service role.
6
7 create or replace function public.take_ai_quota(p_user uuid, p_kind text, p_limit integer)
8 returns boolean
9 language plpgsql
10 security definer
11 set search_path = ''
12 as $$
13 declare
14     used integer;
15 begin
16     if p_kind not in ('slip', 'coach') or p_limit is null or p_limit < 1 then
17         return false;
18     end if;
19     perform pg_advisory_xact_lock(hashtextextended(p_user::text || ':' || p_kind, 0));
20     select count(*) into used
21     from public.ai_usage
22     where user_id = p_user and kind = p_kind and created_at >= now() - interval '24 hours';
23     if used >= p_limit then
24         return false;
25     end if;
26     insert into public.ai_usage (user_id, kind) values (p_user, p_kind);
27     return true;
28 end;
29 $$;
30
31 revoke execute on function public.take_ai_quota(uuid, text, integer) from public, anon, authenticated;
32 grant execute on function public.take_ai_quota(uuid, text, integer) to service_role;
33
34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37 drop constraint if exists transactions_amount_max,
38 add constraint transactions_amount_max check (amount_satang <= 100000000000);
39 alter table public.profiles
40 drop constraint if exists profiles_amounts_max,
41 add constraint profiles_amounts_max check (
42     abs(opening_balance_satang) <= 100000000000
43     and (monthly_budget_satang is null or monthly_budget_satang <= 100000000000)
44 );
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:1

บรรทัด ความหมาย

- | | |
|-------|--|
| 1-5 | คอมเมนต์เล่าบ๊ิกเดิม: เดิมนับจำนวนครั้งวันนี้กับบันทึกครั้งใหม่เป็น 2 คำสั่งแยกกัน ถ้าส่งคำขอพร้อมกันหลายอัน ทุกอันนับได้ "ยังไม่เกิน" แล้วผ่านหมด (race condition) แก้ให้นับและบันทึกในคำสั่งเดียวภายใต้กุญแจล็อก |
| 7-14 | สร้างฟังก์ชัน SQL รับ id ผู้ใช้ ชนิดงาน (slip/coach) และเพดาน คำนวณจริง/เท็จ รับด้วยสิทธิ์เจ้าของ (security definer) |
| 16-18 | คำที่ส่งมาผิด → ไม่อนุญาต |
| 19 | ล็อก ต่อผู้ใช้ต่อชนิดงาน: คำขอที่ 2 ของคนเดียวกันต้องรอคำขอแรกเสร็จก่อน (ล็อกปลดเองเมื่อจบคำสั่ง) |
| 20-22 | นับจำนวนการใช้ใน 24 ชั่วโมงล่าสุด |
| 23-25 | ถึงเพดานแล้ว → ไม่อนุญาต |

บรรทัด	ความหมาย
26-27	ยังไม่ถึง → บันทึกการใช้ 1 ครั้ง แล้วอนุญาต
31-32	ห้ามผู้ใช้ทั่วไปเรียกฟังก์ชันนี้ตรง ๆ ให้เฉพาะเซิร์ฟเวอร์ (service_role)
34-44	(ไฟล์เดียวกัน) เพิ่มเขตแดนจำนวนเงิน \$1,000,000,000 ให้ตารางรายการและโปรไฟล์ ตรงกับ MAX_SATANG ในแอป

เลือก AI ตัวไหน: Claude หรือ Gemini

supabase/functions/_shared/helpers.ts

บรรทัด 1-45

```
1 // Pure helpers for the Edge Functions. No Deno APIs here, so the same file is
2 // unit-tested with vitest (see __tests__/helpers.test.ts).
3
4 export type AiProvider = 'claude' | 'gemini';
5
6 /**
7  * Secret names accepted for each service. The first is the documented one;
8  * the rest are Google's other standard name and common typing slips seen in
9  * the Supabase dashboard (a secret saved as "GEMINI API KEY", 28 ก.ย. 2569).
10 */
11 export const GEMINI_KEY_NAMES = [
12   'GEMINI_API_KEY',
13   'GOOGLE_API_KEY',
14   'GOOGLE_GENERATIVE_AI_API_KEY',
15   'GEMINI API KEY',
16   'GEMINI-API-KEY',
17   'GEMINI_KEY',
18 ];
19 export const CLAUDE_KEY_NAMES = ['ANTHROPIC_API_KEY', 'CLAUDE_API_KEY', 'ANTHROPIC API KEY'];
20
21 /**
22  * The first secret that is set, with spaces, line breaks or quotes from
23  * copy-paste removed. Returns null when none is set.
24 */
25 export function readKey(get: (name: string) => string | undefined | null, names: string[]): string | null {
26   for (const name of names) {
27     const raw = get(name);
28     const clean = raw?.trim().replace(/^[\'"]|[\']$/g, '').trim();
29     if (clean) return clean;
30   }
31   return null;
32 }
33
34 /**
35  * Which AI service answers: Claude when its key is set, otherwise Gemini.
36  * AI_PROVIDER=claude|gemini forces one. Returns null when no usable key is set.
37 */
38 export function pickProvider(env: { forced?: string | null; hasClaude: boolean; hasGemini: boolean }):
39   AiProvider | null {
40   const forced = env.forced?.trim().toLowerCase();
41   if (forced === 'claude') return env.hasClaude ? 'claude' : null;
42   if (forced === 'gemini') return env.hasGemini ? 'gemini' : null;
43   if (env.hasClaude) return 'claude';
44   if (env.hasGemini) return 'gemini';
45   return null;
46 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:1

บรรทัด	ความหมาย
1-2	ไฟล์นี้เป็นฟังก์ชันล้วน ไม่ใช่คำสั่งของ Deno จึงเอาไปทดสอบด้วย vitest ได้เหมือนโค้ดในแอป

บรรทัด	ความหมาย
6–19	รายชื่อ Secret ที่ยอมรับ ตัวแรกคือชื่อที่ถูกต้อง ตัวอื่นเป็นชื่อสำรองและ ชื่อที่พิมพ์ผิดบ่อย คอมเมนต์เล่าว่าเคยมีคนตั้งชื่อ Secret เป็น "GEMINI API KEY" (มีเว้นวรรค) เมื่อ 28 ก.ย. 2569 ทำให้ AI ไม่ทำงาน ทีมจึงให้เซิร์ฟเวอร์รับชื่อเหล่านี้ด้วย
25–32	readKey: ใส่ชื่อที่ละตัว เจอตัวแรกที่มีค่า ตัดช่องว่าง การขึ้นบรรทัด และเครื่องหมายคำพูดที่ติดมาตอนก็อปวางออก
38–45	pickProvider: ถ้าตั้ง AI_PROVIDER บังคับไว้ ใช้ตัวนั้น (ต้องมีกุญแจด้วย) ไม่จันมีกุญแจ Claude ใช้ Claude ก่อน ไม่มีก็ใช้ Gemini ไม่มีเลยคืน null

	Claude (Anthropic)	Gemini (Google)
Secret	ANTHROPIC_API_KEY	GEMINI_API_KEY
รุ่นที่ใช้ (ค่าเริ่มต้น)	claude-haiku-4-5 (เปลี่ยนได้ด้วย SLIP_MODEL, COACH_MODEL)	gemini-3.5-flash-lite แล้วลองรุ่นถัดไปเองถ้าใช้ไม่ได้
ค่าใช้จ่าย	ต้องเติมเครดิต อ่านสลิปใบละประมาณ 10 สตางค์	มีแบบฟรี (มีเพดานต่อนาที/ต่อวัน)
ข้อมูลของผู้ใช้	ไม่นำไปฝึกโมเดล	แบบฟรี Google อาจนำข้อมูลไปปรับปรุงโมเดล (README บอกไว้ตรง ๆ)

ส่งคำขอไปที่ Claude: callClaude

supabase/functions/_shared/common.tsบรรทัด 114–200

```
114 /** Ask the configured AI and return its text reply. */
115 export async function callAI(req: AiRequest): Promise<string> {
116   return (await callAIWithModel(req)).text;
117 }
118
119 /**
120  * Longest wait for one call to the AI service, and for all tries of one request together
121  * (the platform stops a function after 150 s; both slip reads run at the same time).
122  */
123 const FETCH_TIMEOUT_MS = 45_000;
124 const REQUEST_DEADLINE_MS = 110_000;
125
126 /** Ask the configured AI; also says which model answered (the two slip reads compare it). */
127 export async function callAIWithModel(req: AiRequest): Promise<{ text: string; model: string }> {
128   const provider = aiProvider();
129   if (!provider) throw new NotConfiguredError('No AI key is set');
130   const started = Date.now();
131   const deadline = started + REQUEST_DEADLINE_MS;
132   const result = provider === 'claude' ? await callClaude(req, deadline) : await callGemini(req, deadline);
133   // Metadata only: never log images, questions or replies.
134   console.log(JSON.stringify({ ai: provider, model: result.model, task: req.task, variant: req.variant ??
    'primary', ms: Date.now() - started }));
135   return result;
136 }
137
138 /** fetch with a time limit; running out of time counts as the AI being busy (the app waits and retries). */
139 async function timedFetch(url: string, init: RequestInit, deadline: number): Promise<Response> {
140   const left = deadline - Date.now();
141   if (left <= 1000) throw new BusyError('AI request deadline reached');
142   try {
143     return await fetch(url, { ...init, signal: AbortSignal.timeout(Math.min(FETCH_TIMEOUT_MS, left)) });
144   } catch (e) {
145     const name = (e as Error)?.name;
146     if (name === 'TimeoutError' || name === 'AbortError') throw new BusyError(`AI request timed out
    (${url.split('/')[2]})`);
147     throw e;
148   }
```



```

149 }
150
151 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
152
153 async function callClaude(req: AiRequest, deadline: number): Promise<{ text: string; model: string }> {
154   const key = claudeKey();
155   const main = Deno.env.get(req.task === 'slip' ? 'SLIP_MODEL' : 'COACH_MODEL') ?? CLAUDE_DEFAULT_MODEL;
156   const model = req.variant === 'verify' ? (Deno.env.get('VERIFY_MODEL') ?? main) : main;
157   const body: Record<string, unknown> = {
158     model,
159     max_tokens: req.maxTokens,
160     system: req.system,
161     messages: [
162       {
163         role: 'user',
164         content: req.content.map((c) =>
165           c.type === 'text' ? c : { type: 'image', source: { type: 'base64', media_type: c.mediaType, data:
166             c.data } },
167         ),
168       ],
169     ];
170   if (req.schema) {
171     body.output_config = { format: { type: 'json_schema', schema: req.schema } };
172   }
173   const res = await timedFetch(
174     'https://api.anthropic.com/v1/messages',
175     {
176       method: 'POST',
177       headers: {
178         'x-api-key': key,
179         'anthropic-version': '2023-06-01',
180         'content-type': 'application/json',
181       },
182       body: JSON.stringify(body),
183     },
184     deadline,
185   );
186   if (!res.ok) {
187     const detail = await res.text();
188     const msg = `Claude API ${res.status}: ${detail.slice(0, 300)}`;
189     if (isInvalidKey(res.status, detail)) throw new NotConfiguredError(msg);
190     if (res.status === 429 || res.status === 529) throw new BusyError(msg);
191     throw new Error(msg);
192   }
193   const data = await res.json();
194   const text = (data.content ?? []).
195     .filter((b: { type: string }) => b.type === 'text')
196     .map((b: { text: string }) => b.text)
197     .join('');
198   if (!text) throw new Error('Claude API returned no text');
199   return { text, model };
200 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:114`

บรรทัด	ความหมาย
114–117	callAI เรียกตัวที่บอกชื่อรุ่นด้วย แล้วคืนแค่ข้อความ
119–124	เวลาสูงสุด: รอ AI ครั้งละไม่เกิน 45 วินาที และทั้งคำขอไม่เกิน 110 วินาที (Supabase จะตัดฟังก์ชันที่ 150 วินาที)
127–136	เลือก AI แล้วเรียก จับเวลา และเขียน log เฉพาะข้อมูลกำกับ (ตัวไหน รุ่นไหน งานอะไร ใช้กี่มิลลิวินาที) ไม่เคยบันทึกรูป คำถาม หรือคำตอบ
139–149	timedFetch: ส่งคำขอทางอินเทอร์เน็ตแบบมีเวลาจำกัด ถ้าหมดเวลาถือว่า AI ไม่ว่าง (แอปจะรอแล้วลองใหม่)

บรรทัด	ความหมาย
153–156	เลือกรุ่นของ Claude ตามงาน ส่วนการอ่านสลิปครั้งที่ 2 (verify) ใช้รุ่น VERIFY_MODEL ถ้าตั้งไว้
157–169	สร้างเนื้อหาคำขอตามรูปแบบของ Anthropic Messages API: รุ่น, จำนวน token สูงสุด, system prompt และข้อความของผู้ใช้ (รูปแปลงเป็นรูปแบบ base64 ที่ Claude รับ)
170–172	ถ้างานนี้ต้องการ JSON ตามแบบฟอร์ม (อ่านสลิป) ใส่ schema ไปด้วย Claude จะตอบตามแบบฟอร์มเป็นะ
173–185	ส่งไปที่ https://api.anthropic.com/v1/messages แบบระบุใน header x-api-key
186–192	ตอบกลับไม่สำเร็จ: ระบุเจผิด → NotConfigured, 429/529 (ส่งที่เกิน/ระบบเต็ม) → Busy, อื่น ๆ → error ทัวไป
193–199	สำเร็จ: เอาเฉพาะส่วนที่เป็นข้อความมาต่อกัน คั้นพร้อมชื่อรุ่น

ฝั่ง Gemini (callGemini บรรทัด 202–268) หลักการเดียวกันแต่ซับซ้อนกว่า เพราะแบบฟรีติดลิมิตบ่อย: ไล่ลองหลายรุ่นตามลำดับ ถ้ารุ่นไหนใช้ไม่ได้กับกฎเจนี่ข้ามไปรุ่นถัดไป ถ้าทุกรุ่น "ไม่ว่าง" รอตามเวลาที่ Google บอก (ไม่เกิน 12 วินาที) แล้วลองทั้งหมดอีก 1 รอบ ก่อนจะบอกแอปว่า busy การตัดสินใจหลังแต่ละ error อยู่ในฟังก์ชัน geminiStep

supabase/functions/_shared/helpers.tsบรรทัด 75–93

```
75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';
88   if (status === 404) return 'next_model';
89   if (status === 429 && isZeroQuota(body)) return 'next_model';
90   if (status === 400 && usingJsonSchema) return 'plain_schema';
91   if (status === 429 || status === 500 || status === 503) return 'busy';
92   return 'fail';
93 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:75

บรรทัด	ความหมาย
87	ระบุเจผิด (401/403 หรือข้อความ API key not valid) → หยุด บอกว่ายังไม่ได้ตั้งค่า
88	404 = ไม่มีรุ่นนี้ → ลองรุ่นถัดไป
89	429 ที่บอกว่าโควตาเป็น 0 (กฎเจนี่ไม่มีสิทธิ์ใช้รุ่นนี้เลย) → รุ่นถัดไป
90	400 ตอนใช้ JSON schema (รุ่นเก่าบางรุ่นไม่รองรับ) → ลองใหม่โดยเขียนแบบฟอร์มลงใน prompt แทน
91	429/500/503 → ไม่ว่าง ลองรุ่นอื่นก่อน แล้วค่อยรอ
92	อย่างอื่น → ล้มเหลว

ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)

- สร้างกฎเจ: Gemini ที่ aistudio.google.com/apikey (ฟรี) หรือ Claude ที่ console.anthropic.com
- เปิด Supabase Dashboard → **Edge Functions** → **Secrets** (ไม่ใช่ Vault และไม่ใช่หน้า API keys)

3. เพิ่ม Secret ชื่อ GEMINI_API_KEY หรือ ANTHROPIC_API_KEY ใส่ค่ากุญแจ
4. ไม่ต้อง deploy ใหม่ ฟังก์ชันอ่าน Secret ทุกครั้งที่มีคำขอ
5. ถ้าแก้ไขโค้ดใน supabase/functions ต้อง deploy ด้วย `npx supabase functions deploy coach` (หรือ parse-slip)

หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้

หลักการ	ในโค้ด
AI ไม่คิดเลข โค้ดที่ทดสอบแล้วคิด AI แค่อธิบาย	แอปคำนวณ Runway และ purchaseCheck เองก่อนส่ง (FR-5, FR-6) system prompt ห้ามแต่งตัวเลข
AI ผิดได้ ต้องมีคนตรวจ	สลิปที่ไม่มั่นใจเป็นแบบร่าง อ่าน 2 ครั้งด้วย 2 รุ่น ไม่ตรงกันให้ผู้เลือกใช้ (FR-4)
ส่งข้อมูลให้ AI น้อยที่สุด	โค้ชได้เฉพาะยอดรวม รูปที่ส่งอ่านคือรูปที่มี QR สลิปเท่านั้น และไม่เก็บรูปบนเซิร์ฟเวอร์
คุมค่าใช้จ่าย	โควตาต่อวันแบบ atomic, ตัดความยาวคำถาม, จำกัด token คำตอบ
บันทึกการใช้ AI ในการพัฒนา	docs/AI_USAGE_LOG.md บันทึกตาม SRS ว่าใช้ AI ช่วยอะไร แก้อะไร และสมาชิกตรวจอะไร

UX/UI: การออกแบบหน้าจอและ Design System

UI คือหน้าตา (สี ตัวอักษร ปุ่ม) UX คือประสบการณ์ (ใช้งานไหม รู้สึกยังไง) บทนี้อธิบายหลักการออกแบบของ MindPay และโค้ดที่บังคับให้ทุกหน้าทำตาม

หลักการออกแบบของ MindPay

หลักการ	ทำยังไงในแอป
ตัวเลขเดียว เข้าใจทันที	หน้าหลักมีการ์ดเขียวที่บอกยอดเงินและ "เงินพอใช้อีก 16 วัน" ตัวใหญ่ที่สุด ผู้ใช้ไม่ต้องคิดเลขเอง
ทำสิ่งที่ทำบ่อยให้ง่ายที่สุด	ปุ่มสแกนสลิปใหญ่สุดกลางแถบล่าง, สแกนอัตโนมัติตอนเปิดแอป, พูดจตได้, ปุ่มลัด +฿100/500/1,000/5,000 ตอนเพิ่มเงินเข้า
ไม่ดู ไม่ทำให้รู้สึกผิด	น้องกล่าวพูดว่า "ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน" แทน "ใช้เงินเยอะเกินไป!" (system prompt ของโค้ชห้ามตำหนิ)
บอกตรง ๆ ว่ามั่นใจแค่ไหน	สลิปที่ไม่ชัดเป็น "แบบร่าง" พร้อมบอกว่าช่องไหนต้องตรวจ, Runway ใช้คำว่า "ถึงประมาณ"
อธิบายตัวเองได้	การ์ด "คำนวณยังง" ในหน้าเงินพอถึงแสดงสูตรที่ละบรรทัดเหมือนใบเสร็จ
ย้อนกลับได้	ลบรายการแล้วกด "เลิกทำ" ได้ (FR-1), สลิปที่เข้ายอดเองกด "ยังไม่รวมยอด" ได้ (FR-4)
ข้อความผิดพลาดต้องบอกวิธีแก้	"ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" แทน "Invalid input"
ทุกคนใช้ได้ (accessibility)	สีผ่านเกณฑ์อ่านง่าย, ปุ่มใหญ่พอแตะ, มีข้อความสำหรับโปรแกรมอ่านหน้าจอ, ปิดแอนิเมชันได้

Design System คืออะไร

Design System คือ "กฎกลาง + ชิ้นส่วนสำเร็จรูป" ที่ทุกหน้าใช้ร่วมกัน เหมือนชุดตัวต่อเลโก้ที่มีขนาดและสีมาตรฐาน หน้าจอทุกหน้าจึงดูเป็นแอปเดียวกัน และเปลี่ยนธีมสีได้ทั้งแอปในจุดเดียว ของ MindPay อยู่ใน

- `src/ui/theme.ts` ค่ากลาง (token): สี ฟอนต์ ขนาดตัวอักษร ระยะห่าง มุมโค้ง
- `src/ui/components.tsx`, `src/ui/inputs.tsx`, `src/ui/feedback.tsx` ชิ้นส่วน: ตัวหนังสือ ปุ่ม การ์ด ช่องกรอก ข้อความแจ้งเตือน
- `docs/DESIGN_SYSTEM.md` เอกสารกฎ กฎข้อแรก: ห้ามหน้าจอไหนคิดสีหรือขนาดเอง

สี: Green + Gold

ทิศทางแบรนด์คือ **สีเขียว** (เงิน การเติบโต ความมั่นคง) ใช้กับโครงสร้าง และ **สีทอง** ใช้ "น้อย ๆ" กับสิ่งที่สำคัญที่สุดบนหน้า เช่น ยอดเงิน ปุ่มหลัก ใบของต้นไม้เงิน คอมเมนต์บนสุดของ theme.ts เขียนว่า "Never paint a whole screen green"

src/ui/theme.tsบรรทัด 1-21

```
1 /**
2  * MindPay design system tokens.
3  *
4  * Brand: Green (money, growth, stability) + Gold (accent for what matters most).
5  * Rule from the brand direction: green is for structure, gold is used sparingly
6  * for the single most important thing on a screen (the balance, the main action).
7  * Never paint a whole screen green.
8  */
9 import { useColorThemeKey, useSchemeChoice } from './themeMode';
10
```

```

11 export const palette = {
12   forest: '#0E3B2C', // hero surfaces, brand
13   forestDeep: '#082A1F',
14   leaf: '#1F7A52', // primary actions, positive
15   mint: '#DDEEE3',
16   gold: '#C8992A', // accent: balance, main CTA ring, tree leaves
17   goldBright: '#E2B64A',
18   goldSoft: '#F4E7C2',
19   clay: '#B4442F', // critical / expense emphasis
20   amber: '#B7791F', // watch
21 };

```

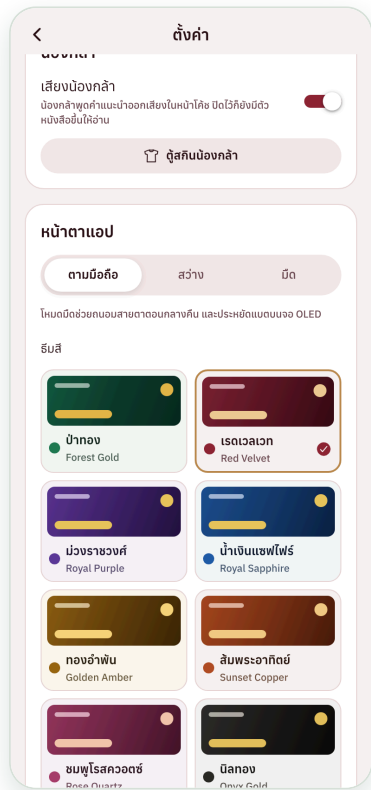
▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:1

สีทั้งหมดถูกตั้งชื่อตาม **หน้าที่** (token) ไม่ใช่ตามสี เช่น ink = สีตัวหนังสือหลัก, primary = สีปุ่มหลัก หน้าจอเรียก theme.ink โดยไม่สนว่าตอนนี้เป็นธีมไหน

Token	ใช้กับ	ปากทอง สว่าง	ปากทอง มืด
bg	พื้นหลังหน้าจอ	#F2F5F1	#0A1410
surface	การ์ด ช่องกรอก	#FFFFFF	#111E18
ink	ตัวหนังสือหลัก	#11231B	#E7EFE9
inkSoft	ตัวหนังสือรอง คำอธิบาย	#4B5F55	#A2B6AA
inkFaint	placeholder ไอคอนเจียบ ๆ (ห้ามใช้กับข้อความที่ต้องอ่าน)	#8A9A91	#6B8076
primary	ปุ่มหลัก สิ่งที่ต้องกดอยู่	#1F7A52	#3FA774
accent	ปุ่มทอง ขอบการ์ดที่ต้องสนใจ	#C8992A	#E2B64A
income / good / watch / critical	สีที่มีความหมาย: เงินเข้า, ดี, ระวัง, วิกฤต เหมือนกันทุกธีม ผู้ใช้จึงไม่สับสน	เขียว / เขียว / ส้มอำพัน / แดง อิฐ	

9 ธีม × สว่าง/มืด



ตั้งค่า → ธีมสี: เลือก 1 ใน 9 ธีม และโหมด ตามมือถือ / สว่าง / มืด

ผู้ใช้เลือกธีมได้ 9 แบบ: ป่าทอง (ค่าเริ่มต้น), แดงเวลเวท, ม่วงราชวงศ์, น้ำเงินแซฟไฟร์, ทองอำพัน, ส้มพระอาทิตย์, ชมพูโรสควอตซ์, ฟ้าทอง และธีมเทศกาลฮาโลวีน แต่ละธีมมีชุดสีสว่างและมืด (COLOR_THEMES ใน theme.ts ระบุที่ 93–204) เลือกแล้วจำไว้ในเครื่อง

ฟังก์ชัน buildTheme สร้างชุดสีเต็ม (ประมาณ 30 token) จากค่าพื้นฐานของแต่ละธีม และคำนวณสีบางตัวให้อ่านง่ายอัตโนมัติ เช่น สีตัวหนังสือบนปุ่มทอง (onAccent) เลือกระหว่างสีเข้มกับสีขาว ตัวที่อ่านออกบนพื้นทองนั้น

หน้าจอเรียกแค่ const theme = useTheme() ได้ชุดสีของธีมและโหมดที่ผู้ใช้เลือกตอนนี้ เปลี่ยนธีมแล้วทุกหน้าเปลี่ยนตามทันที

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ

มาตรฐานสากล WCAG AA กำหนดว่าตัวหนังสือกับพื้นหลังต้องต่างกันอย่างน้อย 4.5 : 1 (1 = สีเดียวกัน, 21 = ดำบนขาว) คนสายตาไม่ดีหรือใช้มือถือกลางแดดจึงอ่านออก

คู่สี (ป่าทอง สว่าง)	อัตราส่วน	ผ่านไหม
ink uu bg	14.93	ผ่าน
inkSoft uu surface	6.85	ผ่าน
ตัวหนังสือขาว uu primary	5.30	ผ่าน
inkFaint uu surface	2.95	ไม่ผ่าน → จึงห้ามใช้กับข้อความ ใช้กับ placeholder และไอคอนตกแต่งเท่านั้น
ทอง accent เป็นตัวหนังสือบนพื้นขาว	2.61	ไม่ผ่าน → จึงมี token accentInk ที่เข้มกว่าไว้ใช้แทน

คำนวณด้วยสูตรเดียวกับฟังก์ชัน contrast() ในแอป

```
src/ui/theme.ts
บรรทัด 261–279

261 function luminance(hex: string): number {
262   const n = parseInt(hex.replace('#', '').slice(0, 6), 16);
263   const ch = (v: number) => {
264     const c = v / 255;
265     return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
266   };
267   return 0.2126 * ch((n >> 16) & 255) + 0.7152 * ch((n >> 8) & 255) + 0.0722 * ch(n & 255);
268 }
269
270 /** WCAG contrast ratio between two hex colours (1 to 21). */
271 export function contrast(a: string, b: string): number {
272   const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x);
273   return (hi + 0.05) / (lo + 0.05);
```

```

274 }
275
276 /** The first colour that reads on `bg` at 4.5:1 or better (the last one otherwise). */
277 export function readableOn(bg: string, ...colors: string[]): string {
278   return colors.find((c) => contrast(c, bg) >= 4.5) ?? colors[colors.length - 1];
279 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:261

บรรทัด ความหมาย

- 261–268** luminance: ความสว่างที่ตาคนรับรู้ของสีหนึ่ง (0 = ดำ ถึง 1 = ขาว) แยกสีแดง เขียว น้ำเงิน แปลงค่า แล้วถ่วงน้ำหนัก เขียวหนักสุด (0.7152) เพราะตาคนไวต่อสีเขียวที่สุด
- 270–274** contrast: เอาสีที่สว่างกว่าเป็นตัวตั้ง (บวก 0.05) หาสีที่มีมืดกว่า (บวก 0.05) ตามสูตรของ WCAG
- 276–279** readableOn: ให้รายการสีมา ค้นสีแรกที่อ่านออกบนพื้นนั้น (≥ 4.5)

src/ui/__tests__/theme.test.ts

บรรทัด 34–58

```

34 for (const spec of COLOR_THEMES) {
35   for (const dark of [false, true]) {
36     it(`${spec.key} ${dark ? 'dark' : 'light'}: every text colour reads on the background it is used on`, ()
=> {
37       const t = buildTheme(spec.key, dark);
38       const pairs: [string, string, string][] = [
39         ['inkSoft on bg', t.inkSoft, t.bg],
40         ['inkSoft on surface', t.inkSoft, t.surface],
41         ['onPrimary on primary', t.onPrimary, t.primary],
42         ['onAccent on accent', t.onAccent, t.accent],
43         ['accentInk on surface', t.accentInk, t.surface],
44         ['accentInk on accentSoft', t.accentInk, t.accentSoft],
45         ['good on goodSoft', t.good, t.goodSoft],
46         ['watch on watchSoft', t.watch, t.watchSoft],
47         ['critical on criticalSoft', t.critical, t.criticalSoft],
48         ['good on bg', t.good, t.bg],
49         ['watch on bg', t.watch, t.bg],
50         ['critical on bg', t.critical, t.bg],
51         ['heroInkSoft on hero', t.heroInkSoft, t.hero],
52         ['heroInk on heroTop', t.heroInk, t.heroTop],
53       ];
54       const low = pairs.filter(([a, b]) => contrast(a, b) < 4.5).map(([n, a, b]) => `${n} ${contrast(a,
b).toFixed(2)}`);
55       expect(low).toEqual([]);
56     });
57   }
58 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/__tests__/theme.test.ts:34

เทสต์ **TC-78** วนตรวจทุกธีม (9) × ทุกโหมด (สว่าง/มืด) = 18 ชุด แต่ละชุดตรวจ 14 คู่สี (บรรทัด 38–53) ถ้าคู่ไหนต่ำกว่า 4.5 เทสต์ล้ม และบอกชื่อคู่พร้อมค่าที่ได้ (บรรทัด 54) ใครเพิ่มธีมใหม่แล้วเลือกสีอ่านยาก npm test จะไม่ผ่าน และ GitHub จะไม่ build แอปให้

ตัวอักษร

src/ui/theme.ts

บรรทัด 301–328

```

301 /**
302  * Two typefaces:
303  * - Noto Serif Thai for money and headings: the "international banking" feel
304  *   from the logo direction, and clear numerals.
305  * - Anuphan for everything else: a modern Thai sans that reads well small.
306  */
307 export const fonts = {

```

```

308 serif: 'NotoSerifThai_600SemiBold',
309 serifBold: 'NotoSerifThai_700Bold',
310 sans: 'Anuphan_400Regular',
311 sansMedium: 'Anuphan_500Medium',
312 sansSemi: 'Anuphan_600SemiBold',
313 sansBold: 'Anuphan_700Bold',
314 };
315
316 export const space = { xs: 4, sm: 8, md: 12, lg: 16, xl: 20, xxl: 28, xxxl: 40 };
317 export const radius = { sm: 8, md: 14, lg: 20, xl: 28, pill: 999 };
318
319 /** Type scale (size / line height). Thai needs generous line height for vowels above and below. */
320 export const type = {
321   display: { fontSize: 38, lineHeight: 52 },
322   h1: { fontSize: 26, lineHeight: 38 },
323   h2: { fontSize: 20, lineHeight: 30 },
324   h3: { fontSize: 17, lineHeight: 26 },
325   body: { fontSize: 15, lineHeight: 24 },
326   small: { fontSize: 13, lineHeight: 20 },
327   micro: { fontSize: 11, lineHeight: 16 },
328 };

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/theme.ts:301`

- **Noto Serif Thai** (มีหัว) ใช้กับตัวเลขเงินและหัวข้อใหญ่ ให้ความรู้สึก "ธนาคารสากล" และตัวเลขอ่านชัด
- **Anuphan** (ไม่มีหัว ก้นสมัย) ใช้กับข้อความอื่นทั้งหมด อ่านง่ายแม้ตัวเล็ก
- บรรทัด 316–317: ระยะห่างมาตรฐาน 4/8/12/16/20/28/40 และมุมโค้ง 8/14/20/28/999 (999 = ปลายมนเป็นเม็ดยา)
- บรรทัด 319–328: ขนาดตัวอักษรคู่กับความสูงบรรทัดเสมอ ภาษาไทยมีสระบนล่างและวรรณยุกต์ ต้องมีความสูงบรรทัดประมาณ 1.4–1.6 เท่าของขนาดตัวอักษร ไม่ขึ้นสระจะชนบรรทัดข้างบน

ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)

`src/ui/components.tsx`

บรรทัด 261–338

```

261 export function Button({
262   label,
263   onPress,
264   kind = 'primary',
265   icon,
266   loading,
267   disabled,
268   style,
269   small,
270   shine,
271 }): {
272   label: string;
273   onPress: () => void;
274   /** onDark: outline button on the dark green hero surfaces */
275   kind?: 'primary' | 'gold' | 'ghost' | 'danger' | 'soft' | 'onDark';
276   icon?: IconName;
277   loading?: boolean;
278   disabled?: boolean;
279   style?: StyleProp<ViewStyle>;
280   small?: boolean;
281   /** A band of light sweeps across (for the one main action on a screen). */
282   shine?: boolean;
283 } {
284   const theme = useTheme();
285   const press = usePressSpring(0.95);
286   const colors = {
287     primary: { bg: theme.primary, fg: theme.onPrimary, border: theme.primary },
288     gold: { bg: theme.accent, fg: theme.onAccent, border: theme.accent },
289     ghost: { bg: 'transparent', fg: theme.ink, border: theme.line },
290     danger: { bg: 'transparent', fg: theme.critical, border: theme.critical },

```



```

291     soft: { bg: theme.surfaceAlt, fg: theme.ink, border: theme.surfaceAlt },
292     onDark: { bg: 'rgba(244,241,230,0.10)', fg: '#F4F1E6', border: 'rgba(244,241,230,0.45)' },
293   }[kind];
294   const off = disabled || loading;
295   return (
296     <AnimatedPressable
297       onPress={() => {
298         Haptics.selectionAsync().catch(() => {});
299         onPress();
300       }}
301       onPressIn={off ? undefined : press.onPressIn}
302       onPressOut={press.onPressOut}
303       disabled={off}
304       // Small buttons are about 40pt tall: the touch area reaches 44pt and more (up and down only,
305       // so buttons side by side never share a touch area).
306       hitSlop={small ? { top: 4, bottom: 4 } : undefined}
307       accessibilityRole="button"
308       aria-disabled={!off}
309       aria-busy={!loading}
310       style={[
311         {
312           backgroundColor: colors.bg,
313           borderColor: colors.border,
314           borderWidth: 1.5,
315           borderRadius: radius.pill,
316           paddingVertical: small ? 8 : 12,
317           paddingHorizontal: small ? 14 : 20,
318           flexDirection: 'row',
319           alignItems: 'center',
320           justifyContent: 'center',
321           gap: 8,
322           opacity: off ? 0.5 : 1,
323           overflow: shine ? 'hidden' : undefined,
324         },
325         style,
326         { transform: [{ scale: press.scale }] },
327       ]}
328     >
329       {shine && !off ? <Shine times={1} delay={900} color={kind === 'gold' ? 'rgba(255,255,255,0.45)' :
'rgba(255,255,255,0.28)'} /> : null}
330       {loading ? (
331         <ActivityIndicator color={colors.fg} />
332       ) : icon ? (
333         <Icons name={icon} size={small ? 16 : 19} color={colors.fg} />
334       ) : null}
335       <Text style={[small ? type.small : type.body, { fontFamily: fonts.sansSemi, color: colors.fg }]}>{label}
</Text>
336     </AnimatedPressable>
337   );
338 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:261

บรรทัด ความหมาย

- 261–283** props ของปุ่ม: ชื่อความ, สิ่งที่ทำเมื่อกด, ชนิด 6 แบบ (ค่าเริ่มต้น primary), ไอคอน, คำสั่งโหลด, ปิดใช้งาน, ขนาดเล็ก, แสงวิ่งผ่าน
- 284–285** สีของริม และแอนิเมชันขยับตัวตอนกด (เหลือ 95%)
- 286–293** ตารางสีของปุ่มแต่ละชนิด: primary = พื้นสีขาวตัวขาว, gold = พื้นทอง, ghost = โปร่งมีขอบ, danger = ขอบแดง (ปุ่มลบ), soft = พื้นเทาอ่อน, onDark = โปร่งบนการ์ดเขียว ทุกสีมาจาก theme
- 294** ปุ่ม "ปิด" ถ้าถูกปิดใช้งานหรือคำสั่งโหลด (กันกดซ้ำ)
- 297–300** กดแล้วมือถือสั่นเบา ๆ (haptic) แล้วทำสิ่งที่ผู้เรียกกำหนด

บรรทัด	ความหมาย
304–306	ปุ่มเล็กสูงประมาณ 40 จุด ขยายพื้นที่แตะขึ้นลงอีก 4 จุด ให้ถึง 44 จุดตามแนวทางของ Apple/Google (นี้วัดกว้างกว่าที่คิด) ขยายเฉพาะแนวตั้ง ปุ่มที่วางติดกันจึงไม่แย่งพื้นที่แตะ
307–309	บอกโปรแกรมอ่านหน้าจอกว่าเป็นปุ่ม และสถานะปิด/กำลังโหลด
310–327	รูปร่าง: ขอบ 1.5, มุมโค้งแบบเบียด, ระยะในตามขนาด, ไอคอนกับข้อความเรียงแนวนอนตรงกลาง, จางลง 50% เมื่อปิดใช้งาน
329	แสงวิ่งผ่าน 1 ครั้ง (ใช้กับปุ่มหลักปุ่มเดียวของหน้า)
330–334	กำลังโหลดแสดงวงหมุนแทนไอคอน
335	ข้อความบนปุ่ม ขนาดตามปุ่ม ฟอนต์หนาทั้งหน้า

ชิ้นส่วน	ใช้เมื่อ
Screen	ทุกหน้า: พื้นหลัง ขอบ เลื่อนช่องที่กำลังพิมพ์ให้พ้นคีย์บอร์ด
Card	กลุ่มเนื้อหา มี 3 โทน: ปกติ, พื้นรอง (alt), ท่องอ่อน (accent)
Button / IconButton	การกระทำ 1 หน้ามีปุ่ม primary แค่ 1 ปุ่ม / ปุ่มไอคอนต้องมี label เสมอ
Segmented	สลับมุมมอง 2–4 แบบ เช่น วันนี้ / 7 วัน / 1 เดือน, รายจ่าย / รายรับ
Chip	ตัวเลือกหลายอัน เช่น หมวดหมู่, จำนวนเงินมัด
Field / AmountField	ช่องกรอก ขอบเปลี่ยนสีตอนพิมพ์และตอนผิด บนเว็บตัวหนังสือ 16 เพื่อไม่ให้ iPhone ซุมหน้า
Badge	ป้ายสถานะสั้น ๆ เช่น "สบาย ๆ"
useToast	ยืนยันผล เช่น "บันทึกรายการแล้ว" มีปุ่ม "เลิกทำ" ได้
ConfirmSheet	ถามยืนยันก่อนทำสิ่งที่ย้อนไม่ได้ เช่น ลบ
EmptyState	ยังไม่มีข้อมูล: นื่องคล้า + คำอธิบาย + ปุ่มให้เริ่ม

Accessibility: ทุกคนใช้ได้

เรื่อง	ทำยังไง
คนตาบอด/สายตาสั้น	ปุ่มทุกปุ่มมี accessibilityLabel ให้ TalkBack (Android) / VoiceOver (iPhone) อ่าน เช่นการ์ดยอดเงินอ่านว่า "ยอดคงเหลือ \$11,359.00 เงินพอใช้อีก 16 วัน"
แยกสีไม่ออก (ตาบอดสี)	สถานะไม่ใช่สีอย่างเดียว มีข้อความกำกับเสมอ เช่น "ใกล้เส้นสำรอง"
เมาจจากการเคลื่อนไหว	เปิด "ลดการเคลื่อนไหว" (Reduce motion) ในมือถือ ทุกแอนิเมชันหยุดและแสดงผลทันที (useReduceMotion())
นิ้วใหญ่ มือสั่น	พื้นที่แตะอย่างน้อย 44 จุด
อ่านยาก	สีผ่าน WCAG AA ทุกธีม (TC-78)

แอนิเมชัน: สวยแต่ไม่กินแบตเตอรี่

แอปมีเอฟเฟกต์เยอะ (ส่วนต่าง ๆ ค่อย ๆ โผล่, ปุ่มแดง, ตันไม้เงินโยก, พลุดอนสำเร็จ) แต่มีกฎ 4 ข้อจาก README:

1. ขยับเฉพาะ transform (ย้าย หมุน ย่อขยาย) กับ opacity (ความจาง) และใช้ **native driver** ให้การขยับทำงานส่วน native ของมือถือ ไม่ผ่าน JavaScript จึงสิ้นแม่มือถือราคาประหยัด

2. ทุกเอฟเฟกต์หยุดเอง ไม่มีอะไรยับยั้งค้างตอนผู้ใช้ไม่ได้ทำอะไร (มี E2E ตรวจสอบว่าหน้าหลักนิ่งภายใน 9 วินาที)
3. เคารพ "ลดการเคลื่อนไหว"
4. ไม่ใช่ native module ใหม่ แอปที่ลงแล้วจึงได้เอฟเฟกต์ใหม่ทาง OTA โดยไม่ต้องลง APK ใหม่

เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)

1. ไม่มีรหัสสี #... หรือ rgba (...) ในหน้าจอ ใช้ token หรือ alpha ()
2. ตัวหนังสือใช้ <T> และเงินใช้ <Money>
3. ปุ่ม ซิป ช่องกรอก การ์ด มาจาก component ไม่สร้างใหม่เอง
4. ลองเปิดหน้านั้น 2-3 รอบ ทั้งสว่างและมืด
5. รับ npm test, npx tsc --noEmit, npx expo lint, npm run e2e:web

1 ทบทวน UX จากที่ไหน

บันทึกใน [docs/AI_USAGE_LOG.md](#) แถว 10 และ 19: ศึกษาแอปเหมียวจวด (ผู้ช่วยที่พูดกับผู้ใช้, "แค่เปิดแอปก็จัดให้") และแนวการออกแบบของ Hugging Face (มาศคอตตัวเดียวหลายอารมณ์, สีอุ่นบนพื้นเรียบ, การ์ดโค้ง เส้นบาง) และหลัก Laws of UX เช่น aesthetic-usability effect (ของสวยทำให้รู้สึกว่ายาง่าย) และ peak-end rule (คนจำช่วงพีคและตอนจบ จึงมีพละตอนสำเร็จ) แล้วออกแบบตัวละครน้องกล้วยของ MindPay เอง ไม่ลอกของใคร

การทดสอบ: พิสูจน์ว่าระบบทำงานถูก

ทีมเรามีการทดสอบ 3 ระดับ: unit test 157 ข้อ, E2E ในเบราว์เซอร์ 145 ข้อ และแผนทดสอบบนมือถือจริง 12 ข้อ บทนี้สอนอ่านและเขียนเทส

ทำไมต้องเขียนเทส

เวลาแก้ไขจุดหนึ่ง อาจทำให้จุดอื่นพังโดยไม่รู้ตัว (เรียกว่า **regression**) ถ้าต้องกดทดสอบทุกหน้าเองทุกครั้งก็แค่ จะใช้เวลาหลายชั่วโมงและลืมบางจุดแน่นอน เทสอัตโนมัติคือ "โปรแกรมที่ตรวจโปรแกรม" รันเสร็จในไม่กี่วินาที และทีมตั้งไว้ว่าถ้าเทสไม่ผ่าน GitHub จะไม่ปล่อยเว็บและ APK ให้ผู้ใช้

157

Unit test (Vitest)
ตรวจฟังก์ชันทีละตัว · รันไม่ถึง 2 วินาที

145

E2E (Playwright)
เปิดเว็บจริงแล้วกดเหมือนคน · ประมาณ 3 นาที

12

Manual test (MT-01-12)
คนทดสอบบนมือถือจริง

Unit test: ตรวจฟังก์ชันทีละตัว

ไฟล์เทสอยู่ในโฟลเดอร์ `__tests__` ชื่อลงท้าย `.test.ts` รันด้วยคำสั่ง `npm test` ใช้เครื่องมือ **Vitest** ตัวอย่างจากเทสของ FR-6:

src/domain/__tests__/domain.test.tsบรรทัด 35-58

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
37
38 let seq = 0;
39 function tx(p: Partial<Transaction> & { day: string; amount: number }): Transaction {
40   const { day, amount, ...rest } = p;
41   return {
42     id: `t${++seq}`,
43     kind: 'expense',
44     amountSatang: amount * 100,
45     categoryKey: 'food',
46     title: 'test',
47     note: null,
48     occurredAt: bkkToIso(day, '12:00'),
49     source: 'manual',
50     status: 'confirmed',
51     slipRef: null,
52     slipImageHash: null,
53     ocrConfidence: null,
54     reviewFlags: [],
55     createdAt: NOW.toISOString(),
56     ...rest,
57   };
58 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

บรรทัด	ความหมาย
35-36	ล็อก "วันนี้" ให้ตายตัว เป็นวันอาทิตย์ที่ 27 ก.ย. 2569 เวลา 14:00 น. ไม่ว่ารันเทสวันไหน ผลก็เหมือนเดิม (ถ้าใช้วันจริง เทสจะผ่านบ้างไม่ผ่านบ้างตามวันที่)
38-58	ฟังก์ชันช่วยสร้างรายการปลอม tx({ day, amount }) ใส่แค่วันกับจำนวนบาท ที่เหลือใช้ค่าตั้งต้น (รายการยืม หมวดอาหาร ยืนยันแล้ว) ...rest ท้ายสุดให้เปลี่ยนช่องอื่นได้ เช่น tx({ day, amount, status: 'draft' })

```

184 it('TC-14 computes days and depletion date', () => {
185   const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000, now: NOW });
186   expect(r.days).toBe(15);
187   expect(r.depletionDay).toBe('2026-10-12');
188   expect(r.status).toBe('healthy');
189 });
190
191 it('TC-15 handles zero average without dividing by zero', () => {
192   const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now: NOW });
193   expect(r.status).toBe('no_spending');
194   expect(r.days).toBeNull();
195   expect(r.depletionDay).toBeNull();
196 });
197
198 it('TC-16 flags balance at or below the low line', () => {
199   const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000, now: NOW });
200   expect(r.status).toBe('below_floor');
201   expect(r.days).toBe(0);
202 });
203
204 it('TC-17 status thresholds', () => {
205   const at = (days: number) =>
206     computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang: 10_000, now:
NOW }).status;
207   expect(at(6)).toBe('critical');
208   expect(at(7)).toBe('watch');
209   expect(at(14)).toBe('healthy');
210 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:184

บรรทัด	ความหมาย
184	it('ชื่อเทสต์', () => {...}) = เทสต์ 1 ข้อ ชื่อขึ้นต้นด้วยรหัส TC ที่ตรงกับเอกสาร Traceability
185	Arrange + Act: เตรียมข้อมูล (ยอด ฿3,500, เลื่อนสำรอง ฿500, ใช้วันละ ฿200) แล้วเรียกฟังก์ชันจริง
186-188	Assert: expect (ค่าจริง) .toBe (ค่าที่ต้องได้) ตรวจสอบว่าได้ 15 วัน, วันที่ 12 ต.ค. 2569 และสถานะ healthy ถ้าไม่ตรงข้อใดข้อหนึ่ง เทสต์ล้มพร้อมบอกว่าได้อะไรแทน
191-196	TC-15 ทดสอบ กรณีพิเศษ: ค่าเฉลี่ยเป็น 0 ต้องไม่หารด้วยศูนย์ ต้องได้ no_spending และวันเป็น null
198-202	TC-16 ทดสอบ ขอบ: ยอดเท่ากับเงินสำรองพอดี ต้องนับว่า "ต่ำกว่าเงินสำรอง"
204-210	TC-17 ทดสอบ ขอบของเกณฑ์: 6 วัน critical, 7 วัน watch, 14 วัน healthy ถ้ามีคนเหลือแค่ < เป็น <= เทสต์นี้จะจับได้ทันที

★ เขียนเทสต์ที่ดี: ทดสอบขอบ ไม่ใช่แค่ค่ากลาง ๆ

มักชอบซ่อนอยู่ตรงขอบ เช่น 0, ค่าเท่ากันพอดี, ค่าว่าง, วันแรกของเดือน, เกียงคืน, ข้อมูลภาษาไทยแปลก ๆ เทสต์ของทีมนึงจึงมีทั้งกรณีปกติ กรณีพิเศษ และกรณีขอบ อาจารย์มักถามว่า "ทดสอบกรณีไหนบ้าง" ให้ตอบเป็น 3 กลุ่มนี้

ไฟล์ unit test หลัก (มีทั้งหมด 18 ไฟล์)

ไฟล์	ทดสอบ
domain.test.ts	เงิน, วันที่ไทย, ยอดคงเหลือ (FR-2), Runway (FR-6), สลิป (FR-4), คำแนะนำโค้ช (FR-5), หมวด
voice.test.ts	แยกประโยคพูดเป็นรายการ ตัวเลขคำไทย (FR-3)

ไฟล์	ทดสอบ
slipAccuracy.test.ts, autoScan.test.ts	ความแม่นยำการอ่านสลิป และสแกนอัตโนมัติ (FR-4)
recurring.test.ts, recap.test.ts	รายการประจำเดือน (FR-1) และสรุปเดือน ปฏิทิน (FR-2)
goals.test.ts	กระปุกออม (FR-6)
buddy.test.ts, klaTalk.test.ts	คำพูดน้องกล้า และการตัดซับไตเติลอ่านออกเสียง (FR-5)
auth.test.ts	ลิงก์อีเมลและข้อความ error ภาษาไทย
audit.test.ts	บีกที่เจอตอนตรวจทั้ง repo (1-3 ต.ค.) กันไม่ให้กลับมาอีก
ui/__tests__/theme.test.ts	สีทุกธีมอ่านง่าย (TC-78)
supabase/functions/_shared/__tests__/helpers.test.ts	ตัวช่วยของเชิร์ฟเวอร์: เลือก AI, ทำความสะอาดผลอ่านสลิป, รวมผลอ่าน 2 ครั้ง

E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง

End-to-End (E2E) test เปิดเว็บเวอร์ชันจริงในเบราว์เซอร์อัตโนมัติ (เครื่องมือ **Playwright**) แล้วกด พิมพ์ เลื่อน ตามขั้นตอนเหมือนคน และตรวจว่าเห็นสิ่งที่ควรเห็น ไฟล์หลักคือ `e2e/web-auth.mjs` (ยาวกว่า 1,000 บรรทัด)

≈ ไม่แตะบัญชีจริง: เชิร์ฟเวอร์ปลอม

ถ้า E2E ใช้ Supabase จริง จะสร้างบัญชีขยะ ส่งอีเมลจริง และเปลืองโควตา AI ทีมเราจึงเขียน `e2e/fake-backend.mjs` เป็น "Supabase ปลอม" ที่ตอบเหมือนของจริงแต่อยู่ในหน่วยความจำ ทุกคำขอที่ไปทอยู๋ของ Supabase จะถูกดักมาตอบด้วยตัวปลอมนี้ (page.route) เหมือนซ่อมละครกับนักแสดงแทน

`e2e/web-auth.mjs`

บรรทัด 29-60

```

29 // ----- helpers
30 const results = [];
31 function check(name, ok, detail = '') {
32   results.push({ name, ok, detail });
33   console.log(`${ok ? 'PASS' : 'FAIL'} ${name}${detail ? ` - ${detail}` : ''}`);
34   // In GitHub Actions a failed check also becomes an annotation on the run.
35   if (!ok && IN_CI) console.log(`::error title=${ENGINE} browser test::${name}${detail ? `
36   (${detail.replace(/\n/g, ' ').slice(0, 300)})` : ''}`);
37 }
38 const browser = await (ENGINE === 'webkit' ? webkit : chromium).launch();
39 const phone = {
40   ...(ENGINE === 'webkit' ? { isMobile: true, hasTouch: true } : {}),
41   // E2E_SCHEME=dark runs everything with the phone in dark mode.
42   colorScheme: process.env.E2E_SCHEME === 'dark' ? 'dark' : 'light',
43 };
44 async function freshPage(tag) {
45   const ctx = await browser.newContext({ viewport: { width: 390, height: 844 }, deviceScaleFactor: 2, locale:
46   'th-TH', ...phone });
47   const page = await ctx.newPage();
48   page.on('pageerror', (e) => console.log(`[${tag}] pageerror`, e.message));
49   page.on('console', (m) => { if (m.type() === 'error') console.log(`[${tag}] console.error`,
50   m.text().slice(0, 200)); });
51   page.on('requestfailed', (r) => { if (!r.url().startsWith(SUPA)) console.log(`[${tag}] request failed`,
52   r.url().slice(0, 160)); });
53   await page.route(`${SUPA}/**`, supabase);
54   // The web QR reader downloads its decoder from a CDN: serve the same file from node_modules.
55   await routeQrDecoder(page);
56   return { ctx, page };
57 }

```

```

54 const shot = (page, name) => page.screenshot({ path: join(SHOTS, `${name}.png`) });
55 const visible = (page, text, timeout = 6000) => page.getByText(text, { exact: false }).first().waitFor({
  state: 'visible', timeout }).then(() => true, () => false);
56 const gone = (page, text, timeout = 6000) => page.getByText(text, { exact: false }).first().waitFor({ state:
  'hidden', timeout }).then(() => true, () => false);
57 const introGone = (page) => page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 8000
  }).catch(() => {});
58 const button = (page, name) => page.getByRole('button', { name, exact: false }).first();
59 /** Waits (up to `timeout`) until the element is visible. */
60 const shown = (locator, timeout = 6000) => locator.waitFor({ state: 'visible', timeout }).then(() => true, ()
  => false);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:29

บรรทัด ความหมาย

30–36	check (ชื่อ, ผ่านไหม, รายละเอียด) บันทึกผลแต่ละข้อ พิมพ์ PASS หรือ FAIL ถ้ารันบน GitHub และไม่ผ่าน จะขึ้นเป็นป้าย error บนหน้า Actions
37–42	เปิดเบราว์เซอร์ Chromium (หรือ WebKit แบบ Safari) โหมดสว่างหรือมืด
43–53	freshPage: เปิดแท็บใหม่ขนาดมือถือ (390×844) ภาษาไทย ดึงคำขอไป Supabase ให้ตัวปลอมตอบ และพิมพ์ error ของหน้าเว็บถ้ามี
54	shot: ถ่ายภาพหน้าจอเก็บไว้ (ภาพหน้าจอหลายภาพในเล่มนี้มาจากตรงนี้)
55–56	visible / gone: รอจนเห็นข้อความ (หรือข้อความหายไป) ภายใน 6 วินาที
58	button: หาปุ่มจากชื่อที่โปรแกรมอ่านหน้าจออ่าน (accessibilityLabel) แอปที่ทำ accessibility ดีจึงเขียนทดสอบง่าย

ตัวอย่าง: ทดสอบแถบ "ลองปรับ" ของ FR-6

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();
584   await slider.scrollIntoViewIfNeeded();
585   // A raw mouse drag does not wait like click() does: make sure the opening animation has fully gone.
586   const introWait = Date.now();
587   await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000 }).catch(() => {});
588   const introMs = Date.now() - introWait;
589   // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590   let box = await slider.boundingBox();
591   for (let i = 0; i < 12; i++) {
592     await page.waitForTimeout(250);
593     const again = await slider.boundingBox();
594     const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x) < 0.5;
595     box = again;
596     if (still) break;
597   }
598   const under = box
599     ? await page.evaluate(({ x, y }) => {
600       const el = document.elementFromPoint(x, y);
601       return el ? `${el.tagName} role=${el.getAttribute('role')} label=${el.getAttribute('aria-label')}` :
        'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
603     : 'no box';

```

```

604     if (box) {
605         // hover() waits until the slider itself receives the pointer (nothing on top of it).
606         await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
607         await page.mouse.down();
608         await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps: 8 });
609         await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps: 8 });
610         await page.mouse.up();
611     }
612     const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613     await page.waitForTimeout(900);
614     await shot(page, '28-runway-slider');
615     check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' && (await
visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under} introWaitMs=${introMs}`));
616     // The arrow keys move it too (keyboard users on the web).
617     await slider.focus();
618     await page.keyboard.press('ArrowRight');
619     const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620     check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0) + 5),
`value=${keyed}`);
621     await page.keyboard.press('Home');
622     await page.fill('#runway-price', '2000');
623     check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ ฿2,000')) &&
(await visible(page, 'ตอนนี้')));
624     await button(page, 'ถามโค้ชเรื่องนี้').click();
625     const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626     check('Runway: "ask the coach" opens the coach with the question ready', asked.includes('฿2,000'), asked);
627     await ctx.close();

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
574–579	เปิดแท็บใหม่ ไปหน้าต้อนรับ รอแอนิเมชันเปิดแอปหาย และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แล้วรอเห็นคำว่า "ยอดคงเหลือ"
580	ไปที่หน้าเงินพอถึง
583	หาแถบเลื่อนจากชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
589–597	รอนหน้าจอหยุดขยับ (การ์ดค่อย ๆ เลื่อนเข้ามา) ไม่จิ้นจะลากพลาด
604–611	จำลองเมาส์: กดที่ปุ่มกลมของแถบ ลากไป 30% แล้ว 60% ของความกว้าง แล้วปล่อย
612–615	อ่านค่าปัจจุบันของแถบ ถ่ายภาพ และตรวจว่าได้ 30 และมีคำว่า "เพิ่มขึ้น"
617–620	ทดสอบคีย์บอร์ด: กดลูกศรขวา ค่าต้องเพิ่ม 5
622–623	พิมพ์ราคา 2000 ในเช็ก่อนจ่าย ต้องเห็น "ถ้าซื้อ ฿2,000" และ "ตอนนี้"
624–626	กด "ถามโค้ชเรื่องนี้" ช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รออยู่ (เชื่อม FR-6 กับ FR-5)

ผลล่าสุด: **145/145 ข้อผ่าน** ทั้งโหมดสว่างและมืด (รันซ้ำตอนทำเล่มนี้ วันที่ 9 ต.ค. 2569 ก็ได้ 145/145 และ unit test 157/157) และมี workflow รันชุดเดียวกันบน WebKit (เอนจินของ Safari) แบบ iPhone โหมดมืดด้วย

Manual test: ทดสอบบนมือถือจริง

บางอย่างทดสอบอัตโนมัติไม่ได้ เช่น ไมโครโฟนจริง แกลเลอรีจริง สลิปจริงจากธนาคาร ทีมจึงมีแผน MT-01 ถึง MT-12 ใน

[docs/TRACEABILITY.md](#) ให้สมาชิกทดสอบบนมือถือแล้วกรอกผล

ID	ขั้นตอน	ผลที่คาดหวัง
MT-02	จดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ"	รายการกลับมา ยอดเท่าเดิม

ID	ขั้นตอน	ผลที่คาดหวัง
MT-08	ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม"	ตอบไทย อ้าง Runway ก่อน/หลัง ไม่ดำหึ
MT-10	บัญชี A ไม่เห็นรายการของบัญชี B	RLS กันไว้
MT-11	บันทึกสลิปจริงลงแคลเลอร์ แล้วเปิดแอป	ภายในไม่กี่วินาทีขึ้น "เพิ่ม 1 รายการจากสลิปใหม่"
MT-12	พูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ บันทึกแล้วยอดลด ฿65

! ตรวจสอบนำเสนอ

ในเอกสาร Traceability ช่อง "ผล" ของ MT ทุกข้อยังเป็น ☐ (ยังไม่ได้กรอก) และในบันทึกการใช้ AI ช่อง "Human verification" ยังเป็น ☐ หลายข้อ ทีมควรทดสอบจริงและกรอกผลก่อนส่งงาน เพราะอาจารย์อาจถามว่า "ทดสอบบนมือถือจริงหรือยัง"

Traceability: ผูก Requirement → โค้ด → ทดสอบ

วิชา Software Engineering เน้น "Golden Thread" คือทุก requirement ต้องโยงไปหาโค้ดที่ทำ และเทสต์ที่พิสูจน์ได้ ทีมเราเก็บไว้ใน [docs/TRACEABILITY.md](#) เป็นตาราง 3 คอลัมน์: Acceptance criteria | Code | Test เช่น

Acceptance criteria (FR-6)	Code	Test
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16

แต่ละเล่มในภาค ข มีตารางแบบนี้ครบทุกข้อของ FR นั้น

ตรวจสอบคุณภาพอื่น ๆ

คำสั่ง	ตรวจสอบอะไร
npm run typecheck	TypeScript ตรวจสอบชนิดข้อมูลทั้งโปรเจกต์ (0 error)
npx expo lint	ESLint ตรวจสอบรูปแบบโค้ดและกฎของ React Hooks
Supabase Advisors	ตัวตรวจสอบความปลอดภัยและประสิทธิภาพของฐานข้อมูล (แจ้งเตือน 0 ปัญหาเรื่องตาราง)
ผู้ตรวจอิสระ	ตอนตรวจทั้ง repo (1–3 ต.ค.) ให้ผู้ตรวจที่ไม่ได้เห็นงานเทียบพฤติกรรมก่อน/หลังแก้ (เช่น ประโยคพูด 260 ประโยค ชื่อผู้รับ 120 ชื่อ) เจอ regression 10 ข้อ แก้ครบพร้อมเทสต์

Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ

ทุกการแก้ไขโค้ดถูกบันทึกเป็นประวัติ และทุกครั้งที่เราส่งโค้ดขึ้น GitHub จะมีหุ่นยนต์ 4 ตัวช่วยทดสอบ สร้างเว็บ สร้าง APK และส่งอัปเดตให้ผู้ใช้

Git: ระบบบันทึกประวัติ

≈ เปรียบเทียบ

Git เหมือนปุ่ม "Save game" ของเกม ทุกครั้งที่ทำสำเร็จหนึ่งด่าน (แก้ฟิเจอร์เสร็จ) เราเซฟไว้พร้อมโน้ตว่าทำอะไร ถ้าพลาดภายหลังก็โหลดเซฟเก่ากลับมาได้ และดูย้อนได้ว่าใครเปลี่ยนอะไรเมื่อไร

คำ	ความหมาย
repository (repo)	โฟลเดอร์โปรเจกต์ที่ Git ดูแล รวมประวัติทั้งหมด (ซ่อนอยู่ในโฟลเดอร์ .git)
commit	"จุดเซฟ" 1 ครั้ง มีรหัส (เช่น 646b485) ข้อความอธิบาย ผู้ทำ และเวลา
branch	"เส้นทางคู่ขนาน" ของโค้ด แยกไปทำฟิเจอร์โดยไม่กระทบเส้นหลัก เส้นหลักของทีมชื่อ main
remote / origin	สำเนาของ repo บนเซิร์ฟเวอร์ (GitHub)
push	ส่ง commit ในเครื่องขึ้น GitHub
pull	ดึง commit ใหม่จาก GitHub ลงเครื่อง
clone	คัดลอก repo ทั้งหมดจาก GitHub ลงเครื่องครั้งแรก
pull request (PR)	คำขอรวม branch หนึ่งเข้า main ให้เพื่อนรีวิวก่อน
merge conflict	สองคนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกของใคร ต้องตัดสินใจเอง
.gitignore	รายชื่อไฟล์ที่ไม่ต้องเก็บใน Git เช่น node_modules (ใหญ่และติดตั้งใหม่ได้), .env (อาจมีความลับ), dist (สร้างใหม่ได้)

ใช้ Git ประจำวัน

Terminal

1

git pull

1. ดึงของล่าสุดก่อนเริ่ม

2

git checkout -b fix-runway-text

2. แยก branch ใหม่ (ไม่บังคับ)

3

... แก้โค้ดใน VS Code ...

4

npm test

3. รันเทสต์ให้ผ่านก่อน

5

git status

4. ดูว่าแก้ไขไฟล์ไหนบ้าง

6

git add src/domain/runway.ts

5. เลือกไฟล์ที่จะเซฟ

7

git commit -m "Runway: say 'ประมาณ' in the date"

6. เซฟพร้อมข้อความ

8

git push -u origin fix-runway-text

7. ส่งขึ้น GitHub แล้วเปิด Pull Request

ใน VS Code ทำแบบเดียวกันได้ด้วยเมาส์ที่เก็บ **Source Control** (Ctrl+Shift+G): ไฟล์ที่แก้ไขขึ้นในรายการ กด + เพื่อเลือก พิมพ์ข้อความ กด Commit แล้วกด Sync Changes

★ ข้อความ commit ที่ดี

บอกว่า "ทำอะไร และทำไม" สั้น ๆ เช่นของทีมเรา: "Runway: arrow keys on the web; steadier drag test", "Audit follow-up: fix regressions found by independent review" อ่านประวัติแล้วรู้เรื่องโดยไม่ต้องเปิดโค้ด ดูประวัติทั้งหมดด้วย `git log --oneline`

ประวัติของ MindPay บน GitHub

repo อยู่ที่ `github.com/2550expo-tech/Socrates-and-Skeletons-` (สาธารณะ ใครก็อ่านได้) branch main มี 66 commit ตั้งแต่ 27 ก.ย. ถึง 3 ต.ค. 2569

วันที่	commit	งานหลักของวันนั้น (จากข้อความ commit)
27 ก.ย.	20	MVP แรก FR-1, FR-2, FR-4, FR-5, FR-6 · build APK อัตโนมัติ · เว็บบน GitHub Pages · EAS Update · AI ใช้ Gemini หรือ Claude · อ่านสลิปใหม่อัตโนมัติตอนเปิดแอป
28 ก.ย.	22	ระบบสมัครแบบรหัสอีเมล, บัญชี, สแกนเองแยกตามช่วงเวลา, แก้ปัญหาการแจ้งเตือน QR มุมขวาล่าง, เทสต์ในเบราว์เซอร์ (รวม Safari)
29 ก.ย.	19	พุดจต (FR-3), บัญชีพุดจตได้, ธีมสี 8 แบบ, เพิ่มเงินเข้า, รายการประจำ, กระปุกออม
30 ก.ย.	2	ธีมฮาโลวีน, เทสต์ความอ่านง่ายของสี
1-3 ต.ค.	3	ตรวจทั้ง repository แก้บั๊ก จัด Design System ให้ตรงกัน

GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่เรา push

CI/CD (Continuous Integration / Continuous Delivery) คือให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เรา push ของทีมเราเขียนเป็นไฟล์ YAML ใน `.github/workflows/` มี 4 ตัว

ไฟล์	ทำอะไร	ผลลัพธ์
<code>web.yml</code>	build เว็บ → unit test → E2E 145 ข้อ → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บที่ <code>2550expo-tech.github.io/Socrates-and-Skeletons-/</code>
<code>android-apk.yml</code>	unit test → typecheck → สร้างโปรเจกต์ Android → build APK (15-25 นาที)	ไฟล์ <code>mind-pay.apk</code> ในหน้า Releases "latest-apk"
<code>eas-update.yml</code>	unit test → ส่งโค้ด JavaScript ใหม่ขึ้น EAS Update	แอปที่ลงแล้วได้โค้ดใหม่ตอนเปิดครั้งถัดไป
<code>e2e-webkit.yml</code>	E2E ชุดเดียวกันบนแอนดรอยด์ Safari แบบ iPhone โหมดมืด	รายงานผล (ไม่บล็อกการเผยแพร่)

git push ขึ้น main

↓ GitHub เปิดเครื่องเสมือน Ubuntu ให้ฟรีตัวละ 1 เครื่อง ทำพร้อมกัน

web.yml

เทสต์ไม่ผ่าน = เว็บไม่เปลี่ยน ผู้ใช้ยังได้เวอร์ชันเดิมที่ติดอยู่

android-apk.yml

เทสต์หรือ typecheck ไม่ผ่าน = ไม่มี APK ใหม่

eas-update.yml

ต้องมี secret `EXPO_TOKEN` และเทสต์ผ่าน

e2e-webkit.yml

ตรวจบน Safari เพิ่ม

`.github/workflows/web.yml`

```

1  # Publishes the web version of MindPay to GitHub Pages on every push to main:
2  #   https://2550expo-tech.github.io/Socrates-and-Skeletons-/
3  # The web version shows the app at phone width, works with real accounts and
4  # demo data, and reads slips from photos the user uploads. Scanning the whole
5  # gallery is only in the phone app. On iPhone: open in Safari -> Share ->
6  # Add to Home Screen, and it opens full screen with the MindPay icon.
7  name: Deploy web app
8
9  on:
10   push:
11     branches: [main]
12   workflow_dispatch:
13
14  permissions:
15   contents: write
16
17  concurrency:
18   group: web-pages
19   cancel-in-progress: true
20
21  jobs:
22   web:
23     runs-on: ubuntu-latest
24     timeout-minutes: 20
25     steps:
26       - uses: actions/checkout@v4
27
28       - uses: actions/setup-node@v4
29         with:
30           node-version: 22
31           cache: npm
32
33       - run: npm ci
34
35       - name: App settings (public Supabase URL and publishable key)
36         run: cp .env.example .env
37
38       - name: Export web build
39         env:
40           EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
41         run: npx expo export --platform web --output-dir dist
42
43       - name: Home-screen icon and full-screen mode (iPhone "Add to Home Screen")
44         env:
45           EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
46         run: node scripts/web-home-screen.mjs dist
47
48       - name: Single-page app fallback for deep links
49         run: cp dist/index.html dist/404.html
50
51       - name: Unit tests
52         run: npm test
53
54       # Browser test of every sign-in path (sign up, email code, links, reset
55       # password, sign out) against a local stand-in for Supabase. If anything
56       # fails, the website is NOT published and the screenshots are attached.
57       - name: Install the test browser
58         run: npx playwright install --with-deps chromium
59
60       - name: Browser tests (sign-in flows)
61         run: node e2e/web-auth.mjs dist e2e-shots
62
63       - name: Keep screenshots when the browser tests fail
64         if: failure()

```

```

65     uses: actions/upload-artifact@v4
66     with:
67       name: e2e-screenshots
68       path: e2e-shots
69
70   - name: Publish to the gh-pages branch
71     uses: peaceiris/actions-gh-pages@v4
72     with:
73       github_token: ${ secrets.GITHUB_TOKEN }
74       publish_dir: ./dist
75       force_orphan: true
76       commit_message: 'Deploy web app'

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → `.github/workflows/web.yml:1`

บรรทัด ความหมาย

1-6	คอมเมนต์ (YAML ใช้ #) บอกว่าเผยแพร่เว็บไปที่ไหน และวิธีเพิ่มลงหน้าจอสโมบิล iPhone
7	ชื่อที่แสดงในแท็บ Actions ของ GitHub
9-12	เมื่อไรจะทำงาน: ทุกครั้งที่ push ขึ้น branch main และกดสั่งเองได้ (workflow_dispatch)
14-15	สิทธิ์เขียนลง repo (ต้องใช้ตอนเผยแพร่ไป branch gh-pages)
17-19	ถ้า push ใหม่ระหว่างที่ตัวเก่ายังทำงาน ยกเลิกตัวเก่า ทำแต่ตัวล่าสุด
21-24	งานชื่อ web รันบนเครื่อง Ubuntu ล่าสุด ไม่เกิน 20 นาที
26	ดึงโค้ดของ commit นี้ลงเครื่อง (uses = ใช้ action สำเร็จรูปที่คนอื่นเขียนไว้)
28-31	ติดตั้ง Node.js 22 และจำ cache ของ npm ให้ครั้งหน้าเร็วขึ้น
33	npm ci ติดตั้งแพ็คเกจตาม package-lock.json แบบเป๊ะ (ต่างจาก npm install ตรงที่ไม่เปลี่ยนเวอร์ชันเลย)
35-36	สร้าง .env จากตัวอย่าง (มีแค่กุญแจสาธารณะ)
38-41	build เว็บลงโฟลเดอร์ dist พร้อมบอกว่าเว็บอยู่ใต้ /Socrates-and-Skeletons-
43-46	เพิ่มไอคอนและตั้งค่าให้ "เพิ่มไปยังหน้าจอสโมบิล" แล้วเปิดเต็มจอเหมือนแอป
48-49	คัดลอกหน้าแรกเป็น 404.html เพราะ GitHub Pages ไม่รู้จักที่อยู่อย่าง /runway ถ้ามีคนเปิดตรง ๆ จะได้หน้า 404 ซึ่งก็คือแอปเราที่พาไปหน้านั้นให้
51-52	รัน unit test 157 ข้อ ไม่ผ่าน = หยุดทำงาน
57-61	ติดตั้งเบราว์เซอร์ทดสอบ แล้วรัน E2E 145 ข้อ
63-68	if: failure() = เฉพาะตอนพัง: เก็บภาพหน้าจอจาก E2E ไว้ให้ดาวน์โหลดดูว่าพังตรงไหน
70-76	ผ่านทุกด่านแล้วจึงเผยแพร่โฟลเดอร์ dist ขึ้น branch gh-pages ซึ่ง GitHub Pages ใช้เป็นเว็บไซต์

อ่าน android-apk.yml (ส่วนสำคัญ)

`.github/workflows/android-apk.yml`

บรรทัด 12-90

```

12 on:
13   push:
14     branches: [main]
15     # Editing documents only does not need a new APK.
16     paths-ignore: ['**/*.md', 'docs/**']
17   workflow_dispatch:
18
19 permissions:
20   contents: write

```

```

21
22 # A build that already started runs to the end; newer pushes wait in line
23 # (only the newest waiting push is kept).
24 concurrency:
25   group: android-apk
26   cancel-in-progress: false
27
28 jobs:
29   apk:
30     runs-on: ubuntu-latest
31     timeout-minutes: 45
32     steps:
33       - uses: actions/checkout@v4
34
35       - uses: actions/setup-node@v4
36         with:
37           node-version: 22
38           cache: npm
39
40       - uses: actions/setup-java@v4
41         with:
42           distribution: temurin
43           java-version: 17
44
45       - name: Install dependencies
46         run: npm ci
47
48       - name: Unit tests
49         run: npm test
50
51       - name: Typecheck
52         run: npx tsc --noEmit
53
54       - name: App settings (public Supabase URL and publishable key)
55         run: cp .env.example .env
56
57       - name: Generate the native Android project
58         run: npx expo prebuild --platform android --no-install
59
60       - name: Give Gradle enough memory
61         # The default Metaspace ran out during Android lint and the build hung until the job timeout.
62         run: |
63           sed -i '/^org.gradle.jvmargs=/d' android/gradle.properties
64           # The generated file has no trailing newline: start with one so the setting is not glued onto the
65           last line.
66           printf '\norg.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8\n' >>
67           android/gradle.properties
68           tail -n 3 android/gradle.properties
69
70       - name: Build release APK
71         working-directory: android
72         # arm64 only: covers practically every Android phone from the last 7 years and builds much faster.
73         # lintVitalAnalyzeRelease is skipped: it is a code-style check that is not needed to produce the APK.
74         run: ./gradlew assembleRelease --no-daemon -PreactNativeArchitectures=arm64-v8a -x
75         lintVitalAnalyzeRelease --stacktrace
76
77       - name: Rename APK
78         run: cp android/app/build/outputs/apk/release/app-release.apk mind-pay.apk
79
80       - uses: actions/upload-artifact@v4
81         with:
82           name: mind-pay-apk
83           path: mind-pay.apk
84
85       - name: Publish to the "latest-apk" release
86         env:
87           GH_TOKEN: ${GITHUB_TOKEN}
88         run: |

```

```
86 gh release delete latest-apk --yes --cleanup-tag || true
87 gh release create latest-apk mind-pay.apk \
88   --title "MindPay (latest APK)" \
89   --notes "Built from ${GITHUB_SHA:7} on $(date -u +'%Y-%m-%d %H:%M UTC'). Open this page on an
Android phone, tap mind-pay.apk, and install. Debug-signed build for testing." \
90   --prerelease
```

▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → .github/workflows/android-apk.yml:12

บรรทัด	ความหมาย
12–17	ทำงานเมื่อ push ขึ้น main ยกเว้นแค่เอกสาร (.md, docs/) ซึ่งไม่ต้อง build APK ใหม่
24–26	ตัวที่เริ่ม build แล้วให้ทำงานจบ ตัวใหม่ต่อคิว (build ใช้เวลานาน ไม่อยากทิ้งกลางทาง)
31	เวลาสูงสุด 45 นาที
40–43	ติดตั้ง Java 17 เพราะเครื่องมือ build ของ Android (Gradle) ต้องใช้
48–52	ด้านตรวจ: unit test และ typecheck ต้องผ่านก่อน
57–58	expo prebuild สร้างไฟล์เตอร์ android ทั้งโปรเจกต์จาก app.json (ถ้าไม่เก็บไฟล์เตอร์นี้ใน Git สร้างใหม่ทุกครั้ง)
60–66	เพิ่มหน่วยความจำให้ Gradle (คอมเมนต์แล้วว่าเคยค้างจนหมดเวลาเพราะหน่วยความจำไม่พอ)
68–72	build APK แบบ release เฉพาะสถาปัตยกรรม arm64 (ครอบคลุมมือถือแทบทุกเครื่องใน 7 ปีหลัง และ build เร็วกว่า)
74–80	เปลี่ยนชื่อเป็น mind-pay.apk และเก็บไว้
82–90	ลบ release "latest-apk" เดิม แล้วสร้างใหม่พร้อมไฟล์ APK ใหม่ (ลิงก์ดาวน์โหลดจึงเหมือนเดิมตลอด) บอกว่า build จาก commit ไหนเมื่อไร

i APK แบบ debug-signed

APK นี้ "เซ็น" ด้วยกุญแจ debug ของ Android ใช้ทดสอบและฟรีเซนต์ได้ แต่ส่งขึ้น Play Store ไม่ได้ ถ้าจะลง Play Store ต้อง build ด้วย EAS Build แบบ production และมีบัญชีนักพัฒนา Google (README อธิบายไว้)

Secrets บน GitHub

คำสั่งที่ workflow ต้องใช้ เก็บใน Settings → Secrets and variables → Actions ของ repo ไม่ได้อยู่ในโค้ด

Secret	ใช้ที่	ได้มาจาก
GITHUB_TOKEN	web.yml, android-apk.yml	GitHub สร้างให้อัตโนมัติทุกครั้ง
EXPO_TOKEN	eas-update.yml	expo.dev → Account settings → Access tokens (ถ้าไม่ตั้ง workflow นี้ข้ามไปเลย ๆ)

ส่วนกุญแจ AI ไม่ได้อยู่บน GitHub เลย อยู่ใน Supabase Secrets (un A10)

ไฟล์เอกสารใน repo

ไฟล์	มีอะไร
README.md	หน้าแรกของ repo: ลิงก์ลองใช้, FR, สถาปัตยกรรม, วิธีติดตั้ง, ตั้งค่า AI, ความปลอดภัย, ข้อจำกัด
docs/TRACEABILITY.md	FR → acceptance criteria → โค้ด → เทสต์ และแผนทดสอบมือ (MT)
docs/AI_USAGE_LOG.md	บันทึกการใช้ AI ตามที่ SRS กำหนด 20 แถว

ไฟล์	มีอะไร
docs/DESIGN_SYSTEM.md	กฎการออกแบบ (บท A11)
docs/SUPABASE_AUTH_SETUP.md	ตั้งค่าอีเมลยืนยันตัวตนของ Supabase
AGENTS.md / CLAUDE.md	คำแนะนำสำหรับผู้ช่วย AI ที่ช่วยเขียนโค้ด (ใช้ Expo 57, คำสั่งที่ควรรับ)

6 FR ทำงานร่วมกันยังไง และคำถามระดับระบบ

ไม่ว่ารับผิดชอบ FR ไหน อาจารย์อาจถามว่า "FR ของเธอเชื่อมกับของเพื่อนยังไง" หรือ "ทำไมเลือกเทคโนโลยีนี้" บทนี้เตรียมคำตอบไว้ให้

สายข้อมูลของทั้งระบบ

FR-1 จัดเอง

หน้าจอดรายการ, เพิ่มเงินเข้า

FR-3 พุดจด

"ข้าว 50 บาท" → รายการ

FR-4 สแกนสลิป

AI อ่านสลิป → รายการหรือแบบร่าง

↓ ทุกทางจบที่ addTx / upsertLocal → ตาราง transactions

รายการทั้งหมด (txs) ใน AppProvider

แหล่งความจริงเดียว นับเฉพาะ status = confirmed

↓ useMoney() คำนวณใหม่ทุกครั้งที่ txs เปลี่ยน

FR-2 ภาพรวม

ยอดคงเหลือ (computeBalance), รายรับ รายจ่าย ตามหมวด ตามช่วงเวลา

FR-6 Money Runway

ใช้ยอดคงเหลือจาก FR-2 + ค่าเฉลี่ย 7 วัน → เงินพอใช้อีกกี่วัน

↓ buildCoachContext ส่งตัวเลขสรุป

FR-5 โค้ชน้องกล้า

อธิบายตัวเลขของ FR-2 และ FR-6 เป็นภาษาคน พุดออกเสียง

ใครใช้ของใคร

FR	ใช้ข้อมูล/ฟังก์ชันจาก	ส่งผลต่อ
FR-1	money.ts (แปลงจำนวนเงิน), categories.ts (หมวด), dates.ts	ทุกรายการที่จัด/แก้/ลบ ทำให้ FR-2, FR-6, FR-5 คำนวณใหม่; หน้าจอดรายการยังเป็นหน้าตรวจสอบสลิปของ FR-4
FR-2	รายการจาก FR-1/3/4, ยอดตั้งต้นจากโปรไฟล์	ยอดคงเหลือเป็นตัวตั้งของ FR-6, ยอดตามหมวดส่งให้โค้ช FR-5
FR-3	categories.ts (เดาหมวด), money/dates, addTx ของ FR-1	สร้างรายการแบบยืนยันแล้ว เข้ายอดทันที
FR-4	Edge Function parse-slip + AI, หน้าจอดรายการของ FR-1 (ตรวจแบบร่าง), categories.ts	รายการจากสลิป: ชัด → เข้ายอด, ไม่ชัด → แบบร่าง (ไม่นับใน FR-2/FR-6 จนยืนยัน)
FR-5	ตัวเลขจาก FR-2 และ FR-6, Edge Function coach + AI	ไม่เปลี่ยนข้อมูล (อ่านอย่างเดียว) แต่ชวนผู้ใช้ไปหน้า Runway/สแกน
FR-6	computeBalance (FR-2), รายจ่ายที่ยืนยันแล้ว (FR-1/3/4), เส้นสำรองจากโปรไฟล์, กระปุกออม	ตัวเลข Runway บนหน้าหลัก, การ์ดเตือนของโค้ช, "เช็กก่อนจ่าย" ส่งคำถามให้ FR-5

ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)

ไฟล์	ใครใช้	ถ้าแก้ผิดจะเกิดอะไร
<code>src/domain/types.ts</code>	ทุก FR	TypeScript จะแจ้ง error ทั่วทั้งแอป
<code>src/domain/money.ts</code>	FR-1, 3, 4, 6	จำนวนเงินเพี้ยนทุกที่

ไฟล์	ใครใช้	ถ้าเกิดผิดพลาดอะไร
<code>src/domain/dates.ts</code>	ทุก FR	รายการไปอยู่ผิดวัน ค่าเฉลี่ย 7 วันผิด
<code>src/domain/summary.ts</code>	FR-2, 5, 6	ยอดคงเหลือผิด → Runway และโค้ชผิดตาม
<code>src/domain/categories.ts</code>	FR-1, 2, 3, 4	หมวดผิด กราฟตามหมวดผิด
<code>src/data/AppProvider.tsx</code>	ทุกหน้า	ข้อมูลไม่โหลด/ไม่อัปเดต
<code>src/app/transaction.tsx</code>	FR-1 และ FR-4 (ตรวจสอบ)	จดยรายการหรือยืนยันสลิปไม่ได้

นี่คือเหตุผลที่ทีมมี unit test ครอบคลุมไฟล์เหล่านี้ทั้งหมด และให้ GitHub รับเทสก่อนปล่อยทุกครั้ง

คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้

Q1	ทำไมเลือก Expo / React Native? โค้ดชุดเดียวได้ทั้ง Android, iPhone และเว็บ ทีมเล็กเวลาน้อยจึงไม่ต้องเขียน 3 รอบ ภาษาเดียว (TypeScript) ทั้งแอปและเซิร์ฟเวอร์ มีเครื่องมือ build APK และส่งอัปเดตอัตโนมัติ
Q2	ทำไมเลือก Supabase? ได้ฐานข้อมูล PostgreSQL, ระบบสมัคร/เข้าสู่ระบบ, และที่รับโค้ดฝั่งเซิร์ฟเวอร์ในทีเดียว มีแพ็คเกจฟรี และ Row Level Security ทำให้แอปคุยกับฐานข้อมูลได้ตรง ๆ อย่างปลอดภัยโดยไม่ต้องเขียนเซิร์ฟเวอร์ API เอง
Q3	ข้อมูลผู้ใช้ปลอดภัยยัง? ทุกตารางเปิด RLS แต่ละคนอ่านเขียนได้เฉพาะแถวของตัวเอง (ทดสอบ MT-10) คุกกี้ในแอปเป็นคุกกี้สาธารณะ คุกกี้แอสและคุกกี้แอส AI อยู่บนเซิร์ฟเวอร์เท่านั้น รูปสลิปไม่ถูกเก็บบนเซิร์ฟเวอร์ โค้ชได้แค่ยอดรวม
Q4	ทำไมเก็บเงินเป็นสตางค์? ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ ($0.1 + 0.2 = 0.30000000000000004$) เก็บเป็นจำนวนเต็มสตางค์จึงบวกลบได้แม่นยำ 100% แปลงเป็นบาทตอนแสดงผลเท่านั้น
Q5	ทำไมต้องจัดวันตามเวลาไทย? ฐานข้อมูลเก็บเวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าไม่แปลง รายการตอนเช้ามีดจะไปอยู่วันก่อนหน้า ยอด "วันนี้" และค่าเฉลี่ย 7 วันจะผิด ทุกจุดจึงใช้ bkkDayKey
Q6	ยอดคงเหลือเก็บไว้ในฐานข้อมูลไหม? ไม่ เก็บเฉพาะรายการ ตัวเลขทั้งหมด (ยอด ค่าเฉลี่ย Runway) คำนวณสดในเครื่องจากรายการทุกครั้งที่มีข้อมูลเปลี่ยน ตัวเลขจึงไม่มีทางขัดกันเอง
Q7	แบบร่าง (draft) คืออะไร ทำไมต้องมี? รายการจากสลิปที่ AI อ่านไม่แน่ใจ รอผู้ใช้ตรวจสอบก่อน ยังไม่นับในยอดใด ๆ เพราะถ้า AI อ่านผิดแล้วนับทันที ทุกตัวเลขในแอปจะผิดตาม
Q8	ใช้ AI ตรงไหน และป้องกัน AI ผิดยัง? 2 ที่: อ่านสลิป (FR-4) และโค้ช (FR-5) ป้องกันด้วย: อ่าน 2 ครั้งด้วย 2 รุ่น, เกณฑ์ความมั่นใจ 80%, ตรวจสอบ QR บนสลิป, แบบร่างรอตรวจ, โค้ชห้ามแต่งตัวเลข ตัวเลขทั้งหมดคำนวณโดยโค้ดที่ทดสอบแล้ว
Q9	ทำไมไม่เรียก AI จากแอปตรง ๆ? คุกกี้แอส AI จะหลุดไปกับแอป Edge Function ถือคุกกี้แอสไว้ ตรวจสอบว่าเข้าสู่ระบบจริง และจำกัดโควตาต่อวันต่อคน
Q10	ทดสอบยัง? 3 ระดับ: unit test 157 ข้อ (ฟังก์ชันคำนวณ), E2E 145 ข้อ (เปิดเว็บจริงเหมือนคน ด้วยเซิร์ฟเวอร์ปลอม), manual test MT-01 ถึง MT-12 บนมือถือจริง ทุกครั้งที่ push GitHub รับเทสก่อนเผยแพร่
Q11	ถ้าอินเทอร์เน็ตหลุดระหว่างพิธีเซนต์? ใช้โหมดทดลอง ข้อมูลอยู่ในเครื่อง ทำงานได้ทุกอย่างยกเว้น AI
Q12	Requirement โยงกับโค้ดยัง? เอกสาร docs/TRACEABILITY.md ผูกทุก acceptance criteria ของทุก FR กับไฟล์/ฟังก์ชันที่ทำ และรหัสเทส (TC/MT) ที่พิสูจน์

Q13 ใช้ AI ช่วยเขียนโค้ดไหม?

ใช้ บันทึกไว้ใน docs/AI_USAGE_LOG.md ตามที่ SRS กำหนด ทุกถวบบอกจุดประสงค์ ผลลัพธ์ สิ่งที่มีแกหรือตัดสินใจ และสิ่งที่สมาชิกต้องตรวจสอบ (ควรตอบตามจริงว่าตัวเองตรวจสอบอะไรไปแล้วบ้าง)

Q14 ถ้ามีผู้ใช้เยอะขึ้น ระบบรับไหวไหม?

การคำนวณทำในเครื่องของแต่ละคน เซิร์ฟเวอร์แค่เก็บข้อมูล มี index ช่วยค้นเร็ว และโหลดเป็นหน้า หน้าละ 1,000 แถว ส่วนที่ต้องระวังคือค่าใช้จ่าย AI ซึ่งมีโควตาต่อวันคุมไว้ และควรเปลี่ยนจาก Gemini แบบฟรีเป็นแบบเสียเงินเมื่อมีผู้ใช้จริง

Q15 อะไรที่ยังไม่ได้ทำ (นอก MVP)?

Accountability Mode (UC-7), ออฟไลน์เต็มระบบ (UC-8), เชื่อม Gmail/LINE, รวบรวมรับในอนาคตเข้า Money Runway และยังไม่ได้ทดสอบบนมือถือหลายรุ่น

คำศัพท์รวม

คำ	ความหมายแบบง่าย
Acceptance criteria	เงื่อนไขที่ต้องผ่าน จึงจะถือว่า requirement นั้นเสร็จ
API / API key	ประตูให้โปรแกรมคุยกัน / รหัสลับที่ใช้ผ่านประตู
APK	ไฟล์ติดตั้งแอป Android
async / await	เขียนงานที่ต้องรอ (เช่นอินเทอร์เน็ต) โดยหน้าจอไม่ค้าง
CI/CD	ให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เกิดเปลี่ยน (GitHub Actions)
Commit / Push / Branch	จุดเชฟ / ส่งขึ้น GitHub / เส้นทางคู่ขนานของโค้ด
Component	ชิ้นส่วนหน้าจอใน React เป็นฟังก์ชันที่คืน JSX
Context / Provider	วิธีแชร์ข้อมูลให้ทุกหน้าใน React (AppProvider)
Design System / Token	กฎและชิ้นส่วนกลางของการออกแบบ / ค่ากลางที่ตั้งชื่อตามหน้าที่ เช่น theme.ink
E2E test	ทดสอบทั้งแอปเหมือนคนกดจริงในเบราว์เซอร์
Edge Function	โค้ดฝั่งเซิร์ฟเวอร์บน Supabase (parse-slip, coach)
EAS Update / OTA	ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้ว โดยไม่ต้องลง APK ใหม่
Expo Router	ระบบเปลี่ยนหน้าที่ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า
Hook	ฟังก์ชันพิเศษของ React ขึ้นต้นด้วย use เช่น useState
JSON	รูปแบบข้อมูลข้อความที่โปรแกรมใช้รับส่งกัน
JSX	หน้าที่เขียนคล้าย HTML ในโค้ด React
Migration	ไฟล์ SQL ที่สร้าง/แก้ไขโครงสร้างฐานข้อมูล เรียงตามวันที่
Native module	ส่วนที่เขียนด้วยภาษาของมือถือ (เช่น โมดูล แกลเลอรี) ต้องลง APK ใหม่ถึงจะมี
props / state	ค่าที่ส่งเข้า component / ความจำของ component
Pure function	ฟังก์ชันที่ใส่ค่าเดิมได้ผลเดิม ไม่ยุ่งกับข้างนอก ทดสอบง่าย
Regression	ของที่เคยทำงาน กลับพังหลังแก้ไขโค้ดส่วนอื่น
Repo (Repository)	1) โพลเดอร์โปรเจกต์ใน Git 2) ชั้น Repo ในแอปที่ซ่อนว่าข้อมูลเก็บที่ไหน
RLS	Row Level Security กฎในฐานข้อมูลให้แต่ละคนเห็นเฉพาะแถวของตัวเอง

คำ	ความหมายแบบง่าย
Satang	หน่วยเงินในระบบ 100 สตางค์ = 1 บาท
Schema (JSON)	แบบฟอร์มบังคับโครงสร้างคำตอบของ AI
Session / Token	สถานะการเข้าสู่ระบบ / บัตรผ่านดิจิทัลที่แนบไปกับทุกคำขอ
SQL	ภาษาสั่งฐานข้อมูล
System prompt	คำสั่งประจำตัวที่บอก AI ว่าต้องเป็นใคร ตอบอย่างไร
Traceability	การโยง requirement → การออกแบบ → โค้ด → เทสต์
TypeScript	JavaScript ที่มีชนิดข้อมูล ช่วยจับผิดตั้งแต่ตอนเขียน
Unit test	ทดสอบฟังก์ชันทีละตัว
WCAG	มาตรฐานความเข้าถึงได้ของเว็บ เช่น ความต่างสีตัวหนังสือ $\geq 4.5:1$

ภาค ข

FR-6 Money Runway

เงินพอถึงวันไหน ภาคนี้เขียนในมุมมองของคนสร้าง FR-6 เอง ตั้งแต่ปัญหาที่อยากแก้ หน้า
จอ หลักการคำนวณ โค้ดจริงที่ละบรรทัด การทดสอบ ไปจนถึงคำถามที่อาจารย์น่าจะถาม

วิธีศึกษา FR-6 ให้เข้าใจจริง

เล่มนี้ละเอียดมาก ไม่ต้องอ่านรวดเดียว บทนี้บอกว่าควรอ่านอะไรก่อนหลัง และรู้ได้ยังงัยว่าเข้าใจแล้ว

ภาค ข ของเล่มนี้เขียนจากมุมมองของฉัน คนที่รับผิดชอบ FR-6 Money Runway ทุกตัวเลขในเล่มตรวจด้วยฟังก์ชันจริงของแอปแล้ว และทุกโค้ดดึงมาจากไฟล์จริง และเปิดใน VS Code ได้ ถ้าเพื่อนต้องช่วยตอบคำถามเรื่อง FR-6 หรือต้องนำเสนอแทนฉัน ให้เริ่มจากบทนี้

เป้าหมาย: ตอบ 5 คำถามนี้ได้ โดยไม่ต้องเปิดเล่ม

- 1 **FR-6 ทำอะไร**

แปลงยอดเงินให้เป็น "จำนวนวัน" และ "วันที่ในปฏิทิน" ว่าถ้ายังใช้เงินแบบ 7 วันที่ผ่านมา เงินจะพอถึงวันไหน เช่น "เงินพอใช้อีก 20 วัน · ถึง 28 ต.ค." (อ่าน: FR-6 ในหน้าเดียว, ปัญหาที่ฉันอยากแก้)
- 2 **สูตรคืออะไร**

ปิดลง [(ยอดคงเหลือ - เงินสำรอง - เงินในกระปุกออม) ÷ ค่าเฉลี่ยรายจ่ายต่อวัน] แล้ววันที่ = วันนี้ + จำนวนวัน (อ่าน: วัตถุประสงค์ 4 อย่าง, สูตรที่ละขั้น)
- 3 **ค่าเฉลี่ยคิดยังไง**

รายจ่ายที่ยืนยันแล้วของวันนี้ + 6 วันก่อน (เวลาไทย)หารด้วย 7 วันในปฏิทิน นับวันที่ไม่ได้ใช้เงินด้วย ผู้ใช้ใหม่หารด้วยจำนวนวันที่ใช้แอปมาจริง (อ่าน: ค่าเฉลี่ยรายจ่ายต่อวัน)
- 4 **มีสถานะอะไรบ้าง**

น้อยกว่า 7 วัน ใกล้เส้นสำรอง (critical) · 7-13 วัน เริ่มต้องระวัง (watch) · 14 วันขึ้นไป สบาย ๆ (healthy) · ยอด ≤ เส้นสำรอง ต่ำกว่าเงินสำรอง (below_floor) · ไม่มีรายจ่ายเลย ยังคำนวณไม่ได้ (no_spending) (อ่าน: สถานะ 5 แบบ และกรณีพิเศษ)
- 5 **รู้ได้ยังงัยว่าถูก**

unit test TC-12 ถึง TC-18 ตรวจทุกเงื่อนไข รวมขอบเขตที่ 6/7/14 วัน และ E2E ที่เปิดเบราว์เซอร์จริงแล้วลากแถบ ใส่ราคา และกดถามได้ชั่งโปรเจกต์ผ่าน 157/157 (อ่าน: การทดสอบ FR-6)

แผนอ่าน 3 ระดับ

ระดับ	เวลา	อ่านบทไหน	ผ่านเมื่อ
1 · เข้าใจภาพรวม สำหรับเพื่อนทุกคนในกลุ่ม	30 นาที	บท A1, A2 แล้วภาค ข: FR-6 ในหน้าเดียว · ปัญหาที่ฉันอยากแก้ · สูตรที่ละขั้น · ทวีร์หน้าจอ 2 บท · สคริปต์นำเสนอ 1 นาที	ตอบ 5 คำถามข้างบนได้ และเปิดแอปแล้วชี้ได้ว่า Runway อยู่ตรงไหน
2 · คิดเลขเองได้ สำหรับคนที่ต้องตอบคำถามอาจารย์	2 ชั่วโมง	ภาค ข ตั้งแต่ต้นจนถึง "แผนที่โค้ดของ FR-6" + ทำโมเดลออกแบบแบบนี้ · ข้อจำกัด · คำถามที่อาจารย์อาจถาม	คิด Runway จากตัวเลขใหม่ด้วยเครื่องคิดเลขได้ถูก และอธิบายเหตุผลของ 7 วัน ปิดลง และเส้นสำรองได้
3 · อ่านโค้ดออก สำหรับคนนำเสนอหรืออยากเข้าใจลึก	ครึ่งวัน	ภาค ก บท A3-A8 แล้วอ่านภาค ข ทุกบทเรียงตามลำดับ เปิด VS Code คู่กัน ลองรันเทสท์ในบท "ใส่โค้ดแบบคอมพิวเตอร์" และทำ mini-runway ในบท "สร้าง FR-6 ตั้งแต่ศูนย์"	ทำแบบฝึกหัดท้ายเล่มได้โดยไม่ดูเฉลย และซักรรทัดในโค้ดที่ทำแต่ละกฎได้

ตัวอย่าง 2 ชุดที่ใช้ทั้งเล่ม (อย่าสับสน)

เล่มนี้ใช้ตัวเลข 2 ชุด แยกให้ออกจะอ่านง่ายขึ้นมาก:

★ ชุดที่ 1 • ตัวอย่างคิดมือ (วันนี้ 8 ต.ค.)

ยอด **฿3,500** • เงินสำรอง **฿500** • ใช้เฉลี่ยวันละ **฿145** (7 วัน รวม **฿1,015**)

→ ใช้ได้จริง $฿3,000 \div 145 = 20.69 \rightarrow$ **20 วัน • ถึง 28**

ต.ค. • สบาย ๆ

ใช้ในบทหลักการ สูตร การไล่โค้ด และสไลด์ส่วนสูตร ตัวเลข กลม ๆ คิดตามง่าย

i ชุดที่ 2 • ภาพหน้าจอจริง (โหมดทดลอง 9 ต.ค.)

ยอด **฿11,359** • เงินสำรอง **฿500** • ใช้เฉลี่ยวันละ **฿660**

→ ใช้ได้จริง $฿10,859 \div 660 = 16.45 \rightarrow$ **16 วัน • ถึง 25**

ต.ค. • สบาย ๆ

ใช้ในบททวิร์หน้าจอ และสไลด์ส่วนหน้าจอ เป็นข้อมูลตัวอย่างที่ แอปสร้างให้ เปิดวันอื่นตัวเลขจะต่างไป

แผนที่: สไลด์หน้าไหน อ่านเพิ่มบทไหน

จัดทำสไลด์นำเสนอ FR-6 ไว้ 25 หน้า (วิธีใช้อยู่ในบทสุดท้าย "นำเสนอ FR-6 ด้วยสไลด์ของจับ") ตารางนี้โยงสไลด์แต่ละหน้ากับบทที่อธิบายเรื่องเดียวกันแบบละเอียด ถ้าอยากเข้าใจสไลด์หน้าไหนลึกขึ้น ให้เปิดบทในคอลัมน์ขวา

สไลด์	เรื่อง	อ่านละเอียดในบท
1–2	หน้าปก • วันนี้จะเล่า 5 เรื่อง	FR-6 ในหน้าเดียว
3–4	฿3,500 เยอะหรือน้อย • แนวคิดหน้าปิดรถยนต์	ปัญหาที่จับอยากแก้
5	Requirement และนิยาม 5 ข้อ	Requirement นิยาม และเงื่อนไขผ่าน
6	ตัวเลข 4 ตัวที่ใช้คำนวณ	วัตถุดิบ 4 อย่างของการคำนวณ
7–8	ค่าเฉลี่ย 7 วัน • กฎผู้ใช้ใหม่	ค่าเฉลี่ยรายจ่ายต่อวัน
9–10	สูตร 5 ขัน • ทำไม่ปิดลง	สูตร Money Runway ที่ละชั้น
11	5 สถานะ	สถานะ 5 แบบ และกรณีพิเศษ
12	หน้าหลัก	ทวิร์หน้าจอ: หน้าหลัก • อ่านโค้ด: การ์ดบนหน้าหลัก และ useMoney
13–14	เก็บเงินพอถึง • เส้นเวลา 30 วัน	ทวิร์หน้าจอ: เก็บ "เงินพอถึง" • อ่านโค้ด: หน้าจอเก็บเงินพอถึง
15–16	ลองปรับ • เช็ก่อนจ่าย	เครื่องมือ "ลองปรับ" "เช็ก่อนจ่าย" และ "ถามโค้ช" • อ่านโค้ด: แถบเลื่อน ต้นไม้เงิน และสิ่งที่ส่งให้โค้ช
17–18	เงินสำรองและกระปุกออม • สถานะบนหน้าจอจริง	เงินสำรอง และกระปุกออม • อ่านโค้ด: กระปุกออม
19	ข้อมูลไหลยังไร	เบื้องหลัง: ข้อมูลไหลยังไร • แผนที่โค้ดของ FR-6
20	โค้ดหัวใจ computeRunway	อ่านโค้ด: runway.ts ทั้งไฟล์ • ไล่โค้ดแบบคอมพิวเตอร์ • พจนานุกรมสัญลักษณ์
21	กรณีพิเศษ	สถานะ 5 แบบ และกรณีพิเศษ
22	การทดสอบ	การทดสอบ FR-6 • สร้าง FR-6 ตั้งแต่ศูนย์ (ขั้นที่ 2, 3, 7)
23	เหตุผลการออกแบบ	ทำไมเงินออกแบบแบบนี้
24	ข้อจำกัดและสิ่งที่จะทำต่อ	ข้อจำกัด และสิ่งที่จะทำต่อ
25	สรุป 3 ประโยค • ถาม-ตอบ	คำถามที่อาจารย์อาจถาม • สคริปต์นำเสนอ 1 นาที

ทริกอ่านโค้ดใน VS Code ที่ใช้บ่อยที่สุด

un A3 และ A8 สอนวิธีใช้ VS Code ตั้งแต่ศูนย์ ส่วนนี้คือปุ่มลัด 7 ตัวที่ฉันใช้ตลอดตอนเขียน FR-6 (บน Mac ใช้ Cmd แทน Ctrl และใช้ Ctrl+ – แทน Alt+←)

อยากทำ	กด	ตัวอย่างกับ FR-6
เปิดไฟล์จากชื่อ และไปบรรทัดที่ต้องการ	Ctrl+P	พิมพ์ runway.ts:72 แล้ว Enter → ไปที่ computeRunway
กระโดดไปที่ฟังก์ชันถูกสร้าง	F12 หรือ Ctrl+คลิก	คลิกที่ bkkDayKey ในบรรทัด 37 → เปิด dates.ts ให้เลย
กลับไปที่เดิม	Alt+←	กลับมาบรรทัด 37 ของ runway.ts
ดูว่าใครเรียกฟังก์ชันนี้บ้าง	Shift+F12	วางเคอร์เซอร์ที่ computeRunway → เห็นว่าในแอปถูกเรียกที่ AppProvider จุดเดียว นอกนั้นคือใน runway.ts เอง (เช็กก่อนจ่าย ลองปรับ) และในไฟล์เกสต์
ดูชนิดข้อมูล	เอาเมาส์ชี้ค้าง	ชี้ที่ days → เห็น number null
ค้นคำในทั้งโปรเจกต์	Ctrl+Shift+F	ค้น เงินพอใช้อีก → เจอหน้าหลัก เก็บเงินพอถึง สรุปรเดือน และคำพูดของน้องกล้า (buddy.ts)
เปิด Terminal ด้านล่าง	Ctrl+`	พิมพ์ npm test

i VS Code บนเว็บ (vscode.dev)

ลิงก์ "เปิดใน VS Code (เว็บ)" ใต้โค้ดทุกก้อนเปิดได้ทันทีโดยไม่ต้องติดตั้งอะไร เหมาะกับการอ่าน ปุ่ม Ctrl+P, Ctrl+Shift+F และการชี้ดูใช้ได้ แต่การกระโดดข้ามไฟล์ (F12) และการรันคำสั่ง (Terminal) ต้องใช้ VS Code ในเครื่องที่โคลนโปรเจกต์แล้ว (un A3)

วิธีอ่านแต่ละบทให้ได้ผล

- 1 อ่านหัวบทและประโยคใต้หัวบทก่อน จะรู้ว่บทนี้ตอบคำถามอะไร
- 2 คิดตัวอย่างตามด้วยเครื่องคิดเลข ทุกครั้งที่เห็นตัวเลข ถ้าคิดแล้วไม่ตรง ให้หยุดหาว่าพลาดตรงไหน ตรงนั้นคือจุดที่ยังไม่เข้าใจ
- 3 อ่านโค้ดคู่กับตาราง "อ่านทีละบรรทัด" อ่านบรรทัดในโค้ดก่อน เดาความหมายเอง แล้วค่อยดูคำอธิบาย
- 4 เล่าให้คนอื่นฟัง ถ้าอธิบายให้เพื่อนที่ไม่ได้เรียนเรื่องนี้เข้าใจได้ภายใน 1 นาที แปลว่าเข้าใจจริง

FR-6 ในหน้าเดียว

ถ้ามีคนถามว่า "FR-6 ของเธอทำอะไร" ให้ตอบจากหน้านี้

FR-6 **Money Runway** คือส่วนของ MindPay ที่ตอบคำถามเดียวให้ชัดที่สุด: "ถ้าฉันยังใช้เงินแบบช่วงที่ผ่านมา เงินจะพอใช้อีกกี่วัน และหมดถึงวันไหน" จนไม่ได้ให้แอปบอกแค่ยอดเงินคงเหลือ แต่แปลงยอดเงินให้เป็น "จำนวนวัน" และ "วันที่ในปฏิทิน" เพราะคนเข้าใจเวลาได้ง่ายกว่าตัวเลขเงินก้อนหนึ่ง

เงินพอใช้อีก (วัน) = ปัดลง [เงินที่ใช้ได้จริง ÷ ค่าเฉลี่ยรายจ่ายต่อวัน]

เงินที่ใช้ได้จริง = ยอดคงเหลือ - เงินสำรอง - เงินในกระปุกออม · ค่าเฉลี่ย = รายจ่ายที่ยืนยันแล้ว 7 วันล่าสุด ÷ 7 (ผู้ใช้ใหม่หารด้วยจำนวนวันที่ใช้มา)
· วันที่เงินพอถึง = วันนี้ + จำนวนวัน

ตัวอย่างสั้น ๆ (วันนี้ 8 ต.ค. 2569)

฿3,500

ยอดคงเหลือ

- ฿500

เงินสำรอง (ค่าเริ่มต้น)

= ฿3,000

ใช้ได้จริง

÷ ใช้เฉลี่ยวันละ ฿145 (7 วันล่าสุด) = 20.69 → ปัดลง

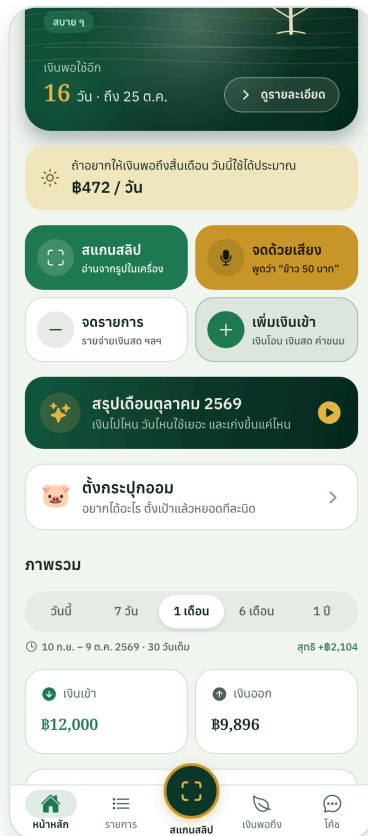
เงินพอใช้อีก 20 วัน · ถึง 28 ต.ค. 2569 · สถานะ "สบาย ๆ"

อยู่ตรงไหนในแอป

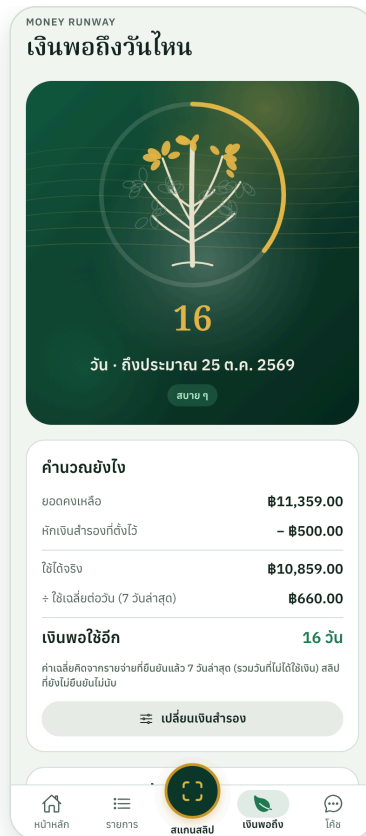
ที่	ผู้ใช้เห็นอะไร
การ์ดเขียวบน หน้าหลัก	"เงินพอใช้อีก 16 วัน · ถึง 25 ต.ค." ป้ายสถานะ ตันไม้เงิน และการ์ด "ถ้าอยากให้เงินพอถึงสิ้นเดือน วันนี้ใช้ได้ประมาณ ฿.../วัน"
แท็บ เงินพอถึง (ไอคอน ใบไม้)	ตัวเลขใหญ่ในวงแหวน การ์ด "คำนวณยังไง" เส้นเวลา 30 วัน แถบ "ลองปรับ" และ "เช็กก่อนจ่าย"
แท็บ โค้ช (น้องกล่า)	การ์ด "สิ่งที่ควรรู้ออนนี้" เทียบเรื่อง Runway ก่อนเรื่องอื่น และโค้ช AI ได้ตัวเลข Runway ไปตอบคำถาม "ซื้อได้ไหม"
ตั้งค่า	ช่อง "เงินสำรองขั้นต่ำ (บาท)" ที่ผู้ใช้ตั้งเองได้
กระปุกออม	เงินที่หยอดไว้ถูกกันออกจากเงินที่ใช้ได้

3 ข้อที่ต้องจำ

1. นับเฉพาะรายการที่ยืนยันแล้ว สลิปที่ AI อ่านแล้วแต่ผู้ใช้ยังไม่กดยืนยัน (แบบร่าง) ยังไม่นับ
2. ปัดลงเสมอ 20.69 วันแสดงเป็น 20 วัน เพราะบอกน้อยไว้ปลอดภัยกว่าบอกเกินจริง
3. คำนวณใหม่ตลอด ทุกครั้งที่ข้อมูลเปลี่ยน และทุกเที่ยงคืนตามเวลาไทย ตัวเลข Runway ไม่ได้เก็บไว้ในฐานข้อมูล แต่คิดสด ๆ ในเครื่อง



การ์ดเขียวบนหน้าหลัก: เงินพอใช้อีก 16 วัน
· ถึง 25 ต.ค.



แท็บเงินพอถึง: ตัวเลขใหญ่ในวงแหวนทอง

ปัญหาที่ฉันอยากแก้

ยอดเงิน ฿3,500 ถือว่าเยอะหรือน้อย? คำตอบคือ "ขึ้นอยู่กับว่าใช้เร็วแค่ไหน"

นักศึกษาส่วนใหญ่ได้เงินเป็นก้อน เช่น ค่าขนมรายเดือนหรือค่าจ้างพิเศษ แล้วค่อย ๆ ใช้ไปทุกวัน เวลาเปิดแอปธนาคารจะเห็นแค่ยอดเงิน เช่น ฿3,500 ตัวเลขนี้ไม่ได้บอกเลยว่าอยู่รอดถึงวันที่เงินก้อนต่อไปเข้าหรือเปล่า

เงินเหลือเล่นสำรอง ฿3,000 ถ้าใช้วันละ	เงินพอใช้อีก
฿100	30 วัน
฿145	20 วัน
฿300	10 วัน
฿500	6 วัน

เงินก้อนเดียวกัน แต่ความหมายต่างกันมาก ฉันจึงออกแบบ FR-6 ให้แปลง "เงิน" เป็น "เวลา"

≈ เปรียบเทียบ: หน้าปิดรถยนต์

รถยนต์รุ่นใหม่ไม่ได้บอกแค่น้ำมันเหลือกี่ลิตร แต่บอกว่า "วิ่งได้อีก 120 กม." โดยเอน้ำมันที่เหลือหารด้วยอัตรากินน้ำมันช่วงหลัง Money Runway ทำแบบเดียวกันกับเงิน

รถยนต์	MindPay FR-6
น้ำมันที่เหลือในถัง	ยอดคงเหลือ
น้ำมันสำรองตอนไฟเตือนขึ้น (ไม่อยากให้ถึงศูนย์)	เงินสำรองขั้นต่ำ (ค่าเริ่มต้น ฿500)
อัตรากินน้ำมันช่วงหลัง	ค่าเฉลี่ยรายจ่ายต่อวัน 7 วันล่าสุด
"วิ่งได้อีก 120 กม."	"เงินพอใช้อีก 20 วัน · ถึง 28 ต.ค."
ขับประหยัดขึ้น ระยะทางเพิ่ม	ใช้น้อยลง จำนวนวันเพิ่ม (แถบ "ลองปรับ")

ทำไมใช้ชื่อ "Runway"

Runway แปลว่ารันเวย์สนามบิน คือระยะทางที่เหลือให้เครื่องบินวิ่งก่อนสุดทาง ในโลกธุรกิจมีคำว่า **cash runway** หมายถึงบริษัทมีเงินพอจ่ายอีกกี่เดือนก่อนเงินหมด ฉันเอาแนวคิดนี้มาย่อให้เป็น "รายวัน" สำหรับคนทั่วไป ในแอปภาษาไทยใช้ชื่อเก็บว่า "เงินพอถึง" และหัวข้อหน้าจอบอกว่า "เงินพอถึงวันไหน" ให้เข้าใจกันโดยไม่ต้องรู้ศัพท์อังกฤษ

เป้าหมายที่ฉันตั้งไว้ตอนออกแบบ

1. ตัวเลขเดียว เข้าใจทันที เปิดแอปปั๊บเห็น "อีก 16 วัน" ไม่ต้องคิดเลขเอง
2. เป็นวันที่ในปฏิทิน "ถึง 25 ต.ค." เอาไปเทียบกับวันเงินเข้าได้เลย
3. เตือนก่อนเงินหมด บังคับเส้นสำรอง ไม่ใช่บังคับศูนย์ และมีสถานะเตือนล่วงหน้า 7 และ 14 วัน
4. ไม่ดู ไม่ทำให้รู้สึกผิด ข้อความใช้คำสุภาพ เช่น "ค่อย ๆ ไปด้วยกัน"
5. ให้ลองคิดล่วงหน้าได้ "ถ้าใช้น้อยลง 20%" หรือ "ถ้าซื้อของ ฿2,000" จะเหลือกี่วัน ก่อนตัดสินใจจริง
6. อธิบายตัวเองได้ มีการ์ด "คำนวณยังไง" ให้ผู้ใช้ตรวจสอบเลขเองได้ ไม่ใช่กล่องดำ

Requirement นิยาม และเงื่อนไขผ่าน

โจทย์ใน SRS สั้นมาก หน้าทีของจับคือทำให้มัน "วัดได้" และ "ทดสอบได้"

1 โจทย์ FR-6 ใน SRS

"ประมาณวันที่ยอดเงินอาจต่ำ จากค่าเฉลี่ยรายจ่าย 7 วัน"

Feedback ที่ทีมได้รับคือคำว่า "Low balance" (เงินต่ำ) ยังกำกวม ต่ำแค่ไหนถึงเรียกว่าต่ำ ศูนย์บาท? ร้อยบาท? ถ้าไม่มีนิยามให้ชัด จะเขียนเทสไม่ได้ว่าถูกหรือผิด ฉะนั้นจึงเขียนนิยาม 5 ข้อไว้ในคอมเมนต์บนสุดของ [src/domain/runway.ts](#) และใน [docs/TRACEABILITY.md](#) ให้ตรงกัน

src/domain/runway.ts

บรรทัด 1-18

```
1 /**
2  * FR-6 Money Runway: "how many more days will my money last?"
3  *
4  * Definitions (these are the testable meanings asked for in the SRS feedback):
5  *
6  * - LOW LINE (runway floor): an amount the user sets, default ฿500.
7  *   "Low balance" means balance <= floor. It is a line to stay above,
8  *   not zero, so the user still has a buffer for emergencies.
9  * - AVERAGE DAILY SPEND: confirmed expenses in the last 7 Bangkok days
10 *   (today and the 6 days before), divided by the number of those days the
11 *   user has been tracking (at most 7). A user who started 3 days ago is
12 *   divided by 3, not 7, so their average is not understated.
13 * - RUNWAY DAYS: floor((balance - floor) / average daily spend).
14 * - DEPLETION DATE: today + runway days = the first day the balance is
15 *   expected to reach the low line.
16 * - ZERO AVERAGE: if there was no spending in the window we cannot divide,
17 *   so the status is "no_spending" and no date is shown.
18 */
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:1

คำ	นิยามที่ฉันใช้
เส้นเงินสำรอง Low line / runway floor	จำนวนเงินที่ผู้ใช้ตั้งเอง ค่าเริ่มต้น ฿500 "เงินต่ำ" หมายถึง ยอดคงเหลือ ≤ เส้นนี้ เป็นเส้นที่ควรอยู่เหนือไว้ ไม่ใช่ศูนย์ เพื่อให้มีเงินกันไว้ยามฉุกเฉิน
ค่าเฉลี่ยรายจ่ายต่อวัน Average daily spend	รายจ่ายที่ยืนยันแล้วใน 7 วันล่าสุดตามเวลาไทย (วันนี้ + 6 วันก่อนหน้า)หารด้วยจำนวนวันที่ผู้ใช้ใช้แอปมาแล้ว แต่ไม่เกิน 7
Runway (วัน)	ปัดลง [(ยอดคงเหลือ - เส้นสำรอง) ÷ ค่าเฉลี่ยต่อวัน] โดยเส้นสำรองในที่นี้รวมเงินในกระเป๋าออกมาด้วย
Depletion date วันที่เงินพอถึง	วันนี้ + Runway คือวันที่คาดว่ายอดเงินจะลงมาถึงเส้นสำรอง
สถานะ	< 7 วัน = critical · < 14 วัน = watch · ≥ 14 วัน = healthy · ยอด ≤ เส้นสำรอง = below_floor · ไม่มีรายจ่ายเลย = no_spending

เงื่อนไขผ่าน (Acceptance criteria) → โค้ด → เทสต์

เงื่อนไข	โค้ด	เทสต์
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ผู้ใช้ใหม่หารด้วยจำนวนวันที่ใช้จริง	averageDailyExpense	TC-13
คำนวณจำนวนวันและวันที่ถูกต้อง	computeRunway	TC-14
ค่าเฉลี่ยเป็นศูนย์ → ไม่หารด้วยศูนย์ แสดง "ยังคำนวณไม่ได้"	computeRunway	TC-15
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16
เกณฑ์สถานะตรงตามนิยาม (6 / 7 / 14 วัน)	computeRunway	TC-17
"เช็ก่อนจ่าย" และ "ใช้ได้วันละเท่าไรถึงสิ้นเดือน"	runwayAfterPurchase, safeDailySpend	TC-18
แถบ "ลองปรับ" 0–50% ลากได้ กดลูกศรได้ เห็นจำนวนวันที่เพิ่ม	runwayWithReduction, Slider.tsx	E2E runway
กระปุกออม: เงินในกระปุกถูกหักออกจากเงินที่ใช้ได้, ตาราง savings_goals มี RLS	goals.ts, useMoney (reserved)	TC-71, E2E goal
ข้อมูลตัวอย่างต้องมี Runway ≥ 10 วันเสมอ ไม่ว่าเปิดวันไหน	sample.ts	TC-33

★ ทำไมต้องนิยามละเอียดขนาดนี้

เพราะ requirement ที่ดีต้อง "ทดสอบได้" ถ้าเขียนแค่ "แจ้งเตือนเมื่อเงินต่ำ" สองคนอาจเข้าใจคนละแบบ แต่ถ้าเขียนว่า "ยอด ≤ เส้นสำรอง ซึ่งค่าเริ่มต้นคือ ฿500" ก็เขียนเทสต์ได้ทันที เช่น ใส่ยอด ฿500 กับเส้นสำรอง ฿500 แล้วต้องได้สถานะ below_floor (ซึ่งคือ TC-16)

วัตถุดิบ 4 อย่างของการคำนวณ

ก่อนเข้าสู่สูตร ต้องรู้ก่อนว่าแต่ละตัวเลขมาจากไหน

① ยอดคงเหลือ (Balance)

ยอดคงเหลือ = ยอดเงินตอนเริ่มใช้ MindPay + รายรับที่ยืนยันแล้ว - รายจ่ายที่ยืนยันแล้ว

ตัวเลขนี้มาจาก FR-2 (ฟังก์ชัน computeBalance ใน [src/domain/summary.ts](#)) ผู้ใช้กรอกยอดเงินตอนตั้งค่าครั้งแรกหลังสมัคร (ช่อง "ตอนนี้มีเงินอยู่เท่าไร? (บาท)") และแก้ไขภายหลังในหน้าตั้งค่า ช่อง "ยอดเงินตอนเริ่มใช้ MindPay (บาท)" ถ้ายอดในแอปไม่ตรงกับธนาคาร แอปโหลดรายการย้อนหลังประมาณ 13 เดือน ส่วนรายการที่เก่ากว่านั้นไม่ได้หายไป แต่ถูกรวมเป็น "ยอดยกมา" (carrySatang) แล้วบวกเข้าไปด้วย

! สำคัญ

รายการ "แบบร่าง" (สลิปที่ AI อ่านไม่ชัด รอผู้ใช้ตรวจ) ไม่นับ ทั้งในยอดคงเหลือและในค่าเฉลี่ย นับเฉพาะ status = confirmed เท่านั้น เพราะถ้า AI อ่านผิดแล้วนับทันที Runway จะผิดตามไปด้วย

② เงินสำรองขั้นต่ำ (Floor หรือ Low line)

เงินก่อนที่ผู้ใช้ "ไม่อยากแตะ" เพื่อฉุกเฉิน เช่น ค่ารถกลับบ้าน ผู้ใช้ตั้งครั้งแรกตอนตั้งค่าเริ่มต้น (ช่อง "เงินสำรองที่ไม่อยากให้ต่ำกว่า (บาท)") และเปลี่ยนภายหลังในหน้าตั้งค่า ช่อง "เงินสำรองขั้นต่ำ (บาท)" ค่าเริ่มต้นคือ ฿500 เก็บในฐานข้อมูลตาราง profiles คอลัมน์ runway_floor_satang (ค่าเริ่มต้น 50000 สตางค์ และมีเงื่อนไขในฐานข้อมูลว่าต้อง ≥ 0 จึงตั้งเป็น ฿0 ได้ถ้าไม่อยากกันเงินไว้เลย)

③ เงินในกระปุกออม (Reserved)

MindPay มีฟีเจอร์ "กระปุกออม" ให้ตั้งเป้าเก็บเงิน เช่น เก็บซื้อหูฟัง เงินที่หยอดลงกระปุกยังอยู่ในบัญชีจริง (จึงยังอยู่ในยอดคงเหลือ) แต่ผู้ใช้ตั้งใจจะไม่ใช้ ถ้านับเป็นเงินใช้ได้ Runway จะดูดีเกินจริง ฉะนั้นให้เงินในกระปุกทุกใบรวมกัน (ฟังก์ชัน reservedSatang) ถูกหักออกเหมือนเป็นเงินสำรองเพิ่ม

④ ค่าเฉลี่ยรายจ่ายต่อวัน (Average daily spend)

ความเร็วในการใช้เงิน คัดจากรายจ่ายที่ยืนยันแล้ว 7 วันล่าสุด เป็นหัวใจของระบบ จึงแยกอธิบายเต็ม ๆ ในบทถัดไป

เงินที่ใช้ได้จริง (spendable) = ① - ② - ③

ในโค้ด จักรวม ② + ③ เป็นตัวเดียวก่อนส่งเข้าสู่สูตร: floorSatang: floor + reserved

★ ตัวอย่าง (ตรวจด้วยฟังก์ชันจริง)

ยอดคงเหลือ ฿3,500 เงินสำรอง ฿500 และหยอดกระปุกซื้อหูฟังไว้ ฿1,000 → เงินที่ใช้ได้จริง = $3,500 - 500 - 1,000 = \text{฿}2,000$ (ไม่ใช่ ฿3,000) ถ้าใช้วันละ ฿145 จะพอใช้ $2,000 \div 145 = 13.79 \rightarrow 13$ วัน ถึง 21 ต.ค. สถานะ "เริ่มต้องระวัง" ทั้งที่ถ้าไม่มีกระปุกจะได้ 20 วัน "สบาย ๆ" ในหน้าจอ คาร์ด "คำนวณยังไง" จะมีบรรทัด "หักเงินที่กันไว้ในกระปุกออม" โผล่ขึ้นมาเองเมื่อมีเงินในกระปุก

ค่าเฉลี่ยรายจ่ายต่อวัน

ถ้าค่าเฉลี่ยผิด ทุกอย่างผิดหมด บทนี้จึงสำคัญที่สุด

หน้าต่าง 7 วัน

ฉันใช้รายจ่ายของ **วันนี้ + 6 วันก่อนหน้า** รวมเป็น 7 วัน นับวันตามเวลาประเทศไทย ถ้าวันนี้คือวันพฤหัสบดี 8 ต.ค. หน้าต่างคือ 2–8 ต.ค. พรุ่งนี้หน้าต่างจะเลื่อนเป็น 3–9 ต.ค. (วันที่ 2 หลุดออก วันที่ 9 เข้ามาแทน) เรียกว่า **moving average** หรือค่าเฉลี่ยเคลื่อนที่

ศ. 2 ต.ค.	ส. 3 ต.ค.	อา. 4 ต.ค.	จ. 5 ต.ค.	อ. 6 ต.ค.	พ. 7 ต.ค.	พฤ. 8 ต.ค. (วันนี้)
฿120	฿0 (ไม่ได้ใช้ ก็นับ)	฿85	฿300	฿0 (ก็นับ)	฿450	฿60

$$120 + 0 + 85 + 300 + 0 + 450 + 60 = \text{฿}1,015 \rightarrow 1,015 \div 7 = \text{฿}145 \text{ ต่อวัน}$$

กฎข้อ 1: วันที่ไม่ได้ใช้เงิน ต้องนับด้วย

ถ้าหารเฉพาะวันที่มีรายจ่าย (5 วัน) จะได้ $1,015 \div 5 = \text{฿}203$ ต่อวัน ซึ่งสูงเกินจริง เพราะในชีวิตจริงผู้ใช้มีวันที่ไม่ใช้เงินจริง ๆ แอปจะบอกว่าเงินหมดเร็วกว่าความจริง ฉันจึงหารด้วย **จำนวนวันในปฏิทิน** ไม่ใช่จำนวนวันที่มีรายการ (เทสต์ TC-12)

กฎข้อ 2: ผู้ใช้ใหม่ หารด้วยจำนวนวันที่ใช้มาจริง

สมมติผู้ใช้เพิ่งเริ่มใช้แอปเมื่อ 6 ต.ค. และจดไว้ 3 วัน: 6 ต.ค. ฿90, 7 ต.ค. ฿150, 8 ต.ค. ฿60 รวม ฿300

วิธีการ	ค่าเฉลี่ย	ถ้าใช้ได้จริง ฿3,000 จะได้
หาร 7 ตรง ๆ (ผิด เพราะ 4 วันแรกเขายังไม่ได้ใช้แอป)	฿42.86	69 วัน (สบายเกินจริง)
หาร 3 ตามจำนวนวันที่ใช้มาจริง (ที่ฉันทำ)	฿100	30 วัน

"วันแรกที่ใช้แอป" คือวันของรายการที่ยืนยันแล้วรายการแรกสุด จะเป็นรายรับหรือรายจ่ายก็ได้ แล้วนับจากวันนั้นถึงวันนี้ (รวมทั้งสองวัน) ถ้าเกิน 7 ก็ใช้ 7 ในหน้าจอจะเห็นจำนวนนี้ในวงเล็บ เช่น "ใช้เฉลี่ยต่อวัน (3 วันล่าสุด)" (เทสต์ TC-13)

กฎข้อ 3: นับเฉพาะ "รายจ่าย" ที่ "ยืนยันแล้ว"

- รายรับไม่เอามาหักลบในค่าเฉลี่ย เพราะค่าเฉลี่ยวัด "ความเร็วในการใช้เงิน" ส่วนรายรับไปเพิ่มยอดคงเหลืออยู่แล้ว
- สลิปแบบร่างที่ยังไม่ยืนยันไม่นับ ในหน้าจอเขียนบอกไว้ใต้การ์ดว่า "สลิปที่ยังไม่ยืนยันไม่นับ" (ทดลองใส่แบบร่าง ฿5,000 ในตัวอย่างข้างบน ค่าเฉลี่ยยังเป็น ฿145 เท่าเดิม)

กฎข้อ 4: ปัดเป็นสตางค์

ผลหารอาจมีทศนิยมยาว ฉันปัดเป็นสตางค์ที่ใกล้ที่สุด (Math . round) เช่น $\text{฿}4,762 \div 7 = \text{฿}680.2857... \rightarrow 68,029 \text{ สตางค์} = \text{฿}680.29$

ทำไมเลือก 7 วัน

- ครบหนึ่งรอบสัปดาห์** มีทั้งวันเรียน/วันทำงานและเสาร์อาทิตย์ ซึ่งมักใช้เงินไม่เท่ากัน ถ้าใช้ 3 วันอาจโดนแต่วันหยุด ตัวเลขจะแกว่งมาก
- สั้นพอที่จะตามนิสัยล่าสุดกัน** ถ้าผู้ใช้เริ่มประหยัดวันนี้ ภายในไม่กี่วันตัวเลขก็ดีขึ้นให้เห็น ถ้าใช้ 30 วัน ความพยายามจะแทบไม่ขยับตัวเลข

- **ตรงกับ SRS** ที่กำหนดไว้ว่า "ค่าเฉลี่ยรายจ่าย 7 วัน"

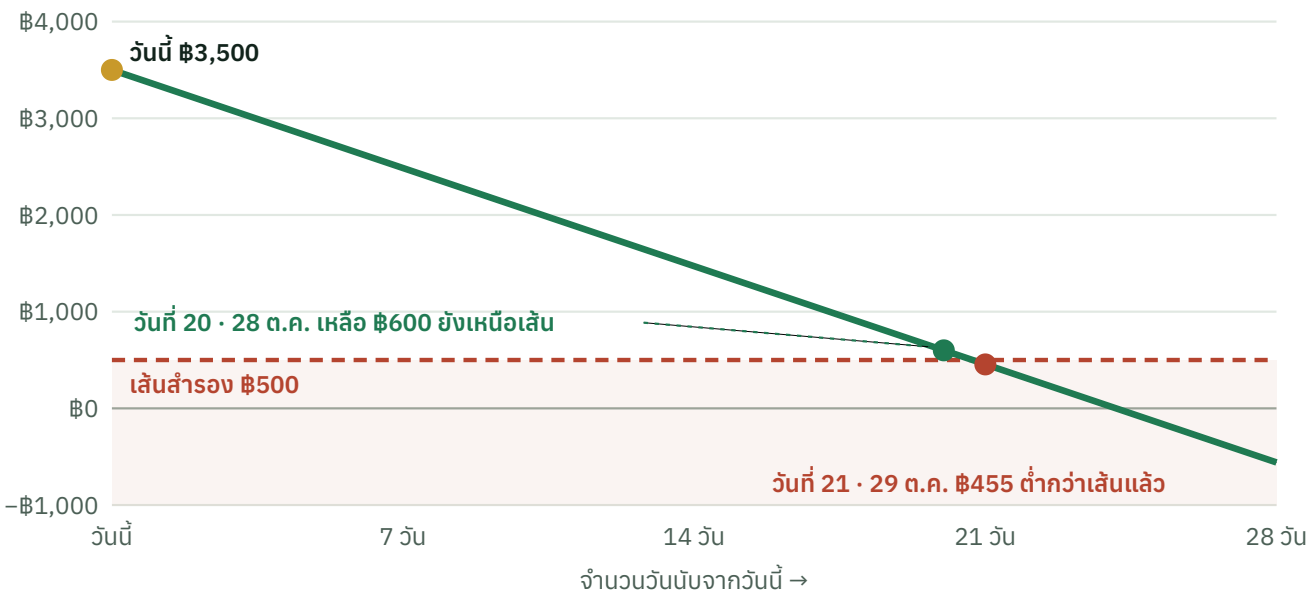
สูตร Money Runway ที่ละขั้น

ทำตามลำดับเดียวกับที่โค้ดใน computeRunway ทำจริง

ข้อมูลตัวอย่าง: วันนี้ 8 ต.ค. 2569 · ยอดคงเหลือ ฿3,500 · เงินสำรอง ฿500 · ไม่มีกระปุกออม · ค่าเฉลี่ย ฿145/วัน (จากบทที่แล้ว)

ขั้น	ทำอะไร	คิดจากตัวอย่าง
1	หาเงินที่ใช้ได้จริง = ยอดคงเหลือ - (เงินสำรอง + เงินในกระปุกลูก)	$3,500 - (500 + 0) = ฿3,000$
2	ถ้าเงินที่ใช้ได้ $\leq 0 \rightarrow$ จบเลย สถานะ below_floor (0 วัน)	$3,000 > 0$ ไปต่อ
3	ถ้าค่าเฉลี่ย $\leq 0 \rightarrow$ จบเลย สถานะ no_spending (หารด้วยศูนย์ไม่ได้)	$145 > 0$ ไปต่อ
4	หารแล้ว ปัดลง เป็นจำนวนเต็ม	$3,000 \div 145 = 20.69 \rightarrow 20$ วัน
5	ถ้าเกิน 365 วัน ให้หยุดที่ 365 และจำไว้ว่า "ถูกตัด" (แสดง 365+)	20 ไม่เกิน
6	วันที่เงินพอถึง = วันนี้ + จำนวนวัน	8 ต.ค. + 20 วัน = 28 ต.ค. 2569
7	ให้สถานะ: < 7 critical, < 14 watch, นอกนั้น healthy	$20 \geq 14 \rightarrow$ สบาย ๆ (healthy)

ภาพให้เห็น: ยอดเงินที่คาดว่าจะใช้วันละ ฿145 ทุกวัน



เส้นตรงสีเขียวลดลงวันละ ฿145 เริ่มจาก ฿3,500 วันนี้ เส้นประสีแดงคือเส้นสำรอง ฿500 ยอดเงินแตะเส้นที่ประมาณวันที่ 20.7 แอปจึงปิดลงเป็น 20 วัน: วันที่ 20 (28 ต.ค.) ยังเหลือ ฿600 อยู่เหนือเส้น แต่วันที่ 21 (29 ต.ค.) เหลือ ฿455 ต่ำกว่าเส้นแล้ว

★ ทำไมปัดลง

บอก "20 วัน" แล้วจริง ๆ ได้ 20.69 วัน ผู้ใช้เหลือเงินเกินมานิดหน่อย ไม่มีใครเดือดร้อน แต่ถ้าปัดขึ้นเป็น "21 วัน" วันที่ 21 เงินจะต่ำกว่าเส้นสำรองแล้ว **เรื่องเงิน บอกน้อยไว้ปลอดภัยกว่าบอกเกิน**

i ตรวจสอบคำตอบเอง

เอาจำนวนวันคูณค่าเฉลี่ยแล้วลบออกจากยอด: $3,500 - 145 \times 20 = \$600 \geq \$500 \checkmark$ แต่ $3,500 - 145 \times 21 = \$455 < \$500$
แปลว่า 20 คือจำนวนวันมากที่สุดที่ยังอยู่เหนือเส้น

สถานะ 5 แบบ และกรณีพิเศษ

ตัวเลขเดียวไม่พอ จึงแปลงจำนวนวันเป็น "ระดับ" ให้ผู้ใช้รู้ว่าควรรู้สึกยังไง

ใกล้เส้นสำรอง	เริ่มต้องระวัง	สบาย ๆ (14 วันขึ้นไป ถึง 365+)
0 critical · 0–6 วัน	7 watch · 7–13 วัน	14 healthy

45 = ต้นไม้/วง

สถานะ (ในโค้ด)	เงื่อนไข	ป้ายในแอป	น้องกลับมาหน้าหลักพูดว่า
healthy	≥ 14 วัน	สบาย ๆ	"เงินพอใช้อีก 20 วัน ต้นไม้เงินแข็งแรงดี สบาย ๆ เลย"
watch	7–13 วัน	เริ่มต้องระวัง	"วันนี้ใช้ได้ราว ๕... เงินจะพอถึงสิ้นเดือนพอดี"
critical	0–6 วัน (ยังเหนือเส้น)	ใกล้เส้นสำรอง	"เงินพอใช้อีก 4 วัน ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน"
below_floor	เงินที่ใช้ได้ ≤ 0	ต่ำกว่าเงินสำรอง	"เงินแตะเส้นเงินสำรองแล้ว ช่วงนี้ใช้เท่าที่จำเป็นก่อนนะ กล้าเป็นกำลังใจให้"
no_spending	ค่าเฉลี่ย = 0	ยังไม่มีรายจ่าย 7 วัน (หน้าเงินพอถึง: "รอข้อมูลรายจ่าย")	"7 วันนี้ยังไม่มีรายจ่าย กล้าเลยยังนับวันไม่ได้ ถ้าจ่ายอะไรอย่าลืมเก็บสลิบไว้ละ"

ข้อความน้องกลับมาจากฟังก์ชัน buddyLine ใน src/domain/buddy.ts (บทอ่านโค้ด) สี่เป็นแค่ตัวช่วย ทุกสถานะมีความกำกับเสมอ คนที่แยกสีไม่ออกก็อ่านเข้าใจ

ต้นไม้เงินและวงแหวน

ในการ์ดเขียวมี "ต้นไม้เงิน" ใบสีทองคือจำนวนวันที่เงินพอใช้ และใบเก็บเงินพอถึงมีวงแหวนรอบต้นไม้ ทั้งสองเต็มเมื่อ **45 วัน** (ประมาณเดือนครึ่ง ถือว่าสบายมาก) สูตรคือ จำนวนวัน ÷ 45 (ไม่เกิน 1) เช่น 15 วัน = 1 ใน 3 ต้นไม้มีใบอย่างน้อย 8% เสมอเมื่อคำนวณได้ จะได้ไม่ดูโหล่งเหมือนต้นไม้ตาย กรณีพิเศษ: ต่ำกว่าเงินสำรอง ในร่องเกือบหมด (ต้นไม้ 4%, วงแหวน 2%) และยังคำนวณไม่ได้ ต้นไม้อยู่ระดับกลาง ๆ 70% ส่วนวงแหวนว่าง แตะต้นไม้แล้วน้องกล้าจะอธิบายว่าใบทองหมายถึงอะไร

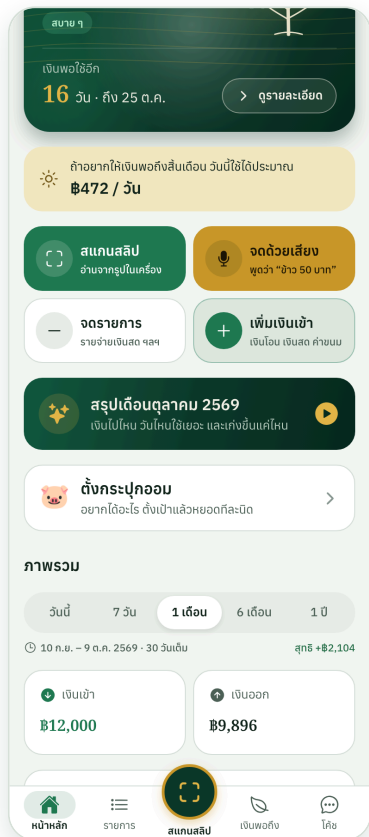
กรณีพิเศษที่ฉันต้องคิดเผื่อ (ตรวจด้วยฟังก์ชันจริงทุกข้อ)

กรณี	ระบบทำอะไร
เงินแตะหรือต่ำกว่าเส้นสำรอง เช่น ยอด ๕500 เส้น ๕500	below_floor, 0 วัน, วันที่ = วันนี้ หน้าจอขึ้น "ยอดเงินแตะเส้นสำรองแล้ว" แทนตัวเลข (เท่ากับเส้นก็นับว่าแตะ เพราะนิยามคือ ≤)
7 วันไม่มีรายจ่ายเลย	no_spending หาดด้วยศูนย์ไม่ได้ จึงไม่แสดงวันที่ ขึ้น "ยังคำนวณไม่ได้" และซ่อนเส้นเวลากับแถบลองปรับ (เพราะไม่มีความเร็วให้ใช้คำนวณ)
เหลือเหนือเส้นนิดเดียว เช่น ใช้ได้ ๕100 แต่ใช้วันละ ๕145	100 ÷ 145 = 0.69 → 0 วัน สถานะ critical (ไม่ใช่ below_floor เพราะยังเหนือเส้นอยู่) วันที่ = วันนี้
เงินเยอะมากเทียบกับรายจ่าย เช่น ๕100,000 ใช้วันละ ๕10	คำนวณได้ 9,950 วัน (ราว 27 ปี) ฉันทัดที่ 365 วัน แสดง "365+" และ "มากกว่า 365 วัน" เพราะประมาณการไกลเกิน 1 ปีไม่มีความหมาย นิัยการใช้เงินเปลี่ยนก่อนแน่นอน
ผู้ใช้ใหม่ เพิ่งใช้ 1–6 วัน	หาดด้วยจำนวนวันที่ใช้จริง (กฎข้อ 2)
ยังไม่มีรายการเลยสักรายการ	ค่าเฉลี่ยเป็น 0 แต่ระบบเช็ก "เงินที่ใช้ได้" ก่อนเสมอ: ถ้ายอดเริ่มต้นเกินเส้นสำรอง → no_spending ถ้ายอดเริ่มต้น ≤ เส้นสำรอง (เช่น ๕0) → below_floor

กรณี	ระบบทำอะไร
มีเงินในกระเป๋าออกมา	หักออกจากเงินที่ใช้ได้ ถ้ายอดเหนือเส้นสำรองแต่เงินในกระเป๋าทำให้ติดลบ ก็เป็น below_floor

ทัวร์หน้าจอ: หน้าหลัก

สิ่งแรกที่ผู้ใช้เห็นเมื่อเปิดแอป (ภาพจากโหมดทดลอง 9 ต.ค. 2569)



การ์ดเขียว (hero card)

- 1 ยอดคงเหลือ ตัวใหญ่ (มาจาก FR-2) และ ป้ายสถานะ เช่น "สบาย ๆ"
- 2 "เงินพอใช้อีก 16 วัน · ถึง 25 ต.ค." ตัวเลขวันนับขึ้นแบบแอนิเมชันตอนเปิด (ปิดได้ถ้าเปิด "ลดการเคลื่อนไหว" ในมือถือ)
- 3 ต้นไม้เงิน ใบทองมากน้อยตามจำนวนวัน แต่แล้วต้นไม้สิ้นและน้องกล้ออธิบาย
- 4 ปุ่ม "ดูรายละเอียด" ไปที่เก็บเงินพอถึง โปรแกรมอ่านหน้าจอจะอ่านทั้งการ์ดว่า "ยอดคงเหลือ ฿11,359.00 เงินพอใช้อีก 16 วัน"

การ์ดทอง "ถ้าอยากให้เป็นเงินพอถึงสิ้นเดือน วันนี้ใช้ได้ประมาณ"

Runway ตอบว่า "ถ้าใช้ความเร็วเดิม จะอยู่ได้กี่วัน" แต่การ์ดนี้ กลับคำถาม: "ถ้าอยากอยู่ให้ถึงสิ้นเดือน ต้องใช้ไม่เกินวันละเท่าไร" (ฟังก์ชัน safeDailySpend)

สถานะปกติ: ยอด ฿11,359 เงินพอใช้อีก 16 วัน ถึง 25 ต.ค. และวันนี้ใช้ได้ประมาณ ฿472

ใช้ได้วันละ = ปัดลงเป็นบาท [เงินที่ใช้ได้จริง ÷ จำนวนวันถึงสิ้นเดือน (นับวันนี้ด้วย)]

ภาพนี้: 9-31 ต.ค. = 23 วัน → $10,859 \div 23 = 472.1 \rightarrow \text{฿}472$ ตัวอย่างคิดมือ (8 ต.ค., ใช้ได้ ฿3,000): 8-31 ต.ค. = 24 วัน → $3,000 \div 24 = \text{฿}125$

★ สองคำถาม คนละมุม

Runway: รู้ความเร็ว อยากรู้ระยะทาง → "ใช้วันละ ฿145 อยู่ได้ 20 วัน" · **ใช้ได้วันละ:** รู้ระยะทาง อยากรู้ความเร็วที่ปลอดภัย → "อยากอยู่ให้ถึง 31 ต.ค. ใช้ได้วันละ ฿125" ถ้า "ใช้ได้วันละ" น้อยกว่าค่าเฉลี่ยที่ใช้อยู่จริง แปลว่าถ้าใช้แบบเดิม เงินจะหมดก่อนสิ้นเดือน ผู้ใช้เห็นสองตัวเลขนี้คู่กันจึงรู้ว่าต้องลดเท่าไร

สถานะต่าง ๆ บนหน้าจอจริง

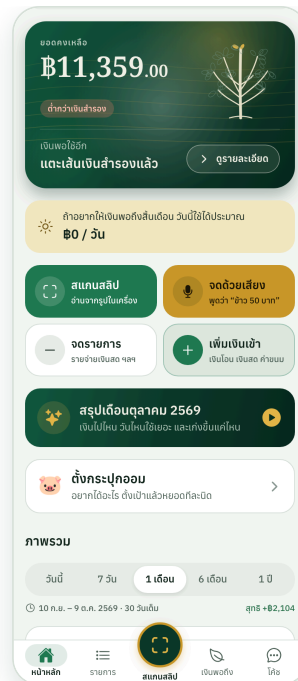
ข้อมูลรายการชุดเดียวกันทั้ง 3 ภาพ จับเปลี่ยนแค่ "เงินสำรองขั้นต่ำ" ในหน้าตั้งค่า ค่าเฉลี่ย ฿660 เท่าเดิม เพราะค่าเฉลี่ยมาจากรายจ่ายเท่านั้น สิ่งที่เปลี่ยนคือ "เงินที่ใช้ได้จริง" ซึ่งเป็นตัวตั้งของการหาร



เงินสำรอง ฿4,429 → ใช้ได้ 6,930
÷ 660 = 10.5 → 10 วัน ถึง 19
ต.ค. "เริ่มต้องระวัง" ใช้ได้วันละ
฿301



เงินสำรอง ฿8,389 → 2,970 ÷
660 = 4.5 → 4 วัน ถึง 13 ต.ค.
"ใกล้เส้นสำรอง" ใบทองเหลืองน้อย



เงินสำรอง ฿11,400 (มากกว่ายอด)
→ ใช้ได้ ฿0 "แตะเส้นเงินสำรองแล้ว"
ใช้ได้วันละ ฿0

ทัวร์หน้าจอ: แท็บ "เงินพอถึง"

หน้าของ FR-6 โดยเฉพาะ ไฟล์ src/app/(tabs)/runway.tsx



ส่วนบน: วงแหวนทอง ต้นไม้เงิน และตัวเลข 16 วัน

- 1 หัวข้อ "MONEY RUNWAY · เงินพอถึงวันไหน"
- 2 วงแหวนทอง เริ่มเมื่อ 45 วัน ตอนนี้ 16 วันจึงเพิ่มประมาณ 1 ใน 3
- 3 ต้นไม้เงิน ออกใบตามจำนวนวัน (ค่อย ๆ งอกและโยกตอนเปิดหน้า)
- 4 ตัวเลข 16 (วันที่เงินพอใช้) นับขึ้นแบบแอนิเมชัน
- 5 "วัน - ถึงประมาณ 25 ต.ค. 2569" ใช้คำว่า "ประมาณ" เพราะเป็นการคาดการณ์ ไม่ใช่คำสัญญา
- 6 ป้ายสถานะ: สบาย ๆ / เริ่มต้องระวัง / ใกล้เคียงสำรอง / ต่ำกว่าเงินสำรอง / รอข้อมูลรายจ่าย

ถ้าสถานะเป็น below_floor ตรงตัวเลขจะเปลี่ยนเป็น "ยอดเงินแตะเส้นสำรองแล้ว" พร้อมยอดคงเหลือและเส้นสำรอง ถ้าเป็น no_spending จะขึ้น "ยังคำนวณไม่ได้" พร้อมเหตุผล



การ์ด "คำนวณยังง" และเส้นเวลา 30 วัน

การ์ด "คำนวณยังง"

ฉันอยากให้ผู้ใช้ **ตรวจสอบเลขเองได้** จึงแสดงสูตรทั้งบรรทัดเหมือนใบเสร็จ:

ยอดคงเหลือ	฿11,359.00
หักเงินสำรองที่ตั้งไว้	- ฿500.00
(ถ้ามี) หักเงินในกระปุกออม	- ...
ใช้ได้จริง	฿10,859.00
÷ ใช้เฉลี่ยต่อวัน (7 วันล่าสุด)	฿660.00
เงินพอใช้อีก	16 วัน

ตรวจ: $10,859 \div 660 = 16.45 \rightarrow$ ปัดลง 16 ✓ ตัวเลขเป็นสีแดงเมื่อสถานะ critical หรือ below_floor ปุ่ม "เปลี่ยนเงินสำรอง" พาไปหน้าตั้งค่า

การ์ด "ถ้ายังใช้เงินแบบนี้ต่อไป" (เส้นเวลา 30 วัน)

แสดงยอดเงินที่คาดว่าจะ 4 จุด: วันนี้ อีก 7 วัน อีก 14 วัน และอีก 30 วัน

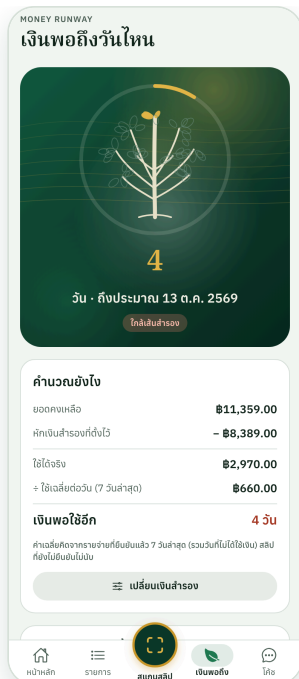
$$\text{ยอดในอีก } d \text{ วัน} = \text{ยอดวันนี้} - \text{ค่าเฉลี่ย} \times d$$

จุด	คิด	ผล
อีก 7 วัน (16 ต.ค.)	$11,359 - 660 \times 7$	฿6,739
อีก 14 วัน (23 ต.ค.)	$11,359 - 660 \times 14$	฿2,119
อีก 30 วัน (8 พ.ย.)	$11,359 - 660 \times 30$	-฿8,441

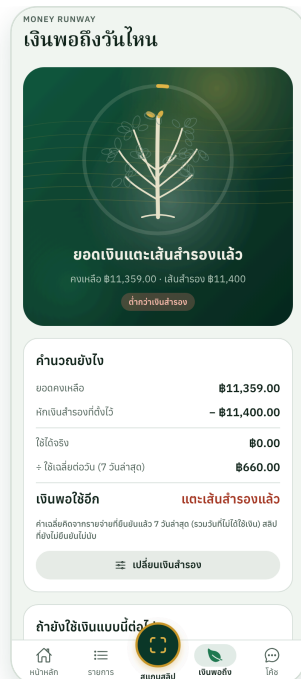
แถบยาวตามสัดส่วนเทียบยอดวันนี้ แถบและตัวเลขกลายเป็น **สีแดง** เมื่อต่ำกว่าหรือเท่ากับเส้นสำรอง ติดลบไม่ได้แปลว่าเป็นหนี้จริง แต่บอกว่า "ถ้าไม่มีเงินเข้าเลย จะไม่พอ" ได้การ์ดเขียนกำกับว่า "ยังไม่รวมรายรับที่จะเข้ามา" การ์ดนี้ซ่อนเมื่อค่าเฉลี่ยเป็น 0



สถานะเริ่มต้องระวัง: 10 วัน ถึง 19 ต.ค.



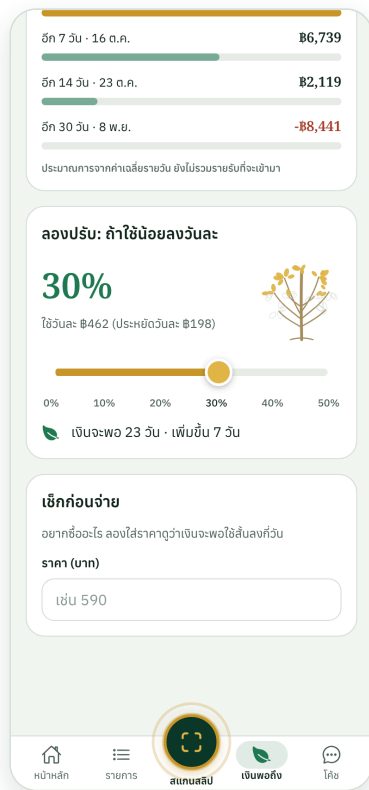
ใกล้เส้นสำรอง: 4 วัน ถึง 13 ต.ค. ตัวเลขในการ์ดเป็นสีแดง



ต่ำกว่าเงินสำรอง: แสดงยอดและเส้นสำรองแทนจำนวนวัน

เครื่องมือ "ลองปรับ" "เช็ก่อนจ่าย" และ "ถามโค้ช"

เครื่องมือ "ถ้า...จะเป็นยังไง" ให้ผู้ใช้ลองก่อนตัดสินใจจริง



ลองปรับ: ถ้าใช้น้อยลงวันละ

แถบเลื่อน 0–50% ทีละ 5% (มีป้ายทุก 10%) ลากด้วยนิ้ว และหรือกดปุ่มลูกศรบนคีย์บอร์ดก็ได้ มือถือสั่นเบา ๆ ทุกขั้น และโปรแกรมอ่านหน้าจออ่านได้ว่า "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"

$$\text{ค่าเฉลี่ยใหม่} = \text{ค่าเฉลี่ยเดิม} \times (1 - p)$$

แล้วคิด Runway ใหม่ด้วยสูตรเดิม ฟังก์ชัน
runwayWithReduction

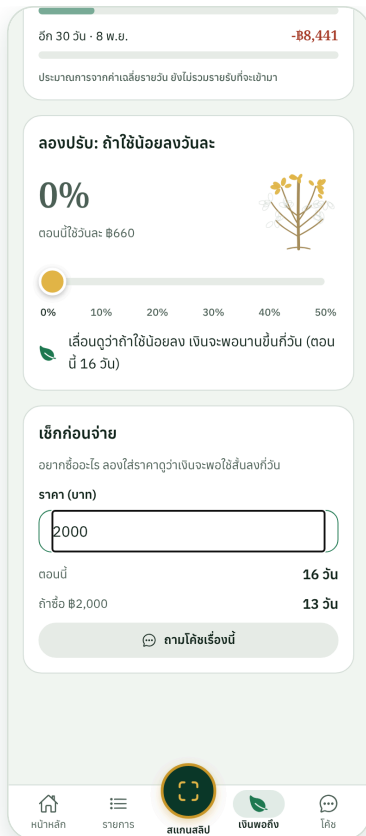
ในภาพ: 30% → $660 \times 0.7 = \text{฿}462$ (ประหยัดวันละ ฿198) → $10,859 \div 462 = 23.5 \rightarrow 23$ วัน · เพิ่มขึ้น 7 วัน ต้นไม้เล็กข้าง ๆ ก็ออกใบเพิ่มตาม แถบนี้ซ่อนเมื่อค่าเฉลี่ยเป็น 0 หรือเงินต่ำกว่าเส้นสำรองแล้ว

ลากแถบไป 30%: ใช้วันละ ฿462 ประหยัดวันละ ฿198 เงินจะพอ 23 วัน เพิ่มขึ้น 7 วัน

ตัวอย่างคิดมือ (ใช้ได้จริง ฿3,000)	0%	10%	20%	30%	40%	50%
ใช้วันละ	฿145	฿130.50	฿116	฿101.50	฿87	฿72.50
เงินพอใช้อีก	20 วัน	22 วัน	25 วัน	29 วัน	34 วัน	41 วัน
ถึงวันที่	28 ต.ค.	30 ต.ค.	2 พ.ย.	6 พ.ย.	11 พ.ย.	18 พ.ย.

i สังเกต

ใช้น้อยลงครั้งหนึ่ง จำนวนวันเพิ่มเกือบเท่าตัว (20 → 41) เพราะจำนวนวัน = เงิน ÷ ความเร็ว ความเร็วลดครึ่ง วันจึงคูณสอง (ที่ไม่ได้ 40 พอดี เพราะเศษที่เคยถูกปิดทิ้งรวมกันได้อีกหนึ่งวัน: $3,000 \div 72.50 = 41.4$)



เช็กก่อนจ่าย

ใส่ราคาของที่ยากซื้อ แอปจะแสดง "ตอนนี้ ... วัน" เทียบกับ "ถ้าซื้อ ฿... ... วัน" กันทีละขั้น (ฟังก์ชัน runwayAfterPurchase) โดยสมมติว่าซื้อวันนี้ แล้วใช้เงินแบบเดิมต่อ

Runway หลังซื้อ = คิดใหม่ด้วย (ยอดคงเหลือ - ราคา)

เส้นสำรองและค่าเฉลี่ยเก่าเดิม

ในภาพ: ซื้อ ฿2,000 → $(11,359 - 2,000 - 500) \div 660 = 13.4 \rightarrow 13$ วัน (จาก 16 ตกไปเป็นช่วง "เริ่มต้องระวัง") · ตัวอย่างคิดมือ: ซื้อ ฿600 → $2,400 \div 145 = 16.55 \rightarrow 16$ วัน ถึง 24 ต.ค. · ถ้าราคามากกว่าเงินที่ใช้ได้ จะขึ้น "แตะเส้นสำรองแล้ว" เป็นสีแดง

เช็กก่อนจ่าย: ใส่ราคา 2000 → ตอนนี้ 16 วัน ถ้าซื้อ ฿2,000 เหลือ 13 วัน

ปุ่ม "ถามโค้ชเรื่องนี้" (เชื่อมไป FR-5)

กดแล้วแอปพาไปเก็บโค้ช พร้อมพิมพ์คำถามรอไว้ให้: "ถ้าฉันซื้อของราคา ฿2,000 วันนี้ จะเป็นยังไง" และแนบราคาไปด้วย เมื่อผู้ใช้กดส่งแอปจะ **คำนวณเองก่อน** แล้วส่ง "ข้อเท็จจริง" ไปให้ AI:

```
// ข้อมูลที่ส่งให้โค้ช (ตัวอย่างจากหน้าจอ)
lowLine: "฿500"
averageDailySpend7d: "฿660"
runway: "16 วัน (ถึง 25 ต.ค. 2569)"
purchaseCheck: { price: "฿2,000", runwayBefore: "16 วัน", runwayAfter: "13 วัน" }
```

★ ทำไมไม่ให้ AI คิดเลขเอง

AI ภาษาอาจคิดเลขพลาดได้ ฉะนั้นให้โค้ดที่ทดสอบแล้วเป็นคนคิด AI มีหน้าที่แค่ "อธิบายเป็นภาษาคน" จากตัวเลขที่ได้รับ ถ้าผู้ใช้พิมพ์คำถามเอง เช่น "หุฟิง 1,990 คุ่มโหม" แอปก็หาราคาในประโยคเองได้ (ฟังก์ชัน priceInQuestion, เทสต์ TC-89) หมายเหตุ: โหมดทดลองยังถาม AI ไม่ได้ ต้องเข้าสู่ระบบด้วยบัญชีจริง

←

ตั้งค่า

เงินของฉัน

ชื่อเล่น

ผู้ทดลองใช้

ยอดเงินตอนเริ่มใช้ MindPay (บาท)

3000

ยอดคงเหลือ = ค่านี้ + รายรับ - รายจ่ายที่ยืนยันแล้ว ถ้ายอดในแอปไม่ตรงกับรายการ ปริณัติ

เงินสำรองขั้นต่ำ (บาท)

500

Money Runway นับวันของยอดเงินเหลือเท่านี้

งบใช้จ่ายต่อเดือน (บาท)

10000

เว้นว่างถ้าไม่ต้องการตั้งงบ

บันทึก

บอingkla

เสียงบอingkla

บอingklaพูดคำแนะนำจากเสียงในหน้าโฮม ปิดไว้ถึงขั้นตัวหนังสือขึ้นได้งาน

☒

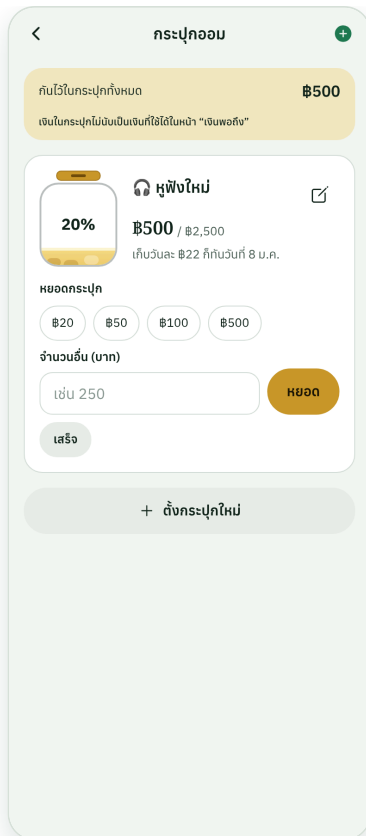
 ถูกระบบบอingkla

หน้าตาแอป

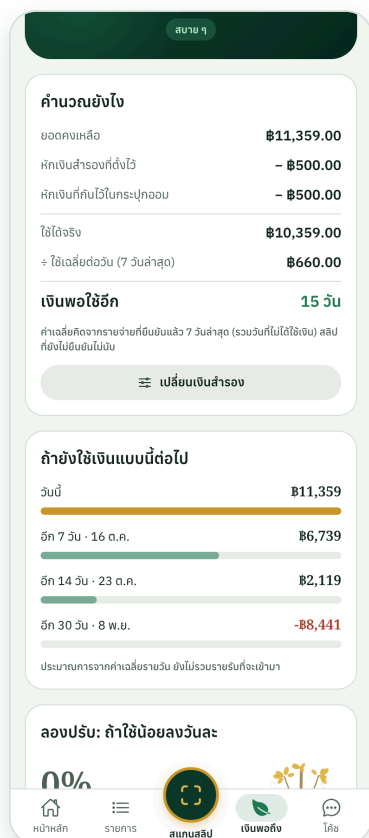
ช่อง "เงินสำรองขั้นต่ำ (บาท)" มีคำอธิบายใต้ช่องว่า "Money Runway นับวันจนยอดเงินเหลือเท่านี้" กดบันทึกแล้วค่าไปเก็บที่ `profiles.runway_floor_satang` ในหน่วยสตางค์ (บัญชีจริง) หรือเก็บในเครื่อง (โหมดทดลอง) และทุกหน้าที่ใช้ Runway ค่าวนใหม่ทันที

ตอนสมัครครั้งแรก หน้าตั้งค่าเงิน (onboarding) ก็ถามค่านี้อีกในช่อง "เงินสำรองที่ไม่อยากให้ต่ำกว่า (บาท)"

135



กระปุกออม "หูฟังใหม่" หยุด 500 จากเป้า 2,500 = 20%



หลังหยุดกระปุก 500: มีบรรทัด "หักเงินที่กันไว้ในกระปุกออม" และเหลือ 15 วัน

กระปุกออม (savings goals)

ผู้ใช้ตั้งเป้า (ชื่อ ไอคอน จำนวนเงิน กำหนดวัน 1/3/6 เดือนหรือไม่กำหนด) แล้วหยุด/ถอนได้ โผล่แก้วเติมทองพร้อมเหรียญหล่น และบอกว่าต้องเก็บวันละเท่าไรถึงกัน (เช่น "เก็บวันละ 22 ก็กันวันที่ 8 ม.ค." คือ 2,000 ที่เหลือ ÷ 92 วัน ปิดขึ้น) ครบเป้าแล้วมีพุดล่อง

การ์ดบนสุดบอกว่า "กันไว้ในกระปุกทั้งหมด 500" และ "เงินในกระปุกไม่นับเป็นเงินที่ใช้ได้ในหน้า เงินพอถึง"

ผลในหน้าเงินพอถึง: การ์ด "คำนวณยังไ้" มีบรรทัดใหม่ "หักเงินที่กันไว้ในกระปุกออม - 500.00" เงินที่ใช้ได้จริงลดเหลือ 10,359 จึงเหลือ $10,359 \div 660 = 15.7 \rightarrow 15$ วัน (จาก 16)

ถ้าลบกระปุกทิ้ง ระบบถามยืนยันก่อน และบอกว่า "เงินที่กันไว้จะกลับมานับเป็นเงินที่ใช้ได้ (ในบัญชีจริงไม่มีอะไรเปลี่ยน)" เพราะกระปุกออมเป็นแค่ "การกันเงินในใจ" ไม่ได้ย้ายเงินออกจากบัญชีธนาคาร ตารางในฐานข้อมูล savings_goals มี RLS ให้เจ้าของเท่านั้น เห็นและแก้ไขได้ (migration

[supabase/migrations/20260929000000_savings_goals.sql](#))

เบื้องหลัง: ข้อมูลไหลยังไง

จากฐานข้อมูลบนคลาวด์ จนเป็นตัวเลข "16 วัน" บนหน้าจอ

ฐานข้อมูล Supabase (PostgreSQL + RLS)

ตาราง profiles (ยอดเริ่มต้น, runway_floor_satang), transactions (ประเภท, สถานะ, จำนวนเงินเป็นสตางค์, เวลา), savings_goals (เงินในกระปุก) · RLS ทำให้แต่ละบัญชีอ่านได้เฉพาะข้อมูลของตัวเอง



ชั้นดึงข้อมูล `src/data/repo.ts`

โหลดและบันทึกข้อมูล มีสองโหมด: บัญชีจริง (คลาวด์) และโหมดทดลอง (เก็บในเครื่อง ใช้ข้อมูลตัวอย่าง)



ที่เก็บข้อมูลกลางของแอป `src/data/AppProvider.tsx`

ถือ txs, profile, goals, carrySatang และ today ให้ทุกหน้าใช้ร่วมกัน และตั้งนาฬิกาให้ today เปลี่ยนตอนเที่ยงคืนเวลาไทย



`useMoney()` ใน `AppProvider.tsx`

คิด balance, average, reserved แล้วเรียก `computeRunway` · ใช้ `useMemo` คือคิดใหม่เฉพาะตอนข้อมูลที่เกี่ยวข้องเปลี่ยน ไม่เปลืองแบตเตอรี่



สูตรล้วน (pure functions) `src/domain/runway.ts`

รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอ ฐานข้อมูล หรืออินเทอร์เน็ต จึงทดสอบได้ง่ายและได้คำตอบเดิมเสมอเมื่อใส่ข้อมูลเดิม



หน้าจอ `src/app/(tabs)/...`

หน้าหลัก (`index.tsx`), แท็บเงินพอถึง (`runway.tsx`), โค้ช (`coach.tsx`), การ์ดสิ่งที่ควรรู้ (`insights.ts`), คำพูดน้องกล้า (`buddy.ts`)

ทำไม Runway ไม่เก็บในฐานข้อมูล

เพราะมันขึ้นกับ "วันนี้" และข้อมูลทุกอย่างที่เปลี่ยนได้ตลอด ถ้าเก็บไว้จะต้องคอยอัปเดตและเสี่ยงไม่ตรงกัน ฉะนั้นจึงให้คิดสดในเครื่องทุกครั้ง ซึ่งเร็วมาก (ข้อมูลไม่ที่ร้อยรายการ) Runway จะคิดใหม่เมื่อ:

- เพิ่ม/แก้/ลบรายการ (FR-1), พุดจด (FR-3), ยืนยันสลิปหรือสลิปเข้ายอดเอง (FR-4)
- แยกเงินสำรองหรือยอดเริ่มต้นในหน้าตั้งค่า
- หยอด/ถอน/ลบกระปุกออม
- ข้ามเที่ยงคืนตามเวลาไทย (หน้าต่าง 7 วันเลื่อน)

เทคนิคที่ 1: เก็บเงินเป็น "สตางค์" (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมแบบเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ใน JavaScript ถ้าพิมพ์ `0.1 + 0.2` จะได้ `0.30000000000000004` ถ้าบวกเงินเป็นพัน ๆ รายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็นสตางค์ทั้งหมด เช่น `฿85.50` เก็บเป็น `8550` สตางค์ จำนวนเต็มบวกลบกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงผล ในโค้ดทุกตัวแปรเงินจึงลงท้ายด้วย Satang เช่น `balanceSatang`, `floorSatang` และค่าเริ่มต้นเงินสำรองเขียนว่า `50_000` (= `฿500`)

เทคนิคที่ 2: นับวันตามเวลาไทย

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าผู้ใช้โอนเงินตอน 06:00 น. ของวันที่ 8 ต.ค. เวลาไทย ในเวลา UTC จะยังเป็น 23:00 น. ของวันที่ 7 ต.ค. ถ้านับวันแบบ UTC รายการนี้จะไปอยู่ผิดวัน หน้าต่าง 7 วันก็จะผิด จนจึงแปลงทุกเวลาเป็น "วันแบบไทย" ด้วยฟังก์ชัน `bkkDayKey` (บวก 7 ชั่วโมงก่อนตัดเอาแค่วันที่) ได้ข้อความรูปแบบ "2026-10-08" ซึ่งเอามาเรียงหรือเทียบกันได้เลย

เทคนิคที่ 3: เริ่มวันใหม่ตอนเที่ยงคืน

ถ้าเปิดแอปค้างไว้ข้ามคืน "วันนี้" ต้องเปลี่ยน หน้าต่าง 7 วันต้องเลื่อน และวันที่เงินพอถึงต้องคิดใหม่ AppProvider คำนวณว่าอีกกี่มีลิวินาทีจะถึงเที่ยงคืนไทย (`msUntilNextBkkMidnight`) แล้วตั้งเวลาไว้ พอถึงเวลาค่า `today` เปลี่ยน `useMoney` ก็คิดใหม่เอง และเมื่อกลับมาเปิดแอปจากพื้นหลังก็เช็ควินาทีวันใหม่อีกครั้ง (อ่านโค้ดที่ละบรรทัดในบท A6)

แผนที่โค้ดของ FR-6

ถ้าอาจารย์ถามว่า "โค้ดส่วนนี้อยู่ตรงไหน" ตอบจากตารางนี้ (แตะชื่อไฟล์เพื่อเปิดใน VS Code)

ไฟล์	หน้าที่ใน FR-6
<code>src/domain/runway.ts</code>	สูตรทั้งหมด: averageDailyExpense, computeRunway, safeDailySpend, endOfMonthDay, runwayAfterPurchase, runwayWithReduction และค่าคงที่ (฿500, 7 วัน, 365 วัน)
<code>src/domain/summary.ts</code>	computeBalance (ยอดคงเหลือ), confirmedOnly (กรองเฉพาะที่ยืนยันแล้ว)
<code>src/domain/goals.ts</code>	reservedSatang (รวมเงินในกระปุกออม), goalProgress, perDayToReach, applyDeposit, goalLine
<code>src/domain/dates.ts</code>	bkkDayKey, addDays, daysBetween, formatThaiDay (วันที่แบบไทย พ.ศ.)
<code>src/data/AppProvider.tsx</code>	useMoney() รวมทุกอย่างแล้วส่งให้ทุกหน้า, depositToGoal
<code>src/app/(tabs)/runway.tsx</code>	เก็บเงินพอถึงทั้งหน้า
<code>src/app/(tabs)/index.tsx</code>	การ์ดเขี้ยวบนหน้าหลัก ป้ายสถานะ และ "วันนี้ใช้ได้ประมาณ"
<code>src/app/(tabs)/coach.tsx</code>	แบบ purchaseCheck ไปกับคำถาม
<code>src/domain/insights.ts</code>	การ์ดเตือน Runway เป็นข้อแรก และข้อมูลที่ส่งให้โค้ช (buildCoachContext)
<code>src/domain/buddy.ts</code>	คำพูดของน้องกล้าตามสถานะ และคำอธิบายต้นไม้ (treeLine)
<code>src/ui/art.tsx</code> , <code>src/ui/Slider.tsx</code>	ต้นไม้เงิน วงแหวน treeHealth และแถบเลื่อน
<code>src/app/settings.tsx</code> , <code>src/app/onboarding.tsx</code>	ช่องเงินสำรองขั้นต่ำ
<code>src/app/goals.tsx</code> , <code>src/app/goal.tsx</code> , <code>src/ui/Jar.tsx</code>	หน้ากระปุกออม หน้าตั้งกระปุก และภาพโหลแก้ว
<code>supabase/migrations/20260927000000_init.sql</code>	คอลัมน์ runway_floor_satang (ค่าเริ่มต้น 50000, ต้อง ≥ 0)
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	ตาราง savings_goals และ RLS
<code>src/domain/__tests__/domain.test.ts</code>	เทสต์ TC-12 ถึง TC-18
<code>src/domain/__tests__/goals.test.ts</code> , <code>src/domain/__tests__/sample.test.ts</code>	TC-71 กระปุกออม, TC-33 ข้อมูลตัวอย่าง
<code>e2e/web-auth.mjs</code>	เทสต์ในเบราว์เซอร์ (แถบเลื่อน เช็กร่อนจ่าย ตามโค้ช กระปุกออม)
<code>docs/TRACEABILITY.md</code>	ตารางโยง FR-6 → โค้ด → เทสต์

อ่านโค้ด: src/domain/runway.ts ทั้งไฟล์

ไฟล์หัวใจของ FR-6 มี 141 บรรทัด 6 ฟังก์ชัน อ่านทีละบรรทัดได้ครบทุกบรรทัด

ส่วนหัว: import ค่าคงที่ และชนิดข้อมูล

src/domain/runway.ts

บรรทัด 19-34

```
19 import { addDays, bkkDayKey, daysBetween } from './dates';
20 import { confirmedOnly } from './summary';
21 import type { Transaction } from './types';
22
23 export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500
24 export const RUNWAY_WINDOW_DAYS = 7;
25 /** Above this we just say "more than a year"; the estimate means little that far out. */
26 export const RUNWAY_CAP_DAYS = 365;
27
28 export type RunwayStatus = 'healthy' | 'watch' | 'critical' | 'below_floor' | 'no_spending';
29
30 export interface AverageSpend {
31   totalSatang: number;
32   daysCounted: number;
33   averageSatang: number;
34 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:19

บรรทัด ความหมาย

- | | |
|-------|---|
| 19 | นำฟังก์ชันวันที่แบบไทยมาใช้ 3 ตัว: บวกวัน, แปลงเวลาเป็นวันไทย, นับจำนวนวันระหว่างสองวัน |
| 20 | นำ confirmedOnly จากไฟล์ของ FR-2 มาใช้ (กรองเฉพาะรายการที่ยืนยันแล้ว) ใช้ตัวเดียวกันทั้งแอป จะได้นิยาม "ยืนยันแล้ว" ตรงกันทุกที่ |
| 21 | นำชนิด Transaction มาใช้ (import type = เอาแค่ชนิด ไม่เพิ่มขนาดแอป) |
| 23 | ค่าคงที่: เงินสำรองตั้งต้น 50,000 สตางค์ = ฿500 (export ให้ไฟล์อื่นใช้ได้) |
| 24 | ขนาดหน้าต่างค่าเฉลี่ย 7 วัน ตั้งชื่อเป็นค่าคงที่ ถ้าวันหนึ่งอยากเปลี่ยนเป็น 14 วัน แก้ที่เดียว (หน้าจอก็ใช้ค่านี้แสดงข้อความ "7 วันล่าสุด") |
| 25-26 | เพดาน 365 วัน คอมเมนต์อธิบายว่าเกินนี้ตอบแค่ "มากกว่าหนึ่งปี" เพราะประมาณการไกลขนาดนั้นไม่มีความหมาย |
| 28 | ชนิดสถานะ เป็นได้ 5 ค่าเท่านั้น (union type) ถ้าพิมพ์ผิดเช่น 'helthy' TypeScript จะฟ้องทันที |
| 30-34 | รูปแบบผลลัพธ์ของค่าเฉลี่ย: ย่อรวมรายจ่าย, จำนวนวันที่ใช้หาร, และค่าเฉลี่ย (ทั้งหมดเป็นสตางค์ ยกเว้นจำนวนวัน) |

ฟังก์ชันที่ 1: averageDailyExpense (ค่าเฉลี่ยรายจ่ายต่อวัน)

src/domain/runway.ts

บรรทัด 36-55

```
36 export function averageDailyExpense(txs: Transaction[], now: Date = new Date()): AverageSpend {
37   const today = bkkDayKey(now);
38   const windowStart = addDays(today, -(RUNWAY_WINDOW_DAYS - 1));
39   const confirmed = confirmedOnly(txs);
40   if (confirmed.length === 0) return { totalSatang: 0, daysCounted: RUNWAY_WINDOW_DAYS, averageSatang: 0 };
41
42   const firstDay = confirmed.map((t) => bkkDayKey(t.occurredAt)).sort()[0];
43   const trackedDays = Math.max(1, daysBetween(firstDay, today) + 1);
44   const daysCounted = Math.min(RUNWAY_WINDOW_DAYS, trackedDays);
45 }
```

```

46 const total = confirmed
47   .filter((t) => t.kind === 'expense')
48   .filter((t) => {
49     const d = bkkDayKey(t.occurredAt);
50     return d >= windowStart && d <= today;
51   })
52   .reduce((s, t) => s + t.amountSatang, 0);
53
54 return { totalSatang: total, daysCounted, averageSatang: Math.round(total / daysCounted) };
55 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:36

บรรทัด	ความหมาย
36	รับรายการทั้งหมด และ "ตอนนี้" (ถ้าไม่ส่งมา ใช้เวลาปัจจุบัน) การรับ now เป็นพารามิเตอร์ทำให้เกสต์กำหนดวันเองได้ ผลลัพธ์จึงคงที่ไม่ว่าวันไหน
37	แปลงเวลาตอนนี้เป็นวันไทย เช่น "2026-10-08"
38	วันเริ่มหน้าต่าง = วันนี้ - 6 วัน (RUNWAY_WINDOW_DAYS - 1) ได้ 7 วันพอดีเมื่อรวมวันนี้ เช่น 2 ถึง 8 ต.ค.
39	กรองเหลือเฉพาะรายการที่ยืนยันแล้ว แบบร่างไม่นับ
40	ถ้าไม่มีรายการที่ยืนยันเลย ตอบว่าค่าเฉลี่ย 0 กันที (computeRunway จะเห็น 0 แล้วให้สถานะ no_spending)
42	หาวันแรกที่ใช้แอป: เอาวันไทยของทุกรายการมาเรียง (sort เรียงข้อความ "ปี-เดือน-วัน" ได้ถูกลำดับเวลา) แล้วหยิบตัวแรกสุด [0]
43	จำนวนวันที่ใช้แอปมา = จากวันแรกถึงวันนี้ + 1 (นับทั้งสองวัน) และอย่างน้อย 1 (กันกรณีแปลก ๆ ที่ได้ 0)
44	จำนวนวันที่จะหาร = ค่าที่น้อยกว่าระหว่าง 7 กับจำนวนวันที่ใช้มา ปกติคือผู้ใช้ใหม่ (TC-13)
46-47	เริ่มรวมยอด: เอาเฉพาะรายจ่าย (รายรับไม่นับ)
48-51	และเฉพาะที่อยู่ในหน้าต่าง: วันของรายการต้อง ≥ วันเริ่ม และ ≤ วันนี้ (เทียบข้อความวันที่ได้ตรง ๆ เพราะรูปแบบ ปี-เดือน-วัน เรียงตามเวลาอยู่แล้ว)
52	รวมจำนวนเงิน เริ่มจาก 0
54	คืนผล: ยอดรวม, จำนวนวันที่หาร, ค่าเฉลี่ย = ยอดรวม ÷ จำนวนวัน ปิดเป็นสตางค์ที่ใกล้ที่สุด วันที่ไม่มีรายจ่ายนับรวมในตัวหารเพราะหารด้วยจำนวนวันในปฏิทิน ไม่ใช่จำนวนรายการ (TC-12)

i ลองไล่โค้ดด้วยตัวอย่าง (วันนี้ 8 ต.ค.)

บรรทัด 38: windowStart = "2026-10-02" · บรรทัด 42: ถ้าเริ่มใช้แอปตั้งแต่ 1 ก.ย. → firstDay = "2026-09-01" · บรรทัด 43: trackedDays = 37 + 1 = 38 · บรรทัด 44: daysCounted = min(7, 38) = 7 · บรรทัด 46-52: total = 101,500 สตางค์ · บรรทัด 54: average = 101,500 ÷ 7 = 14,500 สตางค์ = ฿145

ชนิดของผลลัพธ์ Runway

src/domain/runway.ts

บรรทัด 57-70

```

57 export interface Runway {
58   status: RunwayStatus;
59   /** null when it cannot be computed (no spending) */
60   days: number | null;
61   /** Bangkok day the balance is expected to reach the floor; null if unknown */
62   depletionDay: string | null;
63   /** true when days was capped at RUNWAY_CAP_DAYS */
64   capped: boolean;
65   balanceSatang: number;

```

```

66 floorSatang: number;
67 averageSatang: number;
68 /** Money above the floor that can still be spent */
69 spendableSatang: number;
70 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:57

บรรทัด	ความหมาย
58	สถานะ 1 ใน 5
59–60	จำนวนวัน หรือ null เมื่อคำนวณไม่ได้ (null แปลว่า "ไม่มีค่า" ต่างจาก 0 ที่แปลว่า "ศูนย์วัน")
61–62	วันที่คาดว่าจะอดจะแตะเส้นสำรอง (วันไทย) หรือ null
63–64	จริงเมื่อจำนวนวันถูกตัดที่ 365 (หน้าจจะแสดง "365+")
65–67	เก็บวัตถุที่ใช้คำนวณไว้ในผลด้วย หน้าจจะจะได้แสดงการ์ด "คำนวณยังไง" และฟังก์ชันอื่น (ชื่อของ, ลองปรับ) เอาไปคิดต่อได้
68–69	เงินเหนือเส้นสำรองที่ยังใช้ได้

ฟังก์ชันที่ 2: computeRunway (สูตรหลัก)

src/domain/runway.ts

บรรทัด 72–94

```

72 export function computeRunway(params: {
73   balanceSatang: number;
74   floorSatang: number;
75   averageSatang: number;
76   now?: Date;
77 }): Runway {
78   const { balanceSatang, floorSatang, averageSatang } = params;
79   const today = bkkDayKey(params.now ?? new Date());
80   const spendable = balanceSatang - floorSatang;
81   const base = { balanceSatang, floorSatang, averageSatang, spendableSatang: Math.max(0, spendable) };
82
83   if (spendable <= 0) {
84     return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85   }
86   if (averageSatang <= 0) {
87     return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88   }
89   const raw = Math.floor(spendable / averageSatang);
90   const capped = raw > RUNWAY_CAP_DAYS;
91   const days = capped ? RUNWAY_CAP_DAYS : raw;
92   const status: RunwayStatus = days < 7 ? 'critical' : days < 14 ? 'watch' : 'healthy';
93   return { ...base, status, days, depletionDay: addDays(today, days), capped };
94 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:72

บรรทัด	ความหมาย
72–77	รับ object ที่มี 4 ช่อง: ยอดคงเหลือ, เส้นสำรอง (ที่ useMoney รวมเงินในกระปุกไว้ให้แล้ว), ค่าเฉลี่ย และ now (ไม่บังคับ) คืนค่าเป็น Runway
78	แกะ 3 ค่าออกมาเป็นตัวแปร (destructuring)
79	วันนี้ตามเวลาไทย (?? = ถ้าไม่ส่ง now มา ใช้เวลาปัจจุบัน)
80	เงินที่ใช้ได้จริง = ยอด - เส้นสำรอง
81	เตรียมส่วนที่ผลทุกแบบมีเหมือนกัน เงินที่ใช้ได้ถ้าติดลบให้แสดงเป็น 0 (Math.max(0, ...)) เพราะ "ใช้ได้ -฿200" อ่านแล้วงง

บรรทัด	ความหมาย
83–85	ด้านที่ 1: ใช้ได้ $\leq 0 \rightarrow$ below_floor, 0 วัน, วันที่ = วันนี้ (เท่ากับพอดีนับ เพราะใช้ $\leq$ ตรงกับนิยาม TC-16)
86–88	ด้านที่ 2: ค่าเฉลี่ย $\leq 0 \rightarrow$ no_spending, วันและวันที่เป็น null (กินหารด้วยศูนย์ TC-15) ด้านนี้อยู่หลังด้านที่ 1 ผู้ใช้ที่เงินต่ำกว่าเส้นจึงเห็น "ต่ำกว่าเงินสำรอง" ก่อนเสมอ แม้ไม่มีรายจ่าย
89	หารแล้วปัดลง (Math.floor) ทั้งสองตัวเป็นสตริง หน่วยจึงตัดกันได้จำนวนวันพอดิ
90–91	ถ้าเกิน 365 ให้ใช้ 365 และจำว่าถูกตัด
92	ให้สถานะด้วย ternary ช้อน: น้อยกว่า 7 $\rightarrow$ critical, ไม่จึ้นน้อยกว่า 14 $\rightarrow$ watch, ไม่จึ้น healthy
93	คืนผลทั้งหมด วันที่เงินพอถึง = วันที่ + จำนวนวัน (addDays)

! ถ้าอาจารย์ถามว่า "ถ้าสลับลำดับด้าน 1 กับ 2 จะเกิดอะไร"

ผู้ใช้ใหม่ที่ยังไม่มีรายจ่ายแต่ยอดเริ่มต้นเป็น 0 จะเห็น "ยังคำนวณไม่ได้" แทน "ต่ำกว่าเงินสำรอง" ซึ่งผิดความจริง (เงินเขาต่ำกว่าเส้นแล้วจริง ๆ) ลำดับปัจจุบันจึงถูกต้อง: เช็กรื่องที่ร้ายแรงกว่าก่อน

ฟังก์ชันที่ 3: safeDailySpend (วันนี้ใช้ได้ประมาณเท่าไร)

src/domain/runway.ts

บรรทัด 96–111

```

96 /**
97  * Safe-to-spend: how much per day the user can spend and still stay above
98  * the floor until (and including) `targetDay`. Rounded down to whole baht.
99  */
100 export function safeDailySpend(params: {
101   balanceSatang: number;
102   floorSatang: number;
103   targetDay: string;
104   now?: Date;
105 }): number {
106   const today = bkkDayKey(params.now ?? new Date());
107   const days = Math.max(1, daysBetween(today, params.targetDay) + 1);
108   const spendable = params.balanceSatang - params.floorSatang;
109   if (spendable <= 0) return 0;
110   return Math.floor(spendable / days / 100) * 100;
111 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P $\rightarrow$ src/domain/runway.ts:96

บรรทัด	ความหมาย
96–99	คอมเมนต์: ใช้ได้วันละเท่าไร ถึงยังอยู่เหนือเส้นสำรองจนถึงวันเป้าหมาย (รวมวันนั้น) ปัดลงเป็นบาทเต็ม
100–105	รับยอด เส้นสำรอง วันเป้าหมาย (หน้าหลักส่งวันสิ้นเดือน) และ now
106	วันนี้ตามเวลาไทย
107	จำนวนวันจากวันนี้ถึงวันเป้าหมาย นับวันนี้ด้วย (+1) อย่างน้อย 1 วัน เช่น 8 ถึง 31 ต.ค. = $23 + 1 = 24$ วัน
108–109	เงินที่ใช้ได้ ถ้า ≤ 0 ตอบ 0 เลย (หน้าหลักจะแสดง 0 / วัน)
110	หารจำนวนวัน แล้ว ปัดลงเป็นบาทเต็ม: หาร 100 (เป็นบาท) ปัดลง แล้วคูณ 100 กลับเป็นสตริง เช่น $300,000 \div 24 = 12,500 \rightarrow$ 125

ฟังก์ชันที่ 4: endOfMonthDay (วันสิ้นเดือน)

src/domain/runway.ts

บรรทัด 113–117

```
113 /** Last day of the current Bangkok month, e.g. "2026-09-30". */
114 export function endOfMonthDay(now: Date = new Date()): string {
115   const [y, m] = bkkDayKey(now).split('-').map(Number);
116   return new Date(Date.UTC(y, m, 0)).toISOString().slice(0, 10);
117 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:113

บรรทัด ความหมาย

115 เอาวันที่ "2026-10-08" มาแยกที่ขีด แปลงเป็นตัวเลข ได้ y = 2026, m = 10 (ไม่สนวัน)

116 เทคนิค: Date.UTC(y, m, 0) ใน JavaScript เดือนนับจาก 0 (มกราคม = 0) เดือนที่ m = 10 จึงหมายถึง พฤษภาคม และวันที่ 0 ของเดือนใด ๆ = วันสุดท้ายของเดือนก่อนหน้า ผลจึงเป็น 31 ต.ค. พอได้ใช้กับทุกเดือนรวมถึงกุมภาพันธ์ปีอธิกสุรทิน

ฟังก์ชันที่ 5: runwayAfterPurchase (เช็กก่อนจ่าย)

src/domain/runway.ts

บรรทัด 119–130

```
119 /**
120  * "Before you spend" check: what happens to the runway if the user spends
121  * `amountSatang` now, keeping the same daily habit afterwards.
122  */
123 export function runwayAfterPurchase(current: Runway, amountSatang: number, now: Date = new Date()): Runway {
124   return computeRunway({
125     balanceSatang: current.balanceSatang - amountSatang,
126     floorSatang: current.floorSatang,
127     averageSatang: current.averageSatang,
128     now,
129   });
130 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:119

บรรทัด ความหมาย

119–122 คอมเมนต์: ถ้าซื้อตอนนี้แล้วใช้เงินแบบเดิมต่อ Runway จะเป็นเท่าไร

123 รับ Runway ปัจจุบัน และราคาของ (สตางค์)

124–129 ไม่เขียนสูตรใหม่ แต่ เรียก **computeRunway** ซ้ำ โดยหักราคาออกจากยอด ส่วนเส้นสำรองและค่าเฉลี่ยเท่าเดิม (การใช้ฟังก์ชันเดิมซ้ำ ทำให้กฎทุกข้อ เช่น ปิดลง, below_floor ทำงานเหมือนกันทุกที่)

ฟังก์ชันที่ 6: runwayWithReduction (ลองปรับ)

src/domain/runway.ts

บรรทัด 132–141

```
132 /** What-if: the runway if the daily average were `reducePercent` lower (0..100). */
133 export function runwayWithReduction(current: Runway, reducePercent: number, now: Date = new Date()): Runway {
134   const factor = Math.min(100, Math.max(0, reducePercent)) / 100;
135   return computeRunway({
136     balanceSatang: current.balanceSatang,
137     floorSatang: current.floorSatang,
138     averageSatang: Math.round(current.averageSatang * (1 - factor)),
139     now,
140   });
141 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:132

บรรทัด	ความหมาย
133	รับ Runway ปัจจุบัน และเปอร์เซ็นต์ที่จะลด
134	บังคับให้อยู่ในช่วง 0–100 (<code>Math.min/Math.max</code>) แล้วหาร 100 เป็นสัดส่วน เช่น 30 → 0.3
135–140	เรียก <code>computeRunway</code> เข้าด้วยค่าเฉลี่ยใหม่ = ค่าเฉลี่ยเดิม × (1 – สัดส่วน) ปัดเป็นสตางค์ เช่น $14,500 \times 0.7 = 10,150$ (\$101.50)

ไล่โค้ดแบบคอมพิวเตอร์: ค่าของตัวแปรทุกบรรทัด

ทำตัวเป็นคอมพิวเตอร์ รับฟังก์ชันทีละบรรทัด แล้วจดค่าที่เกิดขึ้นจริง (ทุกค่าในบทนี้ได้จากการรันโค้ดจริง วันที่ = 8 ต.ค. 2569)

บทที่แล้วอธิบายว่าแต่ละบรรทัด "ทำอะไร" บทนี้ตอบอีกคำถามหนึ่งที่สำคัญพอๆกัน: "ตอนรันจริง ตัวแปรแต่ละตัวมีค่าเท่าไร" โปรแกรมเมอร์เรียกวิธีนี้ว่า **trace** (ไล่รอย) ถ้าไล่ตามตารางในบทนี้ได้ด้วยดินสอกับเครื่องคิดเลข แปลว่าเข้าใจ FR-6 ลึกพอจะตอบทุกคำถามของอาจารย์แล้ว

★ วิธีอ่านตาราง trace

คอลัมน์ **บรรทัด** = เลขบรรทัดในไฟล์ `src/domain/runway.ts` · **โค้ด** = สิ่งทีบบรรทัดนั้นคำนวณ (ย่อ) · **ค่าที่ได้** = ค่าของตัวแปรหลังบรรทัดนั้นทำงานเสร็จ · เงินในโค้ดเป็น **สตางค์** เสมอ (100 สตางค์ = 1 บาท) เช่น 350000 = ฿3,500

ฟังก์ชันที่ 1: averageDailyExpense กับรายการ 9 รายการ

ข้อมูลที่ส่งเข้าไป (ตั้งใจใส่ "ตัวหลอก" 3 แบบ เพื่อดูว่าโค้ดกรองถูกไหม):

#	วัน (เวลาไทย)	ประเภท	จำนวน	สถานะ	หมายเหตุ
1	1 ก.ย.	รายรับ	฿5,000	ยืนยันแล้ว	รายการแรกสุด (วันแรกที่ใช้แอป)
2	1 ต.ค.	รายจ่าย	฿999	ยืนยันแล้ว	ตัวหลอก: อยู่นอกหน้าต่าง 7 วัน
3	2 ต.ค.	รายจ่าย	฿120	ยืนยันแล้ว	
4	4 ต.ค.	รายจ่าย	฿85	ยืนยันแล้ว	
5	5 ต.ค.	รายจ่าย	฿300	ยืนยันแล้ว	
6	6 ต.ค.	รายรับ	฿1,000	ยืนยันแล้ว	ตัวหลอก: รายรับไม่นับในค่าเฉลี่ย
7	7 ต.ค.	รายจ่าย	฿450	ยืนยันแล้ว	
8	8 ต.ค.	รายจ่าย	฿60	ยืนยันแล้ว	
9	8 ต.ค.	รายจ่าย	฿5,000	แบบร่าง	ตัวหลอก: สลิปที่ยังไม่ยืนยัน

src/domain/runway.tsบรรทัด 36-55

```
36 export function averageDailyExpense(txs: Transaction[], now: Date = new Date()): AverageSpend {
37   const today = bkkDayKey(now);
38   const windowStart = addDays(today, -(RUNWAY_WINDOW_DAYS - 1));
39   const confirmed = confirmedOnly(txs);
40   if (confirmed.length === 0) return { totalSatang: 0, daysCounted: RUNWAY_WINDOW_DAYS, averageSatang: 0 };
41
42   const firstDay = confirmed.map((t) => bkkDayKey(t.occurredAt)).sort()[0];
43   const trackedDays = Math.max(1, daysBetween(firstDay, today) + 1);
44   const daysCounted = Math.min(RUNWAY_WINDOW_DAYS, trackedDays);
45
46   const total = confirmed
47     .filter((t) => t.kind === 'expense')
48     .filter((t) => {
49       const d = bkkDayKey(t.occurredAt);
50       return d >= windowStart && d <= today;
51     })
52     .reduce((s, t) => s + t.amountSatang, 0);
53 }
```

```
54 return { totalSatang: total, daysCounted, averageSatang: Math.round(total / daysCounted) };
55 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:36

บรรทัด	โค้ด	ค่าที่ได้	อธิบาย
37	today = bkkDayKey(now)	"2026-10-08"	วันนี้ตามเวลาไทย
38	windowStart = addDays(today, -6)	"2026-10-02"	หน้าต่างคือ 2-8 ต.ค. (7 วัน)
39	confirmed = confirmedOnly(txs)	8 รายการ (จาก 9)	รายการที่ 9 (แบบร่าง ฿5,000) หลุดตรงนี้
40	confirmed.length === 0 ?	false	มีรายการ จึงไปต่อ
42	firstDay = ...sort()[0]	"2026-09-01"	รายการแรกสุดคือรายรับ 1 ก.ย. (นับรายรับด้วย เพราะหา "วันแรกที่ใช้แอป")
43	trackedDays = max(1, daysBetween + 1)	37 + 1 = 38	ใช้แอปมาแล้ว 38 วัน (นับทั้ง 1 ก.ย. และ 8 ต.ค.)
44	daysCounted = min(7, 38)	7	ไม่ใช่ผู้ใช้ใหม่ จึงหาร 7
47	.filter(kind === 'expense')	6 รายการ: 1, 2, 4, 5, 7, 8 ต.ค.	รายรับ 2 รายการ (#1, #6) หลุดตรงนี้
48-51	.filter(windowStart ≤ d ≤ today)	5 รายการ: 2, 4, 5, 7, 8 ต.ค.	฿999 ของ 1 ต.ค. หลุดตรงนี้ ("2026-10-01" < "2026-10-02")
52	.reduce((s, t) => s + t.amountSatang, 0)	0 → 12000 → 20500 → 50500 → 95500 → 101500	บวกทีละรายการ เริ่มจาก 0
54	Math.round(101500 / 7)	14500 (= ฿145)	101500 ÷ 7 = 14500 พอดี จึงไม่ต้องปัด

i ผลจริงที่ฟังก์ชันคืนมา

{ "totalSatang": 101500, "daysCounted": 7, "averageSatang": 14500 } ตรงกับที่ได้ด้วยมือทุกช่อง ตัวหลอกทั้ง 3 ตัวถูกรองออกคนละบรรทัด: แบบร่างหลุดที่บรรทัด 39 รายรับหลุดที่บรรทัด 47 และรายการเก่าหลุดที่บรรทัด 48-51

ฟังก์ชันที่ 2: computeRunway ทีละบรรทัด (ยอด ฿3,500 สำรอง ฿500 เหลือ ฿145)

src/domain/runway.ts

บรรทัด 72-94

```
72 export function computeRunway(params: {
73   balanceSatang: number;
74   floorSatang: number;
75   averageSatang: number;
76   now?: Date;
77 }): Runway {
78   const { balanceSatang, floorSatang, averageSatang } = params;
79   const today = bkkDayKey(params.now ?? new Date());
80   const spendable = balanceSatang - floorSatang;
81   const base = { balanceSatang, floorSatang, averageSatang, spendableSatang: Math.max(0, spendable) };
82
83   if (spendable <= 0) {
84     return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85   }
86   if (averageSatang <= 0) {
87     return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88   }
89 }
```



```

89 const raw = Math.floor(spendable / averageSatang);
90 const capped = raw > RUNWAY_CAP_DAYS;
91 const days = capped ? RUNWAY_CAP_DAYS : raw;
92 const status: RunwayStatus = days < 7 ? 'critical' : days < 14 ? 'watch' : 'healthy';
93 return { ...base, status, days, depletionDay: addDays(today, days), capped };
94 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/runway.ts:72`

บรรทัด	โค้ด	ค่าที่ได้	อธิบาย
78	<code>{ balanceSatang, floorSatang, averageSatang } = params</code>	350000, 50000, 14500	แกะค่าจากอ็อบเจกต์ที่ส่งเข้ามา
79	<code>today = bkkDayKey(params.now ?? new Date())</code>	"2026-10-08"	เทสต์ส่ง now มา จึงไม่ใช่เวลาจริง
80	<code>spendable = 350000 - 50000</code>	300000	เงินที่ใช้ได้จริง ฿3,000
81	<code>base = { ..., spendableSatang: max(0, 300000) }</code>	{350000, 50000, 14500, 300000}	ส่วนที่ผลทุกแบบมีเหมือนกัน
83	<code>300000 <= 0 ?</code>	false	ไม่ใช่ below_floor ข้ามบรรทัด 84
86	<code>14500 <= 0 ?</code>	false	ไม่ใช่ no_spending ข้ามบรรทัด 87
89	<code>raw = Math.floor(300000 / 14500)</code>	<code>floor(20.6896...) = 20</code>	ปัดลง ทั้ง 0.69
90	<code>capped = 20 > 365</code>	false	ไม่ถึงเพดาน
91	<code>days = capped ? 365 : raw</code>	20	ใช้ค่า raw
92	<code>20 < 7 ? ... : 20 < 14 ? ... : 'healthy'</code>	'healthy'	ไม่น้อยกว่า 7 และไม่น้อยกว่า 14
93	<code>depletionDay = addDays("2026-10-08", 20)</code>	"2026-10-28"	คืนผลทั้งหมด

ทางที่โค๊ดเดิน ในสถานการณ์อื่น (ผลจริง)

สถานการณ์	spendable (บ.80)	ออกที่ บรรทัด	ผลลัพธ์
ยอด ฿500 · สำรอง ฿500	0	84 (ด้าน 1)	below_floor · 0 วัน · วันที่ = วันนี้ (8 ต.ค.)
ยอด ฿3,500 · ค่าเฉลี่ย ฿0	300000	87 (ด้าน 2)	no_spending · days = null · วันที่ = null
ยอด ฿600 · สำรอง ฿500 · เฉลี่ย ฿145	10000	93	<code>floor(0.69) = 0</code> วัน · critical (ยังเหนือเส้น จึงไม่ใช่ below_floor)
ยอด ฿100,000 · เฉลี่ย ฿10	9950000	93	<code>raw = 9,950</code> → <code>capped = true</code> → 365 วัน · ถึง 8 ต.ค. 2570

ฟังก์ชันที่ 3 และ 4: safeDailySpend กับ endOfMonthDay

`src/domain/runway.ts`

บรรทัด 100–117

```

100 export function safeDailySpend(params: {
101   balanceSatang: number;
102   floorSatang: number;
103   targetDay: string;
104   now?: Date;

```

```

105 }): number {
106   const today = bkkDayKey(params.now ?? new Date());
107   const days = Math.max(1, daysBetween(today, params.targetDay) + 1);
108   const spendable = params.balanceSatang - params.floorSatang;
109   if (spendable <= 0) return 0;
110   return Math.floor(spendable / days / 100) * 100;
111 }
112
113 /** Last day of the current Bangkok month, e.g. "2026-09-30". */
114 export function endOfMonthDay(now: Date = new Date()): string {
115   const [y, m] = bkkDayKey(now).split('-').map(Number);
116   return new Date(Date.UTC(y, m, 0)).toISOString().slice(0, 10);
117 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:100

บรรทัด	โค้ด	ค่าที่ได้	อธิบาย
115	<code>[y, m] = "2026-10-08".split('-').map(Number)</code>	y = 2026, m = 10	"2026-10-08" → ["2026", "10", "08"] → [2026, 10, 8] เอาแค่ 2 ตัวแรก
116	<code>Date.UTC(2026, 10, 0) → toISOString().slice(0, 10)</code>	"2026-10-31"	เดือน 10 ของ JavaScript คือพฤศจิกายน และวันที่ 0 = วันสุดท้ายของเดือนก่อน
106	<code>today</code>	"2026-10-08"	
107	<code>days = max(1, daysBetween(today, "2026-10-31") + 1)</code>	23 + 1 = 24	8 ถึง 31 ต.ค. นับวันนี้ด้วย
108	<code>spendable = 350000 - 50000</code>	300000	
109	<code>300000 <= 0 ?</code>	false	
110	<code>Math.floor(300000 / 24 / 100) * 100</code>	$\text{floor}(125) \times 100 = 12500$	฿125 ต่อวัน (ปัดลงเป็นบาทเต็ม)

ทดสอบ endOfMonthDay กับเดือนอื่นจริง: ก.พ. 2028 (ปีอธิกสุรทิน) → "2028-02-29" · ก.พ. 2027 → "2027-02-28" · 31 ธ.ค. 2026 เวลา 23:30 น. ไทย → "2026-12-31" ถูกทุกกรณีโดยไม่ต้องจำว่าเดือนไหนมีกี่วัน

ฟังก์ชันที่ 5 และ 6: เช็กก่อนจ่าย และลองปรับ

src/domain/runway.ts

บรรทัด 119–141

```

119 /**
120  * "Before you spend" check: what happens to the runway if the user spends
121  * `amountSatang` now, keeping the same daily habit afterwards.
122  */
123 export function runwayAfterPurchase(current: Runway, amountSatang: number, now: Date = new Date()): Runway {
124   return computeRunway({
125     balanceSatang: current.balanceSatang - amountSatang,
126     floorSatang: current.floorSatang,
127     averageSatang: current.averageSatang,
128     now,
129   });
130 }
131
132 /** What-if: the runway if the daily average were `reducePercent` lower (0..100). */
133 export function runwayWithReduction(current: Runway, reducePercent: number, now: Date = new Date()): Runway {
134   const factor = Math.min(100, Math.max(0, reducePercent)) / 100;
135   return computeRunway({
136     balanceSatang: current.balanceSatang,
137     floorSatang: current.floorSatang,
138     averageSatang: Math.round(current.averageSatang * (1 - factor)),
139     now,

```

```
140 });
141 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:119

บรรทัด	โค้ด	ค่าที่ได้	อธิบาย
124-129	computeRunway({ balance: 350000 - 60000, ... })	290000 → spendable 240000 → 16.55 → 16 วัน ถึง 24 ต.ค.	ซื้อของ \$600
124-129	computeRunway({ balance: 350000 - 400000, ... })	-50000 → spendable -100000 → below_floor	ซื้อของ \$4,000 (แพงกว่าเงินที่ใช้ได้)
134	factor = min(100, max(0, 30)) / 100	0.3	ลด 30%
138	Math.round(14500 × (1 - 0.3))	10150	ค่าเฉลี่ยใหม่ \$101.50
135-140	computeRunway({ average: 10150 })	300000 ÷ 10150 = 29.56 → 29 วัน ถึง 6 พ.ย.	เพิ่มจาก 20 วัน 9 วัน
134	min(100, max(0, 150)) / 100	1	ถ้ามีใครส่ง 150% มา ถูกบับเหลือ 100%
138	Math.round(14500 × 0)	0	ค่าเฉลี่ย 0 → computeRunway ตอบ no_spending (ไม่พัง)

ลองปรับ (ผลจริง)	0%	10%	20%	30%	40%	50%
ค่าเฉลี่ยใหม่ (สตางค์)	14500	13050	11600	10150	8700	7250
จำนวนวัน	20	22	25	29	34	41
ถึงวันที่	28 ต.ค.	30 ต.ค.	2 พ.ย.	6 พ.ย.	11 พ.ย.	18 พ.ย.

ลองรันเองใน VS Code (5 นาที)

วิธีที่ดีที่สุดในการเข้าใจโค้ดคือ **เปลี่ยนตัวเลขแล้วดูผล** สร้างไฟล์ทดสอบนี้ในโฟลเดอร์ `src/domain/__tests__` (ชื่ออะไรก็ได้ที่ลงท้าย .test.ts)

src/domain/__tests__/my-runway.test.ts (ไฟล์ทดลอง ไม่ต้อง commit)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 import { describe, expect, it } from 'vitest';
2 import { averageDailyExpense, computeRunway } from '../runway';
3
4 describe('ลองของฉัน', () => {
5   it('ยอด 3,500 สำรอง 500 เฉลี่ย 145 ได้ 20 วัน', () => {
6     const r = computeRunway({
7       balanceSatang: 350_000, // ลองเปลี่ยนเป็น 50_000 แล้วดู status
8       floorSatang: 50_000,
9       averageSatang: 14_500, // ลองเปลี่ยนเป็น 0
10      now: new Date('2026-10-08T05:00:00Z'),
11    });
12    console.log(r); // พิมพ์ผลทั้งหมดออกมาดู
13    expect(r.days).toBe(20);
14    expect(r.depletionDay).toBe('2026-10-28');
15  });
16 });
```

1. เปิด Terminal ใน VS Code (**Ctrl** + **`** ) แล้วพิมพ์ `npx vitest run src/domain/__tests__/my-runway.test.ts --reporter=verbose` (ต้องมี `--reporter=verbose` เพราะ Vitest รุ่นที่โปรเจกต์ใช้ซ่อนข้อความ

console.log ของเทสที่ผ่าน)

2. จะเห็นอีอบเจกต์ที่ console.log พิมพ์ออกมา (days: 20, status: 'healthy', depletionDay: '2026-10-28') ตามด้วย ✓ และ **Tests 1 passed (1)** ฉะนั้นจริงแล้วได้แบบนี้
3. ลองเปลี่ยน balanceSatang เป็น 50_000 แล้วรันใหม่ เทสจะ **ไม่ผ่าน** พร้อมข้อความ AssertionError: expected +0 to be 20 (ได้ 0 วันเพราะกลายเป็น below_floor แต่เทสคาดว่า 20) นี่คือนี่วิธีที่เทสช่วยจับความผิดพลาด
4. ทดลองเสร็จแล้วลบไฟล์ทิ้ง (ไฟล์นี้มีไว้ฝึก ไม่ต้องเพิ่มเข้าโปรเจกต์)

พจนานุกรมสัญลักษณ์: อ่าน runway.ts ออกทุกตัวอักษร

ทุกเครื่องหมายและคำสั่งที่ปรากฏในไฟล์หัวใจของ FR-6 พร้อมตัวอย่างจากไฟล์จริง เปิดหน้านี้ค้างไว้ข้าง VS Code ระหว่างอ่านโค้ด

คนที่ไม่เคยเขียนโค้ดมักติดที่ "เครื่องหมายแปลก ๆ" มากกว่าตรรกะ บทนี้รวบรวม **ทุกสัญลักษณ์** ที่อยู่ใน `src/domain/runway.ts` (141 บรรทัด) เรียงตามทีเจอในไฟล์ แต่ละแถวบอกว่า อ่านออกเสียงว่าจะไร ทำอะไร และยกบรรทัดจริงมาให้ดู

1. โครงของไฟล์

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>/** ... */</code>	คอมเมนต์หลายบรรทัด คอมพิวเตอร์ไม่อ่าน มีไว้ให้คนอ่าน ถ้าขึ้นต้นด้วย <code>/**</code> VS Code จะเอาไปโชว์เป็นคำอธิบายเวลาเอาเมาส์ชี้ชื่อฟังก์ชัน · บรรทัด 1-18 คือนิยาม 5 ข้อของ FR-6
<code>// ...</code>	คอมเมนต์ถึงท้ายบรรทัด · บรรทัด 23: <code>50_000; // ฿500</code>
<code>import { a, b } from './x'</code>	"ยืม" ฟังก์ชันจากไฟล์อื่นมาใช้ · บรรทัด 19: ยืม <code>addDays, bkkDayKey, daysBetween</code> จากไฟล์ <code>dates.ts</code> (<code>./</code> = โฟลเดอร์เดียวกัน)
<code>import type</code>	ยืมเฉพาะ "ชนิดข้อมูล" ใช้ตอนตรวจโค้ดเท่านั้น หายไปตอนแปลงเป็น JavaScript · บรรทัด 21
<code>export</code>	อนุญาตให้ไฟล์อื่นยืมสิ่งนี้ไปใช้ ไม่มี <code>export =</code> ใช้ได้แค่ในไฟล์นี้ · ทุกฟังก์ชันใน <code>runway.ts</code> มี <code>export</code> เพราะหน้าจอลและเทสต์ต้องเรียก
<code>;</code>	จบคำสั่ง (เหมือนจุดจบประโยค)

2. คำคงที่ ตัวแปร และชนิดข้อมูล

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>const ชื่อ = ค่า</code>	สร้างกล่องเก็บค่าที่ เปลี่ยนไม่ได้ · บรรทัด 24: <code>const RUNWAY_WINDOW_DAYS = 7</code> · ชื่อตัวพิมพ์ใหญ่ทั้งหมดเป็นธรรมเนียมว่า "นี่คือค่าตั้ง" (ไม่ได้บังคับ)
<code>50_000</code>	ตัวเลข 50000 ชิดล่างช่วยให้อ่านง่ายเหมือนจุลภาค (คอมพิวเตอร์ไม่สนขีด) = 50,000 สตางค์ = ฿500
<code>type A = 'x' 'y'</code>	ตั้งชื่อชนิดข้อมูลที่ "เป็นได้แค่ค่าเหล่านี้" (อ่านว่า "หรือ") · บรรทัด 28: สถานะเป็นได้ 5 ค่าเท่านั้น พิมพ์ผิดเป็น 'helthy' TypeScript ชิดเส้นแดงทันที
<code>interface ชื่อ { ... }</code>	กำหนด "หน้าตา" ของอ็อบเจกต์ว่าต้องมีช่องอะไร ชนิดอะไร · บรรทัด 30-34 <code>AverageSpend</code> มี 3 ช่องเป็นตัวเลข
<code>ชื่อ: number</code>	เครื่องหมาย : หลังชื่อ = "เป็นชนิด" · <code>totalSatang: number</code> = ต้องเป็นตัวเลข
<code>number null</code>	เป็นตัวเลข หรือ null ("ไม่มีค่า") · บรรทัด 60: <code>days</code> เป็น null ได้เมื่อคำนวณไม่ได้ (ต่างจาก 0 ที่แปลว่าศูนย์วัน)
<code>Transaction[]</code>	[] หลังชนิด = "อาร์เรย์ (รายการหลายตัว) ของ..." · <code>txs: Transaction[]</code> = รายการธุรกรรมหลายรายการ
<code>now?: Date</code>	? หลังชื่อ = ช่องนี้ "ไม่ส่งก็ได้" · บรรทัด 76

3. ฟังก์ชัน

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>function ชื่อ(พารามิเตอร์): ชนิดผล { ... }</code>	ประกาศฟังก์ชัน = เครื่องที่รับของเข้า ทำงาน แล้วส่งผลออก · บรรทัด 36: รับ txs และ now คืน AverageSpend
<code>now: Date = new Date()</code>	ค่าเริ่มต้นของพารามิเตอร์: ถ้าไม่ส่ง now มา ใช้เวลาปัจจุบัน (<code>new Date()</code> = เวลาตอนนี้) · เทสต์ส่งวันที่ตายตัวมาแทน ผลจึงเหมือนเดิมทุกครั้ง
<code>params: { a: number; ... }</code>	รับพารามิเตอร์เป็น "อ็อบเจกต์ก้อนเดียว" ที่มีหลายช่อง อ่านง่ายกว่าส่งตัวเลขเรียงกัน 4 ตัว · บรรทัด 72–77
<code>return ค่า</code>	ส่งผลออกและ จบฟังก์ชันทันที บรรทัดหลังจากนั้นไม่ทำงาน · บรรทัด 84 ถ้าเงินหมด ไม่ไปถึง บรรทัด 89 เลย
<code>(t) => ...</code>	"ฟังก์ชันลูกศร" ฟังก์ชันสั้น ๆ ไม่มีชื่อ อ่านว่า "รับ t แล้วได้ ..." · <code>(t) => t.kind === 'expense'</code> = รับรายการ t แล้วตอบว่าเป็นรายจ่ายไหม

4. อ็อบเจกต์และอาร์เรย์

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>{ a: 1, b: 2 }</code>	อ็อบเจกต์ = กล่องที่มีหลายช่อง แต่ละช่องมีชื่อ · ผลของ <code>computeRunway</code> เป็นอ็อบเจกต์ที่มีช่อง <code>status, days, depletionDay ...</code>
<code>t.kind</code>	จุด = "ช่องของ" · <code>t.amountSatang</code> = จำนวนเงินของรายการ t
<code>const { a, b } = obj</code>	"แกะกล่อง" (destructuring): ดึงช่อง a กับ b ออกมาเป็นตัวแปรในบรรทัดเดียว · บรรทัด 78 แกะ 3 ค่าจาก params
<code>{ balanceSatang, floorSatang }</code>	เขียนย่อ: <code>{ balanceSatang } = { balanceSatang: balanceSatang }</code> เมื่อชื่อช่องกับชื่อตัวแปรเหมือนกัน · บรรทัด 81
<code>{ ...base, status }</code>	<code>... (spread)</code> = "เลขของทุกช่องจาก base ใส่กล่องใหม่" แล้วเพิ่ม/ทับช่องอื่น · บรรทัด 84, 87, 93 ใช้ base ร่วมกัน ไม่ต้องเขียนซ้ำ 3 รอบ
<code>const [y, m] = arr</code>	แกะอาร์เรย์ตามลำดับ: ตัวแรกเป็น y ตัวที่สองเป็น m · บรรทัด 115
<code>arr[0]</code>	ตัวแรกของอาร์เรย์ (นับจาก 0) · บรรทัด 42 <code>.sort()</code> [0] = ตัวที่น้อยที่สุดหลังเรียง = วันแรกสุด
<code>.length</code>	จำนวนสมาชิก · บรรทัด 40 <code>confirmed.length === 0</code> = ไม่มีรายการเลย

5. เมธอดของอาร์เรย์ (ใช้ต่อกันเป็นสาย)

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>.filter(เงื่อนไข)</code>	เก็บเฉพาะตัวที่เงื่อนไขเป็นจริง (เหมือนตะแกรงร่อน) · บรรทัด 47 เก็บแต่รายจ่าย บรรทัด 48–51 เก็บแต่ที่อยู่ในหน้าต่าง
<code>.map(แปลง)</code>	แปลงทุกตัวเป็นค่าใหม่ ได้อาร์เรย์ยาวเท่าเดิม · บรรทัด 42 แปลงรายการเป็น "วันไทย" · บรรทัด 115 <code>.map(Number)</code> แปลงข้อความ "2026" เป็นตัวเลข 2026
<code>.reduce((s, t) => s + t.x, 0)</code>	"ยุบ" ทั้งอาร์เรย์ให้เหลือค่าเดียว: เริ่มที่ 0 แล้วบวกเข้าไปทีละตัว (<code>s</code> = ยอดสะสม, <code>t</code> = ตัวปัจจุบัน) · บรรทัด 52 รวมรายจ่าย 0 → 12000 → ... → 101500
<code>.sort()</code>	เรียงจากน้อยไปมาก (ข้อความ "2026-09-01" เรียงตามเวลาได้ถูก เพราะเขียนปี-เดือน-วัน)

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>.split('-')</code>	หั่นข้อความตรงเครื่องหมาย - · "2026-10-08" → ["2026", "10", "08"]
<code>.slice(0, 10)</code>	ตัดข้อความตำแหน่ง 0 ถึงก่อน 10 · "2026-10-31T00:00:00.000Z" → "2026-10-31"

6. การเปรียบเทียบและการตัดสินใจ

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>===</code>	เท่ากันจริง ๆ (ทั้งค่าและชนิด) · <code>t.kind === 'expense'</code> · <code>ระวัง</code> : ตัวเดียวคือ "ใส่ค่า" ไม่ใช่ "เทียบ"
<code><= >= < ></code>	น้อยกว่าหรือเท่ากับ / มากกว่าหรือเท่ากับ / น้อยกว่า / มากกว่า · <code>ussกั 83 spendable <= 0</code> (เท่ากับศูนย์ก็นับว่าแตะเส้น ตรงนิยาม TC-16)
<code>&&</code>	"และ" ต้องจริงทั้งสองฝั่ง · <code>ussกั 50 d >= windowStart && d <= today</code> = อยู่ในหน้าต่าง
<code>if (เงื่อนไข) { ... }</code>	ถ้าจริงทำในวงเล็บปีกกา ถ้าไม่จริงข้ามไป · <code>ussกั 83 และ 86</code> คือด่านตรวจ 2 ด่าน
<code>ก ? ข : ค</code>	"ternary" = if-else แบบสั้น: ถ้า ก จริงได้ ข ไม่จริงได้ ค · <code>ussกั 91 capped ? 365 : raw</code> · <code>ussกั 92</code> ข้อ 2 ชั้น: น้อยกว่า 7 → critical ไม่เกิน (น้อยกว่า 14 → watch ไม่เกิน healthy)
<code>a ?? b</code>	"ถ้า a ไม่มีค่า (null หรือ undefined) ใช้ b" · <code>ussกั 79 params.now ?? new Date()</code>

7. คณิตศาสตร์และวันที่

สัญลักษณ์	ความหมาย และตัวอย่างจาก runway.ts
<code>Math.floor(x)</code>	ปัดลง เป็นจำนวนเต็ม · <code>floor(20.69) = 20</code> · <code>ussกั 89 และ 110</code>
<code>Math.round(x)</code>	ปัดใกล้สุด (.5 ขึ้น) · <code>ussกั 54</code> ปัดค่าเฉลี่ยเป็นสตริง · <code>ussกั 138</code>
<code>Math.max(a, b) / Math.min(a, b)</code>	ค่าที่มากกว่า / น้อยกว่า · <code>Math.max(0, spendable) = ไม่ให้ติดลบ</code> · <code>Math.min(7, trackedDays) = ไม่เกิน 7</code> · ใช้ทั้งคู่ซ้อนกันเพื่อ "บีบให้อยู่ในช่วง" (<code>ussกั 134: 0 ถึง 100</code>)
<code>-(RUNWAY_WINDOW_DAYS - 1)</code>	วงเล็บคิดก่อน: <code>7 - 1 = 6</code> แล้วใส่ลบ = <code>-6</code> · <code>ussกั 38</code> ย้อนไป 6 วัน
<code>Date.UTC(y, m, d)</code>	สร้างเวลาแบบสากล · เดือนนับจาก 0 (มกราคม = 0) และวันที่ 0 = วันสุดท้ายของเดือนก่อน · <code>ussกั 116</code> ใช้หาวันสิ้นเดือน
<code>.toISOString()</code>	แปลงเวลาเป็นข้อความมาตรฐาน "2026-10-31T00:00:00.000Z"
<code>bkkDayKey / addDays / daysBetween</code>	ฟังก์ชันวันที่ของ MindPay เอง (ไฟล์ <code>dates.ts</code>): แปลงเป็นวันไทย "YYYY-MM-DD" / บวกลบวัน / นับวันระหว่างสองวัน

≈ อ่านโค้ดให้เหมือนอ่านภาษาไทย

`ussกั 92` `const status = days < 7 ? 'critical' : days < 14 ? 'watch' : 'healthy'`; อ่านเป็นประโยคได้ว่า "ให้ status เป็น: ถ้าจำนวนวันน้อยกว่า 7 ก็ critical ไม่เกินถ้าน้อยกว่า 14 ก็ watch นอกนั้น healthy" ลองอ่านทุกบรรทัดของ `runway.ts` ออกเสียงเป็นภาษาไทยแบบนี้ ถ้าอ่านได้ครบ ถือว่าอ่านโค้ดออกแล้ว

อ่านโค้ด: หน้าจอแท็บเงินพอถึง (runway.tsx)

ไฟล์ src/app/(tabs)/runway.tsx 250 บรรทัด แบ่งอ่านเป็น 7 ส่วนตามที่เห็นบนจอ

ส่วนที่ 1: import และฟังก์ชันช่วยแสดงจำนวนวัน

src/app/(tabs)/runway.tsx

บรรทัด 1–26

```
1 /**
2  * FR-6 Money Runway, in full: the number, how it is calculated, what the next
3  * 30 days look like, and two "what if" tools (spend less / check before buying).
4  */
5 import { LinearGradient } from 'expo-linear-gradient';
6 import { router } from 'expo-router';
7 import { useMemo, useState } from 'react';
8 import { View } from 'react-native';
9 import { useApp, useMoney } from '../../data/AppProvider';
10 import { addDays, formatThaiDay } from '../../domain/dates';
11 import { formatBaht, parseBahtToSatang } from '../../domain/money';
12 import { runwayAfterPurchase, runwayWithReduction, RUNWAY_WINDOW_DAYS, type Runway } from
13   '../../domain/runway';
14 import { ContourLines, MoneyTree, ProgressRing, treeHealth } from '../../ui/art';
15 import { Badge, Button, Card, Divider, Ionicons, Money, Row, Screen, T } from '../../ui/components';
16 import { Aurora, GrowBar, Reveal, Sparkles } from '../../ui/effects';
17 import { Field } from '../../ui/inputs';
18 import { useCountUp, useReduceMotion } from '../../ui/motion';
19 import { Slider } from '../../ui/Slider';
20 import { fonts, palette, radius, space, useTheme } from '../../ui/theme';
21 import { TitleDecor } from '../../ui/halloween';
22
23 function daysText(r: Runway) {
24   if (r.status === 'below_floor') return 'แตะเส้นสำรองแล้ว';
25   if (r.days === null) return 'คำนวณไม่ได้';
26   return `${r.capped ? 'มากกว่า ' : ''}${r.days} วัน`;
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/runway.tsx:1

บรรทัด	ความหมาย
1–4	คอมเมนต์: หน้านี้แสดง Money Runway เต็ม ๆ ตัวเลข วิธีคำนวณ 30 วันข้างหน้า และเครื่องมือ "ถ้า..." 2 อย่าง
5	สไตล์ระดับของการ์ดเขียว
6	router สำหรับเปลี่ยนหน้า (ไปตั้งค่า, ไปโค้ช)
7–8	hook ของ React และกล่อง View
9	ข้อมูลกลาง: useApp (วันนี้) และ useMoney (ตัวเลขเงิน)
10–12	ฟังก์ชันจาก domain: บวกวัน, แสดงวันที่ไทย, แปลง/แสดงเงิน, สูตร FR-6 สองตัว และค่าคงที่ 7 วัน
13	ภาพวาด: เส้นสายการ์ด, ต้นไม้เงิน, วงแหวน และ treeHealth
14–20	ชิ้นส่วนหน้าต่างจาก Design System, เอฟเฟกต์ (ออโรรา แท่งโต ค่อย ๆ โผล่ ปรากฏ), ช่องกรอก, ตัวนับเลข, แถบเลื่อน, คำคม, ของตกแต่งหัวข้อ (ธัมฮาโลวีน)
22–26	daysText แปลง Runway เป็นข้อความสั้น ใช้ซ้ำหลายที่ในหน้านี้: ต่ำกว่าเส้น → "แตะเส้นสำรองแล้ว", ไม่มีจำนวนวัน → "คำนวณไม่ได้", ถูกตัด → "มากกว่า 365 วัน", ปกติ → "16 วัน"

ส่วนที่ 2: เตรียมข้อมูลของหน้า

src/app/(tabs)/runway.tsx

บรรทัด 28-49

```
28 export default function RunwayScreen() {
29   const theme = useTheme();
30   const { balance, runway, average, reserved, floor } = useMoney();
31   const { today } = useApp();
32   const [reduce, setReduce] = useState(0);
33   const [price, setPrice] = useState('');
34
35   const reduced = useMemo(() => runwayWithReduction(runway, reduce), [runway, reduce]);
36   const reduceMotion = useReduceMotion();
37   const daysShown = useCountUp(runway.days ?? 0, !reduceMotion && runway.days !== null && !runway.capped,
1100);
38   const reducedDays = useCountUp(reduced.days ?? 0, !reduceMotion && reduced.days !== null, 500, reduced.days
?? 0);
39   /** How full the ring is: 45 days or more is a full, comfortable month and a half. */
40   const fullness = runway.status === 'below_floor' ? 0.02 : runway.days === null ? 0 : Math.min(1, runway.days
/ 45);
41   const priceSatang = parseBahtToSatang(price);
42   const afterBuy = priceSatang ? runwayAfterPurchase(runway, priceSatang) : null;
43
44   const timeline = [0, 7, 14, 30].map((d) => ({
45     d,
46     day: addDays(today, d),
47     balance: balance - runway.averageSatang * d,
48   }));
49   const maxBal = Math.max(1, balance);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/runway.tsx:28

บรรทัด	ความหมาย
28	component หลักของหน้า (export default ให้ Expo Router ใช้เป็นหน้าจอ)
29	สีของธีม
30	ดึงตัวเลขที่คำนวณแล้ว จาก useMoney: ยอด, Runway, ค่าเฉลี่ย, เงินในกระปุก, เส้นสำรอง
31	วันนี้ (ใช้คำนวณวันที่ของเส้นเวลา)
32	state ของแถบลองปรับ เริ่มที่ 0%
33	state ของช่องราคาในเช็กร่อนจ่าย (ข้อความที่พิมพ์)
35	Runway หลังลดค่าใช้จ่าย คำนวณใหม่เฉพาะเมื่อ runway หรือเปอร์เซ็นต์เปลี่ยน (useMemo)
36	ผู้ใช้เปิด "ลดการเคลื่อนไหว" ไหม
37	ตัวเลขวันแบบนับขึ้นใน 1.1 วินาที (ไม่นับถ้าลดการเคลื่อนไหว ไม่มีจำนวนวัน หรือถูกตัด)
38	ตัวเลขวันหลังลด นับขึ้นเร็วกว่า (0.5 วินาที) เวลาเลื่อนแถบ
39-40	ความเต็มของวงแหวน: ต่ำกว่าเส้น = 0.02 (ขีดจำกัดเดียว), คำนวณไม่ได้ = 0, ปกติ = วัน ÷ 45 ไม่เกิน 1
41	แปลงราคาที่พิมพ์เป็นสตริง (null ถ้าไม่ใช่ตัวเลขที่ต้องการ)
42	ถ้ามีราคา คำนวณ Runway หลังซื้อ ไม่มีก็ เป็น null (ไม่แสดงผล)
44-48	เส้นเวลา: 4 จุด (0, 7, 14, 30 วัน) แต่ละจุดมีวันที่ และยอดที่คาด = ยอด - ค่าเฉลี่ย × d
49	ยอดสูงสุดสำหรับคิดความยาวแถบ (อย่างน้อย 1 กันหารด้วยศูนย์)

ส่วนที่ 3: หัวหน้าจอและการ์ดเขียวพร้อมวงแหวน

src/app/(tabs)/runway.tsx

บรรทัด 51-109

```
51   return (
52     <Screen>
53       <Row justify="space-between" align="flex-end" style={{ paddingTop: space.sm }}>
54         <View style={{ gap: 2 }}>
55           <T v="label">Money Runway</T>
56           <T v="h1">เงินพอถึงวันไหน</T>
57         </View>
58         <TitleDecor />
59       </Row>
60
61       <Reveal zoom>
62         <View style={{ borderRadius: radius.xl, overflow: 'hidden' }}>
63           <LinearGradient
64             colors={[theme.heroTop, theme.hero, theme.heroDeep]}
65             locations={[0, 0.55, 1]}
66             start={{ x: 0, y: 0 }}
67             end={{ x: 1, y: 1 }}
68             style={{ padding: space.xl, alignItems: 'center', gap: space.sm }}
69           >
70             <Aurora seed={9} />
71             <ContourLines width={420} height={340} color={theme.heroAccent} />
72             <Sparkles count={12} area={{ top: 4, bottom: 60 }} seed={29} />
73             <ProgressRing value={fullness} size={206} color={theme.heroAccent} track="rgba(244,241,230,0.14)"
width={5}>
74               <MoneyTree
75                 health={treeHealth(runway.status, runway.days)}
76                 size={150}
77                 trunk="#E8E1CC"
78                 leaf={theme.heroAccent}
79                 bare={theme.heroBare}
80                 glow={theme.heroAccent}
81                 grow
82                 sway
83               />
84             </ProgressRing>
85             {runway.status === 'no_spending' ? (
86               <>
87                 <T v="h2" color={theme.heroInk} center>ยังคำนวณไม่ได้</T>
88                 <T v="small" color={theme.heroInkSoft} center>ไม่มีรายจ่ายที่ยืนยันใน {RUNWAY_WINDOW_DAYS} วันล่าสุด จึงยังหา
ค่าเฉลี่ยไม่ได้</T>
89               </>
90             ) : runway.status === 'below_floor' ? (
91               <>
92                 <T v="h2" color={theme.heroInk} center>ยอดเงินแตะเส้นสำรองแล้ว</T>
93                 <T v="small" color={theme.heroInkSoft} center>คงเหลือ {formatBaht(balance)} · เส้นสำรอง
{formatBaht(runway.floorSatang, { decimals: false })}</T>
94               </>
95             ) : (
96               <>
97                 <T v="display" color={theme.heroAccent}>{runway.capped ? '365+' : daysShown}</T>
98                 <T v="h3" color={theme.heroInk} center>วัน · ถึงประมาณ {formatThaiDay(runway.depletionDay!)}</T>
99               </>
100             )}
101             <Badge
102               center
103               onDark
104               label={{ healthy: 'สบาย ๆ', watch: 'เริ่มต้องระวัง', critical: 'ใกล้เส้นสำรอง', below_floor: 'ต่ำกว่าเงิน
สำรอง', no_spending: 'รอข้อบ่งชี้รายจ่าย' }[runway.status]}
105               tone={{ healthy: 'good', watch: 'watch', critical: 'critical', below_floor: 'critical',
no_spending: 'neutral' }[runway.status] as 'good'}
106             />
107           </LinearGradient>
108         </View>
```

บรรทัด	ความหมาย
52	Screen หน้าแบบเลื่อนได้ของ Design System
53–59	หัวหน้า: ป้ายเล็ก "MONEY RUNWAY" และหัวข้อใหญ่ "เงินพอถึงวันไหน" ตามแบบหัวเก็บของ Design System
61	ทั้งการ์ดค่อย ๆ ขยายโผล่ขึ้นมา (Reveal zoom)
62–69	กล่องมุมโค้งใหญ่ ข้างในใส่สีเขียว 3 ระดับจากมุมซ้ายบนไปขวาล่าง (สีจางเข้ม เปลี่ยนตามธีมที่เลือก)
70–72	ของตกแต่ง: แสงออโรรา, เส้นลายคอนทัวร์, ประกายดาว 12 ดวง
73	วงแหวนทอง ขนาด 206 จุด ความเต็ม = fullness จากบรรทัด 40
74–83	ต้นไม้เงิน อยู่กลางวงแหวน ความสมบูรณ์จาก treeHealth ในสีทองของธีม ใบที่ยังไม่ออกเป็นสีจาง ค่อย ๆ งอก (grow) และโยก (sway)
85–89	ถ้า no_spending: ข้อความ "ยังคำนวณไม่ได้" และเหตุผล "ไม่มีรายจ่ายที่ยืนยันใน 7 วันล่าสุด จึงยังหาค่าเฉลี่ยไม่ได้"
90–94	ถ้า below_floor: "ยอดเงินแตะเส้นสำรองแล้ว" พร้อมยอดคงเหลือและเส้นสำรอง
95–100	ปกติ: ตัวเลขใหญ่สุด (display) สีทอง เป็นวันที่นับขึ้น หรือ "365+" และบรรทัด "วัน · ถึงประมาณ 25 ต.ค. 2569" (! หลัง depletionDay บอกว่ามั่นใจว่าไม่ใช่ null เพราะสองกรณีที่เป็น null ถูกจัดการไปแล้วข้างบน)
101–106	ป้ายสถานะ: ใช้ object เป็นตารางแปลงสถานะ → ข้อความ และสถานะ → โทนสี แล้วหีบด้วย [runway.status]

ส่วนที่ 4: การ์ด "คำนวณยังใง"

src/app/(tabs)/runway.tsx

บรรทัด 111–147

```

111     <Card>
112       <T v="h3">คำนวณยังใง</T>
113       <View style={{ gap: space.sm }}>
114         <Row justify="space-between">
115           <T v="small">ยอดคงเหลือ</T>
116           <T v="body" style={{ fontFamily: fonts.sansSemi }}>{formatBaht(balance)}</T>
117         </Row>
118         <Row justify="space-between">
119           <T v="small">หักเงินสำรองที่ตั้งไว้</T>
120           <T v="body" style={{ fontFamily: fonts.sansSemi }}>- {formatBaht(floor)}</T>
121         </Row>
122         {reserved > 0 ? (
123           <Row justify="space-between">
124             <T v="small">หักเงินที่กินไว้ในกระปุกลูก</T>
125             <T v="body" style={{ fontFamily: fonts.sansSemi }}>- {formatBaht(reserved)}</T>
126           </Row>
127         ) : null}
128       <Divider />
129       <Row justify="space-between">
130         <T v="small">ใช้ได้จริง</T>
131         <T v="body" style={{ fontFamily: fonts.sansSemi }}>{formatBaht(runway.spendableSatang)}</T>
132       </Row>
133       <Row justify="space-between">
134         <T v="small">÷ ใช้เฉลี่ยต่อวัน ({average.daysCounted} วันล่าสุด)</T>
135         <T v="body" style={{ fontFamily: fonts.sansSemi }}>{formatBaht(average.averageSatang)}</T>
136       </Row>
137       <Divider />
138       <Row justify="space-between">
139         <T v="h3">เงินพอใช้อีก</T>
140         <T v="h3" color={runway.status === 'below_floor' || runway.status === 'critical' ? theme.critical
: theme.primary}>{daysText(runway)}</T>

```

```

141     </Row>
142   </View>
143   <T v="micro">
144     ค่าเฉลี่ยคิดจากรายจ่ายที่ยืนยันแล้ว {RUNWAY_WINDOW_DAYS} วันล่าสุด (รวมวันที่ไม่ได้ใช้เงิน) สลิปที่ยังไม่ยืนยันไม่นับ
145   </T>
146   <Button label="เปลี่ยนเงินสำรอง" kind="soft" small icon="options-outline" onPress={() =>
147     router.push('/settings')} />
148 </Card>

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/runway.tsx:111

บรรทัด	ความหมาย
112	หัวข้อการด์
114–117	แถว "ยอดคงเหลือ" ซ้อนซ้าย ตัวเลขขวา (justify="space-between")
118–121	แถว "หักเงินสำรองที่ตั้งไว้ – ฿500.00" (ใช้ floor จาก useMoney ซึ่งยังไม่รวมกระปุก จึงแยกบรรทัดได้)
122–127	แถว "หักเงินที่เกินไว้ในกระปุกออม" แสดงเฉพาะเมื่อมีเงินในกระปุก (reserved > 0)
128	เส้นคั่น
129–132	"ใช้ได้จริง" = spendableSatang จาก computeRunway
133–136	"÷ ใช้เฉลี่ยต่อวัน (7 วันล่าสุด)" ตัวเลขในวงเล็บคือ daysCounted ผู้ใช้ใหม่จะเห็น (3 วันล่าสุด)
138–141	ผลลัพธ์ "เงินพอใช้อีก 16 วัน" เป็นสีแดงเมื่อ critical หรือ below_floor ไม่เกินสีหลักของธีม
143–145	หมายเหตุตัวเล็ก: ค่าเฉลี่ยคิดจากรายจ่ายที่ยืนยันแล้ว 7 วันล่าสุด (รวมวันที่ไม่ได้ใช้เงิน) สลิปที่ยังไม่ยืนยันไม่นับ
146	ปุ่ม "เปลี่ยนเงินสำรอง" เปิดหน้าต่างค่า

ส่วนที่ 5: เส้นเวลา 30 วัน

src/app/(tabs)/runway.tsx

บรรทัด 149–174

```

149   {runway.averageSatang > 0 ? (
150     <Card>
151       <T v="h3">ถ้ายังใช้เงินแบบนี้ต่อไป</T>
152       <View style={{ gap: space.md }}>
153         {timeline.map((p) => {
154           const below = p.balance <= runway.floorSatang;
155           return (
156             <View key={p.d} style={{ gap: 4 }}>
157               <Row justify="space-between">
158                 <T v="small" color={theme.ink}>{p.d === 0 ? 'วันนี้' : `อีก ${p.d} วัน · ${formatThaiDay(p.day,
159 { year: false })}`}</T>
160                 <Money satang={p.balance} size="body" decimals={false} color={below ? theme.critical :
161 theme.ink} />
162               </Row>
163               <GrowBar
164                 value={p.balance / maxBal}
165                 color={below ? theme.critical : p.d === 0 ? theme.accent : theme.primary}
166                 opacity={below || p.d === 0 ? 1 : 0.55}
167                 track={theme.surfaceAlt}
168                 delay={200 + timeline.indexOf(p) * 120}
169               />
170             </View>
171           );
172         })}
173       </View>
174       <T v="micro">ประมาณการจากค่าเฉลี่ยรายวัน ยังไม่รวมรายรับที่จะเข้ามา</T>
175     </Card>

```

174) : null}

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/runway.tsx:149

บรรทัด	ความหมาย
149	แสดงการ์ดนี้เฉพาะเมื่อค่าเฉลี่ย > 0 (ถ้าไม่มีความเร็ว ก็คาดอนาคตไม่ได้)
153	วาดทีละจุดของเส้นเวลา (map)
154	จุดนี้ต่ำกว่าหรือเท่ากับเส้นสำรองไหม
156	key = จำนวนวัน (ไม่ซ้ำ) ให้ React แยกแต่ละแถวได้
158	ข้อความซ้าย: "วันนี้" หรือ "อีก 7 วัน · 16 ต.ค." (ไม่แสดงปี)
159	ยอดที่คาดไว้ เป็นสีแดงถ้าต่ำกว่าเส้น
161–167	แถบที่ค่อย ๆ โดตามสัดส่วนยอดเทียบยอดวันนี้: แดงถ้าต่ำกว่าเส้น, ทองสำหรับวันนี้, เขียวจางสำหรับวันอื่น และทีละแถบห่างกัน 0.12 วันนาก็
172	หมายเหตุ: ประมาณการจากค่าเฉลี่ยรายวัน ยังไม่รวมรายรับที่จะเข้ามา (บอกข้อจำกัดตรง ๆ)

ส่วนที่ 6: ลองปรับ

src/app/(tabs)/runway.tsx

บรรทัด 176–220

```
176 {runway.averageSatang > 0 && runway.status !== 'below_floor' ? (  
177   <Card>  
178     <T v="h3">ลองปรับ: ถ้าใช้น้อยลงวันละ</T>  
179     <Row gap={space.md} align="center">  
180       <View style={{ flex: 1, gap: 2 }}>  
181         <T v="display" color={reduce ? theme.primary : theme.inkSoft} style={{ fontVariant: ['tabular-  
182           nums'] }}>  
183           {reduce}%  
184         </T>  
185         <T v="small">  
186           {reduce === 0  
187             ? `ตอนนี้ใช้วันละ ${formatBaht(runway.averageSatang, { decimals: false })}`  
188             : `ใช้วันละ ${formatBaht(reduced.averageSatang, { decimals: false })} (ประหยัดวันละ  
189             ${formatBaht(runway.averageSatang - reduced.averageSatang, { decimals: false })})`  
190           </T>  
191         </View>  
192         <View style={{ alignItems: 'center' }}>  
193           <MoneyTree  
194             health={treeHealth(reduced.status, reduced.days)}  
195             size={86}  
196             trunk={theme.dark ? '#E8E1CC' : '#A88F5E'}  
197             leaf={palette.goldBright}  
198             bare={theme.dark ? '#3D5A4C' : '#C9D3CC'}  
199             grow  
200           />  
201         </View>  
202       </Row>  
203       <Slider  
204         value={reduce}  
205         onChange={setReduce}  
206         min={0}  
207         max={50}  
208         step={5}  
209         label="ใช้น้อยลงวันละกี่เปอร์เซ็นต์"  
210         format={(v) => `${v}%`}  
211         ticks={[0, 10, 20, 30, 40, 50]}  
212       />  
213     </Card>  
214   ) : null}
```

```

212         <Ionicons name="leaf" size={20} color={theme.primary} />
213         <T v="body" style={{ flex: 1 }}>
214             {reduce === 0
215               ? `เลื่อนดูว่าถ้าใช้บ่อยลง เงินจะพอนานขึ้นกี่วัน (ตอนนี้ ${daysText(runway)})`
216               : `เงินจะพอ ${daysText(reduced.days === null ? reduced : { ...reduced, days: reducedDays
217                 })}${reduced.days !== null && runway.days !== null ? ` · เพิ่มขึ้น ${reducedDays - runway.days} วัน` : ''}`}
218             </T>
219         </Row>
220     </Card>
221     ) : null}

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/runway.tsx:176`

บรรทัด	ความหมาย
176	แสดงเมื่อมีค่าเฉลี่ย และยังไม่ต่ำกว่าเส้นสำรอง (ถ้าต่ำกว่าแล้ว ลดเท่าไรก็ยังต่ำกว่า)
181–183	ตัวเลขเปอร์เซ็นต์ใหญ่ ตัวเลขกว้างเท่ากัน (tabular-nums) จะได้ไม่กระตุกตอนเปลี่ยน สีจางตอน 0%
184–188	ตอน 0%: "ตอนนี้ใช้วันละ ฿660" · เมื่อเลื่อน: "ใช้วันละ ฿462 (ประหยัดวันละ ฿198)"
190–199	ต้นไม้เล็กที่ออกใบตาม Runway หลังลด สีลำต้นต่างกันในโหมดสว่าง/มืด
201–210	แถบเลื่อน: ค่าปัจจุบัน, เปลี่ยนแล้วเรียก setReduce, ช่วง 0–50 ทีละ 5, ชื่อสำหรับโปรแกรมอ่านหน้าจอ, รูปแบบ "30%", และป้ายใต้แถบ 0/10/20/30/40/50
211–218	บรรทัดสรุปพร้อมไอคอนใบไม้: ตอน 0% ชวนให้เลื่อน · เมื่อเลื่อน "เงินจะพอ 23 วัน · เพิ่มขึ้น 7 วัน" (ใช้ตัวเลขที่กำลังนับขึ้น)

ส่วนที่ 7: เช็กก่อนจ่าย และส่งต่อให้โค้ช

`src/app/(tabs)/runway.tsx`

บรรทัด 222–250

```

222     <Card>
223       <T v="h3">เช็กก่อนจ่าย</T>
224       <T v="small">อยากซื้ออะไร ลองใส่ราคาดูว่าเงินจะพอใช้สิ้นลงกี่วัน</T>
225       <Field id="runway-price" label="ราคา (บาท)" value={price} onChangeText={setPrice}
226         keyboardType="decimal-pad" placeholder="เช่น 590" />
227       {afterBuy ? (
228         <View style={{ gap: space.sm }}>
229           <Row justify="space-between">
230             <T v="small">ตอนนี้</T>
231             <T v="body" style={{ fontFamily: fonts.sansSemi }}>{daysText(runway)}</T>
232           </Row>
233           <Row justify="space-between">
234             <T v="small">ถ้าซื้อ {formatBaht(priceSatang!, { decimals: false })}</T>
235             <T v="body" style={{ fontFamily: fonts.sansSemi }} color={afterBuy.status === 'below_floor' ||
236               afterBuy.status === 'critical' ? theme.critical : theme.ink}>
237               {daysText(afterBuy)}
238             </T>
239           </Row>
240           <Button
241             label="ถามโค้ชเรื่องนี้"
242             kind="soft"
243             small
244             icon="chatbubble-ellipses-outline"
245             onPress={() => router.navigate({ pathname: '/coach', params: { ask: `ถ้าฉันซื้อของราคา
246               ${formatBaht(priceSatang!, { decimals: false })} วันนี้ จะเป็นยังไง`, price: String(priceSatang), at:
247               String(Date.now()) } )}}
248             />
249         </View>
250       ) : null}

```

บรรทัด	ความหมาย
223–224	หัวข้อ และคำอธิบาย "อยากซื้ออะไร ลองใส่ราคาดูว่าเงินจะพอใช้สิ้นลงกี่วัน"
225	ช่องราคา <code>id="runway-price"</code> (E2E ใช้ <code>id</code> นี้พิมพ์ 2000) เป็นพิมพ์ตัวเลขมีทศนิยม
226	แสดงผลเฉพาะเมื่อพิมพ์ราคาที่ถูกต้องแล้ว
228–231	แถว "ตอนนี้ 16 วัน"
232–237	แถว "ถ้าซื้อ ฿2,000 → 13 วัน" เป็นสีแดงถ้าหลังซื้อเป็น <code>critical</code> หรือ <code>below_floor</code>
238–244	ปุ่ม "ถามโค้ชเรื่องนี้": ไปเก็บโค้ดพร้อมส่ง 3 ค่า: คำถามที่พิมพ์ไว้ให้ (<code>ask</code>), ราคาเป็นสตริง (<code>price</code>) และเวลาที่กด (<code>at</code>) ซึ่งเปลี่ยนทุกครั้ง ถ้ากดถามราคาเดิมซ้ำ หน้าโค้ชจะเติมคำถามให้อีกครั้ง (บันทึกแก้ตอนตรวจ repo)

อ่านโค้ด: การ์ดบนหน้าหลัก และ useMoney

FR-6 ปรากฏบนหน้าหลักผ่านการ์ดเขียวและการ์ด "วันนี้ใช้ได้ประมาณ" ทั้งหมดอยู่ใน src/app/(tabs)/index.tsx

ตารางป้ายสถานะ

src/app/(tabs)/index.tsx

บรรทัด 54–60

```
54 const RUNWAY_BADGE = {
55   healthy: { label: 'สบาย ๆ', tone: 'good' },
56   watch: { label: 'เริ่มต้องระวัง', tone: 'watch' },
57   critical: { label: 'ใกล้เส้นสำรอง', tone: 'critical' },
58   below_floor: { label: 'ต่ำกว่าเงินสำรอง', tone: 'critical' },
59   no_spending: { label: 'ยังไม่มีรายจ่าย 7 วัน', tone: 'neutral' },
60 } as const;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:54

บรรทัด 54–60: แปลงสถานะเป็นข้อความป้ายและโทนสี as const ท้ายสุดบอก TypeScript ว่าค่าเหล่านี้ตายตัว (โทนต้องเป็น good/watch/critical/neutral เท่านั้น) สังเกตว่าบนหน้าหลัก no_spending ใช้ข้อความ "ยังไม่มีรายจ่าย 7 วัน" ส่วนในแถบเงินพอถึงใช้ "รอข้อมูลรายจ่าย"

ตัวเลขที่หน้าหลักดึงมาใช้

src/app/(tabs)/index.tsx

บรรทัด 182–198

```
182 const { profile, txs, goals, repo, refresh, refreshing, loadError, today, newDay } = useApp();
183 const { balance, runway, average, drafts } = useMoney();
184 const [range, setRange] = useState<RangeKey>('1m');
185
186 // `today` changes at 00:00 Bangkok time, so every "วันนี้" number starts over.
187 // eslint-disable-next-line react-hooks/exhaustive-deps
188 const summary = useMemo(() => summarizeRange(txs, range), [txs, range, today]);
189 // eslint-disable-next-line react-hooks/exhaustive-deps
190 const week = useMemo(() => dailyTotals(txs, 7), [txs, today]);
191 const recent = useMemo(
192   () => [...txs].filter((t) => t.status === 'confirmed').sort((a, b) =>
193     b.occurredAt.localeCompare(a.occurredAt)).slice(0, 5),
194   [txs],
195 );
196 const safeToday = safeDailySpend({ balanceSatang: balance, floorSatang: runway.floorSatang, targetDay:
197   endOfMonthDay() });
198 const spentMonth = monthExpense(txs);
199 const readyDrafts = drafts.filter((d) => d.reviewFlags.length === 0).length;
200 const badge = RUNWAY_BADGE[runway.status];
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:182

บรรทัด	ความหมาย
183	ดึงยอด Runway ค่าเฉลี่ย และแบบร่าง จาก useMoney (คำนวณครั้งเดียว ใช้ร่วมกับแถบเงินพอถึง)
195	ใช้ได้วันละเท่าไรถึงสิ้นเดือน: เรียก safeDailySpend โดยวันเป้าหมาย = วันสิ้นเดือน (endOfMonthDay) ใช้เส้นสำรองจาก runway ซึ่งรวมเงินในกระเป๋าแล้ว
198	หาป้ายของสถานะปัจจุบันจากตารางด้านบน

src/app/(tabs)/index.tsx

บรรทัด 217–217


```
217 const daysShown = useCountUp(runway.days ?? 0, !reduce && runway.days !== null && !runway.capped, 900);
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:217

บรรทัด 217: ตัวเลขวันบนหน้าหลักนับขึ้นใน 0.9 วินาที (สั้นกว่าในแถบเงินพอถึงเล็กน้อย)

การ์ดเขียว (hero) ส่วนของ Runway

src/app/(tabs)/index.tsx

บรรทัด 370-456

```
370  {/* Hero: balance + runway (the one place gold is used for the number that matters) */}
371  <Reveal index={1} zoom>
372    <Animated.View style={{ transform: [{ scale: heroScale }] }}>
373      <View
374        style={{
375          borderRadius: radius.xl,
376          overflow: 'hidden',
377          shadowColor: theme.heroDeep,
378          shadowOpacity: theme.dark ? 0 : 0.28,
379          shadowRadius: 18,
380          shadowOffset: { width: 0, height: 10 },
381          elevation: 8,
382        }}
383        accessible
384        accessibilityLabel={`ยอดคงเหลือ ${formatBaht(balance)} ${runway.days !== null ? `เงินพอใช้อีก`
385          ${runway.days} วัน` : ''}`}
386      >
387        <LinearGradient
388          colors={[theme.heroTop, theme.hero, theme.heroDeep]}
389          locations={[0, 0.55, 1]}
390          start={{ x: 0, y: 0 }}
391          end={{ x: 1, y: 1 }}
392          style={{ padding: space.xl, gap: space.md }}
393        >
394          <Aurora />
395          <ContourLines width={420} height={260} color={theme.heroAccent} />
396          <Sparkles trigger={balance} count={9} area={{ top: 4, bottom: 55 }} />
397          <Shine trigger={balance} />
398          {halloween ? <SpookyHeroDecor /> : null}
399          <Row align="flex-start" justify="space-between">
400            <View style={{ flex: 1, gap: 2 }}>
401              <T v="label" color={theme.heroInkSoft}>ยอดคงเหลือ</T>
402              <Money satang={balance} size="display" color={theme.heroInk} countUp />
403              <View style={{ marginTop: space.sm }}>
404                <Badge label={badge.label} tone={badge.tone} onDark />
405              </View>
406            </View>
407            <Animated.View style={{ transform: [{ translateY: treeLift }] }}>
408              <Pressable onPress={tapTree} accessibilityRole="button" accessibilityLabel="ต้นไม้เงิน แต่เพื่อดูความ
409              หมาย" hitSlop={6}>
410                <MoneyTree
411                  health={treeHealth(runway.status, runway.days)}
412                  size={104}
413                  trunk="#E8E1CC"
414                  leaf={theme.heroAccent}
415                  bare={theme.heroBare}
416                  glow={theme.heroAccent}
417                  grow
418                  sway
419                  wiggle={treeTaps}
420                />
421                {treeTaps > 0 ? <Sparkles trigger={treeTaps} count={7} cycles={1} seed={treeTaps} area={{
422                  top: 0, bottom: 70 }} /> : null}
423              </Pressable>
424            </Animated.View>
425          </Row>
```

```

423     <View style={{ height: 1, backgroundColor: alpha(theme.heroInk, 0.15) }} />
424     <Row justify="space-between" align="flex-end">
425       <View style={{ flex: 1 }}>
426         <T v="small" color={theme.heroInkSoft}>เงินพอใช้อีก</T>
427         {runway.days !== null && runway.status !== 'below_floor' ? (
428           <T v="h1" color={theme.heroAccent}>
429             {runway.capped ? '365+ ' : `${daysShown} `}
430             <T v="body" color={theme.heroInk}>วัน · ถึง {formatThaiDay(runway.depletionDay!, { year:
false }}}</T>
431           </T>
432         ) : runway.status === 'below_floor' ? (
433           <T v="h3" color={theme.heroInk}>แตะเส้นเงินสำรองแล้ว</T>
434         ) : (
435           <T v="h3" color={theme.heroInk}>ยังคำนวณไม่ได้</T>
436         )}
437       </View>
438       <Button label="ดูรายละเอียด" small kind="onDark" icon="chevron-forward" onPress={() =>
router.navigate('/runway')} />
439     </Row>
440   </LinearGradient>
441 </View>
442 </Animated.View>
443 </Reveal>
444
445 { /* Today */ }
446 <Reveal index={2}>
447 <Card tone="accent" style={{ paddingVertical: space.md }}>
448   <Row gap={space.md}>
449     <Icons name="sunny-outline" size={22} color={theme.accentInk} />
450     <View style={{ flex: 1 }}>
451       <T v="small" color={theme.ink}>ถ้าอยากให้เห็นพอถึงสิ้นเดือน วันที่ใช้ได้ประมาณ</T>
452       <T v="h3">{formatBaht(safeToday, { decimals: false })} / วัน</T>
453     </View>
454   </Row>
455 </Card>
456 </Reveal>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:370

บรรทัด	ความหมาย
370	คอมเมนต์: การ์ดนี้คือที่เดียวที่ใช้สีทองกับตัวเลขที่สำคัญที่สุด (กฎ Design System)
371–372	การ์ดโพล์เป็นลำดับที่ 2 และย่อลงเล็กน้อยตอนเลื่อนหน้าจอ (parallax)
373–382	กล่องมุมโค้ง มีเงา (เงาไม่แสดงในโหมดมืด)
383–384	accessibility: รวมการ์ดเป็นชิ้นเดียว โปรแกรมอ่านหน้าจออ่าน "ยอดคงเหลือ ฿... เงินพอใช้อีก 16 วัน" ถ้าคำนวณไม่ได้ก็ไม่อ่านส่วนวัน
386–397	พื้นไล่สีเขียวและของตกแต่ง ประกายและแสงวิ่งทำงานเมื่อยอดเงินเปลี่ยน (trigger={balance}) ผู้ใช้ดูรายการแล้วกลับมาจะเห็นการ์ด "จริง"
400–401	ยอดคงเหลือ ตัวใหญ่ บั๊บบ๊น (ของ FR-2)
403	ป้ายสถานะ Runway แบบบนพื้นเข้ม
406–420	ต้นไม้เงินที่แตะได้: แตะแล้วเรียก tapTree (สัน + ประกาย + นื่องคล้ายอธิบาย) ขยับขึ้นเล็กน้อยตอนเลื่อนจอ
423	เส้นคั่นบาง ๆ
426	ข้อความ "เงินพอใช้อีก"
427–431	ถ้ามีจำนวนวันและไม่ต่ำกว่าเส้น: ตัวเลขสีทองขนาด h1 ("365+" ถ้าถูกตัด) ต่อด้วย "วัน · ถึง 25 ต.ค." (ไม่แสดงปี เพราะการ์ดแคบ)
432–433	ต่ำกว่าเส้น: "แตะเส้นเงินสำรองแล้ว"

บรรทัด	ความหมาย
434–436	คำนวณไม่ได้: "ยังคำนวณไม่ได้"
438	ปุ่ม "ดูรายละเอียด" ไปเก็บเงินพอถึง (navigate = สลับแท็บ ไม่ซ่อนหน้า)
446–455	การ์ดทอง: ไอคอนดวงอาทิตย์ "ถ้าอยากให้เงินพอถึงสิ้นเดือน วันนี้ใช้ได้ประมาณ" และตัวเลข safeToday แบบไม่มีสตางค์ "/" วัน"

น้องกล้าพูดตามสถานะ และแตะต้นไม้

src/app/(tabs)/index.tsx บรรทัด 235–250

```

235 function tapTree() {
236   Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light).catch(() => {});
237   setTreeTaps((n) => n + 1);
238   setHop((h) => h + 1);
239   setSaid(treeLine({ status: runway.status, days: runway.days, capped: runway.capped }));
240 }
241 const buddy = buddyLine({
242   status: runway.status,
243   days: runway.days,
244   capped: runway.capped,
245   safeTodaySatang: safeToday,
246   spentTodaySatang: spentOnDay(txs, today),
247   draftsToReview: drafts.length - readyDrafts,
248   hasAnyTransaction: txs.length > 0,
249   newDay: newDay ? { yesterdaySpentSatang: spentOnDay(txs, addDays(today, -1)) } : null,
250 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:235

บรรทัด	ความหมาย
235–240	tapTree: มือถือสั่นเบา ๆ, นับจำนวนครั้งที่แตะ (ทำให้ต้นไม้สั่น), น้องกล้ากระโดด และพูดประโยคจาก treeLine ตามสถานะ Runway
241–250	ประโยคประจำของน้องกล้าบนหน้าหลัก (buddyLine) ส่งสถานะ จำนวนวัน ใช้ได้วันละ ใช้ไปวันนี้ จำนวนสลบที่ต้องตรวจ และถ้าเป็นการเปิดครั้งแรกของวัน ส่งยอดใช้เมื่อวานไปด้วย

useMoney: จุดที่ประกอบทุกอย่างเข้าด้วยกัน

src/data/AppProvider.tsx บรรทัด 442–459

```

442 /** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */
443 export function useMoney() {
444   const { profile, txs, carrySatang, goals, today } = useApp();
445   return useMemo(() => {
446     const opening = profile?.openingBalanceSatang ?? 0;
447     const floor = profile?.runwayFloorSatang ?? 50_000;
448     // Money from before the loaded window (over ~13 months ago) still counts.
449     const balance = computeBalance(opening + carrySatang, txs);
450     const average = averageDailyExpense(txs);
451     // Money in savings goals is set aside: it does not count as money to spend.
452     const reserved = reservedSatang(goals);
453     const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang:
average.averageSatang });
454     const drafts = txs.filter((t) => t.status === 'draft');
455     return { balance, average, runway, drafts, reserved, floor };
456     // `today` makes the runway and averages start over at midnight.
457     // eslint-disable-next-line react-hooks/exhaustive-deps
458   }, [profile, txs, carrySatang, goals, today]);
459 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:442

บรรทัด	ความหมาย
444	ดึงข้อมูลดิบจากสถานะกลาง
446	ยอดตอนเริ่มใช้แอป (ยังไม่มีโปรไฟล์ = 0)
447	เส้นสำรอง จากโปรไฟล์ ถ้ายังไม่มีใช้ 50,000 สตางค์
448–449	ยอดคงเหลือ = computeBalance(ยอดเริ่มต้น + ยอดยกมาของรายการค่ากว่า 13 เดือน, รายการทั้งหมด)
450	ค่าเฉลี่ย 7 วัน
451–452	คอมเมนต์และ เงินในกระปุกทั้งหมด
453	Runway: ส่งเส้นสำรองเป็น floor + reserved (นี่คือจุดเดียวที่ทำให้กระปุกออมมีผลกับ Runway)
454	แบบร่างที่รอตรวจ
455	คืนทั้งหมด รวม reserved และ floor แยกไว้ให้การ์ด "คำนวณยังงัย" แสดงทีละบรรทัด
456–458	คิดใหม่เมื่อ โปรไฟล์ รายการ ยอดยกมา กระปุก หรือวันนี้ เปลี่ยน (คอมเมนต์ eslint ปิดคำเตือนเพราะ today ไม่ได้ถูกใช้ตรง ๆ ในฟังก์ชัน แต่ตั้งใจใส่ไว้ให้คำนวณใหม่ตอนเที่ยงคืน)

อ่านโค้ด: กระจกออม (goals)

กฎใน src/domain/goals.ts และหน้าจอใน src/app/goals.tsx

กฎของกระจกออม

src/domain/goals.ts

บรรทัด 1-75

```

1  /**
2   * Savings goals ("กระจกออม"): money set aside for something the user wants.
3   * The money stays in the user's account, so what is saved is kept out of the
4   * money that can be spent (Money Runway) until the goal is used or removed.
5   * Pure; unit tests in __tests__/goals.test.ts (TC-71).
6   */
7  import { addDays, daysBetween, formatThaiDay } from './dates';
8  import { formatBaht } from './money';
9
10 export interface SavingsGoal {
11   id: string;
12   title: string;
13   emoji: string;
14   targetSatang: number;
15   savedSatang: number;
16   /** "YYYY-MM-DD", or null for no deadline. */
17   dueDay: string | null;
18   /** When the goal was reached (ISO), or null. */
19   doneAt: string | null;
20   createdAt: string;
21 }
22
23 export type GoalInput = Omit<SavingsGoal, 'id' | 'createdAt'>;
24
25 export const GOAL_EMOJIS = ['🎧', '📺', '💻', '✂️', '🎓', '🎁', '🚲', '👟', '🏠', '🐱'];
26
27 /** Quick deadlines offered when creating a goal (days from today; null = none). */
28 export const GOAL_DEADLINES: { label: string; days: number | null }[] = [
29   { label: '1 เดือน', days: 30 },
30   { label: '3 เดือน', days: 91 },
31   { label: '6 เดือน', days: 182 },
32   { label: 'ไม่กำหนด', days: null },
33 ];
34
35 export const deadlineDay = (today: string, days: number | null) => (days === null ? null : addDays(today, days));
36
37 export function goalProgress(g: Pick<SavingsGoal, 'savedSatang' | 'targetSatang'>): number {
38   return g.targetSatang > 0 ? Math.max(0, Math.min(1, g.savedSatang / g.targetSatang)) : 0;
39 }
40
41 /** Money set aside in all goals: kept out of what can be spent. */
42 export function reservedSatang(goals: Pick<SavingsGoal, 'savedSatang'>[]): number {
43   return goals.reduce((s, g) => s + Math.max(0, g.savedSatang), 0);
44 }
45
46 /**
47  * How much to put in per day to reach the goal by its deadline (today counts),
48  * rounded up to whole baht. Null when there is no deadline or it is reached.
49  */
50 export function perDayToReach(g: Pick<SavingsGoal, 'savedSatang' | 'targetSatang' | 'dueDay'>, today: string):
51   number | null {
52   const left = g.targetSatang - g.savedSatang;
53   if (left <= 0 || !g.dueDay) return null;
54   const days = Math.max(1, daysBetween(today, g.dueDay) + 1);

```

```

54   return Math.ceil(left / days / 100) * 100;
55 }
56
57 /** Add (or take out, when negative) money; the goal is reached when the saved amount meets the target. */
58 export function applyDeposit(g: SavingsGoal, deltaSatang: number, now: Date = new Date()): Pick<SavingsGoal,
    'savedSatang' | 'doneAt'> {
59   const saved = Math.max(0, g.savedSatang + deltaSatang);
60   const reached = saved >= g.targetSatang;
61   return { savedSatang: saved, doneAt: reached ? (g.doneAt ?? now.toISOString()) : null };
62 }
63
64 /** One friendly line under a goal. */
65 export function goalLine(g: SavingsGoal, today: string): string {
66   const left = g.targetSatang - g.savedSatang;
67   // What is still to save is rounded up to whole baht, so it never reads "อีก ฿0" while short.
68   const upBaht = (s: number) => formatBaht(Math.ceil(s / 100) * 100, { decimals: false });
69   if (left <= 0) return 'ครบแล้ว! เก่งมาก ถึงเวลาใช้เงินก้อนนี้อย่างสบายใจ';
70   if (!g.dueDay) return `อีก ${upBaht(left)} ก็ครบ`;
71   const days = daysBetween(today, g.dueDay);
72   if (days < 0) return `เลยกำหนดมาแล้ว อีก ${upBaht(left)} ก็ครบ ค่อย ๆ เก็บต่อได้นะ`;
73   const per = perDayToReach(g, today)!;
74   return `เก็บวันละ ${upBaht(per)} ก็ครบวันที่ ${formatThaiDay(g.dueDay, { year: false })}`;
75 }

```

► เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/goals.ts:1

บรรทัด	ความหมาย
1-6	คอมเมนต์: เงินในกระปุกยังอยู่ในบัญชีผู้ใช้ จึงต้องกันออกจากเงินที่ใช้ได้ (Money Runway) จนกว่าจะใช้หรือลบเป้า เทสต์คือ TC-71
10-21	ชนิดของกระปุก 1 ใบ: ชื่อ ไอคอน เป้า (สตางค์) ที่เก็บแล้ว (สตางค์) วันครบกำหนด (หรือ null) วันที่ครบเป้า (หรือ null)
23	ข้อมูลตอนสร้างกระปุก = ทุกช่องยกเว้น id และ createdAt
25	ไอคอนให้เลือก 10 แบบ
28-33	ตัวเลือกกำหนดวัน: 1 เดือน (30), 3 เดือน (91), 6 เดือน (182), ไม่กำหนด
35	แปลงจำนวนวันเป็นวันที่ (null = ไม่กำหนด)
37-39	ความคืบหน้า 0 ถึง 1 = เก็บแล้ว ÷ เป้า (บังคับไม่ให้ติดลบหรือเกิน 1)
41-44	reservedSatang : รวมเงินในกระปุกทุกใบ (ค่าติดลบนับเป็น 0) ค่านี้ส่งไปรวมกับเส้นสำรองใน useMoney
46-55	perDayToReach : ต้องเก็บวันละเท่าไรให้ทันกำหนด = ที่เหลือ ÷ จำนวนวันถึงกำหนด (นับวันนี้) ปิดขึ้น เป็นบวก (ปิดขึ้นเพราะถ้าปิดลงจะเก็บไม่ทัน ตรงข้ามกับ Runway ที่ปิดลงเพื่อความปลอดภัย) ไม่มีกำหนดหรือครบแล้วคืน null
57-62	applyDeposit : หยอด (บวก) หรือถอน (ลบ) ไม่ให้ต่ำกว่า 0 ถ้าครบเป้าบันทึกเวลาที่ครบ (ถ้าเคยครบแล้วใช้เวลาเดิม) ถ้าถอนจนไม่ครบล้างเวลาครบออก
64-75	goalLine : ประโยคใต้กระปุก: ครบแล้ว → ชื่นชม, ไม่มีกำหนด → "อีก ฿... ก็ครบ", เลยกำหนด → ให้กำลังใจ, ปกติ → "เก็บวันละ ฿... ก็ทันวันที่ ..." ยอดที่เหลือปิดขึ้นเป็นบวก จะได้ไม่ขึ้น "อีก ฿0" ทั้งที่ยังขาดอยู่ไม่ที่สตางค์

หยอดหรือถอนจากหน้าจอ

src/app/goals.tsx

บรรทัด 38-58

```

38   async function move(satang: number) {
39     if (busy || satang <= 0) return;
40     const delta = open === 'out' ? -Math.min(satang, goal.savedSatang) : satang;
41     if (!delta) return;
42     setBusy(true);
43     try {
44       const g = await depositToGoal(goal.id, delta);
45       setCustom('');

```

```

46     if (delta > 0) setDrop((n) => n + 1);
47     if (!done && g.savedSatang >= g.targetSatang) {
48       celebrate();
49       toast({ message: `ครบเป้า "${g.title}" แล้ว! เก่งมาก` });
50     } else {
51       toast({ message: delta > 0 ? `หยอด ${formatBaht(delta, { decimals: false })} แล้ว : `ถอน ${formatBaht(-
delta, { decimals: false })} แล้ว` });
52     }
53   } catch {
54     toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
55   } finally {
56     setBusy(false);
57   }
58 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/goals.tsx:38

บรรทัด	ความหมาย
39	กำลังบันทึกอยู่หรือจำนวนไม่ถูกต้อง → ไม่ทำ
40	ถ้ากำลังถอน เปลี่ยนเป็นค่าลบ และถอนได้ไม่เกินที่มีในกระปุก
44	เรียก depositToGoal ใน AppProvider ซึ่งใช้ applyDeposit แล้วบันทึกผ่าน Repo (คลาวด์หรือเครื่อง)
46	หยอดเงิน → เหรียญหล่นลงโหล (นับครั้งเพื่อเล่นแอนิเมชัน)
47–49	ถ้าเพิ่งครบเป้าครั้งนี้ → พลุฉลอง และ "ครบเป้า ... แล้ว! เก่งมาก"
50–52	ไม่เงินแจ้ง "หยอด ฿500 แล้ว" หรือ "ถอน ... แล้ว"
53–54	บันทึกไม่สำเร็จ → แจ้งให้ลองใหม่

พอ depositToGoal อัปเดต goals ใน AppProvider, useMoney ก็คำนวณ reserved และ Runway ใหม่เอง หน้าเงินพอถึงจึงเห็นบรรทัด "หักเงินที่กั้นไว้ในกระปุกออม" กันก็โดยไม่ต้องเขียนโค้ดเชื่อมเพิ่ม

ตารางในฐานข้อมูล

supabase/migrations/20260929000000_savings_goals.sql

บรรทัด 1–30

```

1  -- Savings goals ("กระปุกออม"): money the user sets aside for something they
2  -- want. Saved money still sits in the user's account, so the app keeps it out
3  -- of what can be spent (Money Runway) until the goal is used or removed.
4  -- Same security model as the other tables: each row belongs to one user and
5  -- Row Level Security lets a user read or change only their own rows.
6
7  create table if not exists public.savings_goals (
8    id uuid primary key default gen_random_uuid(),
9    user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
10   title text not null check (char_length(title) between 1 and 60),
11   emoji text not null default '🎯' check (char_length(emoji) <= 8),
12   target_satang bigint not null check (target_satang > 0 and target_satang <= 100000000000),
13   saved_satang bigint not null default 0 check (saved_satang >= 0 and saved_satang <= 100000000000),
14   due_day date,
15   done_at timestamptz,
16   created_at timestamptz not null default now(),
17   updated_at timestamptz not null default now()
18 );
19
20 create index if not exists savings_goals_user_idx on public.savings_goals (user_id, created_at);
21
22 alter table public.savings_goals enable row level security;
23
24 drop policy if exists "own savings goals" on public.savings_goals;

```

```

25 create policy "own savings goals" on public.savings_goals
26   for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);
27
28 drop trigger if exists savings_goals_touch on public.savings_goals;
29 create trigger savings_goals_touch before update on public.savings_goals
30   for each row execute function public.touch_updated_at();

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260929000000_savings_goals.sql:1

บรรทัด ความหมาย

7–18	ตาราง savings_goals: ชื่อ 1–60 ตัวอักษร ไอคอน เป้าต้อง > 0 และไม่เกิน ฿1,000,000,000 ที่เก็บแล้วต้อง ≥ 0 วันกำหนด และวันครบเป้า
20	index ค้นกระปุกของผู้ใช้เรียงตามวันที่สร้าง
22–26	เปิด RLS และ policy: ทำให้ทุกอย่างเฉพาะแกลวของตัวเอง
28–30	trigger อัปเดตเวลาแก้ไขล่าสุด

อ่านโค้ด: แถบเลื่อน ต้นไม้เงิน และสิ่งที่ส่งให้โค้ช

ชิ้นส่วนเสริมที่ทำให้ FR-6 ใช้งานง่ายและเชื่อมกับ FR-5

Slider: แถบเลื่อนที่เขียนเอง

src/ui/Slider.tsx

บรรทัด 1-65

```

1  /**
2   * A slider drawn in JS (no native module, so installed apps get it over the
3   * air): drag the gold thumb or tap the track; it snaps to `step` and gives a
4   * light tick on the phone at each step. Screen readers can adjust it too, and
5   * on the web the arrow keys move it.
6   */
7  import * as Haptics from 'expo-haptics';
8  import { useEffect, useRef, useState } from 'react';
9  import { Animated, Platform, View, type GestureResponderEvent, type LayoutChangeEvent } from 'react-native';
10 import { T } from './components';
11 import { useNative } from './motion';
12 import { space, useTheme } from './theme';
13
14 const THUMB = 30;
15
16 export function Slider({
17   value,
18   onChange,
19   min = 0,
20   max = 100,
21   step = 5,
22   label,
23   format = (v) => `${v}`,
24   ticks,
25 }: {
26   value: number;
27   onChange: (v: number) => void;
28   min?: number;
29   max?: number;
30   step?: number;
31   /** What the slider changes, for screen readers. */
32   label: string;
33   format?: (v: number) => string;
34   /** Values labelled under the track. */
35   ticks?: number[];
36 }) {
37   const theme = useTheme();
38   const [width, setWidth] = useState(0);
39   const [pos] = useState(() => new Animated.Value(0));
40   const [grab] = useState(() => new Animated.Value(0));
41   // Where the track starts on the screen (found when the finger goes down), and the last value sent.
42   const originX = useRef(0);
43   const sent = useRef(value);
44   const travel = Math.max(0, width - THUMB);
45
46   useEffect(() => {
47     if (!width) return;
48     const x = ((value - min) / (max - min || 1)) * Math.max(0, width - THUMB);
49     const anim = Animated.spring(pos, { toValue: x, friction: 9, tension: 120, useNativeDriver: useNative });
50     anim.start();
51     return () => anim.stop();
52   }, [value, width, min, max, pos]);
53
54   const clamp = (v: number) => Math.min(max, Math.max(min, Math.round(v / step) * step));
55   const pick = (pageX: number) => {

```

```

56   if (!travel) return;
57   const v = clamp(min + ((pageX - originX.current - THUMB / 2) / travel) * (max - min));
58   if (v !== sent.current) {
59     sent.current = v;
60     Haptics.selectionAsync().catch(() => {});
61     onChange(v);
62   }
63 };
64 const lift = (to: number) => Animated.spring(grab, { toValue: to, friction: 6, useNativeDriver: useNative
65   }).start();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/Slider.tsx:1`

บรรทัด ความหมาย

1–6	คอมเมนต์: แล่นเลื่อนวาดด้วย JavaScript ล้วน (ไม่ใช้ native module) แอปที่ลงแล้วจึงได้ทาง OTA ลากหรือแตะราง ดัดเข้าขั้นละ step สั้นเบา ๆ ทุกขั้น โปรแกรมอ่านหน้าจอปรับได้ และบนเว็บใช้ลูกศรได้
14	ขนาดปุ่มกลม 30 จุด
16–36	props: ค่า, ฟังก์ชันเมื่อเปลี่ยน, ต่ำสุด, สูงสุด, ขึ้น, ชื่อ (สำหรับโปรแกรมอ่านหน้าจอ), รูปแบบข้อความ, ป้ายใต้ราง
38	ความกว้างจริงของราง (รู้หลังวาดเสร็จ)
39–40	ค่าแอนิเมชัน: ตำแหน่งปุ่ม และการขยายตอนจับ
42–43	จำจุดเริ่มของรางบนจอ และค่าล่าสุดที่ส่งออกไป (useRef เพราะไม่ต้องวาดใหม่)
44	ระยะที่ปุ่มวิ่งได้ = ความกว้าง - ขนาดปุ่ม
46–52	เมื่อค่าเปลี่ยน ให้ปุ่มเดินไปตำแหน่งใหม่แบบสปริง (native driver) ตำแหน่ง = (ค่า - ต่ำสุด) ÷ ช่วง × ระยะวิ่ง
54	clamp : ปิดค่าเข้าขั้นที่ใกล้ที่สุด (เช่น 27 → 25) และไม่ให้เกินช่วง
55–63	pick : แปลงตำแหน่งนิ้วบนจอเป็นค่า ถ้าค่าเปลี่ยนจริงค่อยส่งและแจ้งออกไป (ไม่สั้นรัว ๆ ตอนนิ้วขยับนิดเดียว)
64	ปุ่มขยายเมื่อจับ หดเมื่อปล่อย

`src/ui/Slider.tsx`

บรรทัด 66–115

```

66   return (
67     <View style={{ gap: space.xs }}>
68       <View
69         onLayout={(e: LayoutChangeEvent) => setWidth(e.nativeEvent.layout.width)}
70         accessible
71         accessibilityRole="adjustable"
72         accessibilityLabel={label}
73         accessibilityValue={{ min, max, now: value, text: format(value) }}
74         aria-valuemin={min}
75         aria-valuemax={max}
76         aria-valuenow={value}
77         aria-valuetext={format(value)}
78         accessibilityActions={[{ name: 'increment' }, { name: 'decrement' }]}
79         onAccessibilityAction={(e) => onChange(clamp(value + (e.nativeEvent.actionName === 'increment' ? step
: -step)))}
80         onStartShouldSetResponder={() => true}
81         onMoveShouldSetResponder={() => true}
82         onResponderTerminationRequest={() => false}
83         onResponderGrant={(e: GestureResponderEvent) => {
84           originX.current = e.nativeEvent.pageX - e.nativeEvent.locationX;
85           sent.current = value;
86           lift(1);
87           pick(e.nativeEvent.pageX);
88         }}
89         onResponderMove={(e: GestureResponderEvent) => pick(e.nativeEvent.pageX)}

```

```

90     onResponderRelease={() => lift(0)}
91     onResponderTerminate={() => lift(0)}
92     // Web: arrow keys (and Home/End) move it too, like a built-in slider.
93     {...(Platform.OS === 'web'
94       ? {
95         focusable: true,
96         onKeyDown: (e: { key?: string; nativeEvent?: { key?: string }; preventDefault?: () => void }) =>
97         {
98           const key = e.key ?? e.nativeEvent?.key;
99           const next =
100             key === 'ArrowRight' || key === 'ArrowUp'
101               ? value + step
102             : key === 'ArrowLeft' || key === 'ArrowDown'
103               ? value - step
104             : key === 'Home'
105               ? min
106             : key === 'End'
107               ? max
108             : null;
109           if (next === null) return;
110           e.preventDefault?();
111           const v = clamp(next);
112           if (v !== value) onChange(v);
113         },
114       : {})}
115     style={{ height: THUMB + 12, justifyContent: 'center' }}

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/Slider.tsx:66

บรรทัด ความหมาย

69	วัดความกว้างรางหลังวาด
70–79	accessibility: บอกระบบว่าเป็นตัวปรับค่า (adjustable/slider) มีชื่อ ค่าปัจจุบัน ต่ำสุด สูงสุด เป็นทั้งแบบ React Native และแบบเว็บ (aria-*) และรับคำสั่ง "เพิ่ม/ลด" จาก TalkBack/VoiceOver (ปัดขึ้นลงบนจอ) ทีละ step
80–82	ขอรับการสัมผัสทั้งหมด และไม่ยอมปล่อยให้ ScrollView แยกไปตอนลาก
83–88	นิ้วแตะลง: หาจุดเริ่มของราง จำค่าเดิม ขยายปุ่ม แล้วเลือกค่าตรงจุดที่แตะ (แตะรางแล้วกระโดดไปได้เลย)
89	นิ้วลาก: เลือกค่าตามตำแหน่งนิ้วตลอด
90–91	ปล่อยนิ้วหรือถูกขัดจังหวะ: หดปุ่ม
92–114	เฉพาะเว็บ: กดลูกศรขวา/ขึ้น = เพิ่ม 1 ขึ้น, ซ้าย/ลง = ลด, Home = ต่ำสุด, End = สูงสุด (เหมือนแถบเลื่อนของเบราว์เซอร์) E2E ทดสอบ ว่ากดขวาแล้วเพิ่ม 5

บรรทัด 117–160 วาดราง (เทาอ่อน) ส่วนที่เติมสีทองถึงปุ่ม (ยึดด้วย `scaleX` ซึ่งเร็วกว่าเปลี่ยนความกว้าง) ปุ่มกลมสีทองขอบขาวมีเงา และป้ายตัวเลขใต้ราง ป้ายของค่าปัจจุบันเป็นสีเข้ม

treeHealth: ใบของกีเปอร์เซ็นต์

src/ui/art.tsx

บรรทัด 233–237

```

233 export function treeHealth(status: string, days: number | null): number {
234   if (status === 'below_floor') return 0.04;
235   if (status === 'no_spending' || days === null) return 0.7;
236   return Math.max(0.08, Math.min(1, days / 45));
237 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/art.tsx:233

บรรทัด 234: ต่ำกว่าเส้น → 4% (ในร่องเกือบหมด) · 235: คำนวณไม่ได้ → 70% · 236: ปกติ = วัน ÷ 45 แต่ไม่ต่ำกว่า 8% และไม่เกิน 100% (45 วัน = ต้นไม้เต็ม)

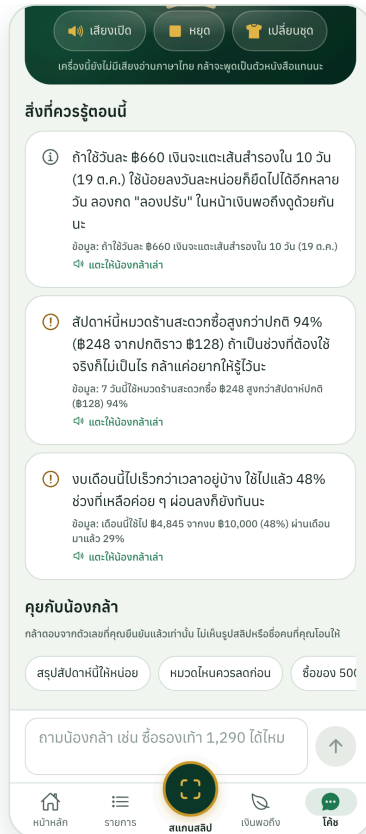
การ์ดเตือน Runway ของโค้ช (insights.ts)

src/domain/insights.tsบรรทัด 84-101

```
84 // Runway first: it is the most important number in the app.
85 if (runway.status === 'below_floor') {
86   const fact = `ยอดคงเหลือ ${formatBaht(runway.balanceSatang)} ต่ำกว่าเส้นเงินสำรอง
    ${formatBaht(runway.floorSatang)}`;
87   out.push({
88     kind: 'runway',
89     severity: 'watch',
90     fact,
91     message: `ตอนนี้เงินต่ำกว่าเงินสำรองที่ตั้งไว้ (${formatBaht(runway.floorSatang)}) แล้วนะ ไม่เป็นไร ช่วงนี้ใช้เฉพาะที่จำเป็นก่อน แล้วลองดูว่ามีรายจ่ายไหนเลื่อนออกไปได้บ้าง`,
92   });
93 } else if (runway.status === 'critical' || runway.status === 'watch') {
94   const fact = `ถ้าใช้วันละ ${baht(runway.averageSatang)} เงินจะแตะเส้นสำรองใน ${runway.days} วัน
    (${formatThaiDay(runway.depletionDay!, { year: false })})`;
95   out.push({
96     kind: 'runway',
97     severity: runway.status === 'critical' ? 'watch' : 'info',
98     fact,
99     message: `${fact} ใช้น้อยลงวันละหน่อยก็ยืดไปได้อีกหลายวัน ลองกด "ลองปรับ" ในหน้าเงินพอถึงดูด้วยกันนะ`,
100   });
101 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/insights.ts:84

บรรทัด	ความหมาย
84	คอมเมนต์: Runway มาก่อน เพราะเป็นตัวเลขที่สำคัญที่สุดในแอป
85-92	ต่ำกว่าเส้น: ข้อเท็จจริง (ยอด vs เส้นสำรอง) และข้อความแบบน้องกล้า "ไม่เป็นไร ช่วงนี้ใช้เฉพาะที่จำเป็นก่อน..." ระดับ watch
93-100	critical หรือ watch: "ถ้าใช้วันละ ฿660 เงินจะแตะเส้นสำรองใน 10 วัน (19 ต.ค.)" ตามด้วยชวนลองแถบ "ลองปรับ" ระดับ watch สำหรับ critical, info สำหรับ watch · สถานะ healthy ไม่มีการ์ดเตือน



แท็บโด้เมื่อสถานะเริ่มต้องระวัง: การ์ดแรกคือค่าเดือน Runway

ข้อมูลที่ส่งให้ AI (buildCoachContext)

src/domain/insights.ts

บรรทัด 177-194

```
177 const r = params.runway;
178 return {
179   today: formatThaiDay(today),
180   balance: formatBaht(params.balanceSatang),
181   lowLine: formatBaht(r.floorSatang, { decimals: false }),
182   averageDailySpend7d: formatBaht(r.averageSatang, {
183     decimals: false }),
184   runway:
185     r.status === 'no_spending'
186       ? 'ยังคำนวณไม่ได้ (ไม่มีรายจ่ายใน 7 วัน)'
187       : r.status === 'below_floor'
188         ? 'ต่ำกว่าเส้นเงินสำรองแล้ว'
189         : ` ${r.capped ? 'มากกว่า' : ''} ${r.days} วัน (ถึง
190           ${formatThaiDay(r.depletionDay!)} )`,
191   monthlyBudget: params.profile.monthlyBudgetSatang ?
192     formatBaht(params.profile.monthlyBudgetSatang, { decimals:
193       false }) : null,
194   spentThisMonth: formatBaht(monthExpense(params.txs, now),
195     { decimals: false }),
196   expenseByCategoryLast7Days: byCat(addDays(today, -6)),
197   expenseByCategoryLast30Days: byCat(addDays(today, -29)),
198   incomeLast30Days: formatBaht(income30, { decimals: false
199     })
200   };
```

▶ เปิดใน VS Code
(เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/domain/insights.ts:177

บรรทัด 181-188 คือส่วนของ FR-6: เส้นสำรอง, ค่าเฉลี่ย 7 วัน, และ Runway เป็นข้อความพร้อมใช้ (เช่น "16 วัน (ถึง 25 ต.ค. 2569)" หรือ "ต่ำกว่าเส้นเงินสำรองแล้ว") ทุกตัวแปลงเป็นข้อความบาทแล้ว AI จึงไม่ต้องคำนวณหรือแปลงหน่วยเอง

แบบ purchaseCheck ตอนถามโด้ (coach.tsx)

src/app/(tabs)/coach.tsx

บรรทัด 122-145

```
122 try {
123   const price = params.price && q === params.ask ? Number(params.price) : priceInQuestion(q);
124   const context = {
125     ...buildCoachContext({ txs, profile, balanceSatang: balance, runway }),
126     ...(price
127       ? {
128         purchaseCheck: {
129           price: formatBaht(price, { decimals: false }),
130           runwayBefore: runway.days === null ? 'คำนวณไม่ได้' : ` ${runway.days} วัน`,
131           runwayAfter: (() => {
132             const r = runwayAfterPurchase(runway, price);
133             return r.status === 'below_floor' ? 'ต่ำกว่าเส้นสำรองทันที' : r.days === null ? 'คำนวณไม่ได้' :
134               ` ${r.days} วัน`;
135           })(),
136         },
137       : {}),
138   };
139   if (repo?.mode === 'demo') {
140     throw new CoachError(`โหมดทดลองยังถาม AI ไม่ได้นะ เข้าระบบด้วยบัญชีจริงแล้วถาม ${BUDDY_NAME} ได้เลย ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้ ${BUDDY_NAME} เล่าให้ฟังได้`);
141   }
```

```

142     const answer = await askCoach({ context, question: q });
143     const id = ++seq.current;
144     setMsgs((m) => [...m, { id, from: 'coach', text: answer }]);
145     tell(answer, { msgId: id });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/coach.tsx:122

บรรทัด	ความหมาย
123	หาราคา: ถ้ามาจากปุ่ม "ถามโค้ชเรื่องนี้" และยังไม่ได้แก้คำถาม ใช้ราคาที่ส่งมา ไม่ขึ้นหาราคาในประโยคเอง (priceInQuestion)
124–125	ข้อมูลสรุปปกติ (buildCoachContext)
126–137	ถ้ามีราคา เพิ่ม purchaseCheck: ราคา, Runway ก่อนซื้อ, และ Runway หลังซื้อ ซึ่ง คำนวณด้วย runwayAfterPurchase ในเครื่อง (ต่ำกว่าเส้นทันที / คำนวณไม่ได้ / X วัน)
139–141	โหมดทดลองถาม AI ไม่ได้ บอกผู้ใช้ตรง ๆ
142–145	ส่งคำถามพร้อมข้อมูลไปที่ Edge Function coach แล้วแสดงคำตอบและให้น้องกล่าอ่านออกเสียง

ช่องเงินสำรองในหน้าตั้งค่า

src/app/settings.tsx

บรรทัด 49–72

```

49  // Satang are kept (the fields show them), and #0 is a fine balance or reserve.
50  const money = (s: string) => parseBahtToSatang(s, { allowZero: true });
51
52  async function save() {
53      const e: Record<string, string> = {};
54      const o = money(opening);
55      const f = money(floor);
56      const b = budget.trim() ? parseBahtToSatang(budget) : null;
57      if (!name.trim()) e.name = 'ใส่ชื่อเล่น';
58      if (o === null) e.opening = 'ใส่เป็นตัวเลข';
59      if (f === null) e.floor = 'ใส่เป็นตัวเลข';
60      if (budget.trim() && b === null) e.budget = 'ใส่เป็นตัวเลข หรือเว้นว่าง';
61      setErrors(e);
62      if (Object.keys(e).length) return;
63      setBusy(true);
64      try {
65          await saveProfile({ displayName: name.trim(), openingBalanceSatang: o!, runwayFloorSatang: f!,
66              monthlyBudgetSatang: b });
67          toast({ message: 'บันทึกการตั้งค่าแล้ว' });
68      } catch {
69          toast({ message: 'บันทึกไม่สำเร็จ', tone: 'error' });
70      } finally {
71          setBusy(false);
72      }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/settings.tsx:49

บรรทัด	ความหมาย
49–50	ฟังก์ชันแปลงเงินของหน้านี้ยอมรับ 0 (ยอดเริ่มต้นหรือเงินสำรอง #0 เป็นค่าที่ถูกต้อง) และเก็บสตางค์ครบ
53–60	ตรวจทุกช่อง ถ้าไม่ใช่ตัวเลข ขึ้น "ใส่เป็นตัวเลข" ใต้ช่องนั้น จบรายเดือนเว้นว่างได้
65	บันทึกโปรไฟล์ รวม runwayFloorSatang → AppProvider อัปเดต profile → useMoney คำนวณใหม่ทุกหน้า
66–68	แจ้งผลสำเร็จหรือไม่สำเร็จ

สร้าง FR-6 ตั้งแต่ศูนย์: ทำตามได้ทีละขั้น

บทก่อน ๆ อ่านโค้ดที่เสร็จแล้ว บทนี้พาสร้าง Money Runway ขึ้นมาใหม่ทีละขั้น ทุกขั้นมีคำสั่งให้พิมพ์ และบอกว่าต้องเห็นอะไรบนจอ

คนที่ไม่เคยเขียนโปรแกรมมักคิดว่าโปรแกรมเมอร์นั่งพิมพ์โค้ดจากบรรทัดแรกไปบรรทัดสุดท้ายรวดเดียว ความจริงไม่ใช่ ฉันคิด FR-6 เป็นขั้น ๆ เริ่มจาก "คำนิยาม" ซึ่งยังไม่ใช้โค้ดเลย แล้วค่อยเขียนเทสต์ เขียนสูตร ต่อเข้ากับข้อมูลจริง ทำหน้าจอลงทดสอบแบบคนใช้ แล้วส่งขึ้น GitHub ทุกขั้นตรวจได้ว่าถูกต้องก่อนขึ้นขั้นถัดไป ถ้าทำตามบทนี้ เพื่อนจะเข้าใจว่า "ทำไมโค้ดหน้าตาเป็นแบบนี้" ไม่ใช่แค่ "โค้ดทำอะไร"

ภาพรวม: 10 ขั้นจากศูนย์ถึงแอปในมือผู้ใช้

ขั้น	ทำอะไร	ไฟล์ที่เกี่ยวข้อง	รู้ได้ยังไงว่าผ่าน
0	เตรียมเครื่อง	ทั้งโปรเจกต์	<code>npm test</code> ผ่าน 157/157
1	เขียนนิยามให้ทดสอบได้	คอมเมนต์บนสุดของ <code>runway.ts</code> , <code>TRACEABILITY.md</code>	ทุกคำนิยามแปลงเป็นเทสต์ได้
2	เขียนเทสต์ก่อน (แดง)	<code>domain.test.ts</code>	รันแล้ว ไม่ผ่าน เพราะยังไม่มีโค้ด
3	เขียนสูตรให้เทสต์ผ่าน (เขียว)	<code>src/domain/runway.ts</code>	รันแล้วผ่านทุกข้อ
4	ตรวจสอบคุณภาพ	ทั้งโปรเจกต์	<code>typecheck</code> และ <code>lint</code> ไม่มี error
5	ต่อสูตรเข้ากับข้อมูลจริง	<code>AppProvider.tsx (useMoney)</code>	ทุกหน้าได้ตัวเลขชุดเดียวกัน
6	ทำหน้าจอลงและเก็บ	<code>(tabs)/runway.tsx</code> , <code>(tabs)/_layout.tsx</code>	เปิดแอปเห็นแท็บ "เงินพอถึง"
7	ทดสอบแบบคนใช้ (E2E)	<code>e2e/web-auth.mjs</code>	ทุกบรรทัดขึ้น PASS
8	ส่งขึ้น GitHub	<code>.github/workflows</code>	เว็บ แอป APK และแอปที่ติดตั้งแล้วอัปเดตเอง
9	ปรับตาม feedback	ดูประวัติใน Git	เทสต์เดิมยังผ่าน และมีเทสต์ใหม่คุมของใหม่

≈ เปรียบเทียบ: สร้างบ้าน

ขั้น 1 คือคุยกับเจ้าของบ้านให้ได้แบบที่ชัด ("ห้องนอน 3×4 เมตร" ไม่ใช่ "ห้องใหญ่ ๆ") ขั้น 2–3 คือลงเสาเข็มแล้วตรวจว่ารับน้ำหนักได้ ขั้น 5–6 คือก่อผนังและทาสี ขั้น 7 คือให้คนลองเดินเข้าออกจริง ขั้น 8 คือส่งมอบกุญแจ ถ้าข้ามขั้น 1 ไปทาสีเลย ทุกอย่างหลังจากนั้นจะต้องรื้อใหม่

ขั้นที่ 0: เตรียมเครื่อง (ทำครั้งเดียว)

ต้องมี Node.js 22 ขึ้นไป, Git และ VS Code (วิธีติดตั้งทีละขั้นอยู่ในบท A3) แล้วเปิดโปรแกรม Terminal (Windows ใช้ PowerShell หรือ Git Bash) พิมพ์ทีละบรรทัด:

Terminalพิมพ์ทีละบรรทัดแล้วกด Enter

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install # ดาวน์โหลดไลบรารีทั้งหมดตาม package.json (ครั้งแรกใช้เวลาหลายนาที)
4 cp .env.example .env # คำการเชื่อม Supabase ของทีม (ถ้าใช้ Command Prompt ของ Windows พิมพ์ copy แทน cp)
5 npm test # ลองรันเทสต์ทั้งหมดดูว่าเครื่องพร้อม
```

ต้องเห็นบรรทัดสรุปท้ายแบบนี้ (เวลาจะต่างกันไปตามเครื่อง):

Test Files 18 passed (18)
Tests 157 passed (157)

จากนั้นเปิด VS Code → **File** → **Open Folder...** → เลือกโฟลเดอร์ Socrates - and - Skeletons - ใน VS Code กด **Ctrl+`** (ปุ่มตัว
หนอนใต้ Esc) เพื่อเปิด Terminal ด้านล่าง ต่อจากนี้พิมพ์คำสั่งในนั้นได้เลยโดยไม่ต้องสลับหน้าต่าง

i ถ้าเห็นข้อความเตือนสีแดง

ตอนรันเทสต์ อาจมีข้อความขึ้นต้นด้วย (!) Your Vite config uses features that are unsupported... นั้น
เป็นแค่ คำเตือน ว่าเครื่องมือเวอร์ชันหน้าจะเปลี่ยนวิธีอ่านไฟล์ตั้งค่า ไม่ใช่ error ผลเทสต์ยังเชื่อถือได้ ให้ดูที่บรรทัด Tests ...
passed เป็นหลัก

ขั้นที่ 1: เขียนนิยามก่อนเขียนโค้ด

โจทย์ใน SRS มีแค่ "ประมาณวันที่ยอดเงินอาจต่ำ จากค่าเฉลี่ยรายจ่าย 7 วัน" และทีมได้ feedback ว่าคำว่า "ต่ำ" กำกวม ก่อนพิมพ์โค้ดสัก
บรรทัด ฉันจึงแปลงทุกคำที่กำกวมให้เป็นประโยคที่ ใส่ตัวเลขแล้วตอบได้ว่าถูกหรือผิด

ประโยคกำกวม (ทดสอบ ไม่ได้)	ประโยคที่ฉันเขียน (ทดสอบได้)	กลายเป็นเทสต์
"เดือนเมื่อเงินต่ำ"	เงินต่ำ = ยอดคงเหลือ ≤ เส้นสำรอง ค่าเริ่มต้น ฿500	ยอด ฿500 เส้น ฿500 → below_floor (TC-16)
"ค่าเฉลี่ย 7 วัน"	รายจ่ายที่ยืนยันแล้ว ของวันนี้ + 6 วันก่อน ตามเวลาไทย ÷ จำนวนวันที่ใช้ แอป (ไม่เกิน 7)	TC-12, TC-13
"ประมาณวันที่"	Runway = ปัดลง[(ยอด - เส้น) ÷ ค่าเฉลี่ย] · วันที่ = วันนี้ + Runway	TC-14
"ถ้าไม่มีข้อมูล"	ค่าเฉลี่ย = 0 → no_spending ไม่แสดงวันที่ (ไม่หารด้วยศูนย์)	TC-15
"สถานะ"	< 7 วัน critical · < 14 วัน watch · ≥ 14 วัน healthy	TC-17 (ลองที่ 6, 7, 14 พอดี)

นิยามชุดนี้อยู่ 2 ที่ และต้องตรงกันเสมอ: คอมเมนต์บนสุดของ [src/domain/runway.ts:1-18](#) (สำหรับคนอ่านโค้ด) และหัวข้อ FR-6 ใน
[docs/TRACEABILITY.md:142-163](#) (สำหรับอาจารย์และคนในทีม ซึ่งมีตารางโยง "เงื่อนไขผ่าน → ฟังก์ชัน → เทสต์" ด้วย)

★ เคล็ดลับเขียนนิยามที่ดี

ลองถามตัวเองว่า "ถ้าให้เพื่อน 2 คนเขียนโค้ดจากนิยามนี้แยกกัน ผลจะออกมาเท่ากันไหม" ถ้าไม่แน่ใจ แปลว่ายังกำกวม เช่น "7 วัน"
ต้องบอกด้วยว่านับวันนี้ไหม ใช้เวลาประเทศไหน และนับรายการแบบร่างไหม

ขั้นที่ 2: เขียนเทสต์ก่อน (ให้มันแดงก่อน)

วิธีที่ฉันใช้เรียกว่า **TDD** (Test-Driven Development) ทำเป็นรอบสั้น ๆ 3 จังหวะ:

① แดง (Red)

เขียนเทสที่บอกว่า "ใส่อะไร ต้องได้อะไร" แล้วรัน ต้องไม่ผ่าน เพราะยังไม่ได้เขียนโค้ด (ถ้าผ่านทั้งที่ยังไม่มีโค้ด แปลว่าเทสไม่ได้ตรวจสอบอะไรจริง)



② เขียว (Green)

เขียนโค้ดให้น้อยที่สุดที่ทำให้เทสผ่าน



③ เก็บกวาด (Refactor)

ปรับโค้ดให้อ่านง่ายขึ้น แล้วรันเทสซ้ำ ต้องยังเขียวอยู่ จากนั้นวนกลับไป ① สำหรับกฎข้อถัดไป

มาลองทำจริงด้วย **Money Runway ฉบับย่อ** ซึ่งเป็นแก่นของ runway.ts ที่ตัดรายละเอียดออก ฉันทันทุกบรรทัดในหัวข้อนี้กับโปรเจกต์จริงแล้ว ผลตรงตามที่เขียนไว้ทุกอย่าง

1 ใน VS Code แถบซ้าย (Explorer) คลิกขวาที่โฟลเดอร์ src/domain/__tests__ → New File → ตั้งชื่อ mini-runway.test.ts

2 พิมพ์ (หรือคัดลอก) โค้ดด้านล่าง แล้วกด **Ctrl+S** บันทึก

src/domain/__tests__/mini-runway.test.ts

ไฟล์ฝึกหัด สร้างเองในเครื่อง (ไม่มีในโปรเจกต์)

```
1 import { describe, expect, it } from 'vitest';
2 import { average7, runway } from '../mini-runway';
3
4 describe('Money Runway ฉบับย่อ', () => {
5   it('เฉลี่ย 7 วัน บัณฑิตที่ไม่ได้ใช้เงินด้วย', () => {
6     expect(average7([120, 0, 85, 300, 0, 450, 60])).toBe(145);
7   });
8   it('ผู้ใช้ใหม่ 3 วัน หายด้วย 3', () => {
9     expect(average7([90, 150, 60])).toBe(100);
10  });
11  it('ยอด 3,500 สำรอง 500 ใช้วันละ 145 ได้ 20 วัน', () => {
12    expect(runway(3500, 500, 145)).toEqual({ days: 20, status: 'healthy' });
13  });
14  it('ยอดเท่ากับเงินสำรองพอดี คือ below_floor', () => {
15    expect(runway(500, 500, 145).status).toBe('below_floor');
16  });
17  it('ไม่มีรายจ่าย ไม่หารด้วยศูนย์', () => {
18    expect(runway(3500, 500, 0)).toEqual({ days: null, status: 'no_spending' });
19  });
20  it('ขอบเขตที่ 6 / 7 / 14 วัน', () => {
21    expect(runway(1100, 500, 100).status).toBe('critical');
22    expect(runway(1200, 500, 100).status).toBe('watch');
23    expect(runway(1900, 500, 100).status).toBe('healthy');
24  });
25 });
```

บรรทัด ความหมาย

- | | |
|------|--|
| 1 | ตั้งเครื่องมือเขียนเทส 3 ตัวจาก Vitest: describe (จัดกลุ่ม) it (เทสหนึ่งข้อ) expect (ตรวจสอบ) |
| 2 | ดึงฟังก์ชัน 2 ตัวจากไฟล์ ../mini-runway (.= ก๊อปปี้ไป 1 โฟลเดอร์ คือ src/domain) ซึ่ง ยังไม่มี ตั้งใจให้เป็นแบบนี้ |
| 4 | ตั้งชื่อกลุ่มเทส ชื่อนี้จะขึ้นในผลลัพธ์ |
| 5-7 | ข้อ 1 คือตัวอย่างหลักของเล่น: 7 วัน มี 2 วันเป็น 0 รวม 1,015 ÷ 7 ต้องได้ 145 พอดี อ่านบรรทัด 6 เป็นภาษาคนว่า "คาดหวังว่า average7 ของลิสต์นี้ ต้องเป็น 145" |
| 8-10 | ผู้ใช้ใหม่มีข้อมูลแค่ 3 วัน (90 + 150 + 60 = 300) ต้องหาร 3 ได้ 100 ไม่ใช่หาร 7 |

บรรทัด	ความหมาย
11-13	สูตรหลัก toEqual ตรวจสอบทั้งข้อผิดพลาดว่าทุกช่วงตรง: $3,000 \div 145 = 20.69 \rightarrow 20$ วัน สถานะ healthy
14-16	ขอบล่าง: ยอดเท่ากับเส้นพอดี้ ต้องนับว่าแตะเส้นแล้ว
17-19	ค่าเฉลี่ย 0 ต้องได้ null (ไม่ใช่ Infinity หรือแอปพัง)
20-24	ทดสอบที่ขอบเขต: เงินเหนือเส้น 600, 700, 1,400 ใช้วันละ 100 = 6, 7, 14 วันพอดี้ ต้องได้ critical, watch, healthy

ใน Terminal ของ VS Code พิมพ์:

Terminal

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 npx vitest run src/domain/__tests__/mini-runway.test.ts
```

ต้องเห็น **สีแดง** แบบนี้ (ตัดมาเฉพาะส่วนสำคัญ):

```
FAIL src/domain/__tests__/mini-runway.test.ts
Error: Cannot find module '../mini-runway' imported from .../src/domain/__tests__/mini-runway.test.ts
> src/domain/__tests__/mini-runway.test.ts:2:1

Test Files  1 failed (1)
Tests      no tests
```

อ่านว่า "หาไฟล์ mini-runway ไม่เจอ (บรรทัด 2 คอลัมน์ 1)" ถูกต้องแล้ว เพราะเรายังไม่ได้สร้าง นี่คือจิ้งหะ แดง

ขั้นที่ 3: เขียนสูตรให้เทสต์ผ่าน (เขียว)

คลิกขวาที่โฟลเดอร์ src/domain → New File → mini-runway.ts แล้วพิมพ์:

src/domain/mini-runway.ts

ไฟล์ฝึกหัด: แก่นของ runway.ts ฉบับย่อ

```
1 // Money Runway ฉบับย่อ: เงินเป็นบวก และรับรายจ่ายรายวันมาเป็นลิสต์
2 export type Status = 'healthy' | 'watch' | 'critical' | 'below_floor' | 'no_spending';
3
4 /** ค่าเฉลี่ยรายจ่ายต่อวันจาก 7 วันล่าสุด (ถ้ามีไม่ถึง 7 วัน หารค่าที่ 0) */
5 export function average7(dailySpend: number[]): number {
6   const last7 = dailySpend.slice(-7);
7   if (last7.length === 0) return 0;
8   const total = last7.reduce((sum, x) => sum + x, 0);
9   return total / last7.length;
10 }
11
12 /** เงินพอใช้อีกกี่วัน และสถานะ */
13 export function runway(balance: number, floor: number, avg: number) {
14   const spendable = balance - floor;
15   if (spendable <= 0) return { days: 0, status: 'below_floor' as Status };
16   if (avg <= 0) return { days: null, status: 'no_spending' as Status };
17   const days = Math.floor(spendable / avg);
18   const status: Status = days < 7 ? 'critical' : days < 14 ? 'watch' : 'healthy';
19   return { days, status };
20 }
```

บรรทัด	ความหมาย
2	ชนิดข้อมูล Status: ค่าได้แค่ 5 คำนี้ ถ้าพิมพ์ผิดเช่น 'helthy' TypeScript จะขีดเส้นแดงทันที
5	ฟังก์ชันรับลิสต์ตัวเลข (รายจ่ายของแต่ละวัน เรียงจากเก่าไปใหม่) คืนตัวเลขหนึ่งตัว

บรรทัด	ความหมาย
6	<code>slice(-7)</code> = เอา 7 ตัวท้ายสุด (7 วันล่าสุด) ถ้ามีไม่ถึง 7 ตัวก็ได้เท่าที่มี นี่แหละคือกฎผู้ใช้ใหม่ฉบับย่อ
7	ลิสต์ว่างคืน 0 กันการหาร $0 \div 0$ (ซึ่งใน JavaScript ได้ NaN แปลว่า "ไม่ใช่ตัวเลข")
8	<code>reduce</code> บวกทุกตัวเข้าด้วยกัน เริ่มจาก 0
9	หารด้วยจำนวนวันในปฏิทิน (รวมวันที่เป็น 0) ไม่ใช่จำนวนวันที่มีรายจ่าย
13	รับ 3 ค่า: ยอด เงินสำรอง ค่าเฉลี่ย
14	เงินที่ใช้ได้จริง = ยอด - เงิน
15-16	ด่านตรวจ 2 ด่านก่อนหาร เรียงเหมือนของจริง: เงินหมดก่อน แล้วค่อยค่าเฉลี่ยศูนย์ (as Status บอก TypeScript ว่าข้อความนี้เป็นชนิด Status)
17	หารแล้วปัดลงด้วย <code>Math.floor</code>
18	เงื่อนไขซ้อน: น้อยกว่า 7 → critical, ไม่จึ้นน้อยกว่า 14 → watch, ไม่จึ้น healthy

บันทึกไฟล์ แล้วรันคำสั่งเดิมอีกครั้ง โดยเติม `--reporter=verbose` ให้แสดงชื่อเทสต์ทุกข้อ:

Terminal
ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 npx vitest run src/domain/__tests__/mini-runway.test.ts --reporter=verbose
```

- ✓ ... > Money Runway ฉบับย่อ > เฉลี่ย 7 วัน นับวันที่ไม่ได้ใช้เงินด้วย
- ✓ ... > Money Runway ฉบับย่อ > ผู้ใช้ใหม่ 3 วัน หารด้วย 3
- ✓ ... > Money Runway ฉบับย่อ > ยอด 3,500 สำรอง 500 ใช้วันละ 145 ได้ 20 วัน
- ✓ ... > Money Runway ฉบับย่อ > ยอดเท่ากับเงินสำรองพอดี คือ `below_floor`
- ✓ ... > Money Runway ฉบับย่อ > ไม่มีรายจ่าย ไม่หารด้วยศูนย์
- ✓ ... > Money Runway ฉบับย่อ > ขอบเขตที่ 6 / 7 / 14 วัน

Test Files 1 passed (1)
Tests 6 passed (6)

เขี้ยวครบ 6 ข้อ นี่คืองังหะเขี้ยว

ทดลองทำบ๊ิกเอง: ทำไมต้องทดสอบที่ขอบ

แก้บรรทัด 18 ของ `mini-runway.ts` จาก `days < 7` เป็น `days <= 7` (บ๊ิกที่คนพลาดบ่อยที่สุด) บันทึก แล้วรันใหม่:

```
FAIL src/domain/__tests__/mini-runway.test.ts > Money Runway ฉบับย่อ > ขอบเขตที่ 6 / 7 / 14 วัน
AssertionError: expected 'critical' to be 'watch' // Object.is equality

Expected: "watch"
Received: "critical"

> src/domain/__tests__/mini-runway.test.ts:22:43

Tests 1 failed | 5 passed (6)
```

อ่านว่า "บรรทัด 22 ของไฟล์เทสต์ คาดว่าจะได้ watch แต่ได้ critical" เทสต์อีก 5 ข้อยังผ่าน เพราะไม่มีข้อไหนลองที่ 7 วันพอดี ถ้าไม่มีเทสต์ขอบ ขอบบ๊ิกนี้จะหลุดไปถึงผู้ใช้ คนที่มีเงินพอ 7 วันจะเห็นป้ายแดง "ใกล้เงินสำรอง" ทั้งที่ควรเป็น "เริ่มต้องระวัง" แกกลับเป็น `<` แล้วรันอีกครั้งให้เขี้ยว

จากฉบับย่อสู่ของจริง (จังหวะเก็บกวาด)

ฉบับย่อพิสูจน์ว่าแก่นของสูตรถูก แต่แอปจริงต้องรับมือโลกจริงมากกว่านั้น ฉันเติมทีละเรื่อง แต่ละเรื่องเพิ่มทดสอบแล้วค่อยแก้ไข (วนรอบแดง → เขียว ช้ำ) จนได้ `src/domain/runway.ts` ที่อ่านทีละบรรทัดไปแล้วในบทก่อน ๆ

เรื่อง	ฉบับย่อ	ของจริง (runway.ts)	ทำไมต้องเพิ่ม
หน่วยเงิน	บาท มีทศนิยม	สตางค์ จำนวนเต็ม (balanceSatang)	ทศนิยมในคอมพิวเตอร์เพี้ยนได้ (bn A5)
ข้อมูลเข้า	ลิสต์รายจ่ายรายวันที่รวมมาแล้ว	รายการ Transaction ทั้งหมด ต้องกรองเอง	ฐานข้อมูลเก็บเป็นรายการ ไม่ได้เก็บเป็นยอดรายวัน
กรองรายการ	ไม่มี	เฉพาะ confirmed · เฉพาะ expense · อยู่ในหน้าต่าง 7 วัน	สลับแบบร่างและรายรับไม่ใช่ "ความเร็วในการใช้เงิน"
วันที่	ไม่มี	นับวันตามเวลาไทย (bkkDayKey)	เครื่องของผู้ใช้อาจตั้งเวลาประเทศอื่น
ผู้ใช้ใหม่	นับจากความยาวลิสต์	นับจากวันของรายการแรกสุด (รายรับหรือรายจ่าย) ถึงวันนี้ ไม่เกิน 7	วันที่ยังไม่ได้ใช้แอป ต่างจากวันที่ไม่ได้ใช้เงิน
ปิดค่าเฉลี่ย	ไม่ปิด	ปิดเป็นสตางค์ (Math . round)	เก็บเป็นจำนวนเต็มต่อได้
ผลลัพธ์	days, status	+ depletionDay, capped, spendableSatang และตัวตั้งทั้ง 3	หน้าจอสองจอใช้แสดงการแจ้งเตือน "คำนวณยังไม่จบ"
เพดาน	ไม่มี	ตัดที่ 365 วัน (capped)	คาดการณ์ใกล้เคียงปีไม่มีความหมาย
ฟังก์ชันเสริม	ไม่มี	safeDailySpend, endOfMonthDay, runwayAfterPurchase, runwayWithReduction	เครื่องมือ "ใช้ได้วันละ" "เช็กก่อนจ่าย" "ลองปรับ"

! อย่าลืมนลบไฟล์ฝึกหัด

ไฟล์ mini-runway ทั้ง 2 ไฟล์มีไว้ฝึกในเครื่องเท่านั้น พิมพ์ `git status --short` จะเห็นบรรทัดขึ้นต้นด้วย ?? แปลว่า "ไฟล์ใหม่ที่ Git ยังไม่รู้จัก" ลบทิ้งได้เลย (คลิกขวา → Delete) อย่า commit ขึ้นโปรเจกต์ของทีม

ขั้นที่ 4: ตรวจสอบคุณภาพก่อนไปต่อ

ทดสอบผ่านยังไม่พอ ก่อนไปขึ้นถัดไปฉันรัน 3 คำสั่งนี้เสมอ แต่ละตัวจับปัญหาคนละแบบ:

Terminal

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 npm test           # ทดสอบยังถูกไหม (unit test ทั้งโปรเจกต์ 157 ข้อ)
2 npm run typecheck  # ชนิดข้อมูลถูกไหม (TypeScript ตรวจสอบทั้งโปรเจกต์)
3 npx expo lint      # เขียนผิดแบบแผนไหม (ESLint)
```

คำสั่ง	จับอะไรได้	ตัวอย่างกับ FR-6
<code>npm test</code>	ตรรกะผิด	เขียน <code><=</code> แทน <code><</code> (เพ่งลองไป)
<code>npm run typecheck</code>	ใช้ข้อมูลผิดชนิด	เอา <code>r.days + 1</code> ทั้งที่ <code>days</code> อาจเป็น <code>null</code>
<code>npx expo lint</code>	รูปแบบที่มักทำให้เกิดบั๊ก	รายการ dependency ของ <code>useMemo</code> ใน <code>useMoney</code> (ดูด้านล่าง)

ตัวอย่าง typecheck: ถ้าเขียนโค้ดคิด "พรุ่งนี้เหลือกี่วัน" แบบนี้

ตัวอย่างที่ผิด

ทดลองแล้ว ได้ error ตามด้านล่างจริง

```
1 const r = computeRunway({ balanceSatang: 350000, floorSatang: 50000, averageSatang: 14500 });
2 export const tomorrow = r.days + 1;
```

error TS18047: 'r.days' is possibly 'null'.

TypeScript เตือนว่า days อาจเป็น null (กรณีไม่มีรายจ่าย) ถ้าปล่อยไว้ ผู้ใช้ที่ยังไม่มีรายจ่ายจะเห็นคำว่า "null วัน" หรือ NaN บนจอ วิธีแก้คือเช็ก่อน เช่น `r.days === null ? 'คำนวณไม่ได้' : r.days + 1` ซึ่งเป็นเหตุผลที่ในหน้าจอจริงมีฟังก์ชัน `daysText` คอยแยกกรณี (บรรทัด 22-26 ของ `runway.tsx`)

ตัวอย่าง lint ที่เกิดกับ FR-6 จริง: ใน `useMoney` ฉันใส่ `today` ไว้ในรายการ dependency ของ `useMemo` ทั้งที่ข้างในไม่ได้ใช้ตัวแปรนี้เลย ถ้าลบบรรทัดปิดคำเตือนออกแล้วรัน lint จะได้ (ลองแล้ว):

```
457:6 warning React Hook useMemo has an unnecessary dependency: 'today'.
      Either exclude it or remove the dependency array react-hooks/exhaustive-deps
```

lint คิดว่าใส่เกินมาโดยไม่ตั้งใจ แต่ฉันตั้งใจ: พอ `today` เปลี่ยนตอนเที่ยงคืน `useMemo` จะคิดยอดและ Runway ใหม่ หน้าต่าง 7 วันจึงเลื่อนเองแม้เปิดแอปค้างไว้ ฉันจึงปิดคำเตือนเฉพาะบรรทัดนั้น พร้อมเขียนคอมเมนต์บอกเหตุผลไว้บรรทัดก่อนหน้า

([src/data/AppProvider.tsx:456-457](#)) กฎคือ ปิดคำเตือนได้ ถ้าเขียนเหตุผลไว้ให้คนถัดไปอ่าน

ขั้นที่ 5: ต่อสูตรเข้ากับข้อมูลจริง

ตอนนี้สูตรถูกแล้ว แต่ยังลอยอยู่ ไม่มีใครเรียกใช้ ฉันต่อกับข้อมูลจริงที่จุดเดียว คือฮุก `useMoney` ใน

[src/data/AppProvider.tsx:442-459](#) (อ่านทีละบรรทัดในบท "อ่านโค้ด: การ์ดบนหน้าหลัก และ `useMoney`")

Supabase / ข้อมูลตัวอย่าง

profiles (ยอดเริ่มต้น เงินสำรอง) · transactions · savings_goals

↓ AppProvider โหลดมาเก็บไว้: profile, txs, carrySatang, goals, today

useMoney()

computeBalance → averageDailyExpense → reservedSatang → computeRunway

↓ คำนวณ { balance, average, runway, drafts, reserved, floor }

หน้าหลัก · แท็บเงินพอถึง · โค้ช · สรุปเดือน และหน้าอื่น ๆ

ทุกหน้าเรียก `useMoney()` ตัวเดียวกัน ตัวเลขจึงตรงกันทุกหน้า

★ ทำไมทำสูตรก่อน แล้วค่อยต่อข้อมูล

เพราะสูตรเป็น pure function ทดสอบได้ในเสี้ยววินาทีโดยไม่ต้องมีอินเทอร์เนต ไม่ต้องมีบัญชี ไม่ต้องเปิดแอป ถ้าทำกลับกัน (ทำหน้าจอก่อน) ทุกครั้งที่อยากรู้ว่าสูตรถูกไหม ต้องเปิดแอป ล็อกอิน จดรายการ แล้วนั่งคิดเลขเทียบ ซ้ำกว่ากันหลายร้อยเท่า

ขั้นที่ 6: ทำหน้าจอ และเพิ่มแท็บ

MindPay ใช้ Expo Router ซึ่งมีกฎง่ายมาก: ไฟล์หนึ่งไฟล์ในโฟลเดอร์ `src/app` = หน้าจอหนึ่งหน้า และชื่อไฟล์คือที่อยู่ของหน้านั้น ส่วนโฟลเดอร์ที่ชื่ออยู่ในวงเล็บอย่าง (`tabs`) เป็นแค่ "กลุ่ม" ไม่ถูกนับเป็นส่วนของที่อยู่ (บท A7)

ไฟล์	กลายเป็น
<code>src/app/(tabs)/index.tsx</code>	หน้าหลัก ที่อยู่ /
<code>src/app/(tabs)/runway.tsx</code>	แท็บเงินพอถึง ที่อยู่ /runway (บนเว็บพิมพ์ต่อท้าย URL ได้เลย)

src/app/(tabs)/_layout.tsx

กรอบของกลุ่ม (tabs): แถบแท็บด้านล่าง ไอคอน และชื่อแท็บ

ฉันจึงสร้างไฟล์ runway.tsx ในโฟลเดอร์ (tabs) แล้วบอกแถบแท็บว่าแท็บนี้ชื่ออะไร ใช้ไอคอนอะไร:

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด

ความหมาย

- 23** ตาราง TABS: ระบุแจกชื่อไฟล์ (ไม่มี .tsx) คำคือชื่อที่แสดงกับไอคอน 2 แบบ Record<string, ...> แปลว่า "อ็อบเจกต์ที่ระบุแจกเป็นข้อความ"
- 26** แท็บของฉัน: ไฟล์ runway → แสดงคำว่า "เงินพอถึง" ไอคอนใบไม้โปร่ง (leaf-outline) และเปลี่ยนเป็นใบไม้ทึบ (leaf) เมื่อกำลังเปิดแท็บนี้อยู่ ใบไม้สื่อถึงต้นไม้เงินที่ออกใบตามจำนวนวัน

src/app/(tabs)/_layout.tsx

บรรทัด 175–186

```
175 export default function TabsLayout() {
176   // Records today's visit and hands out น้องกล้า's skins for finished missions and events.
177   useKlaSync();
178   return (
179     <Tabs tabBar={props} => <TabBar {...props} /> screenOptions={{ headerShown: false }}>
180       <Tabs.Screen name="index" />
181       <Tabs.Screen name="transactions" />
182       <Tabs.Screen name="runway" />
183       <Tabs.Screen name="coach" />
184     </Tabs>
185   );
186 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:175

บรรทัด

ความหมาย

- 175** คอมโพเนนต์กรอบของกลุ่มแท็บ Expo Router เรียกใช้เองเพราะชื่อไฟล์ขึ้นต้นด้วย _layout
- 177** บันทึกว่าวันนี้เปิดแอปแล้ว และแจกสกินของน้องกล้าเมื่อทำภารกิจครบ (ส่วนของ FR-5 ไม่เกี่ยวข้องกับ FR-6 โดยตรง)
- 179** ใช้แถบแท็บที่ตัวเอง (TabBar มีปุ่มสแกนสลิปสีทองตรงกลาง) และซ่อนหัวหน้าจอมาตรฐาน
- 180–183** ลำดับแท็บจากซ้ายไปขวา: หน้าหลัก, รายการ, เงินพอถึง, โค้ช บรรทัด 182 คือแท็บของฉัน (ปุ่มสแกนแทรกอยู่กลางแถบ ระหว่างแท็บที่ 2 กับ 3)

ถ้าเริ่มทำหน้าจอเองจากศูนย์ อย่าเพิ่งทำให้สวย ทำให้ตัวเลขขึ้นจอได้ก่อน แบบนี้ (ลองแล้ว ผ่าน typecheck):

หน้าจอขึ้นแรก (ฉบับฝึก)

ตัวอย่างประกอบ: แค่ 2 บรรทัดข้อความ

```
1 import { Text, View } from 'react-native';
2 import { useMoney } from '../data/AppProvider';
3
4 export default function RunwayScreen() {
5   const { runway } = useMoney();
6   return (
```

```

7   <View style={{ padding: 24, gap: 8 }}>
8     <Text>เงินพอใช้อีก {runway.days ?? '-'} วัน</Text>
9     <Text>สถานะ: {runway.status}</Text>
10  </View>
11  );
12 }

```

บรรทัด ความหมาย

- 1 ดึงกล่อง (View) และตัวหนังสือ (Text) จาก React Native ซึ่งกลายเป็น div/span บนเว็บ และกล่องจริงของ Android บนมือถือ
- 2 ดึงฮุค useMoney จากขั้นที่ 5
- 4 `export default` = นี่คือหน้าจอหลักของไฟล์ Expo Router จะหยิบตัวนี้ไปแสดง
- 5 เรียกฮุค แล้วหยิบเฉพาะ runway ออกมา (destructuring un A5)
- 6–11 คัดหน้าตาเป็น JSX: กล่องเว้นขอบ 24 จุด ข้างในมีข้อความ 2 บรรทัด { } คือช่องใส่ค่าจากตัวแปร ?? '-' แปลว่า "ถ้า days เป็น null ให้แสดงขีดแทน" (นี่คือการแก้ปัญหาที่ typecheck เตือนในขั้นที่ 4)

จากนั้นค่อย ๆ เติมทีละการ์ด บันทึกไฟล์แล้วดูผลทันที การ์ดทั้งหมดตรงกับ "ส่วนที่ 3–7" ในบท "อ่านโค้ด: หน้าจอเก็บเงินพอถึง":

- 1 การ์ดเขียนพร้อมตัวเลขใหญ่ วันที่ และป้ายสถานะ (ตอนแรกยังไม่มียวงแหวน วงแหวนมาทีหลังในรอบปรับ UX)
- 2 การ์ด "คำนวณยังไ้" แบบใบเสร็จ ให้ผู้ใช้ตรวจสอบเลขเองได้
- 3 เส้นเวลา 0 / 7 / 14 / 30 วัน
- 4 เครื่องมือลองปรับ (เวอร์ชันแรกเป็นปุ่ม 4 ปุ่ม ดูขั้นที่ 9)
- 5 เช็กก่อนจ่าย และปุ่มส่งคำถามต่อให้โค้ช

วิธีเปิดแอประหว่างเขียน:

Terminal

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 npx expo start # เปิดเซิร์ฟเวอร์สำหรับพัฒนา (dev server)
```

เมื่อขึ้น QR ใน Terminal: กดปุ่ม **w** เพื่อเปิดในเบราว์เซอร์ หรือสแกน QR ด้วยแอป Expo Go บนมือถือ (เครื่องคอมพิวเตอร์มือถือต้องอยู่ Wi-Fi เดียวกัน) แล้วกด "ลองใช้ด้วยข้อมูลตัวอย่าง" เพื่อดูตัวเลขโดยไม่ต้องสมัคร ทุกครั้งที่กด **Ctrl+S** บันทึกไฟล์ หน้าจอจะเปลี่ยนตามเองภายในไม่กี่วินาที (Fast Refresh) ไม่ต้องปิดเปิดแอปใหม่

ขั้นที่ 7: ทดสอบแบบคนใช้จริง (E2E)

Unit test บอกว่าสูตรถูก แต่ไม่ได้บอกว่าผู้ใช้ "ลากแถบได้จริง" หรือ "กดปุ่มแล้วไปหน้าโค้ชจริง" ฉะนั้นเขียนสคริปต์ให้เบราว์เซอร์อัตโนมัติ (Playwright) ทำแทนคน นี่คือข้อ 14 ของไฟล์ E2E:

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();

```

```

584     await slider.scrollIntoViewIfNeeded();
585     // A raw mouse drag does not wait like click() does: make sure the opening animation has fully gone.
586     const introWait = Date.now();
587     await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000 }).catch(() => {});
588     const introMs = Date.now() - introWait;
589     // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590     let box = await slider.boundingBox();
591     for (let i = 0; i < 12; i++) {
592         await page.waitForTimeout(250);
593         const again = await slider.boundingBox();
594         const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x) < 0.5;
595         box = again;
596         if (still) break;
597     }
598     const under = box
599     ? await page.evaluate(({ x, y }) => {
600         const el = document.elementFromPoint(x, y);
601         return el ? `${el.tagName} role=${el.getAttribute('role')} label=${el.getAttribute('aria-label')}` :
        'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
603     : 'no box';
604     if (box) {
605         // hover() waits until the slider itself receives the pointer (nothing on top of it).
606         await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
607         await page.mouse.down();
608         await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps: 8 });
609         await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps: 8 });
610         await page.mouse.up();
611     }
612     const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613     await page.waitForTimeout(900);
614     await shot(page, '28-runway-slider');
615     check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' && (await
visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under} introWaitMs=${introMs}`));
616     // The arrow keys move it too (keyboard users on the web).
617     await slider.focus();
618     await page.keyboard.press('ArrowRight');
619     const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620     check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0) + 5),
`value=${keyed}`);
621     await page.keyboard.press('Home');
622     await page.fill('#runway-price', '2000');
623     check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ ฿2,000')) &&
(await visible(page, 'ตอนนี้')));
624     await button(page, 'ถามโค้ชเรื่องนี้').click();
625     const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626     check('Runway: "ask the coach" opens the coach with the question ready', asked.includes('฿2,000'), asked);
627     await ctx.close();

```

► เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
572-574	เริ่มเทสต์ข้อ 14 เปิดหน้าต่างเบราว์เซอร์ใหม่ที่สะอาด (ไม่มีข้อมูลค้างจากข้ออื่น)
575-579	เข้าเว็บ รอปุ่ม "ลองใช้ด้วยข้อมูลตัวอย่าง" รอแอนิเมชันเปิดแอปจบ (introGone) กดปุ่ม แล้วรอจนเห็นคำว่า "ยอดคงเหลือ" ซึ่งแปลว่า หน้าหลักโหลดเสร็จ
580-581	ไปที่ที่อยู่ /runway ตรง ๆ (ได้เพราะกฎ "ไฟล์ = หน้า" ของขั้นที่ 6)
583	หาแถบเลื่อนจาก บทบาท (role = slider) และ ชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์" ชื่อนี้คือชื่อเดียวกับที่โปรแกรมอ่านหน้าจอของคนตาบอด อ่านออกเสียง เทสต์นี้จึงตรวจการเข้าถึง (accessibility) ไปด้วยในตัว
584-597	เลื่อนให้แถบอยู่ในจอ รอให้แอนิเมชันเปิดหายไปถึง แล้ววนวัดตำแหน่งแถบทุก 0.25 วินาที จนกว่าตำแหน่งจะนิ่ง (การ์ดค่อย ๆ เลื่อนเข้ามา ถ้าลากตอนยังขยับ จะลากพลาด)

บรรทัด	ความหมาย
598–603	เก็บข้อมูลไว้ช่วยหาสาเหตุถ้าทดสอบฟังก์: ตอนจะกด มีอะไรอยู่ใต้เมาส์
604–611	ลากจริงด้วยเมาส์: กดค้างที่ปุ่มกลม ลากไป 30% แล้วต่อไปที่ 60% ของความยาวราง แล้วปล่อย ทำไม 60% ได้ค่า 30? เพราะรางยาวจาก 0 ถึง 50 ค่าที่ 60% ของรางคือ $0.6 \times 50 = 30$
612–615	อ่านค่าปัจจุบันของแถบจาก aria-valuenow แล้วตรวจ 2 อย่าง: ต้องเป็น "30" และต้องเห็นว่า "เพิ่มขึ้น" (จากข้อความ "เงินจะพอ ... วัน · เพิ่มขึ้น ... วัน") พร้อมถ่ายภาพหน้าจอเก็บไว้
617–620	ทดสอบคีย์บอร์ด: โฟกัสที่แถบ กดลูกศรขวา 1 ครั้ง ค่าต้องเพิ่ม 5 (จาก 30 เป็น 35) สำหรับคนที่ใช้คีย์บอร์ดแทนเมาส์
621–623	กด Home ให้แถบกลับเป็น 0 แล้วพิมพ์ 2000 ในช่องราคา (#runway-price) ต้องเห็นทั้ง "ตอนนี้" และ "ถ้าซื้อ ฿2,000"
624–626	กดปุ่ม "ถามโค้ชเรื่องนี้" แล้วตรวจว่าช่องพิมพ์ในหน้าโค้ช (#coach-input) มีคำถามที่มี "฿2,000" รออยู่ (FR-6 ส่งต่อ FR-5 ได้จริง)
627	ปิดหน้าต่างเบราว์เซอร์ของข้อนี้

วิธีรัน (ครั้งแรกต้องติดตั้งเบราว์เซอร์ทดสอบก่อน):

Terminal

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 npx playwright install chromium # ครั้งแรกครั้งเดียว
2 npm run e2e:web # build เว็บ แล้วรันเทสต์ทั้งหมด (ใช้เวลาหลายนาที)

```

ผลแต่ละข้อจะขึ้นเป็นบรรทัดแบบนี้ (ตามฟังก์ชัน check บรรทัด 31–33 ของไฟล์เดียวกัน):

```

PASS Runway: dragging the slider to 30% less a day shows the extra days - value=30 ...
PASS Runway: the slider also moves with the arrow keys - value=35
PASS Runway: buying ฿2,000 shows the days before and after
PASS Runway: "ask the coach" opens the coach with the question ready - ถ้าฉันซื้อของราคา ฿2,000 วันนี้ จะเป็นยังไง

```

i E2E ไม่แตะฐานข้อมูลจริง

ไฟล์ `e2e/fake-backend.mjs` จำลองเซิร์ฟเวอร์ Supabase ขึ้นมาในเครื่อง (บน A12) เทสต์ข้อนี้ใช้โหมดข้อมูลตัวอย่าง จึงรันกี่รอบก็ได้ ไม่ทำให้บัญชีของใครเลอะ

ขั้นที่ 8: ส่งงานขึ้น GitHub

เมื่อทุกอย่างผ่าน ก็ถึงเวลาเก็บงานเข้า Git และส่งขึ้น GitHub (พื้นฐานอยู่ในบน A13) ทำได้ทั้งใน Terminal และในแผง **Source Control** ของ VS Code (ไอคอนกิ้งไม้ทางซ้าย)

Terminal

แนวปฏิบัติที่แนะนำ: แยก branch แล้วเปิด Pull Request

```

1 git switch -c fr6-whatif-slider # สร้าง branch ใหม่สำหรับงานนี้
2 git status --short # ดูว่าไฟล์ไหนเปลี่ยน (M = แก้ไข, ?? = ไฟล์ใหม่)
3 git add src/ui/Slider.tsx "src/app/(tabs)/runway.tsx" e2e/web-auth.mjs README.md docs/TRACEABILITY.md
4 git commit -m "Runway what-if: a slider instead of four chips"
5 git push -u origin fr6-whatif-slider # ส่ง branch ขึ้น GitHub

```

จากนั้นเปิดหน้า GitHub ของโปรเจกต์ จะมีปุ่ม **Compare & pull request** ให้เพื่อนในทีมช่วยอ่านโค้ด (code review) ก่อนรวมเข้า main

! พุดตามจริง: ทีมเราทำยังไง

ในประวัติของโปรเจกต์ไม่มี merge commit เลย แปลว่าทีม push ขึ้น main ตรง ๆ ทุกครั้ง ซึ่งเร็ว แต่ไม่มีคนอ่านโค้ดก่อนรวม ทีมชดเชยด้วยเทสต์อัตโนมัติที่รันทุกครั้ง push ถ้าอาจารย์ถาม ให้ตอบตรง ๆ แบบนี้ และบอกว่าถ้าทีมใหญ่ขึ้น ควรใช้ branch + Pull Request ตามด้านบน

ทุกครั้งที่มีโค้ดใหม่บน main, GitHub Actions จะทำงานต่อให้เอง 4 งาน (ไฟล์อยู่ในโฟลเดอร์ .github/workflows):

ไฟล์	ทำอะไร	ผลที่ได้
web.yml	build เว็บ → รัน unit test → รัน E2E บน Chromium → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บ MindPay บน GitHub Pages อัปเดต (ถ้าเทสต์ไม่ผ่าน เว็บจะ ไม่ ถูกเผยแพร่ และแนบภาพหน้าจอตอนพังไว้ให้ดู)
e2e-webkit.yml	รัน E2E ชุดเดียวกันบนแอนดรอยด์ของ Safari แบบ iPhone โหมดมืด	รู้ว่าผู้ใช้ iPhone ใช้ได้ด้วย (ไม่ขวางการเผยแพร่เว็บ)
android-apk.yml	build ไฟล์ติดตั้ง Android บนเซิร์ฟเวอร์ของ GitHub (15–25 นาที)	ไฟล์ mind-pay.apk ในหน้า Releases
eas-update.yml	ส่งโค้ด JavaScript ใหม่ไปให้แอปที่ติดตั้งแล้ว (EAS Update)	แอปในมือถือโหลดเองเบื้องหลัง เปิดครั้งถัดไปได้เวอร์ชันใหม่ ไม่ต้องลง APK ใหม่

★ ทำไมแถบเลื่อนของฉันส่งถึงมือถือได้โดยไม่ต้องลงแอปใหม่

เพราะฉันเขียนแถบเลื่อนด้วย JavaScript ล้วน ([src/ui/Slider.tsx](#)) ไม่ได้ติดตั้งไลบรารีที่มีโค้ด native เพิ่ม EAS Update จึงส่งไปให้แอปที่ติดตั้งแล้วได้ทันที ถ้าใช้ไลบรารีแถบเลื่อนสำเร็จรูปที่มีโค้ด native ทุกคนต้องลง APK ใหม่ (ข้อความ commit ของงานนี้เขียนเหตุผลนี้ไว้เอง)

ขั้นที่ 9: ปรับตาม feedback (ประวัติจริงของ FR-6 ใน Git)

ซอฟต์แวร์ไม่เคย "เสร็จ" ในรอบเดียว ลองพิมพ์คำสั่งนี้ดูประวัติของหน้าจอ FR-6:

Terminal

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 git log --format="%h %ad %s" --date=short -- "src/app/(tabs)/runway.tsx"
2 git log --format="%h %ad %s" --date=short -- src/domain/runway.ts
```

ใน VS Code ก็ดูได้: เปิดไฟล์ แล้วดูแผง **Timeline** ที่มุมล่างซ้ายของ Explorer จะเห็นทุกครั้งที่มีไฟล์นี้ถูกแก้ไข ผลจริง (เรียงจากเก่าไปใหม่):

วันที่	commit	เปลี่ยนอะไรใน FR-6	บทเรียน
27 ก.ย.	452e71f	MVP: runway.ts ทั้งไฟล์ เทสต์ TC-12–18 และหน้าจอแรก	สูตรเสร็จพร้อมเทสต์ตั้งแต่วันแรก
27 ก.ย.	88313d6	บรรทัดผลลัพธ์ในการ์ด "คำนวณยังงี้" เป็นสีแดงเมื่อ critical หรือ below_floor (เดิมใช้สีหลักเสมอ)	เจอตอนลองใช้เว็บจริง ไม่ใช่จากเทสต์ (ชื่อ commit: Fix issues found while checking the live web app)
27 ก.ย.	43fd731	ป้ายสถานะอ่านง่ายขึ้นบนการ์ดเขียวเข้ม	สีที่สวยบนพื้นขาว อาจอ่านไม่ออกบนพื้นเข้ม
28 ก.ย.	426a79b	"วันนี้" มาจาก today ของแอป แทนการคิดเองในหน้าจอ	เปิดแอปค้างข้ามเที่ยงคืน ตัวเลขต้องเปลี่ยนเอง
28 ก.ย.	a681223	วงแหวนทอง (เดิมที่ 45 วัน) ตัวเลขนับขึ้น และหยุดเมื่อผู้ใช้เปิด "ลดการเคลื่อนไหว"	แอนิเมชันต้องเคารพผู้ใช้ที่เวียนหัวง่าย
29 ก.ย.	759143d	ลองปรับ: จากปุ่ม 4 ปุ่มเป็นแถบเลื่อน 0–50%	ดูรายละเอียดด้านล่าง

วันที่	commit	เปลี่ยนอะไรใน FR-6	บทเรียน
29 ก.ย.	f310504	กระปุกออม: เจ็บในกระปุกถูกหักออก และมีบรรทัดในการ์ด "คำนวณยังง"	ฟีเจอร์ใหม่ต่อเข้าสู่ตรเดิมผ่าน floorSatang ได้ ไม่ต้องแก้ runway.ts
29-30 ก.ย.	c983c5f, 98a4734	สืจากรั้ม (ไม่เขียนรหัสสืตายตัว) และของตกแต่งรั้มฮาโลวีนข้างหัวข้อ	ใช้ token ของ design system แล้วเปลี่ยนรั้มได้ทั้งแอป
1 ต.ค.	c7bd4b7	ปุ่มถามไค้ชส่งเวลาที่กด (at) ไปด้วย	ถามราคาเดิมซ้ำ ช่องพิมพ์ต้องเดิมให้ใหม่ (คอมเมนต์ใน coach.tsx บรรทัด 76)
3 ต.ค.	9abd1aa	เส้นเวลา: แถบที่ยังเหนือเส้นสำรวจจางลง (ยกเว้นแถบ "วันนี้")	ให้ตาไปหยุดที่แถบวันนี้และแถบสีแดงก่อน

★ **สังเกตบรรทัดที่สองของคำสั่ง**

src/domain/runway.ts มี commit เดียว คือวันแรก (452e71f) หน้าจอถูกแก้ **อีก 10 ครั้ง** หลังวันแรก แต่สุดท้ายไม่ถูกแก้ **อีกเลย** นี่คือนิผลของการแยกสัตรออกจากหน้าจอ: ทีมปรับหน้าตาได้เดิมทีโดยไม่เสี่ยงทำให้ตัวเลขผิด และถ้าวันไหนต้องแก้สัตร เกสต์ TC-12-18 จะฟ้องกันก็ถ้าทำของเดิมพัง

เจาะ 1 การปรับปรุง: จากปุ่ม 4 ปุ่ม เป็นแถบเลื่อน

เวอร์ชันแรกของ "ลองปรับ" เป็นปุ่มเลือก (Chip) 4 ปุ่ม ดูได้จากคำสั่ง `git show 759143d -- "src/app/(tabs)/runway.tsx"` หรือเปิดหน้า commit บน GitHub: [commit 759143d](#)

ก่อน: ปุ่ม 4 ปุ่ม (บรรทัดที่ถูกลบใน commit 759143d) จาก git show

```
1 <Row gap={space.sm} style={{ flexWrap: 'wrap' }}>
2   {[0, 10, 20, 30].map((p) => (
3     <Chip key={p} label={p === 0 ? 'เท่าเดิม' : ` ${p}%`} selected={reduce === p} onPress={() => setReduce(p)} />
4   ))}
5 </Row>
```

หลัง: แถบเลื่อน (บรรทัดที่เพิ่มใน commit เดียวกัน) จาก git show

```
1 <Slider
2   value={reduce}
3   onChange={setReduce}
4   min={0}
5   max={50}
6   step={5}
7   label="ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
8   format={(v) => ` ${v}%`}
9   ticks={[0, 10, 20, 30, 40, 50]}
10 />
```

เรื่อง	ปุ่ม 4 ปุ่ม	แถบเลื่อน
ค่าที่เลือกได้	4 ค่า (0, 10, 20, 30%)	11 ค่า (0 ถึง 50% ทีละ 5)
ความรู้สึกตอนใช้	กดแล้วตัวเลขกระโดด	ลากแล้วเห็นตัวเลขกับตัวไม้ค่อย ๆ เปลี่ยนตามนิ้ว เห็นความสัมพันธ์ "ยิ่งประหยัด ยิ่งอยู่ได้นาน"
สิ่งที่แสดง	ใช้วันละเท่าไร เงินพอกี่วัน และเพิ่มขึ้นกี่วัน	ทั้งหมดนั้น + ประหยัดวันละเท่าไร + ตัวไม้เล็กที่ออกใบตาม
การเข้าถึง	กดได้	ลาก และ กดลูกศร และโปรแกรมอ่านหน้าจออ่านค่าได้ สั้นเบา ๆ ทุกขั้นตอนมือถือ

เรื่อง	ปุ่ม 4 ปุ่ม	แถบเลื่อน
ต้องแก้สูตรใหม่	ไม่ต้อง runwayWithReduction รับเปอร์เซ็นต์ 0-100 ได้อยู่แล้วตั้งแต่วันแรก แก้แค่หน้าจอกับเพิ่ม Slider.tsx และเพิ่มเทสต์ E2E การลาก	

เช็กลิสต์ก่อนบอกว่า "เสร็จ" (Definition of Done)

ทีมที่ดีตกลงกันก่อนว่า "เสร็จ" แปลว่าอะไร ของ FR-6 ฉันใช้รายการนี้:

✓	ข้อ	ตรวจสอบด้วย
☐	นิยามในโค้ดกับใน TRACEABILITY.md ตรงกัน	อ่านเทียบกัน
☐	ทุกเงื่อนไขผ่าน (acceptance criteria) มีเทสต์ และมีเทสต์ที่ขอบเขต	ตาราง "เงื่อนไขผ่าน → โค้ด → เทสต์"
☐	unit test, typecheck และ lint ผ่านทั้งโปรเจกต์ (ไม่ใช่แค่ไฟล์ของตัวเอง)	3 คำสั่งในขั้นที่ 4
☐	หน้าจอกับใช้ได้บนเว็บและมือถือ ทั้งโหมดสว่างและมืด และตอนเปิด "ลดการเคลื่อนไหว"	ลองเอง + E2E บน Chromium และ WebKit
☐	ทุกสถานะมีข้อความกำกับ ไม่ใช่สืออย่างเดียว	ดูภาพหน้าจอกับ 5 สถานะ
☐	E2E ของฟีเจอร์ใหม่ผ่าน และของเดิมไม่พัง	npm run e2e:web
☐	README และ TRACEABILITY.md อัปเดตแล้ว	อ่านทวน
☐	ข้อความ commit บอกว่าทำอะไรและทำไม	git log
☐	GitHub Actions เชี่ยวทุกงาน	แก้ Actions บน GitHub

“ ถ้าอาจารย์ถามว่า "เธอพัฒนา FR-6 ด้วยกระบวนการแบบไหน"

"ฉันแบ่งงานเป็นขั้น เริ่มจากเขียนนิยามให้ทดสอบได้ เพราะ feedback บอกว่าคำว่าเงินต่ำเกินไป ทุกนิยามมีเทสต์ TC-12 ถึง 18 คุณไว้ สูตรเขียนเป็น pure function ในไฟล์ runway.ts แยกจากหน้าจอ แล้วต่อเข้ากับข้อมูลจริงที่จุดเดียวคือ useMoney ค่อยทำหน้าจอกับมี E2E ที่ลากแถบและกดปุ่มเหมือนคนจริง หลังจากนั้นปรับตาม feedback อีกหลายรอบ เช่น เปลี่ยนปุ่ม 4 ปุ่มเป็นแถบเลื่อน และเพิ่มกระปุกออม ซึ่งดูใน Git ได้ว่าหน้าจอกถูกแก้ 10 ครั้ง แต่ runway.ts ไม่ต้องแก้เลยตั้งแต่วันแรก"

การทดสอบ FR-6

ฉันพิสูจน์ยังงี้ว่าสูตรถูก และหน้าจot างงานจริง

157/157

unit test ทั้งโปรเจกต์ผ่าน (รันซ้ำ 9 ท.ค.
2569)

TC-12-18

unit test ของ FR-6 โดยตรง + TC-33, TC-
71

6 ข้อ

E2E ในเบราว์เซอร์ที่เกี่ยวข้องกับ FR-6

Unit test: ทดสอบสูตรที่จะเงื่อนไข

ในเทสต์ ฉันกำหนด "วันนี้" ตายตัวเป็น 27 ก.ย. 2569 (ตัวแปร NOW) เพื่อให้ผลลัพธ์เหมือนเดิมทุกครั้งที่รัน ไม่ว่าจะรันวันไหน

src/domain/__tests__/domain.test.ts

บรรทัด 165-219

```

165 describe('FR-6 money runway', () => {
166   it('TC-12 averages the last 7 days including days with no spending', () => {
167     const txs = [
168       tx({ day: '2026-09-10', amount: 1 }), // tracking started long ago
169       tx({ day: '2026-09-27', amount: 700 }),
170       tx({ day: '2026-09-21', amount: 700 }),
171       tx({ day: '2026-09-20', amount: 5000 }), // outside the window
172     ];
173     const avg = averageDailyExpense(txs, NOW);
174     expect(avg.daysCounted).toBe(7);
175     expect(avg.averageSatang).toBe(20_000);
176   });
177
178   it('TC-13 divides by tracked days for new users', () => {
179     const avg = averageDailyExpense([tx({ day: '2026-09-25', amount: 300 })], NOW);
180     expect(avg.daysCounted).toBe(3);
181     expect(avg.averageSatang).toBe(10_000);
182   });
183
184   it('TC-14 computes days and depletion date', () => {
185     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000, now: NOW });
186     expect(r.days).toBe(15);
187     expect(r.depletionDay).toBe('2026-10-12');
188     expect(r.status).toBe('healthy');
189   });
190
191   it('TC-15 handles zero average without dividing by zero', () => {
192     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now: NOW });
193     expect(r.status).toBe('no_spending');
194     expect(r.days).toBeNull();
195     expect(r.depletionDay).toBeNull();
196   });
197
198   it('TC-16 flags balance at or below the low line', () => {
199     const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000, now: NOW });
200     expect(r.status).toBe('below_floor');
201     expect(r.days).toBe(0);
202   });
203
204   it('TC-17 status thresholds', () => {
205     const at = (days: number) =>
206       computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang: 10_000, now:
NOW }).status;
207     expect(at(6)).toBe('critical');
208     expect(at(7)).toBe('watch');

```

```

209     expect(at(14)).toBe('healthy');
210   });
211
212   it('TC-18 before-you-spend check and safe daily spend', () => {
213     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000, now: NOW });
214     expect(runwayAfterPurchase(r, 100_000, NOW).days).toBe(10);
215     expect(endOfMonthDay(NOW)).toBe('2026-09-30');
216     // ฿3,000 above the floor over 4 days (27-30 Sep) = ฿750/day
217     expect(safeDailySpend({ balanceSatang: 350_000, floorSatang: 50_000, targetDay: '2026-09-30', now: NOW
218     })).toBe(75_000);
219   });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/___tests___/domain.test.ts:165

เทสต์	ใส่ข้อมูล	ต้องได้
TC-12	ใช้แอปมาตั้งแต่ 10 ก.ย. · รายจ่าย ฿700 (27 ก.ย.), ฿700 (21 ก.ย.) และ ฿5,000 (20 ก.ย. ซึ่งอยู่นอกหน้าต่าง)	หาร 7 วัน · ค่าเฉลี่ย ฿200 (= 1,400 ÷ 7) ไม่นับ ฿5,000 และนับวันที่ไม่ได้ใช้เงิน
TC-13	ผู้ใช้ใหม่ รายการแรก 25 ก.ย. ฿300	หาร 3 วัน (25–27 ก.ย.) · ค่าเฉลี่ย ฿100
TC-14	ยอด ฿3,500 · เล้น ฿500 · เฉลี่ย ฿200	15 วัน · ถึง 12 ต.ค. 2569 · healthy
TC-15	ค่าเฉลี่ย ฿0	no_spending · วัน = null · วันที่ = null (ไม่พังเพราะหารศูนย์)
TC-16	ยอด ฿500 · เล้น ฿500 (เท่ากับพอดี)	below_floor · 0 วัน
TC-17	ตั้งให้ได้ 6, 7 และ 14 วันพอดี	critical, watch, healthy ตามลำดับ (ทดสอบที่ขอบของเกณฑ์)
TC-18	จาก TC-14 ชื่อของ ฿1,000 · หาวันสิ้นเดือน · ใช้ได้วันละถึง 30 ก.ย.	10 วัน · 30 ก.ย. · ฿750/วัน (= 3,000 ÷ 4 วัน)
TC-71	กระปุก ฿500 จาก ฿2,500 กำหนด 8 ต.ค. (วันนี้ 29 ก.ย.)	ความคืบหน้า 20%, เก็บวันละ ฿200, รวมเงินในกระปุก 2 ใบถูกต้อง, ถอนไม่ติดลบ
TC-33	สร้างข้อมูลตัวอย่างใน 8 วันที่ต่างกัน (ก.ย. 2569 ถึง มิ.ย. 2570)	ทุกครั้งยอดเกิน ฿2,000 และ Runway อย่างน้อย 10 วัน (คนลองแอปครั้งแรกจะไม่เจอหน้าจอตกลง)

★ ทำไมต้องทดสอบที่ขอบ

บ๊ิกขอบซ่อนอยู่ตรงเกณฑ์ เช่นผลเลขเขียน $\leq$ แทน $<$ แล้ว 7 วันจะกลายเป็น critical แทน watch TC-17 จึงลอง 6 กับ 7 และ TC-16 ลองยอด "เท่ากับ" เล้นสำรวจพอดี ไม่ได้ลองแค่ค่ากลาง ๆ

E2E: ทดสอบเหมือนคนใช้จริงในเบราร์เซอร์

สคริปต์ [e2e/web-auth.mjs](#) เปิดเว็บเวอร์ชันจริงด้วยเบราร์เซอร์อัตโนมัติ (Playwright) จำลองเซิร์ฟเวอร์ แล้วกดตามขั้นตอนเหมือนคน (อ่านโค้ดที่ละบรรทัดในบท A12) ข้อที่เกี่ยวกับ FR-6:

1. ลากแถบ "ลองปรับ" ไปที่ 30% แล้วต้องเห็นคำว่า "เพิ่มขึ้น"
2. กดปุ่มลูกศรขวามันก็ขึ้น 5% แถบต้องขยับเพิ่ม 5%
3. ใส่ราคา ฿2,000 ใน "เช็ก่อนจ่าย" ต้องเห็นทั้ง "ตอนนี้" และ "ถ้าซื้อ ฿2,000"
4. กด "ถามโค้ชเรื่องนี้" แล้วช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รอยอยู่

5. สร้างกระปุกออมแล้วหยอดเงิน หน้าเงินพอถึงต้องมีบรรทัด "หักเงินที่กันไว้ในกระปุกออม"

6. ข้ามเที่ยงคืน: ตัวเลข "วันนี้" เริ่มใหม่เอง (ทดสอบรวมกับ FR-2/FR-4)

คำสั่งรัน unit test: `npm test · E2E: npm run e2e:web`

เช็กลิสต์ทดสอบเองบนมือถือ (ทำได้ใน 10 นาที)

#	ทำ	ต้องเห็น
1	เปิดแอป กด "ลองใช้ด้วยข้อมูลตัวอย่าง"	จำนวนวันบนหน้าหลักเท่ากับในแท็บเงินพอถึง
2	เปิดการ์ด "คำนวณยังไง" แล้วกดเครื่องคิดเลขตาม	ใช้ได้จริง ÷ ค่าเฉลี่ย แล้วปิดลง ได้เท่ากับแอปบอก
3	จดรายจ่ายใหม่ ฿700 วันนี้	ค่าเฉลี่ยเพิ่มประมาณ ฿100 (700 ÷ 7) ยอดลด ฿700 จำนวนวันลดลง
4	ตั้งค่า → เงินสำรอง ใส่ค่าสูงกว่ายอดคงเหลือ	"ยอดเงินแตะเส้นสำรองแล้ว" ป้าย "ต่ำกว่าเงินสำรอง" แล้วตั้งกลับเป็น 500
5	ลากแถบลองปรับจาก 0% ไป 50%	จำนวนวันเพิ่มขึ้นทุกชั้น ต้นไม้เล็กลงใบเพิ่ม
6	เช็ก่อนจ่าย ใส่ราคามากกว่า "ใช้ได้จริง"	"แตะเส้นสำรองแล้ว" สีแดง
7	สร้างกระปุกออม หยอด ฿500	มีบรรทัดหักเงินในกระปุก จำนวนวันลดลง
8	บัญชีจริง: สแกนสลิปที่ AI อ่านไม่ชัด (เป็นแบบร่าง)	Runway ยังไม่เปลี่ยนจนกว่าจะกดยืนยัน

FR-6 เชื่อมกับ FR อื่นยังไง

FR-6 อยู่ "ปลายน้ำ" ใช้ข้อมูลจาก FR อื่นแทบทั้งหมด และส่งผลต่อ FR-5

FR	ความเกี่ยวข้องกับ Money Runway
FR-1 จัดการรายการ	ทุกรายการที่เพิ่ม แก้ ลบ เปลี่ยนทั้งยอดคงเหลือและค่าเฉลี่ย Runway จึงเปลี่ยนกันที่ (แผนทดสอบ MT-02: จดรายจ่าย ฿85 แล้ว Runway ต้องเปลี่ยน)
FR-2 ภาพรวม	ยอดคงเหลือ (computeBalance) เป็นตัวตั้งของสูตร และการด Runway อยู่บนหน้าหลักร่วมกับภาพรวม กราฟ "รายจ่าย 7 วันล่าสุด" มีเส้นประเป็นค่าเฉลี่ยที่ใช้คำนวณ Runway
FR-3 พุดจด	พุด "ข้าว 50 บาท" แล้วกดบันทึก เป็นรายจ่ายที่ยืนยันแล้ว เข้าค่าเฉลี่ยทันที
FR-4 สแกนสลิปอัตโนมัติ	สลิปที่อ่านชัดเข้ายอดเลย สลิปที่อ่านไม่ชัดรอยืนยันและยังไม่บันใน Runway (MT-11: บันทึกสลิปใหม่แล้ว Runway เปลี่ยนภายในไม่กี่วินาที)
FR-5 โค้ชน้องกล้า	โค้ชได้ lowLine, averageDailySpend7d และ runway ไปตอบทุกคำถาม และได้ purchaseCheck เมื่อถามเรื่องชื่อของ การ์ด "สิ่งที่คุณรู้ตอนนี้" เอาเรื่อง Runway ขึ้นก่อนเสมอเมื่อสถานะ watch, critical หรือ below_floor (MT-08: ถาม "ชื่อของ 500 บาทวันนี้ได้ไหม" ต้องอ้าง Runway ก่อน/หลัง)
กระปุกออม	เงินในกระปุกถูกหักออกจากเงินที่ใช้ได้
เหรียญ "ต้นไม้งาม"	ได้เหรียญเมื่อสถานะ Runway เป็น "สบาย ๆ" (healthy)

“สรุปเป็นประโยคเดียว

FR-1, FR-3 และ FR-4 คือ "ทางเข้าข้อมูล" FR-6 คือ "สมอง" ที่แปลงข้อมูลเป็นคำตอบว่าเงินพอถึงวันไหน และ FR-5 คือ "ปาก" ที่อธิบายคำตอบนั้นให้ผู้ฟังแบบเป็นมิตร

ทำไมฉันออกแบบแบบนี้

ทุกการตัดสินใจมีทางเลือกอื่น นี่คือเหตุผลที่ฉันเลือก

ตัดสินใจ	ทางเลือกอื่น	เหตุผลที่เลือกแบบนี้
ค่าเฉลี่ย 7 วัน	3 วัน, 30 วัน	ครบรอบสัปดาห์ (วันธรรมดา + วันหยุด) ตามนิสัยล่าสุดกัน และตรง SRS · 3 วันแคว้งเกินไป 30 วันซ้ำเกินไป
นับวันที่ไม่ใช้เงินเป็น 0	หารเฉพาะวันที่มีรายการ	ได้ความเร็วจริงตามปฏิทิน ไม่สูงเกินจริง
ผู้ใช้ใหม่หารด้วยวันที่ใช้จริง	หาร 7 เสมอ	หาร 7 ทำให้ค่าเฉลี่ยต่ำเกินจริงและ Runway ยาวเกินจริงในสัปดาห์แรก ซึ่งเป็นช่วงที่ผู้ใช้ตัดสินใจว่า จะเชื่อแอปไหม
ปัดลง	ปัดปกติ, แสดงทศนิยม	ปลอดภัยกว่า และ "15 วัน" อ่านง่ายกว่า "15.83 วัน"
นับถึงเส้นสำรอง ไม่ใช่ 0	นับถึงศูนย์	ทำให้นิยาม "เงินต่ำ" ทดสอบได้ และเตือนก่อนถึงเงินก้อนสุดท้ายที่จำเป็น
เงินสำรอง ฿500 และให้ผู้ใช้ปรับเอง	บังคับค่าเดียว	฿500 พอเป็นค่ารถหรือค่าข้าวฉุกเฉินสำหรับนักศึกษา แต่แต่ละคนต่างกัน จึงให้ปรับได้ตั้งแต่ ฿0
เกณฑ์ 7 / 14 วัน	เกณฑ์เป็นเปอร์เซ็นต์ของยอด	คิดเป็นสัปดาห์เข้าใจง่าย: ไม่ถึงสัปดาห์ = รับปรับ, ไม่ถึงสองสัปดาห์ (ราวครึ่งเดือน ซึ่งเป็นรอบเงินเข้าของหลายคน) = เริ่มระวัง เกณฑ์อยู่ในนิยาม จึงปรับภายหลังได้โดยแก้จุดเดียวและแก้เกสตาม
ตัดที่ 365 วัน	แสดงเลขจริงทุกค่า	"4,000 วัน" ดูไม่น่าเชื่อและไม่ค่อยมีประโยชน์ นิสัยเปลี่ยนก่อนแน่
ไม่นับแบบร่าง	นับสลิปทุกใบทันที	ถ้า AI อ่านยอดผิด Runway จะผิดตาม ให้คนยืนยันก่อนจึงแม่นยำกว่า
หักเงินในกระเป๋าออม	ไม่หัก	เงินที่ตั้งใจเก็บไม่ใช้เงินใช้ ถ้าไม่หัก Runway จะหลอกว่าสบาย
เก็บเงินเป็นสตางค์	เก็บเป็นบาททศนิยม	จำนวนเต็มบวกกลมมน่ายำ ไม่มีปัญหา 0.1 + 0.2
คิดสดในเครื่อง	คิดบนเซิร์ฟเวอร์แล้วเก็บผล	ตัวเลขไม่ค้าง ไม่ต้องรอเน็ต ไม่ต้องตั้งงานคำนวณตอนเที่ยงคืนบนเซิร์ฟเวอร์
แยกสูตรเป็น pure functions	เขียนคำนวณในหน้าจอ	ทดสอบได้ง่ายโดยไม่ต้องเปิดแอป ใช้สูตรเดียวกันได้ทุกหน้า (หน้าหลัก เก็บเงินพอถึง โฉ้ย)
ให้โค้ดคิดเลข AI แค่อธิบาย	ให้ AI คิด Runway เอง	ตัวเลขถูกต้องตรวจสอบได้ AI ทำหน้าที่ที่ถนัดคือการอธิบายเป็นภาษาคน
ต้นไม้/วงแหวนเติมที่ 45 วัน	เติมที่ 30 หรือ 365	45 วัน (เดือนครึ่ง) ถือว่าสบายมาก ถ้าเติมที่ 365 ต้นไม้ของคนส่วนใหญ่จะโผล่ตลอด
แถบเลื่อนเขียนเอง	ใช้ไลบรารีแถบเลื่อนแบบ native	ไม่ต้องลง APK ใหม่ (ส่งทาง OTA ได้) ควบคุม accessibility และคีย์บอร์ดบนเว็บได้เอง

ข้อจำกัด และสิ่งที่จะทำต่อ

การรู้วาระบบ "ยังทำอะไรไม่ได้" สำคัญพอ ๆ กับรู้ว่าทำอะไรได้

ข้อจำกัด	ผลที่เกิด และที่จับทำไว้แล้ว
ยังไม่รวมรายรับในอนาคต	ถ้าพรั้งนี้ค่าขนมเข้า Runway วันนี้จะดูสั้นกว่าความจริง ฉันทเขียนบอกไว้ใต้เส้นเวลาว่า "ยังไม่รวมรายรับที่จะเข้ามา" และอยู่ในหัวข้อข้อจำกัดของ README ด้วย
รายจ่ายก้อนใหญ่ครั้งเดียว ถึงค่าเฉลี่ย	ตัวอย่าง (ตรวจด้วยโค้ดจริง): ปกติใช้วันละ ฿100 ทุกวัน แต่มีวันหนึ่งซื้อเคสมือถือ ฿3,500 เพิ่ม ค่าเฉลี่ย 7 วันจะกลายเป็น $(100 \times 7 + 3,500) \div 7 = ฿600$ แทนที่จะเป็น ฿100 Runway จะสั้นลงมากไป 7 วัน แล้วกลับเป็นปกติเมื่อรายการนั้นหลุดหน้าต่าง
สมมติว่าใช้เท่ากันทุกวัน	ชีวิตจริงไม่เป็นเส้นตรง (บางวันจ่ายค่าหอ) ผลจึงเป็น "ประมาณ" แอปใช้คำว่า "ถึงประมาณ" และ "ประมาณการ"
ขึ้นกับการจดให้ครบ	ถ้าจ่ายเงินสดแล้วไม่จด ค่าเฉลี่ยจะต่ำกว่าจริง Runway จะยาวเกินจริง ทีมจึงทำ FR-3 พุดจด และ FR-4 สแกนสลิปอัตโนมัติให้จดง่ายที่สุด
ยอดในแอปอาจไม่ตรงธนาคาร	ถ้ามีรายการตกหล่น ผู้ใช้แค่ "ยอดเงินตอนเริ่มใช้ MindPay" ในตั้งค่าเพื่อปรับให้ตรงได้
บิลประจำยังไม่ถูกหักล่วงหน้า	ค่าหอที่จะครบกำหนดสัปดาห์หน้ายังไม่ถูกหักไว้ แม้แอปจะรู้จักรายการประจำแล้ว (FR-1)

ไถ่เตียพัฒนาต่อ (ยังไม่ได้ทำ)

- 1. รวมรายรับที่คาดว่าจะเข้า เช่น ค่าขนมทุกวันที่ 1 ให้เส้นเวลา มี "ขึ้นบันได" ขึ้นตอนเงินเข้า
- 2. กันบิลประจำล่วงหน้า หักค่าหอ ค่าเน็ตที่จะถึงก่อนวันที่เงินพอถึงออกจากเงินที่ใช้ได้
- 3. ลดผลของรายจ่ายผิดปกติ เช่น ใช้ค่ามัธยฐาน หรือให้ผู้ใช้ตี๊ก "รายการนี้ไม่ใช่ปกติ"
- 4. แจ้งเตือนเมื่อสถานะเปลี่ยน เช่น จากสบาย ๆ เป็นเริ่มต้องระวัง
- 5. ให้เลือกช่วงค่าเฉลี่ยเอง 7 / 14 / 30 วัน สำหรับคนที่รายจ่ายแกว่งมาก

★ ถ้าโดนถามเรื่องข้อจำกัด

ตอบตรง ๆ ว่ารู้และตั้งใจเลือก: MVP เน้นสูตรที่เข้าใจง่าย ตรวจสอบได้ และทดสอบได้ก่อน เพราะถ้าผู้ใช้ไม่เข้าใจที่มาของตัวเลขก็จะไม่เชื่อ ส่วนความแม่นยำขั้นสูง (รายรับในอนาคต บิลประจำ) ต่อยอดได้จากโครงสร้างเดิม เพราะสูตรแยกเป็น pure functions แล้ว

คำถามที่อาจารย์อาจถาม พร้อมคำตอบ

ฝึกตอบให้คล่อง แต่ละข้อมีคำตอบสั้นที่พูดได้ใน 20–30 วินาที

Q1	ทำไมใช้ค่าเฉลี่ย 7 วัน ไม่ใช่ 30 วัน? 7 วันครบหนึ่งรอบสัปดาห์ มีทั้งวันธรรมดาและวันหยุด ตามนิสัยล่าสุดได้เร็ว ถ้าผู้ใช้เริ่มประหยัดจะเห็นผลภายในไม่กี่วัน ส่วน 30 วันช้าเกินไป และ SRS กำหนดไว้ที่ 7 วัน
Q2	ถ้าผู้ใช้เพิ่งเริ่มใช้แอป 2 วัน คิดยังไง? หารด้วย 2 ไม่ใช่ 7 เพราะนับจากวันของรายการแรกที่ยืนยันแล้ว ถ้าหาร 7 ค่าเฉลี่ยจะต่ำเกินจริงและบอกว่าเงินพอนานเกินจริง มีเทสต์ TC-13
Q3	ถ้า 7 วันไม่มีรายจ่ายเลย จะหารด้วยศูนย์ไหม? ไม่ ไล่เช็คก่อนหาร ถ้าค่าเฉลี่ยเป็น 0 จะเป็นสถานะ no_spending ไม่แสดงวันที่ แสดงว่า "ยังคำนวณไม่ได้" พร้อมเหตุผล มีเทสต์ TC-15
Q4	ทำไมปิดลง? เพื่อความปลอดภัย 20.69 วันถ้าปิดขึ้นเป็น 21 วันที่ 21 ยอดจะต่ำกว่าเส้นสำรองแล้ว บอกน้อยไว้ผู้ใช้ไม่เดือดร้อน บอกเกินอาจเงินขาด
Q5	"เงินต่ำ" นิยามว่าอะไร? ยอดคงเหลือ $\leq$ เส้นเงินสำรองที่ใช้ตั้ง ค่าเริ่มต้น 500 (ถ้ามีกระปุกออม เส้นนี้รวมเงินในกระปุกด้วย) เท่ากับพอดีนับว่าต่ำ ทดสอบใน TC-16
Q6	ทำไมไม่นับถึงศูนย์บาท? เงินศูนย์บาทคือสายเกินไปแล้ว เส้นสำรองทำให้เตือนก่อนถึงเงินก้อนสุดท้ายที่จำเป็น เช่น ค่ารถกลับบ้าน แต่ถ้าผู้ใช้อยากนับถึงศูนย์ก็ตั้งเป็น 0 ได้
Q7	สลิปที่ยังไม่ยืนยันนับไหม? ไม่นับ นับเฉพาะรายการที่ยืนยันแล้ว ถ้า AI อ่านยอดผิดแล้วนับทันที Runway จะผิด หน้าจอเขียนบอกไว้ใต้การ์ดคำนวณ
Q8	รายรับที่จะเข้าเดือนหน้านับไหม? ยังไม่นับ เป็นข้อจำกัดที่ระบุไว้ใน README และบนหน้าจอ ("ยังไม่รวมรายรับที่จะเข้ามา") อยู่ในแผนพัฒนาต่อ
Q9	ถ้าวันหนึ่งชื่อของแพคเกจผิด ตัวเลขจะเพี้ยนไหม? เพี้ยนชั่วคราว 7 วัน เพราะค่าเฉลี่ยสูงขึ้น แล้วจะกลับเป็นปกติเมื่อรายการนั้นหลุดหน้าต่าง เป็นข้อจำกัดที่รู้ และมีโอเคเสียแก่ เช่น ใช้ค่ามัธยฐาน
Q10	เกณฑ์ 7 กับ 14 วันมาจากไหน? ก็กำหนดในนิยามที่ตอบ feedback ของ SRS คิดเป็นสัปดาห์ให้เข้าใจง่าย: ไม่ถึง 1 สัปดาห์ต้องรีบปรับ ไม่ถึง 2 สัปดาห์ (ราวครึ่งเดือน) เริ่มระวัง ปรับภายหลังได้โดยแค่จุดเดียว และมีเทสต์ที่ขอบ (TC-17)
Q11	ทำไมเก็บเงินเป็นสตางค์? ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ $0.1 + 0.2$ ได้ 0.30000000000000004 เก็บเป็นจำนวนเต็มสตางค์จึงแม่นยำ แปลงเป็นบาทตอนแสดงผลเท่านั้น
Q12	ตัวเลขอัปเดตเมื่อไหร่? ทันทีที่ข้อมูลเปลี่ยน (จด แก่ ลบ ยืนยันสลิป แก้ตั้งค่า หยอดกระปุก) และทุกเที่ยงคืนเวลาไทย เพราะคิดสดในเครื่องด้วย useMoney ไม่ได้เก็บผลไว้
Q13	AI โค้ชคำนวณ Runway เองหรือเปล่า? ไม่ โค้ดในแอปคำนวณแล้วส่งตัวเลขให้ AI อธิบาย เพราะโค้ดผ่านการทดสอบแล้ว ส่วน AI อาจคิดเลขพลาด และข้อมูลที่ส่งเป็นตัวเลขสรุป ไม่ส่งรูปสลิปหรือข้อความที่โอนให้
Q14	พิสูจน์ยังไงว่าถูก? มี unit test 7 ข้อ (TC-12 ถึง TC-18) ที่ล็อกวันที่ไว้ ทดสอบทั้งกรณีปกติ กรณีพิเศษ และขอบของเกณฑ์ มี E2E ที่กดหน้าจอจริงในเบราว์เซอร์ และการ์ด "คำนวณยังไง" ให้ผู้ใช้กดเครื่องคิดเลขตรวจสอบได้
Q15	ข้อมูลเงินของผู้ใช้ปลอดภัยไหม? ฐานข้อมูลเปิด RLS ทุกตาราง แต่ละบัญชีอ่านได้เฉพาะข้อมูลของตัวเอง (MT-10) และการคำนวณ Runway ทำในเครื่องผู้ใช้

Q16 กระเป๋าออมเกี่ยวกับ FR-6 ยังไง?

เงินในกระปุกยังอยู่ในบัญชีจริง แต่ผู้ใช้ตั้งใจไม่ใช้ จึงถูกรวมเข้ากับเส้นสำรองใน useMoney (floor + reserved) แล้วหักออกจากเงินที่ใช้ได้ ไม่
จิ้น Runway จะดูดีเกินจริง

Q17 ถ้าแก้โค้ดให้ใช้ 14 วันต้องแก้ตรงไหน?

แก้ค่าคงที่ RUNWAY_WINDOW_DAYS ใน src/domain/runway.ts บรรทัด 24 จุดเดียว ข้อความบนหน้าจอใช้ค่าคงที่นี้อยู่แล้ว แล้วแก้
เทสต์ TC-12 ให้ตรงนิยามใหม่ และอัปเดต SRS/Traceability

แบบฝึกหัดเชิงความเข้าใจ

ลองคิดเองก่อน แล้วค่อยดูเฉลยหน้าถัดไป ทุกข้อ "วันนี้" = 10 ต.ค. 2569 และไม่มีกระปุกลูกอม ยกเว้นที่บอก

1. ยอดคงเหลือ ฿2,000 เงินสำรอง ฿500 ใช้แอปมานานแล้ว รายจ่าย 7 วันล่าสุด (4–10 ต.ค.) คือ 100, 0, 50, 200, 0, 150, 200 บาท เงินพอใช้อีกกี่วัน ถึงวันไหน สถานะอะไร
2. ต่อจากข้อ 1 ถ้าวันนี้ซื้อรองเท้า ฿400 จะเหลือกี่วัน สถานะอะไร
3. ต่อจากข้อ 1 ถ้าเลื่อนแถบ "ลองปรับ" ไปที่ 20% จะใช้วันละเท่าไร และเงินพอใช้อีกกี่วัน
4. ต่อจากข้อ 1 การ์ดบนหน้าหลัก "ถ้าอยากให้เงินพอถึงสิ้นเดือน วันนี้ใช้ได้ประมาณ" จะขึ้นวันละเท่าไร
5. ผู้ใช้ใหม่เริ่มใช้แอปเมื่อวาน (9 ต.ค.) จดรายจ่ายไว้ 9 ต.ค. ฿180 และวันนี้ ฿60 ยอดคงเหลือ ฿1,000 เงินสำรอง ฿500 ค่าเฉลี่ยเท่าไร เงินพอใช้อีกกี่วัน สถานะอะไร
6. ยอดคงเหลือ ฿600 เงินสำรอง ฿500 แต่หยุดกระปุกลูกอมไว้ ฿200 สถานะอะไร
7. ใช้แอปมาหลายเดือน แต่ 7 วันล่าสุดไม่มีรายจ่ายเลย ยอด ฿600 เงินสำรอง ฿500 สถานะอะไร หน้าจอแสดงอะไร
8. (คิดลึก) ยอด ฿600 เงินสำรอง ฿500 ใช้วันละ ฿145 สถานะเป็น below_floor หรือ critical เพราะอะไร

เฉลย (ตรวจด้วยการรันฟังก์ชันจริงใน src/domain/runway.ts แล้ว)

ข้อ	วิธีคิด	คำตอบ
1	รวม $700 \div 7 = \text{฿}100/\text{วัน}$ · ใช้ได้ $2,000 - 500 = 1,500$ · $1,500 \div 100 = 15$ · 10 ต.ค. + 15	15 วัน ถึง 25 ต.ค. · healthy ($15 \geq 14$)
2	ใช้ได้ $1,500 - 400 = 1,100$ · $\div 100 = 11$	11 วัน ถึง 21 ต.ค. · watch
3	$100 \times 0.8 = \text{฿}80$ · $1,500 \div 80 = 18.75 \rightarrow$ ปัดลง 18	ใช้วันละ $\text{฿}80$ · 18 วัน (เพิ่ม 3 วัน) ถึง 28 ต.ค.
4	10–31 ต.ค. นับวันนี้ด้วย = 22 วัน · $1,500 \div 22 = 68.18 \rightarrow$ ปัดลงเป็นบาท	$\text{฿}68$ ต่อวัน
5	ใช้มา 2 วัน (9–10 ต.ค.) จังหาร 2: $(180 + 60) \div 2 = \text{฿}120$ · ใช้ได้ $500 \div 120 = 4.17 \rightarrow 4$	เฉลี่ย $\text{฿}120$ · 4 วัน ถึง 14 ต.ค. · critical
6	เส้นสำรองรวมกระเป๋า = $500 + 200 = 700$ · ใช้ได้ $600 - 700 = -100 \leq 0$	below_floor (ต่ำกว่าเงินสำรอง) ทั้งที่ยอดยังเกิน $\text{฿}500$
7	ค่าเฉลี่ย 0 หारไม่ได้ (ยอดเหนือเส้น จึงไม่ใช่ below_floor)	no_spending · "ยังคำนวณไม่ได้" ไม่มีวันที่
8	ใช้ได้ $100 > 0$ จึงยังไม่ใช่ below_floor · $100 \div 145 = 0.69 \rightarrow 0$ วัน < 7	critical 0 วัน วันที่ = วันนี้ (ยังเหนือเส้นแต่ไม่พอหนึ่งวันเต็ม)

สคริปต์นำเสนอ 1 นาที และคำศัพท์

อ่านออกเสียงประมาณ 60–70 วินาที เปิดเก็บ "เงินพอถึง" ประกอบไปด้วย

“ พูดยานี้ได้เลย ”

"FR-6 ที่ฉันรับผิดชอบคือ Money Runway ตอบคำถามว่า เงินจะพอใช้ถึงวันไหน เพราะยอดเงินอย่างเดียวไม่ได้บอกว่าจะอยู่รอดไหม ขึ้นอยู่กับว่าใช้เร็วแค่ไหน"

สูตรคือ เอายอดคงเหลือ ลบเงินสำรองที่ผู้ใช้อยู่แล้ว ค่าเริ่มต้น 500 บาท และลบเงินในกระปุกออม ได้เงินที่ใช้ได้จริง แล้วหารด้วยค่าเฉลี่ยรายจ่าย 7 วันล่าสุด ปิดลงเป็นจำนวนวัน แล้วบวกกับวันนี้เป็นวันที่ เช่น มีเงินใช้ได้ 3,000 ใช้วันละ 145 ได้ 20 วัน คือพอถึง 28 ตุลาคม

ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย ผู้ใช้ใหม่หารด้วยวันที่ใช้จริง และนับเฉพาะรายการที่ยืนยันแล้ว จากนั้นแบ่งเป็นสภาวะ ต่ำกว่า 7 วันใกล้เส้นสำรอง ต่ำกว่า 14 วันเริ่มต้องระวัง และกรณีพิเศษอย่างไม่มีรายจ่ายก็ไม่หารด้วยศูนย์

ในหน้าจอก็มีการคำนวณยังงี้ให้ตรวจสอบได้ เส้นเวลา 30 วัน แถบลองปรับว่าถ้าใช้น้อยลงจะพอเพิ่มกี่วัน และเช็กรายจ่ายที่ส่งต่อให้โค้ช AI อธิบายได้ ทั้งหมดมี unit test 7 ข้อและ E2E ในเบราวเซอร์ ข้อจำกัดคือยังไม่รวมรายรับในอนาคต ซึ่งเป็นแผนต่อไปค่ะ/ครับ"

คำศัพท์ของ FR-6

คำ	ความหมายแบบง่าย
Runway	จำนวนวันที่เงินพอใช้ก่อนแตะเส้นสำรอง
Floor / Low line / เงินสำรอง	เงินขั้นต่ำที่ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
Spendable / ใช้ได้จริง	ยอดคงเหลือ – เงินสำรอง – เงินในกระปุกออม
Depletion date	วันที่คาดว่าเงินจะลงมาถึงเส้นสำรอง ("เงินพอถึง")
Moving average	ค่าเฉลี่ยที่หน้าตาต่างเลื่อนไปทุกวัน (7 วันล่าสุดเสมอ)
Confirmed / Draft	รายการที่ยืนยันแล้ว / แบบร่างที่ยังรอตรวจ (ไม่นับ)
Reserved	เงินที่กั้นไว้ในกระปุกออมทุกใบรวมกัน
Safe daily spend	ใช้ได้วันละเท่าไรให้เงินพอถึงสิ้นเดือน
Satang (สตางค์)	หน่วยที่ใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท
Pure function	ฟังก์ชันที่ใส่ข้อมูลเดิมได้ผลเดิมเสมอ ไม่ยุ่งกับสิ่งอื่น ทดสอบง่าย
Bangkok day key	วันที่ตามเวลาไทยในรูปแบบ ปี-เดือน-วัน เช่น 2026-10-08

นำเสนอ FR-6 ด้วยสไลด์ของฉัน

คู่มือใช้สไลด์ 25 หน้า: แต่ละหน้าพูดอะไร ใช้เวลาเท่าไร ชีตตรงไหน เดโมสดยังไง และถ้าโดนถามจะตอบว่าอะไร

ฉันทำสไลด์นำเสนอ FR-6 ไว้ 25 หน้า ขนาดจอกว้าง 16:9 สี่เดียวกับแอป (เขี้ยวปากเข้ากับทอง) ตัวหนังสือใหญ่อ่านได้จากหลังห้อง ทุกหน้ามี **โน้ตผู้พูด** (speaker notes) ภาษาไทยที่เขียนเป็นประโยคพูดไว้แล้ว บทนี้สรุปทุกอย่างที่ต้องรู้ก่อนขึ้นพูด

25

สไลด์ แบ่ง 5 ส่วน

10 นาที

แผนเวลาหลัก (มีแบบ 5 นาทีด้วย)

2 ชุด

ตัวเลข: คัดมือ ฿3,500 และหน้าจอจริง
฿11,359

ก่อนใช้สไลด์: 4 เรื่องที่ต้องทำ

- 1 เปิดลิงก์สไลด์ claude.ai/artifact/NBXzLkY1ljtKXwQXUZZT7k ด้วยบัญชีของเจ้าของ ลิงก์นี้ **เป็นส่วนตัว** จนกว่าเจ้าของจะกดแชร์ ถ้าเพื่อนเปิดไม่ได้ แปลว่ายังไม่ได้แชร์
- 2 ใส่ชื่อจริงในหน้าปก หน้าแรกมีช่อง [ชื่อ-นามสกุล ผู้นำเสนอ • รหัสนักศึกษา] รอให้ใส่ชื่อของคนที่น่าเสนอ
- 3 เก็บไฟล์สำรองไว้ในเครื่อง ดาวโหลดสไลด์เป็น PDF หรือ PowerPoint ไว้ในแฟลชไดรฟ์ เผื่อห้องเรียนไม่มีอินเทอร์เน็ต
- 4 อ่านโน้ตผู้พูดทุกหน้า 1 รอบ ไม่ต้องท่องจำ แต่ให้รู้ว่าแต่ละหน้า "ต้องพูดเรื่องอะไรให้ได้" (ตารางด้านล่าง)

แผนเวลา 10 นาที

ส่วน	สไลด์	เวลา	เวลาสะสม	ผู้ฟังต้องได้อะไรกลับไป
1 · เปิดเรื่อง	1–4	1:30	1:30	ยอดเงินอย่างเดียวยังไม่พอ FR-6 แปลงเงินเป็นเวลา
2 · สูตร	5–11	3:00	4:30	คิด Runway เองได้: ใช้ได้จริง ÷ ค่าเฉลี่ย 7 วัน แล้วปิดลง
3 · หน้าจอ	12–18	2:30	7:00	ผู้ใช้เห็นอะไร ตรวจสอบเลขเองได้ และลองคิดส่วนหน้าได้
4 · เบื้องหลัง	19–21	1:30	8:30	คิดสดในเครื่อง โค้ดหัวใจ 13 บรรทัด และกรณีพิเศษ
5 · คุณภาพ	22–25	1:30	10:00	พิสูจน์ด้วยเทสต์ มีเหตุผลทุกการตัดสินใจ รู้ข้อจำกัด

★ ถ้าได้เวลาแค่ 5 นาที

ใช้ 12 หน้านี้: **1** (0:15) → **3** (0:30) → **5** (0:30) → **7** (0:30) → **9** (0:30) → **10** (0:20) → **13** (0:30) → **15** (0:25) → **16** (0:25) → **20** (0:30) → **22** (0:25) → **25** (0:10) รวม 5:00 พอดี หน้าที่ยาวให้กดผ่านเร็ว ๆ หรือซ่อนไว้ ถ้าได้แค่ 1 นาที ใช้สคริปต์ในบท "สคริปต์นำเสนอ 1 นาที และคำศัพท์"

สไลด์ที่ละหน้า: พูดอะไร ชีตตรงไหน

คอลัมน์ "ต้องพูดให้ได้" คือแก่นของหน้านั้น ถ้าตื่นเต้นจนลืมโน้ต พูดแค่นี้ก็ครบ ส่วนคอลัมน์ขวาคือคำถามที่หน้านั้นมักกระตุ้นให้คนถาม พร้อมคำตอบสั้น

#	สไลด์ · เวลา	ต้องพูดให้ได้ / ชัดตรงไหน	ถ้าโดนถาม
1	ปก 0:20	แนะนำตัว + "FR-6 ตอบว่าเงินจะพอใช้อีกกี่วัน และถึงวันไหน" ชีเลข 16 ในวงแหวนทองของภาพหน้าจอ	—
2	วันนี้จะเล่า 5 เรื่อง 0:15	อ่านชื่อ 5 ส่วนเร็ว ๆ ไม่ต้องอธิบาย	—
3	฿3,500 เยอะหรือน้อย? 0:30	ถามผู้ฟังก่อน 1 วินาที แล้วชี้ตาราง: เงินเท่ากัน ใช้วันละ ฿100 อยู่ได้ 30 วัน ใช้วันละ ฿500 อยู่ได้แค่ 6 วัน · แลวไฮไลต์ ฿145 คือตัวอย่างหลัก	ทำไมไม่บอกแค่อยอดเงิน → ยอดเดียวกันความหมายต่างกันตามความเร็วที่ใช้
4	หน้าปิดรถยนต์ 0:25	น้ำมันในถัง = ยอด · ไฟเตือน = เงินสำรอง · อัตรากินน้ำมัน = ค่าเฉลี่ย 7 วัน · "วิ่งได้อีก 120 กม." = "เงินพอใช้อีก 20 วัน"	ทำไมชื่อ Runway → รันเวย์คือระยะที่เหลืก่อนสุดทาง ธุรกิจเรียก cash runway
5	Requirement 0:30	SRS มีประโยคเดียว feedback บอกว่า "Low balance" คำควม ฉันจึงเขียนนิยาม 5 ข้อไว้ทั้งในโค้ดและ TRACEABILITY ทุกข้อกลายเป็นเทสต์ได้	เงินต่ำคือเท่าไร → ยอด ≤ เงินสำรอง ค่าเริ่มต้น ฿500 ตั้งเองได้
6	ตัวเลข 4 ตัว 0:25	ยอด (จาก FR-2) – เงินสำรอง – กระปุกลอย = ใช้ได้จริง ฿3,000 · ตัวที่ 4 ค่าเฉลี่ยคือหัวใจ	ยอดมาจากไหน → ยอดเริ่มต้น + รายรับ – รายจ่าย เฉพาะที่ยืนยันแล้ว
7	ค่าเฉลี่ย 7 วัน 0:30	ชี 2 วันที่เป็น ฿0 "ก็ต้องนับ" · $1,015 \div 7 = ฿145$ · ถ้าหาร 5 ได้ ฿203 สูงเกินจริง · TC-12	ทำไมนับวันที่ไม่ได้ใช้เงิน → เป็นวันจริงในชีวิตไม่นับแล้วเงินจะดูหมดเร็วเกินจริง
8	ผู้ใช้ใหม่ 0:25	ใช้มา 3 วัน ฿300: หาร 7 ได้ 69 วัน (หลอก) · หาร 3 ได้ 30 วัน (จริง) · TC-13	วันแรกนับจากอะไร → วันของรายการที่ยืนยันแล้วรายการแรกสุด รายรับหรือรายจ่ายก็ได้
9	สูตร 5 ชั้น 0:30	ไล่ตามกล่อง: ฿3,000 → 20.69 → 20 วัน → 28 ต.ค. → สบาย ๆ · ชีแถบล่าง: 2 ด้านก่อนหาร และเพดาน 365	ถ้าค่าเฉลี่ยเป็น 0 → ไม่หาร แสดง "ยังคำนวณไม่ได้" (TC-15)
10	ทำไมปิดลง 0:20	ชีกราฟ: วันที่ 20 เหลือ ฿600 ยังเหนือเส้น · วันที่ 21 เหลือ ฿455 ต่ำกว่าเส้นแล้ว · "บอกน้อยไว้ ปลอดภัยกว่าบอกเกิน"	ทำไมไม่ปิดเศษตามปกติ → 20.69 ปิดปกติได้ 21 ซึ่งเกินความจริง
11	5 สถานะ 0:20	< 7 วัน ใกล้เส้นสำรอง · < 14 วัน เริ่มต้องระวัง · ≥ 14 สบาย ๆ + กรณีพิเศษ 2 แบบ · สีเป็นแค่ตัวช่วย มีข้อความเสมอ	ทำไม 7 กับ 14 → คิดเป็นสัปดาห์ เข้าใจง่าย และมีเทสต์ที่ขอบ (TC-17)
12	หน้าหลัก 0:25	เปลี่ยนไปใช้ตัวเลขชุดหน้าจอรจริง · ชี "เงินพอใช้อีก 16 วัน · ถึง 25 ต.ค." · การ์ดทอง: ถ้าอยากถึงสิ้นเดือน ใช้ได้วันละ ฿472	การ์ดทองต่างจาก Runway ยังไง → Runway: ใช้เท่าเดิมอยู่ได้กี่วัน · การ์ดทอง: อยากถึงสิ้นเดือนต้องใช้วันละไม่เกินเท่าไร
13	เก็บเงินพอถึง 0:25	วงแหวนเต็มที่ 45 วัน · การ์ด "คำนวณยังไง" = ใบเสร็จ: $10,859 \div 660 = 16.45 \rightarrow 16$ · "ไม่ใช่กล่องดำ"	ทำไมวงแหวนเต็มที่ 45 → เดือนครึ่ง ถือว่าสบายมากแล้ว
14	เส้นเวลา 30 วัน 0:20	ยอดใน d วัน = ยอด – ค่าเฉลี่ย × d · อีก 30 วันติดลบ ฿8,441 · เขียนกำกับว่ายังไม่รวมรายรับ	ติดลบแปลว่าเป็นหนี้ไหม → ไม่ใช่ แปลว่าถ้าไม่มีเงินเข้าเลยจะไม่พอ
15	ลองปรับ 0:25	เลื่อน 30%: ฿660 → ฿462 (ประหยัด ฿198) → 23 วัน เพิ่ม 7 วัน · ตาราง: ใช้น้อยลงครึ่งหนึ่ง วันเกือบเท่าตัว (20 → 41)	แบบนี้ทำยังงัย → เขียนเองด้วย JavaScript ลาก/แตะ/กดลูกศรได้ โปรแกรมอ่านหน้าจออ่านได้ มี E2E คุม
16	เช็ก่อนจ่าย 0:25	ใส่ ฿2,000: 16 วัน สบาย ๆ → 13 วัน เริ่มต้องระวัง · ปุ่มถามโค้ช (FR-5) · ชีกล่อง purchaseCheck: แอปพลิเคชันเอง AI แค่อธิบาย	ทำไมไม่ให้ AI คิดเลข → AI ภาษาคิดเลขพลาดได้ โค้ดที่ผ่านเทสต์แล้วแม่นยำกว่า
17	เงินสำรองและกระปุก 0:15	เงินสำรองตั้งได้ตั้งแต่ ฿0 · หยอดกระปุก ฿500 → บสรัก "หักเงินที่กินไว้ในกระปุกออม" โฟลเอง → 16 เหลือ 15 วัน	ทำไมต้องหักเงินในกระปุก → ยังอยู่ในบัญชี แต่ตั้งใจไม่ใช้ ไม่หักแล้ว Runway จะดูดีเกินจริง

#	สไลด์ · เวลา	ต้องพูดให้ได้ / ชัดตรงไหน	ถ้าโดนถาม
18	สถานะบนหน้าจอจริง 0:15	ข้อมูลชุดเดียวกัน เปลี่ยนแค่เงินสำรอง → 10 วัน / 4 วัน / ต่ำกว่าเงินสำรอง · ค่าเฉลี่ย ฿660 เท่าเดิมทุกภาพ	—
19	ข้อมูลไหลยังไ 0:30	3 ทางเข้า (FR-1, 3, 4) → กรองที่ยืนยันแล้ว → ยอด + ค่าเฉลี่ย → computeRunway ใน useMoney → 3 ที่แสดง · ไม่เก็บในฐานข้อมูล คิดใหม่ทันทีที่ข้อมูลเปลี่ยนและตอนเที่ยงคืน	ถ้าไม่เก็บใน DB → ตัวเลขไม่ค้าง ไม่ต้องรอเซิร์ฟเวอร์ ไม่ต้องมิงงานคำนวณตอนเที่ยงคืน
20	โค้ดหัวใจ 0:40	ชี้ทีละจุด: บรรทัด 80 ใช้ได้จริง · 83–88 ด้าน 2 ด้าน · 89 ปิดลง · 90–91 เพดาน 365 · 92 สถานะ · ปิดด้วย "เป็น pure function จึงทดสอบง่าย"	pure function คืออะไร → ใส่ข้อมูลเดิมได้ผลเดิมเสมอ ไม่แตะหน้าจอหรือฐานข้อมูล
21	กรณีพิเศษ 0:20	เลือกพูด 3 แถว: ยอดเท่ากับเส้นพอดี · ไม่มีรายจ่าย 7 วัน · เงินเยอะมาก (365+)	เหลือเหนือเส้นนิดเดียว → 0 วัน สถานะ critical (ยังไม่ใช้ below_floor เพราะยังเหนือเส้น)
22	การทดสอบ 0:30	157/157 ผ่าน · TC-12-18 ล็อกวันที่ไว้ที่ 27 ก.ย. 2569 · เน้น TC-17 ทดสอบที่ขอบ 6/7/14 · E2E ลากแถบ ใส่ราคา ตามโค้ช	ทดสอบที่ขอบคืออะไร → ลองค่าที่เกณฑ์พอดี เพราะ $b < a$ กับ $b \leq a$ ซ่อนอยู่ตรงนั้น
23	เหตุการณ์การออกแบบ 0:25	เลือกพูด 3 ใน: ค่าเฉลี่ย 7 วัน · ปิดลงเสมอ · โค้ดคิดเลข AI แค่อธิบาย	ถ้าไม่ใช้ 30 วัน → ซ้ำเกินไป ประหยัดแล้วตัวเลขแถบไม่ขยับ
24	ข้อจำกัด 0:20	ยังไม่รวมรายรับในอนาคต · รายจ่ายก้อนใหญ่ถึงค่าเฉลี่ย 7 วัน · ใกล้เคียงต่อยอดทำได้เพราะสูตรแยกเป็น pure function	แก้รายจ่ายก้อนใหญ่ยังงัย → ใช้ค่ามัธยฐาน หรือให้ผู้ใช้ดีกว่า "ไม่ใช่ปกติ"
25	สรุป 0:15	อ่าน 3 หัวการ: แปลงเงินเป็นเวลา · ปลอดภัยไว้ก่อน · อธิบายตัวเองได้ แล้วขอบคุณ เชิญถาม	ดูตาราง "ตอบใน 1 ประโยค" ด้านล่าง

! ระวังสลับตัวเลข 2 ชุด

สไลด์ 3–11 ใช้ตัวอย่างคิดมือ (฿3,500 · ฿145 · 20 วัน · 28 ต.ค.) ส่วนสไลด์ 12–18 ใช้หน้าจอจริงในโหมดทดลอง (฿11,359 · ฿660 · 16 วัน · 25 ต.ค.) ตอนเปลี่ยนจากสไลด์ 11 ไป 12 ให้พูดเชื่อกว่า "ต่อไปดูหน้าจอจริง ตัวเลขจะเป็นข้อมูลตัวอย่างในแอป แต่สูตรเดียวกัน" ผู้ฟังจะไม่งง

เดโมสด 2 นาที (ถ้าอาจารย์ให้โชว์แอปจริง)

ใช้แทนสไลด์ 12–16 หรือต่อท้ายสไลด์ 18 ก็ได้ เปิดเว็บเวอร์ชันจริงที่ 2550expo-tech.github.io/Socrates-and-Skeletons- (หรือแอปในมือคือ) ไว้ก่อนเริ่มพูด

#	ทำ	พูด
1	กด "ลองใช้ด้วยข้อมูลตัวอย่าง"	"นี่คือข้อมูลตัวอย่างที่แอปสร้างให้ ตัวเลขทั้งหมดคิดสดจากรายการเหล่านี้"
2	ชีการ์ดเขียบบนหน้าหลัก	"เปิดแอปปั๊บ เห็นเลยว่าเงินพอใช้อีกกี่วัน ถึงวันไหน"
3	แตะแท็บ "เงินพอถึง" (ไอคอนใบไม้) เลื่อนลงที่การ์ด "คำนวณยังงัย"	อ่านตัวเลขบนจอ แล้วกดเครื่องคิดเลขในมือถือให้ดู: ใช้ได้จริง ÷ ค่าเฉลี่ย แล้วปิดลง "ได้เท่ากับเลขใหญ่ข้างบน"
4	เลื่อนลงที่เส้นเวลา	"ถ้าใช้แบบนี้ต่อไป แถบสีแดงคือวันที่ต่ำกว่าเส้นสำรองแล้ว"
5	ลากแถบ "ลองปรับ" ไปที่ 30%	"ถ้าใช้น้อยลง 30% เงินพอเพิ่มอีก ... วัน ต้นไม้เล็กออกใบตาม"
6	พิมพ์ 2000 ในช่อง "เช็กก่อนจ่าย"	"ก่อนซื้อ ... วัน ถ้าซื้อเหลือ ... วัน รู้ก่อนจ่ายจริง"

#	ทำ	พูด
7	(ถ้ามีเวลา) กด "ถามโค้ชเรื่องนี้"	"คำถามพร้อมราคาถูกส่งไปรอที่หน้าโค้ชแล้ว แอปคิดตัวเลขก่อนส่งให้ AI แค่อธิบาย"
8	(ถ้ามีเวลา) หน้าหลัก → ไอคอนเฟือง "ตั้งค่า" มุมขวบน → ใส่เงินสำรองสูง ๆ → กด "บันทึก" → กลับไปดูหน้าเงินพอถึง	"สถานะเปลี่ยนทันที เพราะคิดใหม่ทุกครั้งที่ข้อมูลเปลี่ยน" แล้ว ตั้งกลับเป็น 500

1 3 เรื่องที่ต้องรู้ก่อนเดโม

ตัวเลขไม่เท่าสไลด์: ข้อมูลตัวอย่างถูกสร้างตามวันที่กด "ลองใช้ด้วยข้อมูลตัวอย่าง" ครั้งแรก จึงต่างจากภาพในสไลด์ (ถ่ายวันที่ 9 ต.ค.) ไม่ต้องจำตัวเลข อ่านจากจอแล้วคิดด้วยสูตรเดิม เทสต์ TC-33 รับประกันว่าข้อมูลที่สร้างวันไหนก็ตาม มี Runway อย่างน้อย 10 วัน หน้าจอจะไม่แดงตอนเดโม

สร้างข้อมูลตัวอย่างใหม่ในวันนำเสนอ: ข้อมูลตัวอย่างถูกเก็บไว้ในเครื่องหลังสร้างครั้งแรก ถ้าซ่อมด้วยเบราว์เซอร์เดียวกันไว้หลายวันก่อน รายการจะเก่า ค่าเฉลี่ย 7 วันอาจเหลือน้อยจนตัวเลขแปลก ก่อนเดโมให้ไปที่ ตั้งค่า → "ออกจากโหมดทดลอง" แล้วกด "ลองใช้ด้วยข้อมูลตัวอย่าง" ใหม่ (หรือเปิดหน้าต่างแบบไม่ระบุตัวตน) แอปจะสร้างข้อมูลชุดใหม่ของวันนั้น

โค้ชในโหมดทดลองตอบไม่ได้: ถ้ากดส่งคำถามในหน้าโค้ช แอปจะตอบว่า "โหมดทดลองยังถาม AI ไม่ได้นะ เข้าสู่ระบบด้วยบัญชีจริง..." เพราะ AI ทำงานผ่านเซิร์ฟเวอร์ที่ต้องเข้าสู่ระบบ ถ้าอยากใช้คำตอบของ AI ต้องเดโมด้วยบัญชีจริงที่มีรายการแล้ว ไม่เกินหยุดที่ข้อ 7 (คำถามไปรอในช่องพิมพ์) ก็พอ

ขยายจอ: บนเว็บกด Ctrl กับ + ให้ตัวหนังสือใหญ่ขึ้น คนหลังห้องจะเห็นตัวเลข

แผนสำรองเมื่อเกิดเรื่องไม่คาดคิด

อินเทอร์เน็ตล่ม เปิดสไลด์หรือเว็บไม่ได้	เปิดไฟล์ PDF หรือ PowerPoint ที่ดาวน์โหลดไว้ สไลด์ 12–18 เป็นภาพหน้าจอจริงทั้งหมด ใช้แทนเดโมได้เลย
อาจารย์ขอลูกโค้ดจริง	เปิดลิงก์ "เปิดใน VS Code (เว็บ)" ใต้โค้ด runway.ts ในบท "อ่านโค้ด: runway.ts ทั้งไฟล์" หรือเปิด GitHub แล้วไปที่ src/domain/runway.ts บรรทัด 72–94
อาจารย์ขอให้รันเทสต์ใหญ่	เปิด Terminal ในโฟลเดอร์โปรเจกต์ พิมพ์ npm test सोमैतिका चिबुर्ग Tests 157 passed (157) (ต้องเตรียมเครื่องตามขั้นที่ 0 ในบท "สร้าง FR-6 ตั้งแต่ศูนย์" ไว้ก่อน)
ตื่นเต้นจนลืมว่าจะพูดอะไร	ดูคอลัมน์ "ต้องพูดให้ได้" ในตารางข้างบน หรือข้ามไปสไลด์ 25 แล้วพูด 3 ประโยคสรุป
เวลาหมดก่อนจบ	กระโดดไปสไลด์ 25 ทันที พูด 3 ประโยค แล้วจบ ตีกว่าพูดเร็วจนไม่มีใครฟังทัน
โดนถามเรื่องที่ไม่รู้คำตอบ	พูดตรง ๆ ว่า "ขอตรวจในโค้ดก่อนแล้วจะตอบภายหลัง" ห้ามเดา อาจารย์ให้คะแนนความซื่อตรงมากกว่าคำตอบที่มั่ว

ตอบใน 1 ประโยค (ท่องให้คล่อง)

คำตอบเต็มอยู่ในบท "คำถามที่อาจารย์อาจถาม พร้อมคำตอบ" ส่วนนี้คือฉบับย่อสำหรับตอบทันที

- ทำไม 7 วัน?** ครบรอบสัปดาห์ ตามนิสัยล่าสุดกัน และตรงกับ SRS
- ทำไมปิดลง?** บอกน้อยไว้ปลอดภัยกว่า วันที่ปิดขึ้นไม่เงินจะต่ำกว่าเส้นแล้ว
- ไม่มีรายจ่ายเลย?** ไม่หารด้วยศูนย์ แสดง "ยังคำนวณไม่ได้" (TC-15)
- ผู้ใช้ใหม่?** หารด้วยจำนวนวันที่ใช้แอปมาจริง ไม่เกิน 7 (TC-13)
- เงินต่ำคืออะไร?** ยอด $\leq$ เงินสำรองที่ตั้งเอง ค่าเริ่มต้น ฿500 (TC-16)

6	สลิปที่ยังไม่ยืนยัน? ไม่ับ จนกว่าผู้ใช้กดยืนยัน เพราะ AI อาจอ่านผิด
7	เก็บ Runway ในฐานะข้อมูลใหม่? ไม่ คิดสดในเครื่องทุกครั้ง ตัวเลขจึงไม่ค้าง
8	AI คิด Runway เองใหม่? ไม่ โค้ดที่ผ่านเทสต์คิด AI แค่อธิบายเป็นภาษาคน
9	รายรับในอนาคต? ยังไม่ับ เป็นข้อจำกัดที่เขียนบอกไว้บนจอ และอยู่ในแผนต่อไป
10	ชื่อของแพคเกจเดียว? ค่าเฉลี่ยสูงชั่วคราว 7 วันแล้วกลับปกติ แก้ได้ด้วยค่ามัธยฐาน
11	ทำไมเก็บเป็นสตางค์? จำนวนเต็มไม่เพี้ยน 0.1 + 0.2 ในคอมพิวเตอร์ไม่ได้ 0.3 พอดี
12	พิสูจน์ว่าถูกยัง? unit test TC-12-18 ที่ขอบเขต + E2E ในเบราวเซอร์ + ผู้ใช้ตรวจเองได้จากการ์ด "คำนวณยัง?"
13	กระบวนการพัฒนา? นิยามที่ทดสอบได้ → เทสต์ → pure function → useMoney → หน้าจอ → E2E → ปรับตาม feedback (บน "สร้าง FR-6 ตั้งแต่ศูนย์")

เทคนิคการพูดที่ช่วยได้จริง

- ใช้ตัวอย่างเดียวตลอด ฿3,500 สำรอง ฿500 ใช้วันละ ฿145 ได้ 20 วัน ผู้ฟังจำได้ ไม่ต้องคิดตัวเลขใหม่ทุกหน้า
- พูดตัวเลขข้างหลังครึ่งหนึ่ง และชี้ไปที่ตัวเลขบนจอทุกครั้งที่พูดถึง
- เปิดด้วยคำถาม "฿3,500 เยอะหรือน้อย" แล้วหยุด 1 วินาที ให้ผู้ฟังคิดตาม
- พูดว่า "ฉันเลือก... เพราะ..." ทุกครั้งที่เล่าการออกแบบ อาจารย์วิชา Software Engineering อยากรู้ได้ยืนเหตุผล ไม่ใช่แค่ว่าทำอะไร
- บอกข้อจำกัดเองก่อนโดนถาม แสดงว่าเข้าใจระบบจริง (สไลด์ 24)

เช็กลิสต์ก่อนขึ้นพูด

เมื่อไร	ทำอะไร
คืนก่อนวันนำเสนอ	☐ ใส่ชื่อในหน้าปก ☐ ซ้อมจับเวลา 1 รอบให้ได้ประมาณ 10 นาที ☐ ดาวโหลดสไลด์เป็น PDF/PowerPoint ใส่แฟลชไดรฟ์ ☐ อ่านตาราง "ตอบใน 1 ประโยค" 2 รอบ
30 นาทีก่อน	☐ เปิดสไลด์ในเครื่องที่จะใช้ ลองกดเลื่อนหน้า ☐ เปิดเว็บ MindPay ในแท็บถัดไป สร้างข้อมูลตัวอย่างใหม่ของวันนั้น (ออกจากโหมดทดลองแล้วกดลองใช้ใหม่) รอไว้ ☐ ชาร์จมือถือ (ถ้าจะเดโมในแอป)
1 นาทีก่อน	☐ ปิดการแจ้งเตือนในคอมและมือถือ ☐ ขยายจอให้เต็ม ☐ หายใจลึก ๆ แล้วเริ่มด้วยคำถาม "฿3,500 เยอะหรือน้อย"